4 Il processore

In un problema importante, nessun dettaglio è insignificante.

Proverbio francese

4.1 Introduzione

Nel Capitolo 1 abbiamo visto che le prestazioni di un calcolatore sono determinate da tre fattori chiave: numero di istruzioni, durata del ciclo di clock e *numero di cicli di clock per istruzione* (CPI). Il compilatore e l'architettura dell'insieme delle istruzioni, che abbiamo esaminato nel Capitolo 2, determinano il numero di istruzioni richieste da un dato programma. Tuttavia, sia il periodo di clock sia il numero di cicli di clock per istruzione dipendono dalla particolare implementazione del processore. In questo capitolo, costruiremo un'unità di elaborazione dati (*datapath*) e l'unità di controllo (*control unit*) associata a due differenti implementazioni hardware dell'insieme di istruzioni RISC-V.

In particolare, il capitolo fornisce una spiegazione dei princìpi e delle tecniche adottate nella costruzione di un processore, partendo, in questo paragrafo, da un livello di astrazione elevato e da una visione semplificata. Passeremo poi a descrivere passo dopo passo come ideare un'unità di elaborazione e una versione semplificata di processore sufficienti a implementare un insieme di istruzioni come quello del MIPS. Nella parte centrale del capitolo descriveremo un'implementazione più realistica del RISC-V, basata sulla **pipeline**. Infine, seguirà il Paragrafo 4.11 nel quale saranno sviluppati i concetti necessari a implementare insiemi di istruzioni più complessi, come l'x86.

Per i lettori interessati all'interpretazione ad alto livello delle istruzioni e del loro impatto sulle prestazioni di un programma, i concetti che stanno alla base della pipeline sono presentati in questo paragrafo iniziale e nel Paragrafo 4.6. Gli sviluppi più recenti sono descritti nel Paragrafo 4.11, mentre il paragrafo

successivo è dedicato alla descrizione delle architetture Core i7 Intel e Cortex-A8 ARM. Nel Paragrafo 4.13 viene mostrato come sfruttare il parallelismo a livello di istruzioni per raddoppiare le prestazioni della funzione che effettua la moltiplicazione di due matrici, discussa nel Paragrafo 3.9. Questi paragrafi consentono di comprendere ad alto livello il funzionamento di una pipeline.

I Paragrafi 4.3, 4.4 e 4.7 saranno particolarmente utili ai lettori più interessati al funzionamento del processore e alle sue prestazioni, mentre coloro che vogliono comprendere come si costruisce un processore dovrebbero leggere anche i Paragrafi 4.2, 4.8–4.10. Per i lettori interessati alla progettazione moderna dell'hardware, il Paragrafo 4.14 descrive i linguaggi di progettazione dell'hardware e gli strumenti CAD impiegati per implementare l'hardware e per descrivere l'implementazione di una pipeline. Questo paragrafo, inoltre, propone altri esempi di come siano eseguite le istruzioni dall'hardware basato su pipeline.

Un'implementazione di base del RISC-V

Esamineremo ora un'implementazione che comprende il seguente sottoinsieme di istruzioni di base del RISC-V:

- le istruzioni di riferimento alla memoria *load word* (`lw`) e *store word* (`sw`);
- le istruzioni aritmetico-logiche `add`, `sub`, `and` e `or`;
- le istruzioni di salto condizionato dal risultato di un test di uguaglianza, *branch if equal* (`beq`).

Un tale sottoinsieme non comprende tutte le istruzioni sugli interi (per es., sono assenti le istruzioni di scorrimento, moltiplicazione e divisione) e non include neppure alcuna istruzione sui numeri in virgola mobile, ma consente di illustrare i principi chiave utilizzati per costruire un'unità di elaborazione e per progettare l'unità di controllo relativa. L'implementazione delle restanti istruzioni è molto simile.

Nell'esaminare l'implementazione, avremo l'opportunità di comprendere come l'architettura dell'insieme delle istruzioni determini molti aspetti dell'implementazione stessa, e come la scelta tra le varie strategie di implementazione influenzi sia la frequenza di clock sia il CPI del calcolatore. Molti principi chiave introdotti nel Capitolo 2, quale *La semplicità favorisce la regolarità*, trovano diretta applicazione nell'implementazione delle istruzioni. Inoltre, molti concetti introdotti in questo capitolo per implementare il sottoinsieme di istruzioni RISC-V sono gli stessi su cui si basa la realizzazione della maggior parte dei calcolatori, dai server ad alte prestazioni ai microprocessori universali, ai processori embedded.

Una panoramica sull'implementazione

Nel Capitolo 2 abbiamo visto l'insieme di base delle istruzioni RISC-V, comprese le istruzioni aritmetico-logiche sugli interi, le istruzioni di accesso alla memoria e le istruzioni di salto condizionato. Molti componenti necessari per realizzare queste istruzioni sono gli stessi, indipendentemente dal tipo di istruzione. Per ogni istruzione i primi due passi sono identici.

1. Inviare il contenuto del *program counter* (PC) alla memoria che contiene il programma e prelevare l'istruzione dalla memoria; questa operazione viene anche indicata con il termine inglese di *fetch*, letteralmente "raggiungere".
2. Leggere il contenuto di uno o due registri utilizzando i campi dell'istruzione per selezionare i registri. Per l'istruzione *load word* (`lw`) abbiamo bisogno di leggere un solo registro, ma la maggior parte delle altre istruzioni richiede la lettura di due registri.

 © 978-88-08-19966-9

Dopo questi due passi, le azioni richieste per completare l'esecuzione delle istruzioni dipendono dalla loro tipologia. Fortunatamente, per ognuna delle tre classi (istruzioni di accesso alla memoria, aritmetico-logiche e di salto), le azioni da compiere sono, a grandi linee, le stesse, indipendentemente dal particolare codice operativo dell'istruzione. La semplicità e la regolarità dell'insieme di istruzioni RISC-V semplificano l'implementazione rendendo simile l'esecuzione di istruzioni di classi diverse.

Per esempio, tutti i tipi di istruzioni, eccetto i salti incondizionati, utilizzano l'unità aritmetico-logica (ALU) dopo aver letto i registri; le istruzioni di accesso alla memoria utilizzano la ALU per il calcolo dell'indirizzo, le istruzioni aritmetico-logiche per l'esecuzione della loro operazione e i salti condizionati per effettuare i confronti. Dopo avere utilizzato la ALU, le azioni richieste per completare l'esecuzione variano a seconda del tipo di istruzione. Un'istruzione di accesso alla memoria avrà bisogno di accedere alla memoria sia per scrivere i dati (con una store) sia per leggerli (mediante una load). Un'istruzione aritmetico-logica, invece, dovrà scrivere i dati prodotti dalla ALU nel registro destinazione. Infine, in un'istruzione di salto condizionato l'indirizzo dell'istruzione successiva potrà cambiare in base al risultato del confronto; in assenza di salto, il PC verrà incrementato di 4 per puntare all'istruzione successiva.

La **Figura 4.1** mostra uno schema ad alto livello di astrazione dell'implementazione di un RISC-V, focalizzato sulle varie unità funzionali e sulle loro interconnessioni. Sebbene questa figura mostri che la maggior parte del flusso dei dati "scorre" attraverso il processore, omette due importanti aspetti dell'esecuzione di un'istruzione.

Per prima cosa, in diversi punti, la Figura 4.1 mostra dati che provengono da due diverse sorgenti e arrivano alla stessa unità funzionale. Per esempio, il valore scritto nel PC può provenire da uno dei due sommatori ("Add" nella

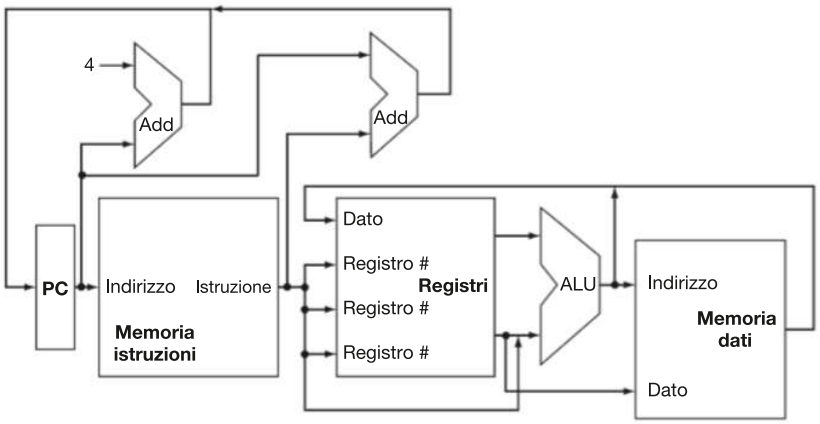


Figura 4.1 Uno schema astratto dell'implementazione di un sottoinsieme delle istruzioni RISC-V che mostra le principali unità funzionali e le principali connessioni tra loro. Tutte le istruzioni iniziano utilizzando il PC per fornire il loro indirizzo alla memoria istruzioni. Dopo che l'istruzione è stata caricata (*fetch*), il numero d'ordine dei registri contenenti gli operandi può essere letto nei campi opportuni dell'istruzione stessa. Una volta caricati gli operandi, si può elaborare il loro contenuto per determinare un indirizzo di memoria (nel caso di load o store), per eseguire un calcolo effettivo (operazioni aritmetico-logiche su interi) o per eseguire un confronto (salto condizionato). Se l'istruzione è di tipo aritmetico-logico, il risultato dalla ALU deve venire scritto in un registro. Se l'operazione è una load o una store, il risultato della ALU viene utilizzato come indirizzo, rispettivamente, per leggere dalla memoria il dato da scrivere nel register file o per scrivere il contenuto di un registro in memoria. I salti condizionati richiedono l'utilizzo dell'uscita della ALU per determinare l'indirizzo dell'istruzione successiva; tale indirizzo può provenire dalla ALU, nella quale PC e offset vengono sommati, oppure da un sommatore apposito che incrementa il PC di 4. Le linee che collegano tra loro le unità funzionali rappresentano i bus, formati da gruppi di segnali di controllo. Le frecce indicano la direzione in cui l'informazione scorre. Dato che le linee dei segnali possono incrociarsi senza necessariamente essere connesse, la connessione tra due linee viene esplicitamente segnalata con un punto.

figura); il dato scritto nel register file può provenire sia dalla ALU sia dalla memoria. Inoltre, il secondo input della ALU può provenire da un registro o dal campo immediato dell'istruzione. In pratica, queste linee multiple per i dati non possono semplicemente essere connesse tra loro, ma dobbiamo introdurre un circuito logico che selezioni da quale, fra le differenti sorgenti collegate, debba essere preso il dato, connettendo la sorgente opportuna con la sua destinazione. Questa selezione viene comunemente effettuata con un dispositivo chiamato *multiplexer*, che funge da "selettore dei dati" (*data selector*).

Il circuito logico del multiplexer viene descritto in dettaglio nell'Appendice A; esso porta in uscita il contenuto di uno degli ingressi a seconda del valore presente sulle sue linee di controllo. Le linee di controllo sono impostate principalmente in base alle informazioni prelevate dall'istruzione stessa che viene eseguita.

Il secondo aspetto omesso nella figura è che i segnali di controllo di molte unità funzionali presenti nello schema dipendono dal tipo di istruzione. Per esempio, la memoria dati deve essere letta da una load, mentre deve essere scritta da una store. Il register file deve essere scritto solamente nel caso di un'istruzione di load o di un'istruzione aritmetico-logica. Naturalmente, la ALU deve poi poter scegliere l'operazione corretta tra le tante che ha a disposizione, come abbiamo visto nel Capitolo 2 (l'Appendice A descrive in dettaglio la progettazione dei circuiti della ALU). Come nei multiplexer, le linee di controllo impostate sulla base del contenuto dei vari campi dell'istruzione dirigono l'elaborazione.

La **Figura 4.2** mostra l'unità di elaborazione della Figura 4.1 integrata con i tre multiplexer richiesti e con le linee di controllo per le principali unità funzionali. L'*unità di controllo*, che ha come ingresso l'istruzione, viene utilizzata per determinare come impostare le linee di controllo per le unità funzionali e per due dei tre multiplexer. Il terzo multiplexer, che determina se nel PC debba essere scritto il valore PC + 4 o l'indirizzo di destinazione del salto, viene controllato dall'uscita "Zero" della ALU; questa uscita è impostata a 1 quando l'uguaglianza tra i due operandi è verificata, e quindi riporta il risultato del confronto richiesto dall'istruzione `beq`. La regolarità e la semplicità dell'insieme delle istruzioni RISC-V consentono di utilizzare un processo di decodifica semplice per impostare le linee di controllo.

Nel resto del capitolo raffineremo questo schema introducendo dei dettagli che richiederanno l'aggiunta di altre unità funzionali, l'aumento del numero di connessioni tra le unità, e, naturalmente, il miglioramento dell'unità di controllo affinché essa sia in grado di stabilire quali azioni debbano essere compiute per ogni diverso tipo di istruzione. I Paragrafi 4.3 e 4.4 descrivono una semplice implementazione che utilizza un singolo ciclo di clock, molto lungo, per ogni istruzione ed è basata sulla forma generale dell'unità di elaborazione descritto nelle **Figure 4.1** e **4.2**. In questa CPU ogni istruzione inizia l'esecuzione su un fronte del segnale di clock e completa l'esecuzione sul fronte di clock successivo.

Sebbene sia facile da comprendere, questo approccio non è pratico, dal momento che il ciclo di clock deve essere allungato di molto per dare il tempo alle istruzioni più lente di terminare l'esecuzione. Dopo aver progettato l'unità di controllo di questo semplice calcolatore, esamineremo implementazioni più veloci con tutte le lore complessità, compresa la gestione delle eccezioni.

Autovalutazione

Quanti dei cinque componenti classici di un calcolatore (unità di controllo, processore, memoria, input e output) sono mostrati nelle Figure 4.1 e 4.2?

 © 978-88-08-19966-9

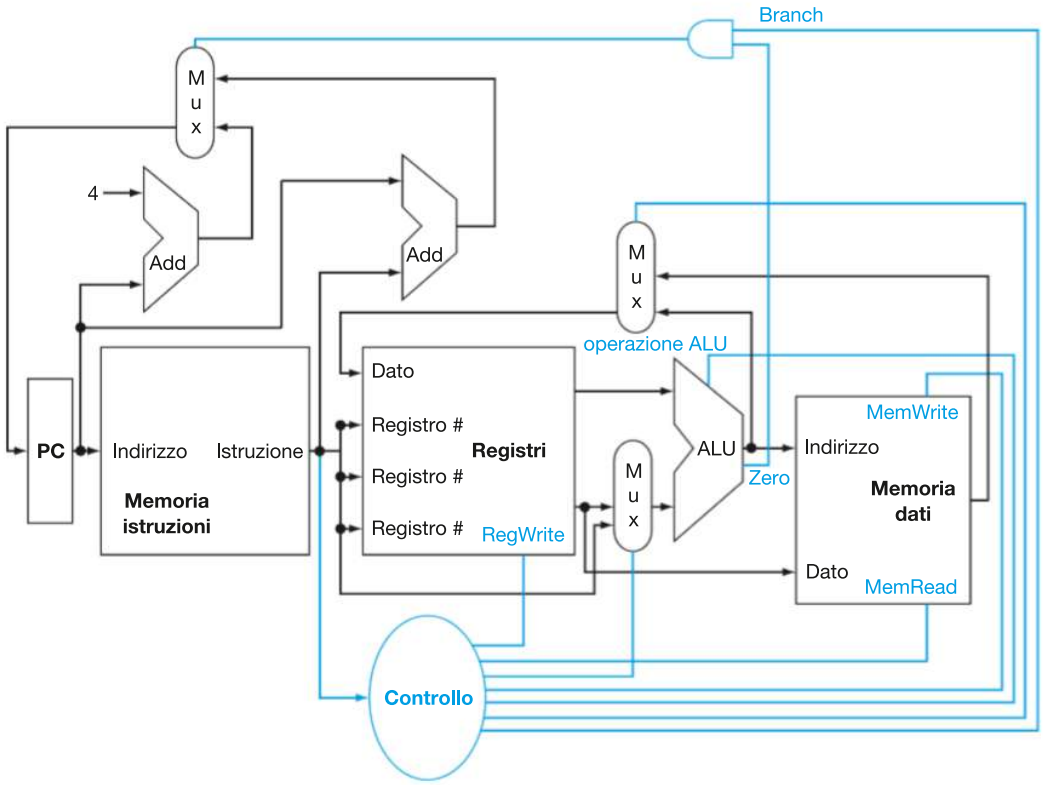


Figura 4.2 Implementazione base del sottoinsieme di istruzioni RISC-V, comprendente anche i multiplexer necessari e le linee di controllo. Il multiplexer "Mux" in alto controlla ciò che viene scritto nel PC: PC + 4 oppure l'indirizzo di destinazione del salto condizionato; questo multiplexer viene controllato da una porta logica che esegue l'AND tra il segnale di Zero in uscita dalla ALU e un segnale di controllo che indica che l'istruzione è un'istruzione di salto condizionato. Il multiplexer posizionato centralmente nella parte alta della figura, la cui uscita ritorna al register file, viene utilizzato per portare l'uscita della ALU (in caso di istruzione aritmetico-logica) oppure l'uscita della memoria dati (in caso di load) al register file per la scrittura. Infine l'ultimo multiplexer, posizionato più in basso, viene utilizzato per determinare se il secondo ingresso della ALU provenga dai registri (per un'istruzione aritmetico-logica oppure per un'istruzione di salto condizionato) o dal campo costante dell'istruzione stessa (per un'operazione di load o store). Le linee di controllo aggiunte hanno un significato evidente e determinano le operazioni che devono essere eseguite dalla ALU oppure se la memoria debba essere letta o scritta o ancora se si debba eseguire un'operazione di scrittura sul register file. Le linee di controllo sono mostrate in blu in modo da risultare più visibili.

4.2 Convenzioni del progetto logico

Nell'analizzare il progetto di un calcolatore, dobbiamo decidere come funzionerà la logica hardware che implementa il calcolatore e come esso dovrà essere sincronizzato. Questo paragrafo riprende alcuni concetti fondamentali di logica digitale che verranno utilizzati largamente in tutto il capitolo. Il lettore che non abbia una buona preparazione nel campo della progettazione dei circuiti digitali, troverà utile leggere l'Appendice A prima di proseguire oltre.

Gli elementi funzionali che costituiscono l'unità di elaborazione del RISC-V sono costituiti da due diverse classi di elementi logici: elementi che operano sui dati ed elementi che contengono lo stato. Gli elementi che operano sui dati sono di tipo **combinatorio**, il che significa che in ogni istante i loro output dipendono solamente dagli input ricevuti nello stesso istante. Dati gli stessi valori di ingresso, un elemento combinatorio produrrà sempre il medesimo valore in uscita. La ALU nella Figura 4.1, e descritta nell'Appendice A, costituisce un esempio di elemento combinatorio: dato un certo insieme di valori in ingresso, essa produce sempre la stessa uscita perché non possiede alcun elemento interno di memoria.

Elemento combinatorio: un elemento in grado di eseguire delle operazioni, per esempio una porta AND o una ALU.

Gli elementi dell'unità di elaborazione che non sono combinatori contengono lo *stato*. Un elemento può contenere lo stato solo se ha al suo interno elementi di memoria. Chiameremo questi **elementi di stato**, perché se spegniamo e riaccendiamo il calcolatore, possiamo far ripartire il flusso di lavoro del calcolatore esattamente dallo stesso punto caricando gli elementi di stato con i valori che essi contenevano prima di avere spento. Inoltre, salvando e ripristinando gli elementi di stato, il calcolatore si comporterebbe come se non fosse mai stato spento. Di conseguenza, gli elementi di stato caratterizzano completamente il funzionamento del calcolatore. Nella Figura 4.1 la memoria istruzioni, la memoria dati e i registri sono esempi di elementi di stato.

Un elemento di stato possiede almeno due ingressi e un'uscita. Gli ingressi richiesti sono il valore da scrivere nell'elemento e il clock, che determina quando scrivere. L'uscita di un elemento di stato è il valore contenuto al suo interno, scritto in un ciclo di clock passato. Per esempio, uno degli elementi di stato più semplici, dal punto di vista logico, è il *flip-flop* di tipo D (*vedi* Appendice A), che ha esattamente questi due ingressi (il valore del dato e il clock) e un'uscita. Oltre ai flip-flop, l'architettura RISC-V utilizza altri due tipi di elementi di stato: le memorie e i registri, che compaiono anch'essi nella Figura 4.1. Il clock viene utilizzato per determinare quando l'elemento di stato deve essere scritto, mentre la lettura può avvenire in qualsiasi istante.

I componenti logici che contengono lo stato sono anche detti *sequenziali*, in quanto le loro uscite dipendono sia dagli ingressi sia dal valore del loro stato interno. Per esempio, l'uscita dell'unità funzionale che contiene i registri dipende sia dal numero del registro specificato sugli ingressi sia dal valore contenuto da quel particolare registro, che era stato scritto in precedenza. L'Appendice A descrive in modo dettagliato il funzionamento e la struttura degli elementi combinatori e di quelli sequenziali.

Elemento di stato: un elemento di memoria, per esempio un registro o una memoria.

Metodologia di temporizzazione

La **metodologia di temporizzazione** definisce quando i segnali possono essere scritti e quando possono essere letti. È importante temporizzare le operazioni di lettura e scrittura, perché se un segnale venisse letto e contemporaneamente scritto, il valore letto potrebbe corrispondere al valore precedente la scrittura, a quello successivo, o addirittura a una combinazione dei due. Ovviamente i progettisti dei calcolatori non possono accettare questa imprevedibilità nei risultati, che viene evitata proprio grazie alla metodologia di temporizzazione.

Per semplicità, supporremo di utilizzare una metodologia di **temporizzazione sensibile ai fronti** (*edge-triggered*); essa garantisce che il valore memorizzato all'interno di un elemento logico sequenziale venga aggiornato solamente in corrispondenza di un fronte del segnale di clock, che è una veloce transizione dal livello basso al livello alto e viceversa (**Figura 4.3**). Poiché solo gli elementi di stato possono memorizzare i dati, un qualsiasi circuito combinatorio deve ricevere l'input da un insieme di elementi di stato e può scrivere il risultato dell'elaborazione in un secondo insieme di elementi di stato. Gli ingressi sono i valori che erano stati scritti in un ciclo di clock precedente, mentre gli output del circuito combinatorio sono valori che potranno essere utilizzati in un ciclo di clock successivo. La Figura 4.3 mostra un blocco di logica combinatoria posto tra due elementi di stato. Esso lavora all'interno di un singolo ciclo di clock: tutti i segnali si devono propagare a partire dall'elemento di stato 1 e devono attraversare la logica combinatoria fino a raggiungere l'elemento di stato 2 nel tempo di un periodo di clock. Il tempo richiesto perché i segnali raggiungano l'elemento di stato 2 definisce quindi la durata minima del periodo di clock. Per semplicità, non si riporterà esplicitamente il **segnale di controllo** della scrittura per quegli elementi di stato che vengono scritti

Metodologia di temporizzazione: l'approccio utilizzato per determinare quando un dato è valido e stabile in relazione al segnale di clock.

Temporizzazione sensibile ai fronti: una tecnica di temporizzazione nella quale tutti i cambiamenti di stato avvengono su un fronte del segnale di clock.

Segnale di controllo: un segnale utilizzato per la selezione in un multiplexer o per dirigere il funzionamento di un'unità funzionale; si distingue dai segnali che portano i dati, che contengono l'informazione che viene elaborata dalle unità funzionali.

 © 978-88-08-19966-9

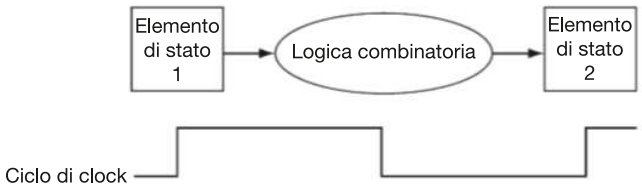


Figura 4.3 Logica combinatoria, elementi di stato e clock sono strettamente correlati. In un sistema digitale sincrono, il segnale di clock determina quando gli elementi di stato scrivono la loro memoria interna. Gli ingressi a un elemento di stato devono raggiungere un valore stabile (cioè devono aver raggiunto un valore che non cambierà almeno fino a dopo il fronte di clock successivo) prima che il fronte attivo del clock provochi l'aggiornamento dello stato. In questo capitolo, supporremo che tutti gli elementi di stato, comprese le memorie, siano sensibili al fronte di salita, cioè cambino contenuto quando il clock passa dallo stato basso allo stato alto.

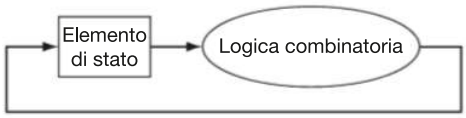


Figura 4.4 Una metodologia sensibile ai fronti permette di leggere e scrivere un elemento di stato nello stesso ciclo di clock, senza creare una competizione che potrebbe causare indeterminazione nel contenuto dell'elemento. Ovviamente, il ciclo di clock deve avere una durata sufficiente a garantire che gli ingressi siano stabili quando arriva il fronte attivo del clock. Non si può verificare alcuna retroazione all'interno del singolo ciclo di clock, grazie all'aggiornamento sul fronte del clock dell'elemento di stato. Se fosse possibile aggiornare l'elemento di stato prima del fronte di clock, il circuito non potrebbe funzionare correttamente. I circuiti presentati in questo capitolo e nel prossimo si basano sulla metodologia di temporizzazione sensibile ai fronti e su strutture simili a quella mostrata in questa figura.

a ogni fronte attivo del clock. Viceversa, se un elemento di stato non viene aggiornato a ogni ciclo di clock, allora si renderà necessario un segnale esplicito di controllo della scrittura. Sia il segnale di clock sia il segnale di controllo della scrittura sono ingressi, e il contenuto dell'elemento di stato può essere modificato solamente sul fronte attivo del clock, quando cioè il segnale di controllo della scrittura è asserito.

Asserito: un segnale associato a un livello logico alto o vero.

Deasserito: un segnale associato a un livello logico basso o falso.

Utilizzeremo il termine **asserito** per indicare un segnale che si trova nello stato logico alto e il termine *asserire* per indicare che un segnale deve essere portato al livello logico alto; utilizzeremo il termine **non asserito** per indicare un segnale che si trova nello stato logico basso e il termine *non asserire* per indicare che un segnale deve essere portato al livello logico basso. Abbiamo introdotto i termini "asserire" e "non asserire" perché un 1 può rappresentare a volte un valore logico alto e altre volte un valore logico basso.

La metodologia sensibile ai fronti permette di leggere il contenuto di un registro, inviare il valore attraverso uno o più blocchi di logica combinatoria e scrivere lo stesso registro nello stesso ciclo di clock, come mostrato nella Figura 4.4. Non occorre specificare se la scrittura avvenga sul fronte di salita (da basso ad alto) o di discesa (da alto a basso) del segnale di clock, dal momento che gli ingressi a un circuito logico combinatorio possono cambiare solo sul fronte attivo del clock. In questo testo utilizzeremo il fronte di salita del clock. Con la metodologia di temporizzazione sensibile ai fronti non si corre il rischio di innescare una retroazione all'interno dello stesso ciclo di clock, e il circuito logico illustrato nella Figura 4.4 funziona correttamente. Nell'Appendice A sono discussi brevemente alcuni vincoli temporali aggiuntivi (come i tempi di set-up e di hold) e vengono presentate altre metodologie di temporizzazione.

Quasi tutti gli elementi di stato e combinatori dell'architettura RISC-V a 32 bit hanno ingressi e uscite di ampiezza pari a 32 bit, essendo questa l'ampiezza della maggior parte dei dati elaborati dal processore. Quando incontreremo

unità con ingressi o uscite di ampiezza diversa da 32 bit la loro presenza sarà indicata esplicitamente. Nelle figure, i bus, ossia le linee che trasportano segnali di ampiezza maggiore di 1 bit, saranno indicati con linee più spesse. In alcuni casi due o più bus verranno uniti per formare un bus di ampiezza maggiore; per esempio, possiamo ottenere un bus da 32 bit unendo due bus da 16 bit. In questi casi, etichette sulle linee indicheranno chiaramente che due bus sono stati concatenati per ottenere un unico bus. Verranno inoltre aggiunte delle frecce per indicare la direzione del flusso dei dati tra gli elementi. Infine, un diverso **colore** consentirà di distinguere i segnali di controllo dai dati, distinzione che diverrà più chiara procedendo con la lettura del capitolo.

Autovalutazione

Vero o falso? Poiché il register file può essere letto e scritto nello stesso ciclo di clock, le unità di elaborazione RISC-V che utilizzano la metodologia di temporizzazione sensibile ai fronti devono possedere più di una copia del register file.

Approfondimento. Esiste anche una versione dell'architettura RISC-V a 64 bit. Naturalmente, la maggior parte dei cammini interni di questa architettura è a 64 bit.

4.3 Realizzazione di un'unità di elaborazione

Un modo ragionevole per iniziare la costruzione di un'unità di elaborazione consiste nell'esaminare i componenti principali che servono per eseguire i diversi tipi di istruzioni RISC-V. Procediamo dall'inizio, analizzando quali **elementi dell'unità di elaborazione** (*datapath elements*) siano richiesti da ciascuna classe di istruzioni per poi approfondire i diversi elementi procedendo attraverso i diversi livelli di **astrazione**. Quando mostreremo i diversi elementi dell'unità di elaborazione, mostreremo anche i relativi segnali di controllo associati a quel livello di astrazione.

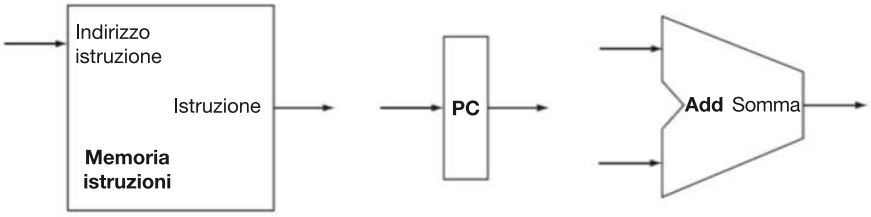
Elemento dell'unità di elaborazione: un'unità funzionale utilizzata per eseguire operazioni o memorizzare i dati all'interno del processore. Nell'implementazione RISC-V, gli elementi dell'unità di elaborazione sono la memoria dati, la memoria istruzioni, il register file, l'unità aritmetico-logica (ALU) e i sommatori.

La **Figura 4.5a** mostra il primo elemento di cui abbiamo bisogno: un'unità di memoria in cui salvare le istruzioni del programma che sia in grado di fornire in uscita l'istruzione associata all'indirizzo dato in ingresso. La **Figura 4.5b** mostra anche il **program counter** (**PC**) che, come abbiamo già visto nel Capitolo 2, è un registro utilizzato per memorizzare l'indirizzo dell'istruzione corrente. Infine, è necessario un sommatore per incrementare di 4 il PC e ottenere l'indirizzo dell'istruzione successiva. Tale sommatore è un elemento combinatorio e può essere costruito a partire dalla ALU descritta in dettaglio nell'Appendice A; è sufficiente impostare i segnali di controllo della ALU in modo che venga specificata sempre un'operazione di somma. Identificheremo negli schemi circuitali una ALU di questo tipo, configurata permanentemente come sommatore, con l'etichetta *Add*, come mostrato nella **Figura 4.5c**; essa non può eseguire le altre operazioni tipiche di una ALU.

Program counter (PC): il registro che contiene l'indirizzo dell'istruzione in esecuzione di un programma.

Per eseguire una qualsiasi istruzione occorre anzitutto prelevare l'istruzione stessa dalla memoria. Per prepararsi a eseguire l'istruzione successiva, bisogna incrementare il program counter in modo che punti a quell'istruzione, cioè 4 byte dopo. La **Figura 4.6** mostra come si possono combinare i tre elementi della Figura 4.5 per formare quella parte dell'unità di elaborazione che preleva le istruzioni e incrementa il PC per ottenere l'indirizzo dell'istruzione successiva del programma.

 © 978-88-08-19966-9



(a) Memoria istruzioni **(b)** Program counter (PC) **(c)** Adder

Figura 4.5 Sono necessari due elementi di stato per memorizzare e accedere alle istruzioni, e serve un sommatore per calcolare l'indirizzo dell'istruzione successiva. Gli elementi di stato sono la memoria delle istruzioni e il program counter (PC). La memoria delle istruzioni fornisce solamente un accesso in lettura, perché essa non viene scritta dall'unità di elaborazione. Per questo tratteremo la memoria istruzioni come un circuito logico combinatorio: l'uscita in ogni istante corrisponde al contenuto della locazione specificata dall'indirizzo in ingresso, e non è necessario alcun segnale di controllo esplicito. La memoria delle istruzioni dovrà essere scritta solamente nella fase di caricamento del programma; non è difficile aggiungere questa funzionalità e quindi, per semplicità, in questo libro non la considereremo. Il program counter è un registro a 32 bit che viene scritto al termine di ogni ciclo di clock e non richiede quindi un segnale di controllo esplicito per la scrittura. Il sommatore è una ALU configurata in modo da eseguire sempre la somma dei suoi due ingressi su 32 bit e fornire in uscita il risultato della somma su 32 bit.

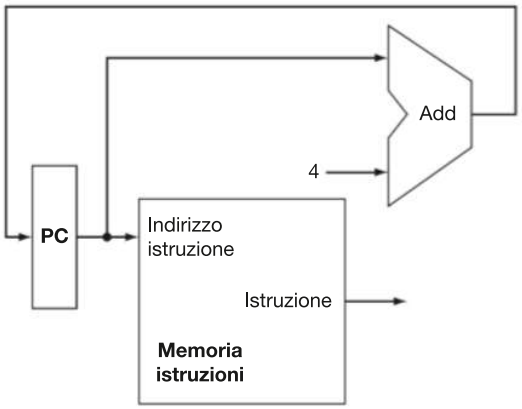


Figura 4.6 Parte dell'unità di elaborazione utilizzata per prelevare le istruzioni e incrementare il program counter. L'istruzione prelevata viene utilizzata dalle altre parti dell'unità di elaborazione.

Consideriamo ora le istruzioni in formato R (*vedi* Figura 2.19). Tutte le istruzioni di questo tipo leggono due registri, eseguono un'operazione con la ALU sul contenuto di questi due registri e, alla fine, scrivono il risultato in un registro. Chiameremo queste *istruzioni di tipo R* o *istruzioni aritmetico-logiche*, dato che eseguono operazioni aritmetiche o logiche. Questa classe di istruzioni comprende `add`, `sub`, `and` e `or`, introdotte nel Capitolo 2. Un tipico esempio di istruzione di questo tipo è `add x1, x2, x3`, che legge il contenuto di `x2` e `x3` e scrive il risultato dell'addizione nel registro `x1`.

I registri universali a 32 bit del processore sono raccolti in una struttura detta **register file** (banco di registri), un insieme di registri in cui ciascun registro può essere letto o scritto specificando il numero a esso associato all'interno dell'insieme. Il register file contiene lo stato dei registri del calcolatore. Avremo inoltre bisogno di una ALU per operare sui valori letti dai registri.

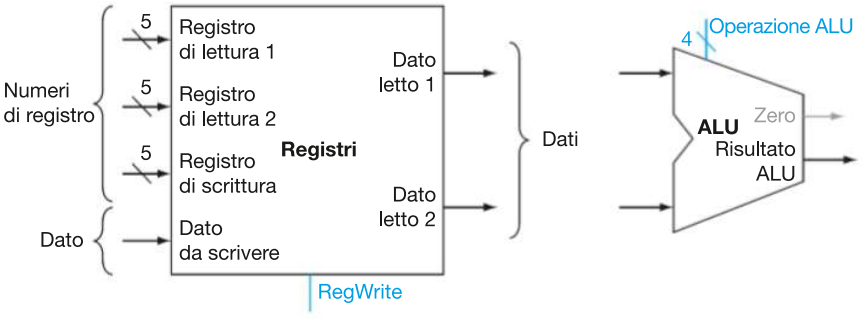
Register file: un elemento dello stato che contiene un insieme di registri che possono essere letti o scritti fornendo il numero del registro sul quale si vuole operare.

Poiché le istruzioni di tipo R hanno tre registri come operandi, per ciascuna istruzione dovremo leggere due dati di una parola ciascuno dal register file e poi scrivere il risultato, sempre nel register file. Per ogni parola letta dai registri è necessario prevedere un ingresso al register file che specifichi il *numero*

del registro da leggere e un'uscita dal register file sulla quale viaggerà il valore letto dal registro. Per scrivere un dato di una parola serviranno due ingressi: il primo ingresso deve specificare il *numero del registro di scrittura* e il secondo deve fornire il *dato* da scrivere nel registro. Il register file fornisce in qualunque momento in uscita il contenuto del registro il cui numero d'ordine è specificato sull'ingresso relativo al *numero del registro di lettura*. La scrittura, invece, viene controllata da un segnale di controllo esplicito, "RegWrite", che deve essere asserito perché il registro possa essere scritto in corrispondenza del fronte del clock. Quindi serviranno complessivamente quattro ingressi (tre per i numeri del registro e uno per il dato da scrivere) e due uscite (entrambe per i dati), come mostrato nella **Figura 4.7a**. Gli ingressi che specificano il numero dei registri hanno ampiezza di 5 bit, in modo da poter specificare uno dei 32 registri ($32 = 2^5$), mentre i bus dei dati in ingresso e in uscita sono di 32 bit ciascuno.

La **Figura 4.7b** mostra una ALU che riceve due input su 32 bit e produce un risultato su 32 bit; inoltre, produce un segnale a un bit che vale 1 se il risultato dell'operazione è 0. I segnali di controllo della ALU, su 4 bit, vengono descritti in dettaglio nell'Appendice A; li rivedremo brevemente quando dovremo definire il circuito che ne imposta il valore opportuno.

Consideriamo ora le istruzioni RISC-V di caricamento di un registro (load) e di trasferimento alla memoria (store). Esse hanno la forma generale `lw x1, offset(x2)` e `sw x1, offset(x2)`, e calcolano un indirizzo di memoria sommando il contenuto del registro base, in questo caso `x2`, al campo offset di 12 bit, contenuto nell'istruzione stessa, considerato dotato di segno. Se l'istruzione è una store, il dato da memorizzare deve essere letto dal register file dove risiede in `x1`, mentre se è una load, il valore letto dalla memoria deve essere scritto all'interno del register file nel registro specificato, in questo caso `x1`. Quindi, per eseguire queste istruzioni occorreranno sia il register file sia la ALU, come riportato nella Figura 4.7.



(a) Registri **(b)** ALU

Figura 4.7 I due elementi richiesti per implementare le operazioni di tipo R sono il register file e la ALU. Il register file contiene tutti i registri e dispone di due porte di lettura e una di scrittura. Lo schema circuitale di un register file a più porte è discusso nel Paragrafo A.8 dell'Appendice A. In ogni istante il register file fornisce in uscita il contenuto dei registri corrispondenti agli ingressi "Registro di lettura", senza richiedere alcun segnale di controllo; viceversa, la scrittura di un registro deve essere indicata esplicitamente asserendo il segnale di controllo della scrittura, "RegWrite". Si ricordi che la scrittura avviene sul fronte attivo del clock, per cui tutti gli ingressi interessati (cioè il dato da scrivere, il numero di registro e il segnale di controllo) devono essere validi sul fronte attivo del clock. Essendo la scrittura attiva sui fronti, l'architettura implementata nel nostro progetto può tranquillamente leggere e scrivere lo stesso registro nello stesso ciclo di clock: la lettura fornirà il dato contenuto nel registro scritto in un ciclo di clock precedente, mentre il valore scritto potrà essere letto a partire dal ciclo di clock successivo. Gli ingressi che specificano al register file il numero d'ordine dei registri hanno ampiezza di 5 bit, mentre i bus che trasportano i dati sono ampi 64 bit. L'operazione che la ALU deve eseguire viene specificata dal segnale "Operazione ALU", che è ampio 4 bit per la ALU descritta nell'Appendice A. L'uscita "Zero" della ALU, verrà utilizzata tra breve per implementare i salti condizionati.

Estensione del segno: aumento della dimensione di un dato ottenuta replicando il bit più significativo, che rappresenta il segno del dato originale, nei bit più significativi del dato di destinazione.

Indirizzo di destinazione del salto: l'indirizzo che viene specificato in un salto condizionato; esso diventerà il nuovo indirizzo contenuto nel program counter (PC) se il salto condizionato viene eseguito. Nell'architettura RISC-V, l'indirizzo di destinazione di un salto è ottenuto mediante la somma del campo offset contenuto nell'istruzione stessa con l'indirizzo dell'istruzione di salto condizionato.

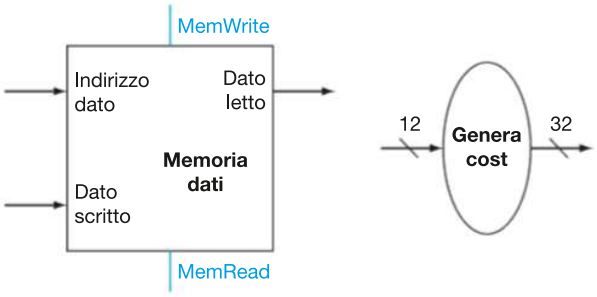
Inoltre, saranno necessarie anche un'unità per l'**estensione del segno** del campo offset su 12 bit, contenuto nell'istruzione, da estendere a 32 bit, con segno, e un'unità di memoria dati in cui scrivere o da cui leggere il dato. La memoria dati viene scritta dalle istruzioni di store, e sarà quindi dotata dei segnali di controllo sia di lettura sia di scrittura; riceverà in ingresso, inoltre, l'indirizzo e il dato che deve essere scritto. La **Figura 4.8** mostra questi due nuovi elementi.

L'istruzione `beq` ha tre operandi: due registri il cui contenuto viene confrontato per determinare se è uguale, e un offset di 16 bit utilizzato per calcolare l'**indirizzo di destinazione del salto** a partire dall'indirizzo dell'istruzione stessa. La forma di questa istruzione è `beq x1, x2, offset`; per eseguirla, occorre calcolare l'indirizzo di destinazione del salto, sommando il campo offset dell'istruzione (dopo averlo esteso a 32 bit con segno) al PC. Nella definizione delle istruzioni di salto condizionato ci sono due dettagli (Capitolo 2) ai quali è bene prestare attenzione.

- L'architettura dell'insieme delle istruzioni specifica che l'indirizzo di base per il calcolo dell'indirizzo di salto è quello dell'istruzione di salto stessa.
- L'architettura stabilisce anche che il campo offset sia spostato di 1 bit a sinistra, in modo tale che l'offset non codifichi lo spiazzamento in numero di byte, ma in numero di mezze parole. Di fatto, tale spostamento aumenta lo spazio di indirizzamento dell'offset di un fattore 2 rispetto alla codifica dello spiazzamento in byte.

Per poter gestire quest'ultima complicazione è necessario far scorrere il campo offset di 1 bit a sinistra.

Oltre a calcolare l'indirizzo di destinazione del salto, bisogna determinare se l'istruzione da eseguire dopo sia quella presente nella posizione di memoria successiva oppure quella contenuta all'indirizzo di destinazione del salto. Quando la condizione di salto è vera (cioè quando i due operandi sono uguali),



(a) Unità di memoria dati **(b)** Unità di estensione del segno

Figura 4.8 Le due unità necessarie per l'implementazione delle istruzioni di load e di store: oltre al register file e alla ALU mostrate nella Figura 4.7, sono richieste anche un'unità di memoria dati e un'unità di estensione del segno. L'unità di memoria è un elemento di stato che ha come ingressi l'indirizzo del dato e il dato da scrivere, e possiede una sola uscita per il dato letto. I segnali di controllo per la lettura e la scrittura sono separati, anche se all'interno di un ciclo di clock solo uno dei due può essere asserito. L'unità di memoria ha bisogno di un segnale di lettura esplicito, dal momento che, diversamente da quanto avviene per il register file, leggere il contenuto di un indirizzo non valido può causare dei problemi, come vedremo nel Capitolo 5. L'unità di generazione della costante riceve in ingresso un numero a 12 bit per una load, una store e un salto condizionato, e fornisce in uscita un valore esteso a 32 bit dotato di segno (Capitolo 2). Si suppone che la scrittura della memoria sia attiva su un fronte del segnale di clock. I chip di memoria standard, in realtà, possiedono un segnale di abilitazione alla scrittura che permette, quando asserito, l'esecuzione dell'operazione di scrittura. Sebbene il segnale di abilitazione alla scrittura nei chip di memoria reali non sia sensibile ai fronti, l'unità di elaborazione attiva sui fronti presentata in questo capitolo può essere facilmente adattata per funzionare anche con i chip di memoria reali. Per una discussione più approfondita sul funzionamento dei chip di memoria reali *vedi* Paragrafo A.8 dell'Appendice A.

l'indirizzo di destinazione del salto diventa il nuovo valore del PC e si parla di **salto preso** (*branch taken*, salto eseguito); se il contenuto degli operandi è diverso, il valore del PC incrementato di 4 sostituisce il valore corrente nel PC, proprio come in tutte le altre istruzioni; in questo caso si parla di **salto non preso** (*branch not taken*, salto non eseguito).

Per i salti condizionati, quindi, l'unità di elaborazione deve eseguire due operazioni: calcolare l'indirizzo di destinazione del salto e confrontare il contenuto di due registri. Inoltre, come si vedrà tra poco, i salti hanno anche un effetto sulla parte dell'unità di elaborazione incaricata di prelevare le istruzioni. La parte dell'unità di elaborazione che gestisce i salti è visualizzata nella **Figura 4.9**. Per il calcolo dell'indirizzo di destinazione, si utilizzano un'unità di estensione del segno, analoga a quella mostrata nella Figura 4.8, e un sommatore. Per eseguire il confronto possiamo utilizzare il register file mostrato nella Figura 4.7a, che ci permette di leggere il contenuto dei due registri contenenti gli operandi; osserviamo che, in questo caso, non è richiesto di scrivere il risultato nel register file. Il confronto tra i due operandi viene fatto dalla ALU descritta nell'Appendice A.

Dato che la ALU fornisce un segnale in uscita che indica se il risultato è pari a 0, si possono inviare i due operandi alla ALU specificando tramite i segnali di controllo di effettuare una sottrazione: se l'uscita Zero della ALU viene asserita, i due operandi sono uguali. Segnaliamo esplicitamente che, anche se l'uscita Zero indica sempre che il risultato di un'operazione è 0, noi la

Salto preso: un salto condizionato nel quale la condizione di salto è soddisfatta; in questo caso nel program counter (PC) viene scritto l'indirizzo di destinazione del salto. Le istruzioni di salto incondizionato possono essere assimilate a dei salti condizionati che vengono sempre eseguiti.

Salto non preso: un salto condizionato in cui la condizione di salto non è soddisfatta; in questo caso nel program counter (PC) viene scritto l'indirizzo dell'istruzione successiva all'istruzione di salto condizionato.

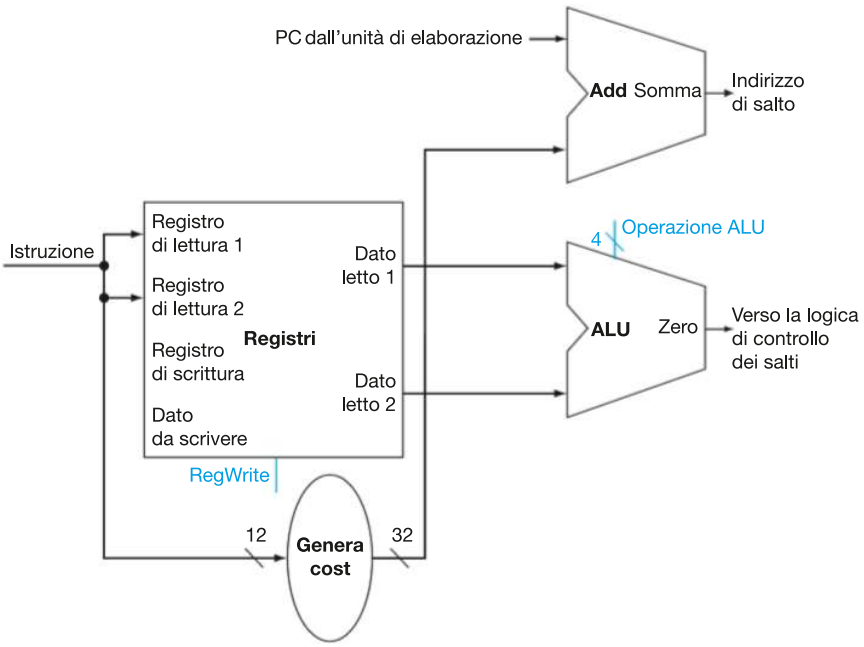


Figura 4.9 La porzione del cammino di elaborazione dei dati per i salti condizionati impiega la ALU per valutare la condizione di salto e un sommatore a parte per determinare l'indirizzo di destinazione del salto, calcolato come somma del PC e il contenuto dei 12 bit del campo immediato dell'istruzione (lo spiazzamento del salto), fatti scorrere a sinistra di 1 bit ed estesi a 32 bit con segno. L'unità etichettata "Shift a sinistra 1" corrisponde semplicemente a un particolare instradamento dei segnali che rappresentano lo spiazzamento; l'instradamento è effettuato in modo tale che, a valle dell'estensione del segno, venga aggiunto un segnale uguale a 0_{due} all'estremità inferiore del campo offset: poiché l'entità dello spostamento è costante, non è richiesto alcun componente dedicato. Sapendo che lo spiazzamento viene esteso a partire dal 12° bit, questa operazione di scorrimento scarterà solamente dei "bit di segno". La logica di controllo deciderà, in funzione dell'uscita Zero della ALU, se nel PC debba essere caricato il contenuto del PC incrementato di 4 oppure l'indirizzo di destinazione del salto.

 © 978-88-08-19966-9

utilizzeremo solamente per il test di uguaglianza dei salti condizionati. In seguito, vedremo precisamente come debbano essere collegati i segnali di controllo della ALU all'interno dell'unità di elaborazione.

L'istruzione di salto condizionato opera sommando al PC i 12 bit dell'istruzione fatti scorrere a sinistra di 1 posizione. Questo scorrimento si ottiene semplicemente concatenando degli 0 all'offset dell'istruzione di salto condizionato, come già descritto nel Capitolo 2.

Progettazione di un'unità di elaborazione unificata

Ora che abbiamo esaminato i componenti dell'unità di elaborazione necessari a eseguire i diversi tipi di istruzioni, possiamo combinarli in una singola unità di elaborazione aggiungendo i segnali di controllo opportuni. L'unità di elaborazione più semplice cercherà di eseguire tutte le istruzioni in un singolo ciclo di clock. Quindi, nessuna risorsa dell'unità di elaborazione può essere utilizzata più di una volta per ogni istruzione e ogni elemento funzionale utilizzato più di una volta deve essere duplicato: perciò servirà una memoria delle istruzioni separata da quella dei dati. Anche se dovremo duplicare alcune unità funzionali, molti elementi potranno essere condivisi dal flusso di esecuzione di istruzioni diverse.

La condivisione da parte di istruzioni di tipo diverso di uno stesso elemento potrebbe richiedere la presenza di collegamenti multipli agli ingressi di quell'elemento; in questo caso occorrerà introdurre un multiplexer, con i suoi segnali di controllo, per poter selezionare l'input corretto all'unità funzionale tra tutti gli input possibili.

Costruzione di un'unità di elaborazione

ESEMPIO

L'unità di elaborazione per le istruzioni aritmetico-logiche (o di tipo R) e quella per le istruzioni di accesso alla memoria sono molto simili. Le differenze principali sono le seguenti.

- Le istruzioni aritmetico-logiche utilizzano la ALU con gli ingressi provenienti da due registri. Le istruzioni di memoria utilizzano la ALU per calcolare l'indirizzo della memoria dati, ma prendono il secondo ingresso alla ALU dal campo offset a 12 bit dell'istruzione stessa, con l'estensione del segno.
- Il valore da scrivere nel registro destinazione proviene dalla ALU (per le istruzioni di tipo R) o dalla memoria (per l'istruzione load).

Si mostri come costruire un'unità di elaborazione relativa alla parte di calcolo per le istruzioni di accesso a memoria e per le istruzioni aritmetico-logiche, utilizzando un solo register file e una sola ALU per entrambi i tipi di istruzioni; si introducano i multiplexer necessari.

SOLUZIONE

Per creare un'unità di elaborazione che utilizzi un solo register file e una sola ALU, dobbiamo implementare un meccanismo che possa prelevare il secondo operando della ALU da due sorgenti diverse; lo stesso dobbiamo fare per il dato da scrivere nel register file.

Occorre quindi inserire un primo multiplexer all'ingresso della ALU e un secondo multiplexer all'ingresso del register file. La **Figura 4.10** mostra la parte di esecuzione dell'unità di elaborazione che supporta questi due tipi di istruzioni.

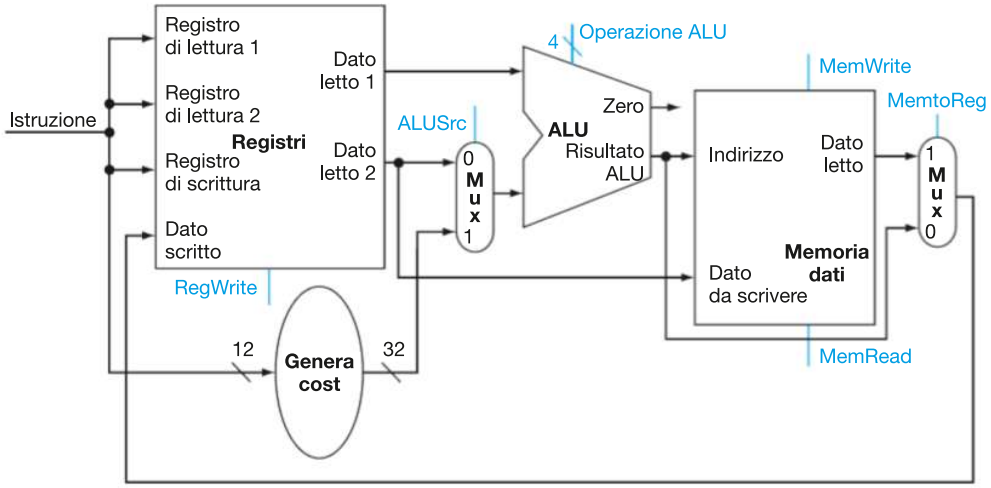


Figura 4.10 Unità di elaborazione per le istruzioni di accesso alla memoria e per le istruzioni di tipo R. Questo esempio mostra come si possa ottenere un'unica unità di elaborazione utilizzando i componenti delle Figure 4.7 e 4.8 con l'aggiunta di multiplexer. In particolare, sono necessari due multiplexer.

A questo punto, possiamo combinare tra loro tutti i pezzi introdotti finora per costruire una semplice unità di elaborazione per il nucleo dell'architettura RISC-V: possiamo aggiungere la parte relativa al prelevamento delle istruzioni (fetch, Figura 4.6), la parte di esecuzione delle istruzioni di tipo R e delle istruzioni di accesso alla memoria (Figura 4.10) e, infine, la parte dell'unità di elaborazione per i salti condizionati (Figura 4.9). La **Figura 4.11** mostra l'unità di elaborazione che otteniamo dalla composizione delle diverse parti. L'istruzione di salto condizionato utilizza la ALU principale per il confronto degli operandi, contenuti in due registri; occorre perciò continuare a utilizzare il sommatore della Figura 4.9 per calcolare l'indirizzo di destinazione del salto. Un altro multiplexer consente di selezionare quale tra l'indirizzo dell'istruzione successiva (PC + 4) e l'indirizzo di destinazione del salto debba essere scritto nel PC.

Autovalutazione

I. Quale delle seguenti affermazioni è corretta per un'istruzione di load (Figura 4.10)?
   a. Il segnale MemtoReg deve essere asserito per fare in modo che il dato proveniente dalla memoria sia inviato al register file.
   b. Il segnale MemtoReg deve essere asserito per fare in modo che il registro di destinazione corretto sia inviato al register file.
   c. Non è importante il valore a cui viene impostato il segnale MemtoReg per le istruzioni di load.

II. L'unità di elaborazione a singolo ciclo, descritta a grandi linee in questo paragrafo, *deve* avere due memorie separate per le istruzioni e i dati, perché:
   a. il formato dei dati e delle istruzioni nel RISC-V è diverso e quindi occorrono due memorie distinte;
   b. avere due memorie separate risulta meno costoso;
   c. il processore opera in un singolo ciclo di clock e quindi non può utilizzare una memoria con una sola porta di uscita per due accessi diversi all'interno dello stesso ciclo.

 © 978-88-08-19966-9

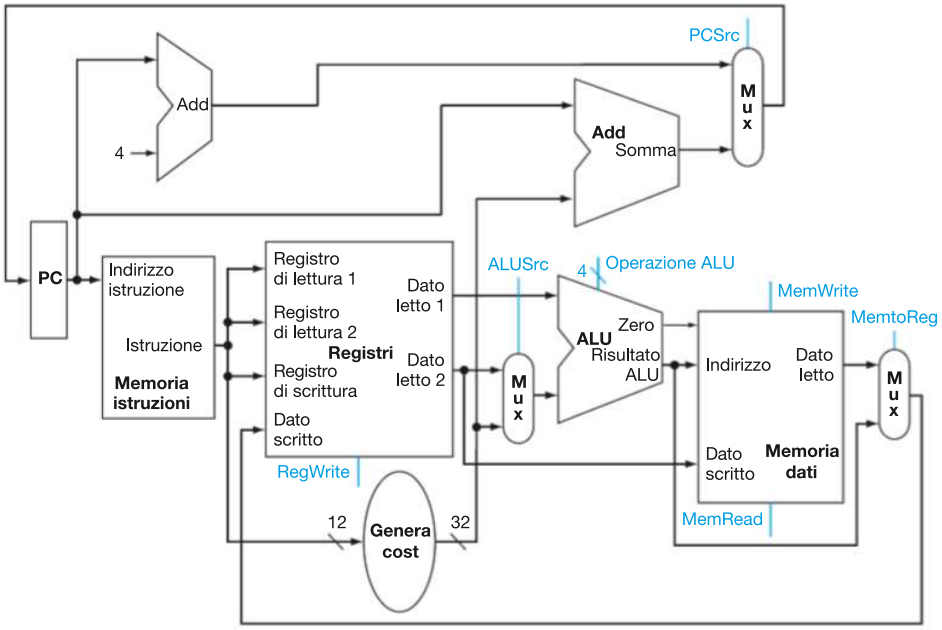


Figura 4.11 Una semplice unità di elaborazione per il nucleo dell'architettura RISC-V che combina tra loro gli elementi richiesti dalle istruzioni appartenenti ai diversi tipi. I componenti sono mostrati nelle Figure 4.6, 4.9 e 4.10. Questa unità di elaborazione può eseguire le istruzioni di base (load/store di un registro, operazioni con la ALU e salti condizionati) in un singolo ciclo di clock. Per integrare i salti condizionati si è resa necessaria solamente l'aggiunta di un altro multiplexer.

Ora che abbiamo completato quest'unità di elaborazione dati elementare, possiamo aggiungere l'unità di controllo: essa dovrà poter accettare dei valori di input e generare un segnale di scrittura per ciascun elemento di stato, un segnale di selezione per ciascun multiplexer e i segnali di controllo per la ALU. Il controllo della ALU è, per molti versi, particolare e conviene perciò progettarlo prima delle altre parti dell'unità di controllo.

Approfondimento. L'unità logica che genera il campo immediato deve scegliere se estendere il segno del campo immediato su 12 bit come segue. Si prendono i bit 31:20 per le istruzioni load, bit 31:25 e 11:7 per le istruzioni store o bit 31, 7, 30:25 e 11:8 per le istruzioni di salto condizionato. Dato che l'input è sempre su 32 bit per tutte le istruzioni, l'unità logica può utilizzare i bit del codice operativo dell'istruzione per selezionare il campo appropriato. Nei RISC-V il bit 6 del codice operativo è pari a 0 per le istruzioni di trasferimento dati e pari a 1 per le istruzioni di salto condizionato, il bit 5 del codice operativo è pari a 0 per le istruzioni di load e pari a 1 per le istruzioni di store. Quindi i bit 5 e 6 controllano un multiplexer 3:1, contenuto all'interno dell'unità logica di generazione del campo immediato che seleziona il gruppo di 12 bit appropriato alle istruzioni di load, store e salto condizionato.

4.4 Uno schema semplice di implementazione

In questo paragrafo esamineremo quella che può essere considerata una semplice e possibile implementazione hardware del sottoinsieme RISC-V che abbiamo scelto; utilizzeremo l'unità di elaborazione dati costruita nel paragrafo precedente, a cui aggiungeremo la relativa unità di controllo. Questa semplice implementazione consente di eseguire le istruzioni *load word* (`lw`), *store word* (`sw`), *branch if equal* (`beq`) e le istruzioni aritmetico-logiche `add`, `sub`, `and` e `or`.

Unità di controllo della ALU

La ALU del RISC-V descritta nell'Appendice A prevede le 4 seguenti combinazioni dei suoi quattro input di controllo:

Linee di controllo della ALU	Operazione
0000	AND
0001	OR
0010	somma
0110	sottrazione

A seconda del tipo di istruzione, la ALU eseguirà una di queste quattro operazioni. Per le istruzioni load e store la ALU deve eseguire una somma per calcolare l'indirizzo di memoria; per le istruzioni di tipo R deve invece eseguire una delle quattro operazioni (AND, OR, somma o sottrazione) in funzione del valore dei 7 bit del campo funz7 (bit 31:25) e dei 3 bit del campo funz3 (bit 14:12) dell'istruzione (Capitolo 2). Per l'istruzione di salto condizionato, la ALU deve eseguire una sottrazione tra i due operandi e controllare se il risultato è 0.

I 4 bit di controllo della ALU possono essere generati utilizzando una piccola unità di controllo che riceve in ingresso i campi funz7 e funz3 dell'istruzione e un campo di controllo su 2 bit, chiamato ALUOp; ALUOp indica se l'operazione da eseguire è una somma (ALUOp = 00) per le istruzioni di load e store, una sottrazione (01) per le beq, oppure indica se l'operazione viene determinata dal contenuto dei campi funz7 e funz3 (10). L'uscita dell'unità di controllo della ALU è un segnale di 4 bit che controlla direttamente la ALU, generando una delle combinazioni di 4 bit mostrate nella tabella precedente.

Nella **Figura 4.12** sono riportati i valori da assegnare agli ingressi di controllo della ALU in funzione del segnale di controllo ALUOp di 2 bit e del contenuto dei campi funct7 e funct3; più avanti vedremo come l'unità di controllo principale imposti i bit di ALUOp.

L'utilizzo di livelli multipli di decodifica (in questo caso l'unità di controllo principale imposta i bit ALUOp, poi utilizzati come ingressi dell'unità di controllo della ALU che, a sua volta, genera i segnali effettivi di controllo della ALU) è una tecnica di progettazione utilizzata di frequente. Più livelli di controllo possono ridurre le dimensioni dell'unità di controllo principale; inoltre, l'uso di più unità di controllo di dimensioni ridotte può ridurre la latenza

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
lw	00	load di 1 parola	XXXXXXX	XXX	somma	0010
sw	00	store di 1 parola	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001

Figura 4.12 Il valore dei bit di controllo della ALU dipende, per le istruzioni di tipo R, dai bit di ALUOp e dai diversi codici funzione. Il codice operativo, riportato nella prima colonna, determina il valore dei bit di ALUOp. Si noti che quando il codice ALUOp vale 00 o 01, l'operazione che la ALU deve eseguire non dipende dal contenuto dei campi funz7 e funz3; in questo caso si dice che il contenuto del campo funzione *è indifferente* e viene indicato con delle X. Invece, quando ALUOp vale 10 vengono utilizzati i campi funz7 e funz3 per determinare il valore degli ingressi di controllo alla ALU *Vedi* l'Appendice A per maggiori dettagli.

dell'unità di controllo. Queste ottimizzazioni sono importanti, poiché spesso l'unità di controllo è l'elemento critico per la definizione della durata del ciclo di clock.

Ci sono diversi modi per implementare la corrispondenza tra i 2 bit del campo ALUOp e i bit dei campi funz con i 4 bit di controllo della ALU che selezionano l'operazione. Dato che interessa solamente un piccolo sottoinsieme dei possibili valori che possono essere assunti dai campi funz, e che questi ultimi vengono utilizzati solamente quando i bit di ALUOp sono pari a 10, possiamo ricorrere a un piccolo circuito logico che riconosca il sottoinsieme dei valori possibili e imposti di conseguenza i bit di controllo della ALU in modo corretto.

Tabella della verità: utilizzata nella logica booleana, è la rappresentazione di una funzione logica. In una tabella della verità vengono elencate tutte le possibili combinazioni dei valori che possono assumere le variabili in ingresso e, per ogni combinazione, viene associata l'uscita relativa.

Come primo passo della progettazione di questo circuito, è utile costruire una *tabella della verità* che contenga le combinazioni di interesse dei campi funz e del segnale di ALUOp, come mostrato nella **Figura 4.13**. Tale **tabella della verità** mostra i 4 bit di controllo della ALU in funzione di questi tre campi in ingresso. Dato che la tabella della verità completa è molto lunga e che non siamo interessati al valore che assumono i segnali di controllo della ALU per le combinazioni che non servono (cioè, sono indifferenti), vengono riportate solamente le righe in corrispondenza delle quali deve essere specificato un preciso valore dei segnali di controllo. Nel resto di questo capitolo adotteremo la convenzione di riportare nelle tabelle della verità solamente gli elementi che devono essere asseriti, tralasciando quelli che sono non asseriti o che sono indifferenti. Questa scelta ha uno svantaggio, che sarà discusso nel Paragrafo B.2 dell'Appendice B .

Termine indifferente: una variabile di ingresso di una funzione logica che non ha effetto sull'uscita della funzione. I termini indifferenti possono assumere un valore arbitrario.

Poiché in molti casi non serve conoscere i valori assunti da alcuni input, essi sono considerati **termini indifferenti**, il che consente di compattare le tabelle. Un termine indifferente di una tabella della verità, rappresentato da una X in una delle colonne associate alle variabili in ingresso, indica che l'uscita non dipende dal valore dell'ingresso associato a quella colonna. Per esempio, quando i bit di ALUOp valgono 00, come nella prima riga della tabella della Figura 4.13, i segnali di controllo della ALU vengono sempre impostati a 0010, indipendentemente dal contenuto dei campi funz; in tal caso gli ingressi dei campi funzione sono indifferenti. Incontreremo in seguito altri tipi di termini indifferenti; il lettore che non abbia familiarità con il concetto di termine indifferente è invitato a consultare l'Appendice A per maggiori dettagli.

Una volta costruita la tabella della verità, questa può essere ottimizzata e convertita in un circuito costituito da porte logiche connesse tra loro: tale processo è di natura completamente meccanica. Perciò, invece di mostrare qui

ALUOp		Campo funz7							Campo funz3			Operazione
ALUOp1	ALUop2	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Figura 4.13 Tabella della verità dei 4 bit di controllo della ALU (detti "operazione"). Gli ingressi della tabella della verità sono i campi ALUOp e i campi funz. Vengono riportate solamente le righe della tabella per cui i segnali di controllo della ALU devono essere asseriti. Sono mostrate anche alcune righe associate a valori indifferenti: per esempio, ALUOp non utilizza il codice 11, quindi la tabella della verità conterrà le combinazioni 1X e X1 di ALUOp anziché 10 e 01. Anche se mostriamo tutti e dieci i bit dei campi funz, notate che gli unici bit che assumono valori diversi per le quattro istruzioni di formato R sono i bit 30, 14, 13 e 12. Quindi, occorrono solamente questi quattro bit dei campi funzione come input all'unità di controllo della ALU.

questi passi, abbiamo preferito inserire la descrizione del processo di ottimizzazione e il risultato finale nel Paragrafo B.2 dell'Appendice B .

Progettazione dell'unità di controllo principale

Ora che abbiamo terminato di descrivere la progettazione di una ALU che utilizza come ingressi di controllo il campo funzione e un segnale su 2 bit, possiamo tornare ad analizzare il resto dell'unità di controllo. Iniziamo con l'identificare i diversi campi delle istruzioni e i segnali di controllo richiesti dall'unità di elaborazione mostrata nella Figura 4.11. Per capire come connettere i campi di un'istruzione all'unità di elaborazione, conviene rivedere prima il formato delle tre classi di istruzioni: tipo R, salto condizionato e load/store. Questi formati sono mostrati nella **Figura 4.14**.

Ci sono alcune considerazioni importanti da fare in merito a questo formato delle istruzioni, su cui ci baseremo in fase di progettazione.

- Il campo **codice operativo** o **codop** (Capitolo 2) è sempre contenuto nei bit 6:0. A seconda del codice operativo, il campo funz3 (bit 14:12) e funz7 (31:25) servono come campi di estensione del codice operativo.
- Il primo registro operando si trova sempre nei bit in posizione 19:15 (rs1) per le istruzioni di tipo R e le istruzioni di salto condizionato. Questo campo specifica anche il registro base per le istruzioni di load e di store.
- Il secondo registro operando si trova sempre nei bit in posizione 24:20 (rs2) per le istruzioni di tipo R e le istruzioni di salto condizionato. Questo campo specifica anche il registro il cui contenuto viene copiato in memoria nelle istruzioni di store.
- Il secondo operando può anche essere costituito dai 12 bit di offset delle istruzioni di branch o di load/store.
- Il registro destinazione si trova sempre nei bit in posizione 11:7 (rd) per le istruzioni di tipo R e di load.

Codice operativo (codop): il campo che denota il tipo di operazione e il formato di un'istruzione.

Il primo principio di progettazione introdotto nel Capitolo 2 – *La semplicità favorisce la regolarità* – risulta particolarmente utile nella progettazione dell'unità di controllo dell'unità di elaborazione.

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediato[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop

Figura 4.14 Le quattro classi di istruzioni (tipo R, load, store e salto condizionato) utilizzano quattro diversi formati di istruzione. (a) Formato delle istruzioni di tipo R (codice operativo = 51_{dec}). Queste hanno tre operandi contenuti nei registri: rs1, rs2 e rd; i campi rs1 e rs2 contengono il numero dei due registri sorgenti e rd contiene il numero del registro destinazione. L'operazione che la ALU deve eseguire è codificata nei campi funz3 e funz7 e verrà decodificata dall'unità di controllo della ALU, progettata nel paragrafo precedente. Le istruzioni di tipo R implementate sono `add`, `sub`, `and` e `or`. (b) Formato di tipo I delle istruzioni di load (codice operativo = 3_{dec}) di tipo I. Il registro rs1 è il registro base il cui contenuto viene sommato al campo immediato di 12 bit per ottenere l'indirizzo del dato in memoria. Il campo rd è il registro destinazione per il valore letto dalla memoria. (c) Formato di tipo S delle istruzioni di store (codice operativo = 35_{dec}) di tipo S. Il registro rs1 è il registro base il cui contenuto viene sommato al campo immediato di 12 bit per ottenere l'indirizzo del dato in memoria (il campo immediato è suddivido in un gruppo di 7 e in un gruppo di 5 bit). Il campo rs2 è il registro sorgente il cui valore viene copiato nella memoria. (d) Formato di tipo SB delle istruzioni branch if equal (codice operativo = 99_{dec}). I registri rs1 e rs2 vengono confrontati. Il campo indirizzo immediato di 12 bit viene preso, il suo bit di segno esteso, fatto scorrere a sinistra di una posizione e sommato al PC per calcolare l'indirizzo di destinazione del salto. Le Figure 4.17 e 4.18 forniscono il razionale di questo insolito posizionamento dei bit nel formato SB.

Interfaccia hardware/software

Rispetto al MIPS, i formati delle istruzioni del RISC-V sembrano più complicati ma in realtà semplificano l'hardware e possono perfino diminuire la durata del periodo di clock in alcune implementazioni RISC-V, specialmente quelle in pipeline che vedremo nel Paragrafo 4.6. Dato che i compilatori, gli assemblatori e i debugger nascondono ai programmatori i dettagli del formato delle istruzioni, perché non aiutare l'hardware? Il primo esempio è il formato dell'istruzione di store. La Figura 4.15 mostra i formati delle istruzioni MIPS per il trasferimento di dati con la memoria e di quelle aritmetiche e il loro impatto sul cammino di esecuzione. Il MIPS richiede un multiplexer 2:1 per specificare su quale registro scrivere il dato letto dalla memoria, che in questo caso non serve. Questo multiplexer può trovarsi facilmente sul cammino critico e quindi aumenterebbe il periodo di clock. Per mantenere sempre il registro destinazione nei bit da 7 a 11 per tutte le istruzioni, il formato S del RISC-V ha dovuto suddividere il campo immediato in due gruppi di bit: i bit da 25 a 31 contengono il campo immediato[11:5] e i bit da 7 a 11 il campo immediato[4:0]. Sembra strano, confrontando la gestione del campo immediato con quella del MIPS nel quale i bit del campo immediato sono tutti contigui, ma l'assemblatore del RISC-V nasconde questa complessità e l'hardware ne beneficia.

Il secondo esempio sembra ancora più strano. La Figura 4.16 mostra che il RISC-V ha due formati, SB e UJ, i quali hanno gli stessi campi della stessa dimensione dei formati S e U e contengono anch'essi una costante, con la differenza che i bit sono scambiati. Anche i formati SB e UJ semplificano l'hardware procurando un lavoro maggiore all'assemblatore. Le figure successive mostrano come l'hardware genera il campo immediato nel RISC-V. La Figura 4.17 mostra quali bit dell'istruzione corrisponderebbero ai bit del campo immediato di questi tipi di istruzioni, se le istruzioni di salto condizionato utilizzassero il formato S, invece del formato SB, e se le istruzioni di salto incondizionato utilizzassero il formato U, invece del formato UJ. L'ultima riga mostra il numero di input per ogni bit di output, che determina il numero di porte del multiplexer utilizzato per generare il campo immediato.

La Figura 4.18 mostra invece il formato vero delle istruzioni di salto condizionato e incondizionato e il numero ridotto di bit specifici. I formati SB e UJ riducono la dimensione del multiplexer per i bit da 12 a 19 del campo immediato da 3:1 a 2:1 e per i bit da 10 a 1 del campo immediato da 4:1 a 2:1. Anche qui, i progettisti del RISC-V hanno definito dei formati che appaiono strani ma che sono efficienti, semplificando i multiplexer 18:1. Il risparmio non è significativo per i processori di fascia alta, ma molto prezioso per i processori di fascia bassa e tutto il costo è supportato dagli assemblatori (e dagli autori).

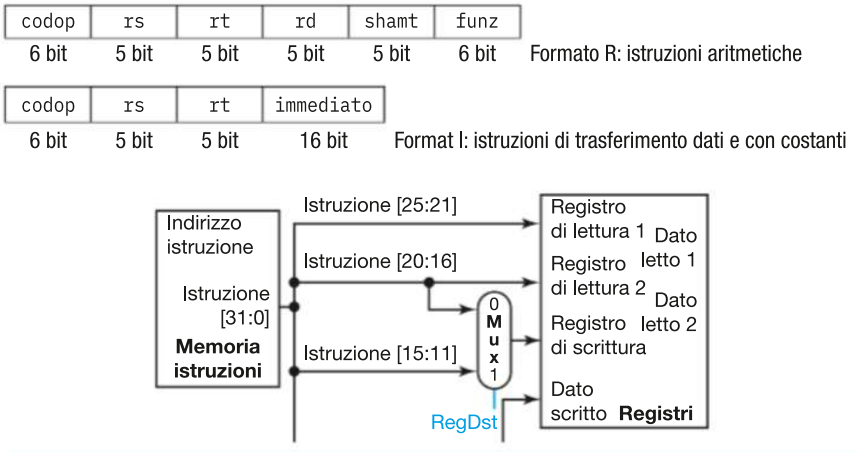


Figura 4.15 Il MIPS, il format delle istruzioni aritmetiche e di quelli di trasferimento dei dati, e il loro impatto sul cammino di esecuzione del MIPS. Nelle istruzioni aritmetiche MIPS che utilizzano il formato R, `rd` è il registro destinazione, `rs` il registro del primo operando e `rt` quello del secondo operando. Nelle istruzioni di load e che utilizzano una costante, `rs` contiene sempre il primo operando, ma `rt` diventa il registro di scrittura. Da qui nasce l'esigenza di un multiplexer 2:1 per scegliere tra `rt` e `rd` il numero del registro corretto su cui scrivere il risultato.

Nome (Dimensione campo)	Campo						Commenti
	7 bit	5 bit	5 bit	3 bit	5 bit	7 bit	
Tipo R	funz7	rs2	rs1	funz3	rd	codop	Formato istruzioni aritmetiche
Tipo I	imm[11:0]		rs1	funz3	rd	codop	Load e aritmetiche con costante
Tipo S	imm[11:5]	rs2	rs1	funz3	imm[4:0]	codop	Store
Tipo SB	imm[12,10:5]	rs2	rs1	funz3	imm[4:1,11]	codop	Salti cond
Tipo UJ	imm[20, 10:1, 11, 19:12]				rd	codop	Formato salti incond
Tipo U	imm[31:12]				rd	codop	Formato upper immed

Figura 4.16 I formati RISC-V. La Figura 4.16 introduce i formati R, I, S e U, che sono intuitivi.

		Campo immediato generato bit per bit																													
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Istruzione	Formato	Input del campo immediato bit per bit																															
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20
Store	S	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i11	i10	i9	i8	i7
Salto cond	S	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i30	i29	i28	i27	i26	i25	i24	“	“	“	“	0
Salto incond	U	“	“	“	“	“	“	“	“	“	“	“	“	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	i19	i18	i17	i16	i15	i14	i13	i12	“
Load Upper imm	U	“	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	i19	i18	i17	i16	i15	i14	i13	i12	0	0	0	0	0	0	0	0	0	0	0	“
Num input		1	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3	3	3	3	3	4	4	4	4	4	4	4	4	4	4	4	3

Figura 4.17 Gli input del campo immediato (costante) quando i salti condizionati utilizzano il formato S e i salti incondizionati il formato U.

		Campo immediato generato bit per bit																															
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Istruzione	Formato	Input del campo immediato bit per bit																															
Load, Aritm imm	I	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i31	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20
Store	S	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i11	i10	i9	i8	i7
Salto cond	SB	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	“	i7	“	“	“	“	“	“	“	“	“	“	0
Salto incond	UJ	“	“	“	“	“	“	“	“	“	“	“	“	i19	i18	i17	i16	i15	i14	i13	i12	i20	“	“	“	“	“	“	“	“	“	“	“
Load Upper imm	U	“	i30	i29	i28	i27	i26	i25	i24	i23	i22	i21	i20	“	“	“	“	“	“	“	“	0	0	0	0	0	0	0	0	0	0	0	“
Num input		1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	2	2	2	2	2	2	3	3	3	3	3

Figura 4.18 Gli input del campo immediato (costante) quando i salti condizionati utilizzano il formato SB e i salti incondizionati il formato UJ, che sono quelli che utilizza il RISC-V.

Utilizzando le osservazioni riportate sopra, possiamo ora aggiungere all'unità di elaborazione elementare che abbiamo costruito le etichette sui campi dell'istruzione. La Figura 4.19 mostra queste aggiunte, un blocco che rappresenta l'unità di controllo della ALU, i segnali di scrittura degli elementi di stato, il segnale di lettura della memoria dati e i segnali di controllo dei multiplexer. Dato che tutti i multiplexer hanno due ingressi, ciascuno di essi richiede un solo segnale di controllo.

La Figura 4.19 mostra i sei segnali di controllo di 1 bit ciascuno e il segnale di controllo ALUOp di 2 bit. Abbiamo già spiegato come funziona questo segnale, ed è utile definire informalmente come debbano operare gli altri segnali di controllo prima di determinare il modo in cui devono essere impostati durante l'esecuzione delle istruzioni. La Figura 4.20 descrive il funzionamento di questi sei segnali di controllo.

 © 978-88-08-19966-9

Ora che abbiamo visto il funzionamento di ciascuno dei segnali di controllo, possiamo passare a esaminare come debbano essere impostati. L'unità di controllo può impostare tutti i segnali tranne uno, basandosi esclusivamente sul codice operativo dell'istruzione stessa. L'eccezione è costituita dal segnale di controllo PCSrc: esso deve essere asserito se l'istruzione è una branch

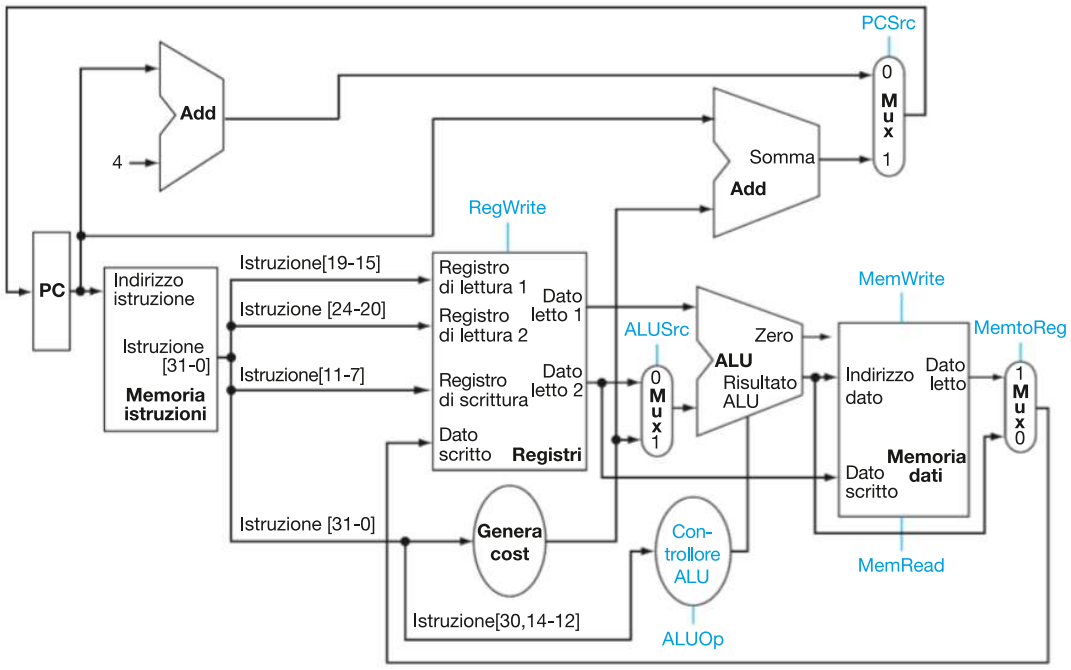


Figura 4.19 L'unità di elaborazione della Figura 4.11 con tutti i multiplexer e i segnali di controllo necessari. I segnali di controllo sono mostrati in blu. È stato introdotto anche il blocco che rappresenta il controllore della ALU. Il PC non richiede un segnale di controllo della scrittura esplicito, dal momento che viene scritto al termine di ogni ciclo di clock; la logica di controllo dei salti determina se nel PC debba essere scritto il valore precedente del PC incrementato di 4 oppure l'indirizzo di destinazione del salto.

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nullo	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di PC + 4	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nullo	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nullo	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

Figura 4.20 Effetto di ciascuno dei sette segnali di controllo. Quando il segnale di controllo a 1 bit di un multiplexer a due vie è asserito, il multiplexer seleziona l'ingresso etichettato con 1; in caso contrario, cioè se il segnale di controllo non è asserito, il multiplexer seleziona l'ingresso etichettato con 0. Si ricordi che tutti gli elementi di stato ricevono il clock come ingresso implicito e che il clock controlla le operazioni di scrittura. Far commutare il clock mediante un segnale esterno a un elemento di stato potrebbe causare problemi di temporizzazione (*vedi* Appendice A per la discussione di questo problema).

if equal, decisione che l'unità di controllo è in grado di prendere, *ma anche* se l'uscita Zero della ALU, utilizzata per il confronto di uguaglianza, è vera. Per generare il segnale PCSrc occorre dunque collegare in AND un segnale proveniente dall'unità di controllo, che verrà chiamato *Branch*, con l'uscita Zero della ALU.

Questi otto segnali di controllo, i sei della Figura 4.20 e i due di ALUOp, possono ora essere impostati sulla base dei segnali di ingresso all'unità di controllo, che sono i sette bit del codice operativo (bit 6:0). La **Figura 4.21** mostra l'unità di elaborazione completa, con l'unità di controllo e i relativi segnali.

Prima di iniziare a scrivere un insieme di equazioni o una tabella della verità per specificare il modo di operare dell'unità di controllo, è utile cercare di definire informalmente il suo funzionamento. Dato che il valore di ciascuna linea di controllo dipende solamente dal codice operativo, possiamo dire se ciascun segnale debba valere 0, 1 o sia indifferente (X) per i diversi valori che il codice operativo può assumere. La **Figura 4.22** riporta il valore di ciascun segnale di controllo in funzione del valore del codice operativo, ed è ottenuta direttamente dalle informazioni contenute nelle Figure 4.12, 4.20 e 4.21.

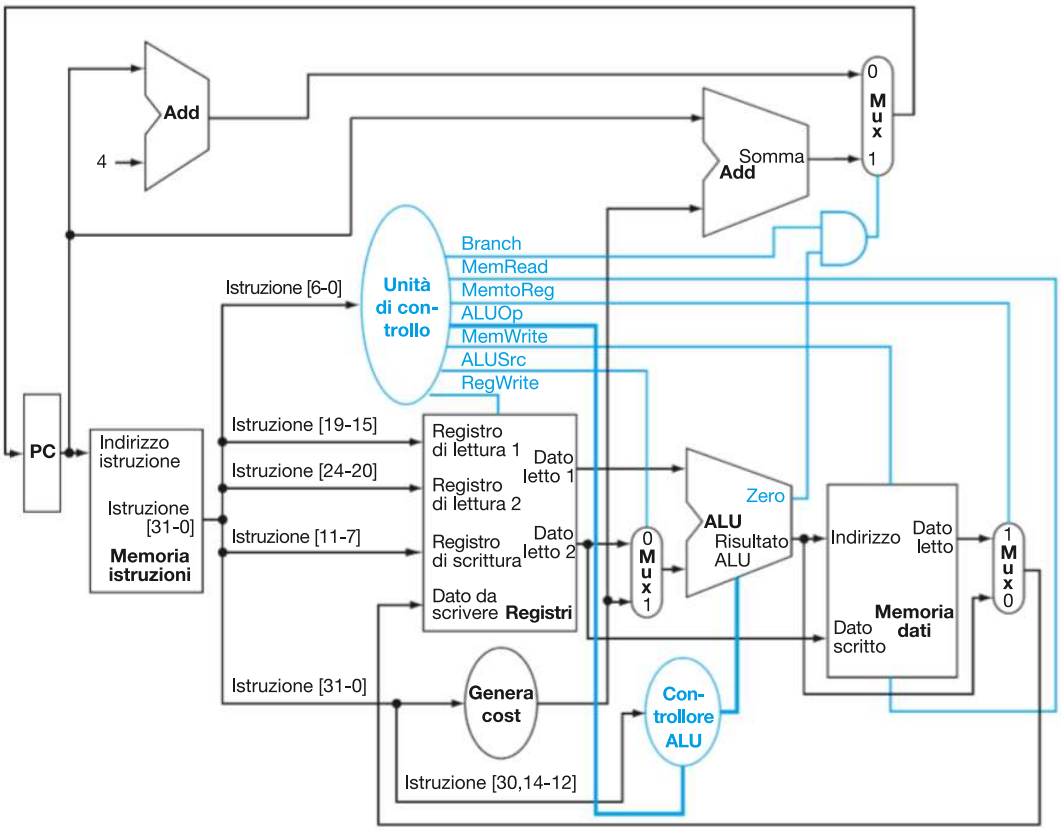


Figura 4.21 L'unità di elaborazione costruita finora, con la sua unità di controllo. L'input dell'unità di controllo è il campo a 7 bit dell'istruzione che costituisce il codice operativo. L'uscita dell'unità di controllo è formata da due segnali a 1 bit utilizzati per controllare i multiplexer (ALUSrc e MemtoReg), tre segnali a 1 bit per controllare lettura e scrittura del register file e della memoria dati (RegWrite, MemRead e MemWrite), un segnale a 1 bit utilizzato come segnale di controllo per i salti condizionati (Branch) e un segnale di controllo a 2 bit per la ALU (ALUOp). Una porta AND viene utilizzata per combinare il segnale di Branch e l'uscita Zero della ALU; l'uscita di questa porta determina da dove prendere il valore successivo del PC e genera il segnale di controllo PCSrc per il multiplexer posizionato in alto (Figura 4.15). Si noti che PCSrc viene derivato e non proviene direttamente dall'unità di controllo; per questo motivo, ometteremo di indicarlo nelle figure successive.

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Figura 4.22 Il valore assunto dalle linee di controllo è completamente determinato dal campo del codice operativo dell'istruzione. La prima riga della tabella corrisponde alle istruzioni in formato R (`add`, `sub`, `and` e `or`), per le quali i campi registro sorgente sono rs1 e rs2 e il campo registro destinazione è rd; questo stabilisce come impostare il segnale ALUSrc. Inoltre, le istruzioni di tipo R scrivono un registro (RegWrite = 1), ma non leggono né scrivono nella memoria dati. Quando il segnale di controllo Branch è impostato a 0, il PC viene aggiornato incondizionatamente a PC + 4; altrimenti, il contenuto del PC viene sostituito con l'indirizzo della destinazione del salto, ma solo quando anche l'uscita Zero della ALU è alta. Il campo ALUOp per le istruzioni di tipo R è impostato a 10 per indicare che i segnali di controllo della ALU dovranno essere generati a partire dai campi funz. La seconda e la terza riga di questa tabella riportano l'impostazione dei segnali di controllo per `lw` e `sx`, per le quali ALUSrc e ALUOp assumono il valore richiesto perché venga calcolato l'indirizzo in memoria, mentre MemRead e MemWrite determinano il tipo di accesso alla memoria. Infine, in una load, RegWrite viene impostato in modo tale che il dato letto dalla memoria venga scritto nel registro rd. Nell'istruzione di salto condizionato il campo ALUOp indica la sottrazione (01), utilizzata per controllare l'uguaglianza degli operandi. Si noti che il segnale MemtoReg risulta irrilevante quando il segnale RegWrite è uguale a 0: dal momento che l'istruzione non scrive nel register file, il valore del dato presente sulla porta di scrittura non viene utilizzato; quindi il valore di MemtoReg nelle ultime due righe viene sostituito con una X per indicare che è indifferente. La scelta di impostare questi segnali di controllo con un valore indifferente deve essere fatta dal progettista, poiché dipende dalla conoscenza del funzionamento dell'unità di elaborazione.

Funzionamento dell'unità di elaborazione

Con le informazioni contenute nelle Figure 4.20 e 4.22 potremmo progettare la logica dell'unità di controllo; tuttavia, prima di procedere esaminiamo l'utilizzo che ciascuna istruzione fa dell'unità di elaborazione. Nelle prossime figure sarà riportato il flusso di elaborazione attraverso l'unità di elaborazione di tre istruzioni appartenenti alle tre diverse classi. I segnali di controllo asseriti e gli elementi attivi dell'unità di elaborazione saranno evidenziati. Si noti che un multiplexer il cui segnale di controllo valga 0 compie un'azione ben definita, anche se il suo segnale di controllo non viene evidenziato. I segnali a più bit saranno evidenziati se risulta asserito almeno uno dei segnali associati ai singoli bit.

La **Figura 4.23** mostra il modo in cui l'unità di elaborazione esegue un'istruzione di tipo R, per esempio `add x1, x2, x3`. Sebbene l'esecuzione avvenga in un singolo ciclo di clock, possiamo pensare che l'esecuzione proceda attraverso una sequenza di quattro passi, che possiamo identificare analizzando il flusso delle informazioni:

1. L'istruzione viene prelevata dalla memoria e si incrementa il PC;
2. I due registri `x2` e `x3` vengono letti dal register file, mentre l'unità di controllo principale calcola il valore da attribuire alle linee di controllo;
3. La ALU elabora i dati letti dal register file, utilizzando alcuni bit del codice operativo per selezionare l'operazione della ALU;
4. Il risultato calcolato dalla ALU viene scritto nel registro destinazione (`x1`) del register file.

In modo analogo, si può illustrare l'esecuzione di un'istruzione di caricamento di un registro, per esempio

```
lw x1, offset(x2)
```

utilizzando lo stesso approccio della Figura 4.23. Le unità funzionali attive e i segnali di controllo asseriti per l'elaborazione di un'istruzione di load sono mostrati nella **Figura 4.24**. Così come avevamo scomposto l'istruzione di tipo R in quattro passi sequenziali, possiamo scomporre l'istruzione di load in cinque passi sequenziali:

1. L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC;
2. Viene letto il contenuto di un registro (`x2`) dal register file;

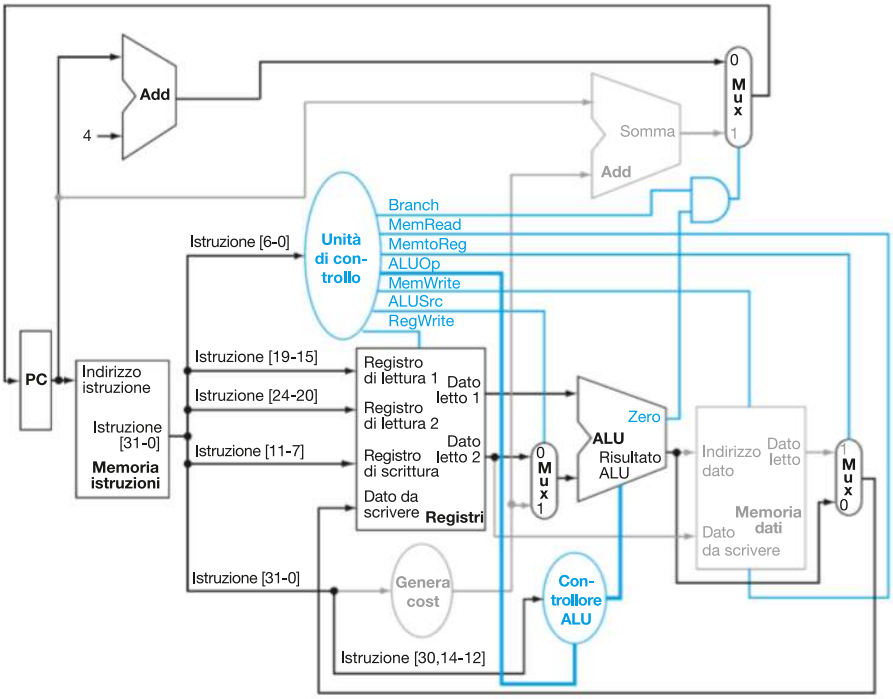


Figura 4.23 L'unità di elaborazione durante l'esecuzione di un'istruzione di tipo R, come `add x1, x2, x3`. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu.

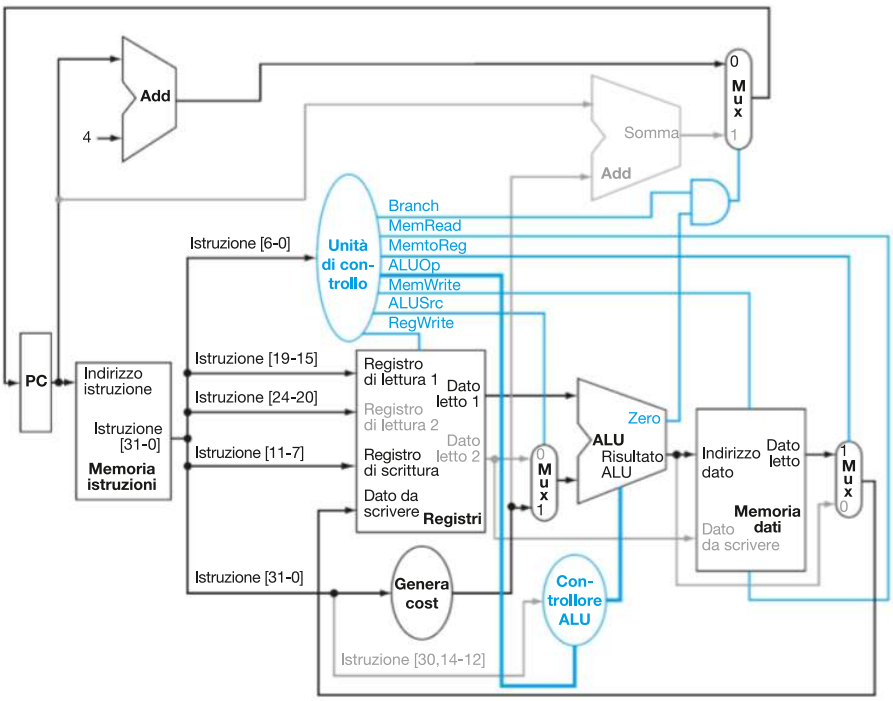


Figura 4.24 L'unità di elaborazione durante l'esecuzione di un'istruzione di load. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu. Un'istruzione di store viene eseguita in modo molto simile; le differenze principali sono tre: i segnali di controllo della memoria devono specificare una scrittura (del contenuto del secondo registro letto), il secondo registro letto contiene il dato da scrivere in memoria e non è presente l'operazione di scrittura del dato nel register file.

3. La ALU somma il valore letto dal register file ai 12 bit del campo `offset` (campo immediato) dell'istruzione, considerati dotati di segno ed estesi a 32 bit;
4. La somma calcolata dalla ALU viene utilizzata come indirizzo per la memoria dati;
5. Il dato proveniente dall'unità di memoria dati viene scritto nel register file nel registro `x1`.

Infine, possiamo utilizzare la stessa tecnica per esaminare l'esecuzione di un'istruzione di salto condizionato, per esempio `beq x1, x2, offset`. Essa si comporta in modo molto simile a un'istruzione di tipo R, con la differenza che l'uscita della ALU viene utilizzata per scegliere se scrivere nel PC il valore PC + 4 oppure l'indirizzo di destinazione del salto. I quattro passi sequenziali dell'esecuzione di una `beq` sono mostrati nella **Figura 4.25**:

1. L'istruzione viene prelevata dalla memoria istruzioni e si incrementa il PC;
2. Vengono letti dal register file i due registri `x1` e `x2`;
3. La ALU esegue la sottrazione del contenuto dei due registri letti dal register file; il valore del PC viene sommato ai 12 bit del campo `offset` dell'istruzione, considerati dotati di segno, estesi a 32 bit e fatti scorrere di una posizione a sinistra; il risultato costituisce l'indirizzo di destinazione del salto;
4. La linea Zero in uscita dalla ALU viene utilizzata per determinare da quale sommatore prendere l'indirizzo successivo da scrivere nel PC.

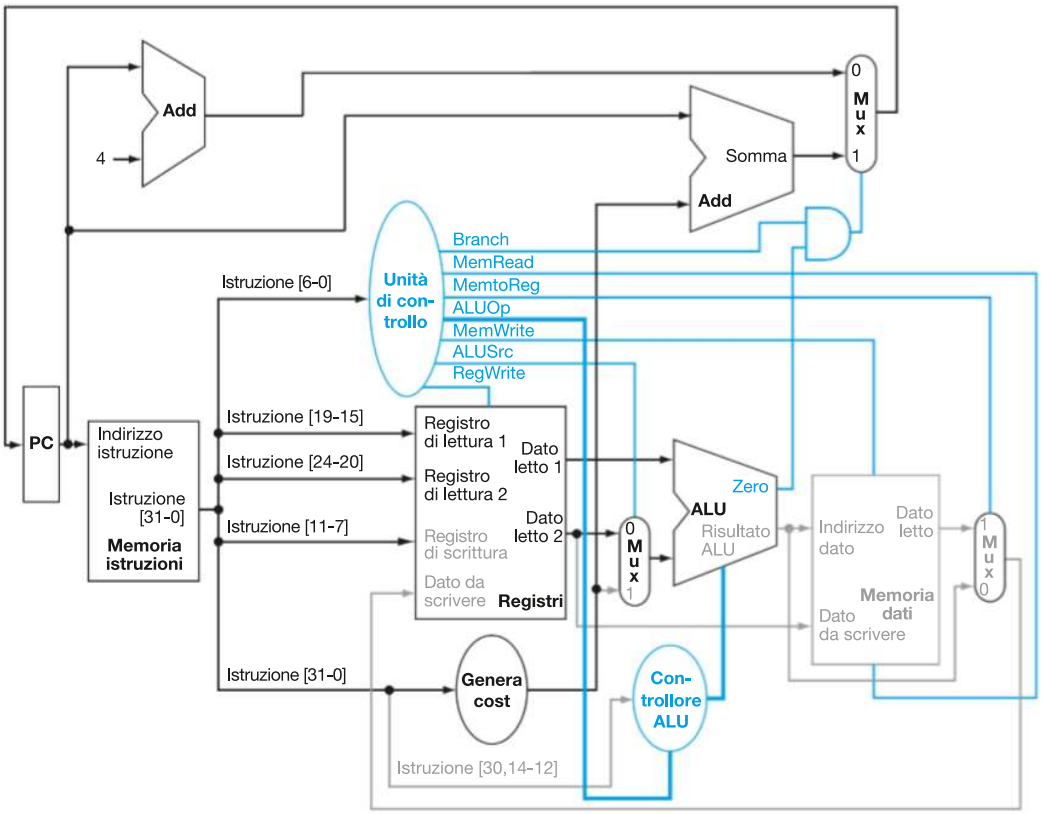


Figura 4.25 L'unità di elaborazione durante l'esecuzione di un'istruzione branch equal. Le linee di controllo, le unità funzionali dell'unità di elaborazione e le connessioni attive sono evidenziate in blu. Dopo aver utilizzato il register file e la ALU per eseguire il confronto, l'uscita Zero della ALU viene impiegata per scegliere il valore successivo del program counter tra i due possibili candidati.

Input o output	Nome del segnale	Formato R	ld	sd	beq
Input	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Output	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp01	1	0	0	0
	ALUOp02	0	0	0	1

Figura 4.26 La funzione logica dell'unità di controllo per l'implementazione a singolo ciclo di clock è completamente specificata da questa tabella della verità. Le prime sette righe della tabella riporta le combinazioni del codice operativo associate alle quattro classi di istruzioni esaminate, una per ogni colonna; i segnali di controllo vengono impostati in funzione di questi ingressi. La parte inferiore della tabella fornisce invece le uscite in funzione del codice operativo. Quindi l'uscita RegWrite viene asserita in corrispondenza di due diverse combinazioni degli input. Se si considerano solamente i quattro codici operativi riportati, la tabella della verità si può semplificare utilizzando la condizione di indifferenza per alcuni ingressi: per esempio, si può identificare un'istruzione in formato R con l'espressione Op4 · Op5, dato che questa è sufficiente a distinguere le istruzioni in formato R dalle `lw`, `sw` e `beq`. Tuttavia non sfrutteremo queste semplificazioni, dato che in un'implementazione completa dell'insieme delle istruzioni anche i restanti codici operativi vengono utilizzati nell'implementazione completa del RISC-V.

Completamento dell'unità di controllo

Dopo aver esaminato i diversi passi di esecuzione delle istruzioni, possiamo procedere all'implementazione dell'unità di controllo. La funzione di controllo può essere determinata in modo esatto a partire dal contenuto della Figura 4.22. Le uscite dell'unità di controllo sono i segnali di controllo, mentre i suoi input sono costituiti dai 7 bit del codice operativo. Possiamo quindi scrivere una tabella della verità per ciascuna delle uscite, basata sulla codifica binaria dei codici operativi. La Figura 4.26 mostra la funzione logica implementata nell'unità di controllo sotto forma di una grande tabella della verità che contiene tutte le uscite e i bit del codice operativo come ingressi. Tale tabella specifica completamente la funzione di controllo e può essere implementata direttamente mediante porte logiche in modo automatico. Quest'ultimo passaggio viene descritto nel Paragrafo C.2 dell'Appendice C.

Perché oggi non si utilizzano più implementazioni a singolo ciclo?

Sebbene un processore a singolo ciclo funzioni correttamente, esso non viene più utilizzato nelle implementazioni moderne, perché troppo inefficiente. Per capirne il motivo, si noti che, in questa implementazione, il periodo di clock deve avere la stessa durata per tutte le istruzioni; quindi, il periodo di clock è determinato dal cammino di esecuzione più lungo all'interno del processore (cammino critico). Tale cammino sarà molto probabilmente associato a un'istruzione di load, dato che essa utilizza cinque unità funzionali in sequenza: la memoria istruzioni, il register file, la ALU, la memoria dati e, di nuovo, il register file. Anche se il CPI è pari a 1 (Capitolo 1), le prestazioni complessive di un'implementazione a singolo ciclo saranno probabilmente scarse, poiché la durata del periodo di clock risulterà troppo lunga.

Il prezzo da pagare per un'implementazione a singolo ciclo con una durata del clock fissa è significativo, ma potrebbe comunque essere considerato

accettabile per un insieme limitato di istruzioni, come quello considerato. I primi calcolatori, dotati di un insieme di istruzioni semplice, utilizzavano questa implementazione. Tuttavia, se si volesse implementare un'unità di elaborazione in virgola mobile o un insieme di istruzioni comprendente istruzioni più complesse, un progetto a singolo clock non funzionerebbe altrettanto bene.

Dato che dobbiamo assicurare che la durata del ciclo di clock sia pari alla durata dell'istruzione più lunga da eseguire, non si possono sfruttare quelle tecniche che consentono di ridurre il tempo di esecuzione delle istruzioni più comuni ma che non riducono la durata dell'esecuzione dell'istruzione più lenta: un'implementazione a un solo ciclo è in contrasto quindi con una delle grandi idee introdotte nel Capitolo 2, che richiede di **rendere veloci le situazioni più comuni**.

SITUAZIONI COMUNI

Autovalutazione

Nel Paragrafo 4.6 illustreremo una tecnica di implementazione diversa, chiamata pipeline. Essa utilizza un'unità di elaborazione simile a quella a singolo ciclo ma molto più efficace, perché consente un throughput sensibilmente maggiore. Le pipeline sono più efficienti grazie alla sovrapposizione temporale dell'esecuzione di diverse istruzioni.Osserviamo i segnali di controllo nella Figura 4.26. Si possono combinare fra loro? Esiste qualche segnale di controllo che può essere sostituito con l'inverso di un altro? (Suggerimento: si prendano in considerazione i segnali indifferenti.) Se sì, è possibile sostituire un segnale di controllo con un altro senza utilizzare un invertitore?

4.5 Un'implementazione multi-ciclo

Nel paragrafo precedente, abbiamo suddiviso l'esecuzione di ciascuna istruzione in una sequenza di passi che corrispondono alle unità funzionali richieste a ogni passo. Possiamo utilizzare questa suddivisione per creare un'**implementazione multi-ciclo**. In un'implementazione multi-ciclo, ciascun passo dell'esecuzione richiederà 1 ciclo di clock e sarà consentito utilizzare le unità funzionali più di una volta per ogni istruzione, a patto che vengano utilizzate in cicli di clock diversi. Questa condivisione delle unità funzionali consente di ridurre la quantità di hardware richiesto. La possibilità di eseguire le istruzioni in un numero diverso di cicli di clock e la condivisione delle unità funzionali all'interno dell'esecuzione della singola istruzione sono il maggiore vantaggio dell'implementazione multi-ciclo del MIPS. Questo paragrafo, disponibile on-line, descrive l'implementazione multi-ciclo del MIPS.

Sebbene questa implementazione possa ridurre i costi dell'hardware, quasi tutti i chip oggigiorno implementano la pipeline, che non ha l'obiettivo di aumentare le prestazioni sulla singola istruzione rispetto a quella a singolo ciclo di clock. Per questo motivo, i lettori non interessati a questo argomento, possono saltare l'implementazione multi-ciclo e andare direttamente alla pipeline. Tuttavia, dato che alcuni docenti vedono un vantaggio didattico nello spiegare l'implementazione multi-ciclo prima della pipeline, la descrizione di questa implementazione viene offerta on-line.

4.6 Introduzione alla pipeline

Il tempo non va sprecato.
Proverbio americano

La **pipeline** è una tecnica di implementazione hardware di un insieme di istruzioni che prevede la sovrapposizione temporale dell'esecuzione delle diverse istruzioni; attualmente è utilizzata quasi dalla totalità dei calcolatori.

In questo paragrafo utilizzeremo spesso delle analogie per introdurre i termini e le problematiche legate alla pipeline. I lettori non interessati a una trattazione approfondita possono limitarsi a leggere questo paragrafo e poi passare ai Paragrafi 4.11 e 4.12, che contengono un'introduzione alle tecniche di pipeline avanzate adottate da alcuni fra i processori più recenti, quali il Core i7 Intel e il Cortex-A53 ARM. Chi, invece, è interessato a esaminare a fondo la struttura dei calcolatori dotati di pipeline, potrà considerare questo paragrafo una buona introduzione ai concetti che verranno trattati nei Paragrafi 4.7–4.10.

Pipeline: una tecnica di implementazione hardware grazie alla quale l'esecuzione di più istruzioni viene sovrapposta nel tempo, in maniera simile a quanto avviene in una catena di montaggio.

Chiunque abbia dovuto fare il bucato molte volte ha sicuramente già utilizzato, in un certo senso, la tecnica della pipeline. Fare il bucato, secondo un approccio *non basato sulla pipeline*, richiederebbe i passi seguenti:

1. Mettere un carico di biancheria sporca nella lavatrice;
2. Una volta terminato il ciclo di lavaggio, mettere la biancheria ancora bagnata nell'asciugatrice;
3. Dopo che l'asciugatrice ha terminato, mettere la biancheria asciugata sul tavolo da stiro e stirarla;
4. Quando l'operazione di stiratura è finita, chiedere al proprio coinquilino di riporre la biancheria nell'armadio.

Non appena questo bucato sarà terminato, si potrà cominciare con il carico successivo di biancheria sporca.

Un approccio basato sul concetto di *pipeline* richiede molto meno tempo. Come mostrato nella **Figura 4.27**, quando la lavatrice finisce il primo ciclo di lavaggio e la biancheria viene trasferita all'asciugatrice, si può riempire subito la lavatrice con il secondo carico di biancheria sporca. Dopo che il primo

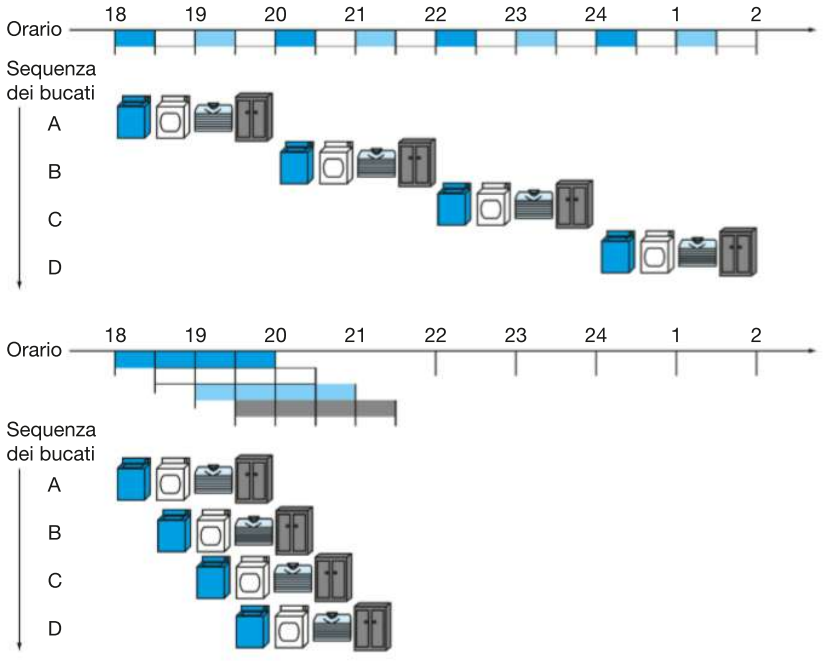


Figura 4.27 Analogia della pipeline con il bucato. Anna, Bruno, Carla e Dario hanno, ciascuno, dei panni sporchi da lavare, asciugare, stirare e riporre negli armadi. Ogni operazione richiede 30 minuti. Fare i bucati in sequenza richiede quindi 8 ore per completare i quattro cicli di lavaggio, asciugatura, stiratura e sistemazione della biancheria, mentre il bucato con pipeline richiede solamente 3,5 ore. Lo stadio di avanzamento dei diversi carichi all'interno della pipeline, al variare del tempo, è indicato dalla posizione sull'asse dei tempi delle quattro unità funzionali; queste vengono indicate per ogni bucato per chiarezza, ma in realtà c'è una sola unità funzionale di ogni tipo.

 © 978-88-08-19966-9

carico di biancheria è stato asciugato, si può iniziare l'operazione di stiratura, e si può spostare il carico bagnato nell'asciugatrice e mettere il successivo carico di biancheria sporca in lavatrice. Una volta stirata la biancheria del primo carico, il vostro coinquilino potrà prendere gli abiti dal tavolo da stiro e riporli nell'armadio; nel frattempo inizierà la stiratura del secondo carico, il terzo carico si troverà nell'asciugatrice e potrete mettere il quarto carico nella lavatrice. A questo punto tutti i passaggi, detti *stadi* nel linguaggio delle pipeline, sono attivi contemporaneamente: purché si abbiano risorse separate per ciascuno stadio, il bucato può essere messo in pipeline.

Il paradosso delle pipeline è che il tempo necessario per lavare, asciugare, stirare e mettere nell'armadio un singolo calzino non viene affatto ridotto; il motivo per cui le pipeline sono più veloci quando si hanno molti carichi di biancheria è che tutte le unità lavorano in parallelo e quindi si possono fare più bucati nello stesso intervallo di tempo: in sostanza la pipeline aumenta il *throughput* (Capitolo 1) della nostra lavanderia. Perciò, la pipeline non diminuisce il tempo per completare il ciclo di lavoro di un singolo bucato, ma quando ci sono più bucati da fare in sequenza, il miglioramento del throughput fa sì che il tempo complessivo diminuisca.

Se tutti gli stadi richiedessero circa lo stesso tempo e ci fosse abbastanza lavoro da fare, l'incremento di velocità dovuto alla pipeline sarebbe pari al numero dei suoi stadi, in questo caso quattro: lavaggio, asciugatura, stiratura e sistemazione della biancheria. Quindi, fare il bucato sfruttando la pipeline è potenzialmente quattro volte più veloce rispetto a farlo senza pipeline: 20 bucati richiederebbero circa 5 volte il tempo richiesto da un singolo bucato, mentre 20 bucati fatti in sequenza richiedono 20 volte il tempo di un singolo bucato. Nella Figura 4.27 la pipeline risulta solo 2,3 volte più veloce, perché ci sono solamente 4 bucati in tutto. Si noti che all'inizio e alla fine dell'attività la pipeline della Figura 4.27 non è completamente piena, e le fasi di avvio e di fine possono diminuire significativamente le prestazioni se il numero di bucati non è abbastanza grande rispetto al numero degli stadi della pipeline. Se ci fossero molti più di 4 bucati, allora tutti gli stadi sarebbero occupati per la maggior parte del tempo e il miglioramento del throughput sarebbe molto vicino a 4.

Gli stessi principi si applicano ai processori, dove ciò che si vuole porre in pipeline è l'esecuzione delle istruzioni. Le istruzioni RISC-V richiedono tipicamente cinque passi:

1. Fetch dell'istruzione (cioè caricamento dell'istruzione dalla memoria);
2. Lettura dei registri e decodifica dell'istruzione;
3. Esecuzione di un'operazione o calcolo di un indirizzo;
4. Accesso a un operando nella memoria dati (se richiesto);
5. Scrittura del risultato in un registro (se richiesto).

Quindi la pipeline RISC-V, che esploreremo in questo capitolo, ha cinque stadi. L'esempio seguente mostra come la pipeline acceleri l'esecuzione delle istruzioni proprio come avviene per il bucato.

Confronto fra le prestazioni di un processore a singolo ciclo e quelle di un processore con pipeline

ESEMPIO

Per rendere concreto il discorso, consideriamo una pipeline ben precisa. In questo esempio, e nel resto del capitolo, limiteremo la nostra attenzione a otto istruzioni: load di una parola (`lw`), store di una parola (`sw`), somma (`add`), sottrazione (`sub`), AND (`and`), OR (`or`) e branch if equal (`beq`).

(continua)

(continua)

Confrontiamo il tempo medio necessario per l'esecuzione di queste istruzioni in un'implementazione a singolo ciclo, in cui tutte le istruzioni richiedono un ciclo di clock, con il tempo medio richiesto per un'implementazione con pipeline.

La durata delle operazioni delle unità funzionali più importanti è, per questo esempio, di 200 ps per l'accesso alla memoria, 200 ps per le operazioni della ALU e 100 ps per la lettura e la scrittura del register file.

Nel modello a singolo ciclo tutte le istruzioni impiegano esattamente un ciclo di clock, quindi la durata del ciclo di clock va dilatata in modo tale da poter contenere l'esecuzione dell'istruzione più lenta.

SOLUZIONE

Il tempo richiesto da ciascuna delle sette istruzioni è riportato nella **Figura 4.28**. Il progetto a singolo ciclo deve permettere la completa esecuzione dell'istruzione più lenta, la `lw` nella Figura 4.28, per cui il tempo richiesto da ciascuna istruzione sarà di 800 ps. In modo simile a quanto mostrato nella Figura 4.27, la **Figura 4.29** confronta l'esecuzione con e senza pipeline di tre istruzioni di load: il tempo trascorso tra la prima e la quarta istruzione nell'architettura senza pipeline è pari a 3×800 ps, ossia 2400 ps.

Tutti gli stadi della pipeline richiedono un ciclo di clock, per cui il periodo di clock deve essere abbastanza lungo da contenere l'operazione più lenta. Come nel progetto a singolo ciclo si deve considerare la durata dell'istruzione più lenta (che è di 800 ps), anche se alcune istruzioni potrebbero richiedere solo 500 ps, nel caso della pipeline si deve considerare la durata dello stadio più lento (che è di 200 ps), anche se alcuni stadi richiedono solo 100 ps.

La pipeline offre comunque un miglioramento delle prestazioni di quattro volte: l'intervallo di tempo tra la prima e la quarta istruzione è infatti di 3×200 ps, ossia 600 ps.

Dalla trattazione precedente è possibile ricavare una formula che fornisce l'incremento della velocità prodotto dalla pipeline: se gli stadi sono perfettamente bilanciati, il tempo di esecuzione delle istruzioni in un calcolatore con pipeline (in condizioni ideali) è pari a:

$$\text{Tempo tra due istruzioni}_{\text{con pipeline}} = \frac{\text{Tempo tra due istruzioni}_{\text{senza pipeline}}}{\text{Numero di stadi della pipeline}}$$

Tipo di istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub e or)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionato (beq)	200 ps	100 ps	200 ps			500 ps

Figura 4.28 Tempo di esecuzione totale per le diverse istruzioni calcolato a partire dal tempo di elaborazione richiesto dalle singole unità funzionali. Il calcolo assume che i multiplexer, l'unità di controllo, l'accesso al PC e l'unità di estensione del segno non introducano ritardi.

 © 978-88-08-19966-9

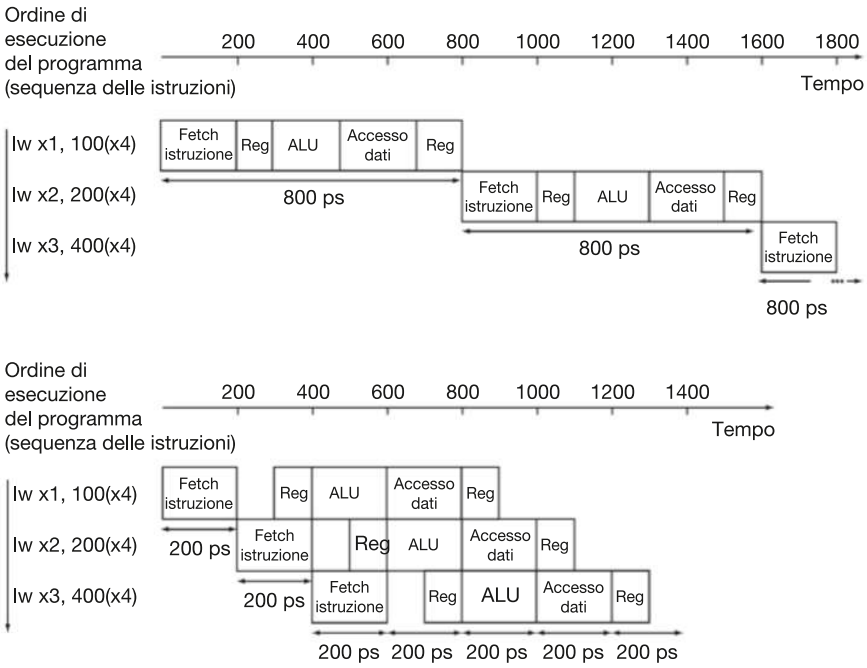


Figura 4.29 Confronto tra un'esecuzione a singolo ciclo di clock senza pipeline (in alto) e un'esecuzione con pipeline (in basso). In entrambi i casi si utilizzano gli stessi componenti hardware, il cui tempo di elaborazione è riportato nella Figura 4.28. Si trova una diminuzione del tempo medio di esecuzione delle istruzioni di un fattore 4: esso scende da 800 ps a 200 ps. Si confronti questa figura con la Figura 4.27: per il bucato avevamo ipotizzato che tutti gli stadi avessero la stessa durata. Se l'asciugatrice fosse stata più lenta, avrebbe determinato la durata degli altri stadi. Anche nel calcolatore la durata dei diversi stadi di elaborazione è determinata dalla risorsa funzionale più lenta: dal calcolo nella ALU o dall'accesso alla memoria. Assumiamo che la scrittura nel register file avvenga nella prima metà del ciclo di clock e che la lettura dal register file abbia luogo nella seconda metà; tale assunzione sarà valida per tutto questo capitolo.

In condizioni ideali, con un numero molto grande di istruzioni, l'incremento di velocità dovuto alla pipeline è pari al numero dei suoi stadi: una pipeline a cinque stadi sarebbe quindi quasi cinque volte più veloce.

La formula suggerisce che una pipeline a cinque stadi dovrebbe diminuire di quasi cinque volte il tempo di esecuzione rispetto alla versione senza pipeline, che nel nostro esempio era di 800 ps; cioè, dovrebbe avere un clock di 160 ps. Tuttavia, l'esempio mostra anche che gli stadi possono non essere perfettamente bilanciati; la pipeline introduce inoltre dei tempi (la cui origine sarà più chiara tra poco) che si aggiungono al tempo di esecuzione. In definitiva, il tempo di esecuzione per la singola istruzione in un calcolatore con pipeline sarà maggiore del minimo teorico, quindi l'incremento di velocità sarà minore del numero di stadi.

È bene notare che nell'esempio appena discusso non siamo riusciti a ottenere neppure il miglioramento di un fattore 4 che avevamo dichiarato. Se osserviamo il tempo totale di esecuzione delle tre istruzioni, infatti, in presenza di pipeline ci sono voluti 1400 ps rispetto ai 2400 ps dell'implementazione a singolo ciclo. Naturalmente, questo è dovuto al fatto che il numero di istruzioni non è grande. Che cosa accadrebbe se aumentassimo il numero di istruzioni?

Possiamo aumentare il numero di istruzioni a 1 000 003 (ne aggiungiamo 1 000 000 alle tre considerate prima). In presenza di pipeline, ogni istruzione contribuirebbe a incrementare di 200 ps il tempo di esecuzione totale, che passerebbe quindi a 1 000 000 × 200 ps + 1400 ps, ovvero a 200 001 400 ps. In

assenza di pipeline ogni istruzione in più richiederebbe 800 ps, dando un tempo di esecuzione totale di 1 000 000 × 800 ps + 2400 ps, cioè di 800 002 400 ps. In queste condizioni, quindi, il rapporto tra il tempo di esecuzione del processore senza pipeline e quello del processore dotato di pipeline si avvicina al rapporto tra la durata delle istruzioni nei due tipi di calcolatore:

$$\frac{800\,002\,400 \text{ ps}}{200\,001\,400 \text{ ps}} \simeq \frac{800 \text{ ps}}{200 \text{ ps}} \simeq 4{,}00$$

La tecnica della pipeline migliora le prestazioni *aumentando il throughput delle istruzioni*, ma *non riduce il tempo di esecuzione della singola istruzione*. Tuttavia, il throughput delle istruzioni rimane la metrica più importante, dal momento che i programmi reali eseguono miliardi di istruzioni.

Progettazione dell'insieme di istruzioni per architetture dotate di pipeline

Anche da questa semplice introduzione alla pipeline è possibile evidenziare alcuni elementi chiave dell'insieme delle istruzioni del RISC-V, che è stato progettato espressamente per l'esecuzione in pipeline.

Anzitutto, tutte le istruzioni RISC-V hanno la stessa lunghezza: tale proprietà semplifica notevolmente la fase di fetch delle istruzioni nel primo stadio della pipeline e la loro decodifica nel secondo stadio. In un insieme di istruzioni come quello dell'x86, in cui la lunghezza delle istruzioni varia da 1 byte a 15 byte, la progettazione della pipeline sarebbe notevolmente più complicata. Le implementazioni più recenti dell'architettura x86, in realtà, traducono le istruzioni x86 in microistruzioni più semplici che somigliano a quelle del RISC-V e inviano quindi alla pipeline queste semplici microistruzioni, invece delle istruzioni originali x86 (Paragrafo 4.11).

In secondo luogo, le istruzioni RISC-V adottano un numero ridotto di formati diversi e in tutti i formati i registri sorgente sono sempre specificati nella stessa posizione, qualunque sia l'istruzione.

In terzo luogo, nel RISC-V gli operandi residenti in memoria possono comparire solo nelle istruzioni di load e di store: questo vincolo permette di utilizzare lo stadio di esecuzione per calcolare l'indirizzo di memoria, per accedere poi alla memoria nello stadio successivo. Se si potessero effettuare delle operazioni sugli operandi residenti in memoria, come avviene nell'x86, gli stadi 3 e 4 dovrebbero essere espansi in tre stadi: uno stadio per il calcolo dell'indirizzo, uno per l'accesso alla memoria dati e un terzo per l'esecuzione delle operazioni. Vedremo tra poco quali siano gli inconvenienti di pipeline più lunghe.

Hazard nelle pipeline

In una pipeline si possono verificare situazioni in cui l'istruzione successiva non può essere eseguita nel ciclo di clock immediatamente successivo. Tali eventi sono detti *hazard* (o *criticità*) e possono essere di tre tipi.

Hazard strutturali

Il primo tipo di hazard è l'**hazard strutturale**, che si verifica quando le risorse hardware presenti non sono in grado di supportare la combinazione di istruzioni che si vorrebbe eseguire nello stesso ciclo di clock. Nel caso del bucato, si verificherebbe un hazard strutturale se si utilizzasse un'unica macchina per

Hazard strutturale: una situazione in cui un'istruzione non può essere eseguita nel ciclo di clock previsto perché l'hardware non supporta la combinazione di istruzioni da eseguire.

 © 978-88-08-19966-9

lavare e asciugare anziché due macchine separate, oppure se il nostro coinquilino fosse impegnato in qualche altro compito e non potesse riporre gli abiti nell'armadio. In questi casi, la nostra attenta programmazione basata sulla pipeline diventerebbe inutile.

Come già osservato, l'insieme di istruzioni RISC-V è stato progettato appositamente per essere eseguito in pipeline, facendo in modo che per i progettisti fosse semplice evitare gli hazard strutturali nel progetto dell'unità di elaborazione. Supponiamo, tuttavia, di disporre di una singola memoria anziché di due: se nella pipeline della Figura 4.29 ci fosse una quarta istruzione, la prima istruzione dovrebbe accedere ai dati in memoria, mentre la quarta istruzione dovrebbe essere prelevata dalla stessa memoria nel medesimo ciclo di clock. Senza due memorie distinte, l'architettura con pipeline presenterebbe in questo caso un hazard strutturale.

Hazard sui dati

Hazard sui dati: si verifica quando un'istruzione non può essere eseguita in un certo ciclo di clock perché i dati di cui ha bisogno l'istruzione non sono ancora disponibili.

Un **hazard sui dati** si verifica quando la pipeline deve essere messa in stallo perché uno stadio deve attendere che termini l'elaborazione in un altro stadio della pipeline. Ritornando all'esempio della lavanderia, supponiamo di trovare un calzino spaiato sul tavolo da stiro. Una possibilità è di cercare il calzino mancante nell'armadio della vostra stanza; naturalmente, il bucato che ha terminato l'asciugatura (ed è pronto per essere stirato) dovrà attendere che abbiate terminato la ricerca, e così pure il bucato che è uscito dalla fase di lavaggio ed è pronto per essere asciugato.

In un calcolatore dotato di pipeline, gli hazard sui dati nascono quando un'istruzione dipende dal risultato di un'istruzione precedente che si trova ancora all'interno della pipeline (questa è una dipendenza che non esiste realmente quando si fa il bucato). Supponiamo, per esempio, di avere un'istruzione di somma seguita da una sottrazione che utilizza il risultato della somma (x19):

```
add  x19, x0, x1
sub  x2, x19, x3
```

Se non si prendessero delle contromisure, gli hazard sui dati potrebbero causare tanti stalli della pipeline: l'istruzione `add`, infatti, non scrive il proprio risultato prima del quinto stadio, e ciò significa che si dovrebbero perdere tre cicli di clock.

Anche se possiamo affidare ai compilatori il compito di evitare questo tipo di hazard, il risultato non sarebbe soddisfacente: queste dipendenze sono troppo frequenti e il ritardo che viene introdotto è troppo lungo per poter sperare che il compilatore possa risolvere il problema.

Propagazione: detta anche bypass è un metodo per risolvere gli hazard sui dati che consiste nel prelevare i dati mancanti da un buffer interno all'unità di elaborazione, invece di aspettare che arrivino da uno dei registri visibili al programmatore o dalla memoria.

La soluzione più utilizzata è basata sull'osservazione che non è necessario che termini l'istruzione per cercare di risolvere un hazard sui dati: nella sequenza di istruzioni appena vista, non appena il risultato della somma viene generato dalla ALU, questo potrebbe essere già utilizzato come ingresso alla ALU per l'operazione di sottrazione. La tecnica che prevede l'aggiunta di un circuito che fornisce in un punto dell'esecuzione il dato mancante, propagandolo da una risorsa interna, viene chiamata **propagazione** o **bypass**.

Propagazione nel caso di due istruzioni

ESEMPIO

Nel caso delle due istruzioni sopra riportate, mostrare quali stadi della pipeline dovrebbero essere collegati con la tecnica della propagazione. Si utilizzi lo schema riportato nella **Figura 4.30** per rappresentare

(continua)

(continua)

l'unità di elaborazione durante l'elaborazione nei cinque stadi della pipeline. Si faccia una copia dell'unità di elaborazione per ciascuna istruzione e la si allinei sull'asse dei tempi in modo analogo all'esempio del bucato della Figura 4.27.

SOLUZIONE

La **Figura 4.31** mostra i collegamenti necessari a propagare il valore di `x1`, preso al termine della fase di esecuzione dell'istruzione `add`, come input nella fase di esecuzione dell'istruzione `sub`.

Da questa rappresentazione grafica dell'esecuzione nella pipeline si evince che la propagazione funziona solamente se lo stadio a cui il dato viene propagato è successivo nel tempo allo stadio dal quale viene prelevato. Per esempio, non ci potrebbe essere una propagazione valida tra l'uscita della fase di accesso alla memoria di un'istruzione e l'ingresso della fase di esecuzione dell'istruzione successiva, poiché ciò vorrebbe dire propagare un dato a ritroso nel tempo.

La tecnica della propagazione funziona molto bene e verrà descritta in dettaglio nel Paragrafo 4.8, ma non è in grado di evitare tutti gli stalli di una pipeline. Per esempio, supponiamo che la prima istruzione della Figura 4.31 sia un'istruzione di caricamento dalla memoria in `x1` (load), anziché una `add`. Come si può facilmente vedere esaminando la Figura 4.31, il dato desiderato sarebbe disponibile solamente *dopo* il quarto stadio della prima istruzione,

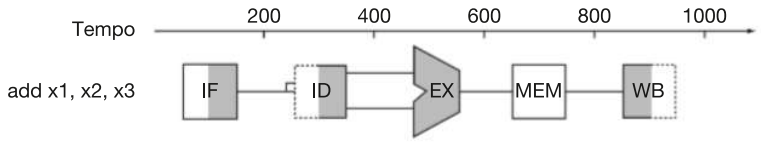


Figura 4.30 Rappresentazione grafica di un'istruzione nella pipeline ispirata alla pipeline del bucato della Figura 4.27. In tutto il capitolo utilizzeremo dei simboli grafici per rappresentare le risorse fisiche della pipeline; all'interno dei simboli verrà riportata un'abbreviazione, corrispondente al nome dello stadio associato. Le sigle dei cinque stadi sono le seguenti: *IF* per lo stadio di prelevamento dell'istruzione (da *instruction fetch*), con il quadrato associato che rappresenta la memoria istruzioni; *ID* per lo stadio di decodifica delle istruzioni e lettura dal register file (da *instruction decode*), con il simbolo associato che rappresenta il register file (che in questo stadio viene letto); *EX* per lo stadio di esecuzione (da *execution*), con il simbolo associato che rappresenta la ALU; *MEM* per lo stadio di accesso alla memoria dati, con il quadrato associato che rappresenta la memoria dati; *WB* per lo stadio di scrittura del risultato nel register file (da *write-back*), con il simbolo associato che rappresenta il register file (che in questo stadio viene scritto). L'ombreggiatura indica che l'elemento funzionale viene utilizzato dall'istruzione; per esempio, MEM viene rappresentato senza ombreggiatura perché l'istruzione `add` non accede alla memoria dati. L'ombreggiatura della metà destra del register file o della memoria indica che l'elemento viene letto in quello stadio, mentre l'ombreggiatura della metà sinistra indica che l'elemento viene scritto; per esempio, la metà destra di ID risulta ombreggiata nel secondo stadio perché il register file viene letto, mentre nel quinto stadio la metà sinistra di WB è ombreggiata per indicare che il register file viene scritto.

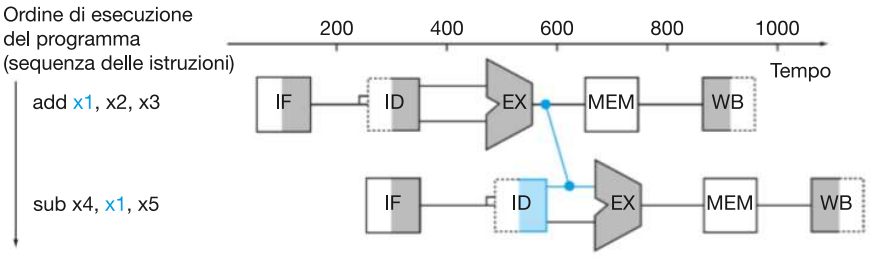


Figura 4.31 Rappresentazione grafica della propagazione. Il segmento in blu che connette le due copie della pipeline mostra il cammino che segue la propagazione del dato dall'uscita dello stadio EX della `add` all'input dello stadio EX della `sub`, dove sostituisce il contenuto del registro `x1` letto dal register file nel secondo stadio della `sub`.

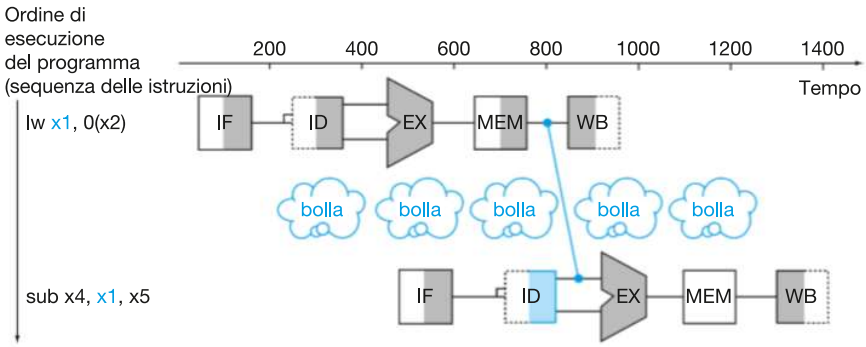


Figura 4.32 Anche se si fa uso della propagazione, è necessario uno stallo quando un'istruzione in formato R, che segue un'istruzione load, deve utilizzare il dato trasferito dalla memoria dalla load. Senza lo stallo, la propagazione del dato dall'uscita dello stadio di accesso alla memoria all'ingresso dello stadio di esecuzione procederebbe a ritroso nel tempo, il che è impossibile. Questa figura è in realtà una semplificazione, non potendo sapere se lo stallo sarà necessario o meno fino a quando l'istruzione di sottrazione non verrà letta e decodificata. Nel Paragrafo 4.8 mostreremo in dettaglio che cosa accade realmente in questo tipo di hazard.

Hazard sui dati di una load: è un tipo particolare di hazard sui dati, originato dal fatto che il dato letto dalla memoria da un'istruzione load non è ancora disponibile quando servirebbe a un'altra istruzione.

Stallo della pipeline: detto anche **bolla**, è uno stallo richiesto per risolvere un hazard.

ossia troppo tardi per essere utilizzato come *input* del terzo stadio della sottrazione. Di conseguenza, anche disponendo del meccanismo della propagazione, dovremmo imporre uno stallo della durata di un ciclo di clock a causa di un **hazard sui dati di una load** (*load-use data hazard*) come mostrato nella Figura 4.32. Questa figura mostra una situazione particolarmente importante della pipeline, chiamata formalmente **stallo della pipeline** ma spesso indicata con il termine **bolla** (*bubble*)[1].

In seguito vedremo altre situazioni di stallo della pipeline. Nel Paragrafo 4.8 mostreremo come possono essere gestiti i casi più difficili (come quello appena illustrato), utilizzando dei circuiti dedicati al riconoscimento degli hazard e alla messa in stallo della pipeline, oppure utilizzando un software che sia in grado di riordinare il codice per evitare gli hazard dovuti all'utilizzo dei dati delle load, come mostrato dal prossimo esempio.

Il meccanismo della propagazione illustra un altro elemento chiave dell'architettura RISC-V, oltre alle tre caratteristiche descritte in precedenza in questo paragrafo. Un'istruzione RISC-V scrive un solo risultato e lo fa sempre nell'ultimo stadio della pipeline. La propagazione sarebbe molto più difficile se ogni istruzione producesse più risultati da propagare o se questi venissero scritti prima del termine dell'esecuzione dell'istruzione.

Riordinare il codice per evitare stalli nella pipeline

ESEMPIO

Si consideri il seguente frammento di codice C:

```
a = b + e;
c = b + f;
```

Di seguito è riportato il codice RISC-V corrispondente, assumendo che tutte le variabili siano contenute in memoria e possano essere indirizzate come spiazzamento rispetto a x31:

(continua)

[1] Più propriamente, stallo è la condizione in cui si vuole mettere la pipeline, la bolla è il modo per ottenere questa condizione, come sarà chiaro tra poco.

(continua)

```
lw   x1,0(x31)   // load b
lw   x2,4(x31)   // load e
add  x3,x1,x2    // b + e
sw   x3,12(x31)  // store a
lw   x4,8(x31)   // load f
add  x5,x1,x4    // b + f
sw   x5,16(x31)  // store c
```

Identificare gli hazard nel codice riportato sopra e riordinare le istruzioni in modo da evitare stalli nella pipeline.

SOLUZIONE

Entrambe le istruzioni `add` presentano un hazard, perché uno dei loro operandi dipende dall'istruzione `lw` immediatamente precedente. Si noti che la propagazione elimina alcuni potenziali hazard, tra cui la dipendenza della prima `add` dalla prima `lw` e tutti gli hazard sulle istruzioni di store. Spostando di due posizioni verso l'alto la terza istruzione `lw`, si possono eliminare entrambi gli hazard:

```
lw   x1,0(x31)
lw   x2,4(x31)
lw   x4,8(x31)
add  x3,x1,x2
sw   x3,12(x31)
add  x5,x1,x4
sw   x5,16(x31)
```

Su un processore con pipeline dotato di propagazione, la sequenza riordinata richiederà due cicli di clock in meno della versione originale.

Approfondimento. Il termine *forwarding* (propagazione) deriva dall'idea che il risultato è passato da un'istruzione a quella successiva. Il termine inglese *bypassing* (scavalcamento) sottolinea il passaggio diretto di un risultato all'unità che ne ha bisogno, senza passare per il register file.

Hazard sul controllo

Il terzo tipo di hazard è rappresentato dagli **hazard sul controllo** o **hazard sui salti condizionati**. Questi hazard avvengono quando bisogna prendere una decisione in funzione del risultato dell'esecuzione di un'istruzione e altre istruzioni sono già state avviate all'esecuzione. Riferendoci all'esempio del bucato, supponiamo che ai nostri amici sia affidato l'infelice compito di lavare le divise di una squadra di calcio: a seconda di quanto sono sporche, occorre determinare la quantità di detersivo e la temperatura dell'acqua in modo da effettuare un lavaggio abbastanza energico da pulire le divise, ma non troppo energico per evitare che le divise si usurino in fretta. Nella pipeline per il bucato sarebbe necessario attendere il completamento del secondo stadio, quando si possono esaminare le divise asciutte, prima di poter decidere se sia necessario modificare le impostazioni della lavatrice. Che cosa si può fare in questo caso?

Hazard sul controllo: detti anche **hazard sui salti condizionati**, si verificano quando un'istruzione non può essere eseguita nel ciclo di clock corretto della pipeline, perché un'altra istruzione è stata caricata al suo posto; ossia, il flusso degli indirizzi delle istruzioni non è quello che la pipeline si aspetta.

Nel seguito esamineremo due possibili soluzioni agli hazard sul controllo applicate al caso del bucato e forniremo la soluzione equivalente per i calcolatori.

Stallo: lavare il bucato in modo sequenziale fino a quando il primo carico è asciutto; se il risultato ottenuto non è quello desiderato, ripetere i cicli di lavaggio e di asciugatura finché non si ottengono le impostazioni corrette.

Questa scelta prudente è certamente corretta, ma è lenta.

In un calcolatore l'equivalente di un'operazione di scelta è l'istruzione di salto condizionato (branch). Si noti che la lettura dalla memoria istruzioni dell'istruzione che segue la branch viene fatta nel ciclo di clock immediatamente successivo al prelevamento della branch; tuttavia la pipeline non ha alcun modo di sapere quale sarà l'istruzione successiva perché *ha appena ricevuto* l'istruzione di branch dalla memoria! Esattamente come nel caso del bucato, una soluzione potrebbe essere: mettere in stallo la pipeline immediatamente dopo aver caricato dalla memoria un'istruzione di branch e attendere fino a che la pipeline non abbia determinato il risultato del confronto prima di calcolare l'indirizzo dell'istruzione successiva.

Ipotizziamo di avere inserito sufficienti risorse hardware addizionali da poter confrontare il contenuto di due registri, calcolare l'indirizzo di destinazione del salto e aggiornare il PC già nel secondo stadio (Paragrafo 4.9 per i dettagli). Anche disponendo di tali circuiti aggiuntivi, in presenza di salti condizionati la pipeline si comporterebbe come quella della **Figura 4.33**: l'istruzione, che deve essere eseguita subito dopo la `beq` nel caso in cui i due operandi di quest'ultima risultino diversi, richiede uno stallo pari a un ciclo di clock, cioè 200 ps, prima di poter essere caricata dalla memoria istruzioni.

Impatto sulle prestazioni di uno stallo dovuto ai salti condizionati

ESEMPIO

Stimare l'impatto sul numero di *cicli di clock per istruzione* (CPI) degli stalli dovuti alle istruzioni di salto condizionato. Si assuma che tutte le altre istruzioni abbiano un CPI pari a 1.

SOLUZIONE

La Figura 3.22 del Capitolo 3 mostra che i salti condizionati costituiscono il 10% delle istruzioni eseguite dagli SPECint2006. Dato che le altre istruzioni eseguite hanno un CPI pari a 1 e i salti condizionati richiedono un ciclo di clock aggiuntivo dovuto allo stallo, si otterrebbe un CPI totale pari a 1,10, e conseguentemente un rallentamento di 1,10 rispetto al caso ideale.

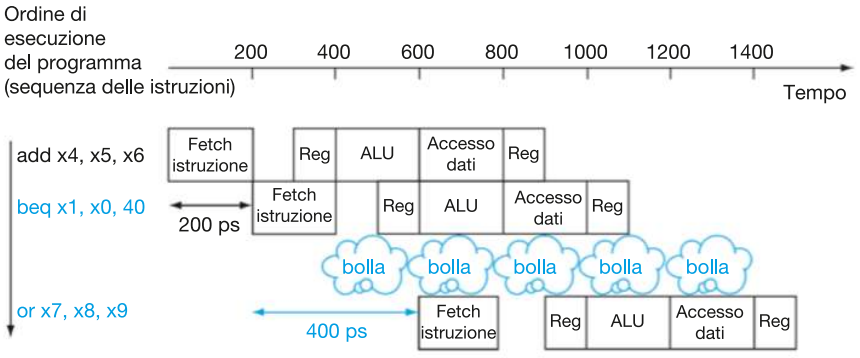


Figura 4.33 Esempio di pipeline messa in stallo a ogni istruzione di salto condizionato come possibile soluzione agli hazard sul controllo. In questo esempio si suppone che il salto venga eseguito e che l'istruzione di destinazione del salto sia la `or`. Dopo l'istruzione di salto condizionato si verifica uno stallo della pipeline, o bolla, della durata di uno stadio. In realtà il processo di generazione di uno stallo è leggermente più complicato, come vedremo nel Paragrafo 4.9; l'effetto sulle prestazioni è comunque lo stesso dell'inserimento di una bolla.

Se non si può decidere nel secondo stadio l'esito del confronto associato ai salti condizionati, come spesso accade nelle pipeline più lunghe, si genera un rallentamento ancora maggiore della pipeline quando occorre metterla in stallo per una branch. Il costo di questa soluzione sarebbe troppo elevato (per quasi tutti i calcolatori) per poterla prendere in considerazione. Si ricorre, quindi, a una seconda soluzione per risolvere gli hazard sul controllo, che è basata su una delle grandi idee del Capitolo 1:

> ***Predizione***: se siamo ragionevolmente sicuri di conoscere le impostazioni ottimali della lavatrice per lavare le divise, possiamo *supporre* che queste impostazioni funzioneranno e iniziare a lavare il secondo carico di divise mentre il primo si sta ancora asciugando.

In tal modo, se le impostazioni funzionano, non si rallenta la pipeline. In caso contrario, occorrerà ripetere il lavaggio del primo carico di divise che era stato lavato utilizzando le impostazioni che avevamo supposto ottimali.

In effetti, i calcolatori utilizzano estensivamente la **predizione** nella gestione dei salti condizionati. L'approccio più semplice consiste nel predire sempre che il salto non venga preso (*untaken branch*). Se la predizione si rivela corretta, la pipeline procede al massimo della velocità, e solo quando il salto deve essere preso si verifica uno stallo. I due casi possibili sono esemplificati nella **Figura 4.34**.

Una versione più sofisticata di **predizione dei salti** consente di prevedere se un salto debba essere eseguito o meno. Tornando all'analogia con il bucato, le uniformi scure richiederebbero certe impostazioni di lavaggio e quelle chiare altre. Nei programmi software, consideriamo i salti che si trovano al

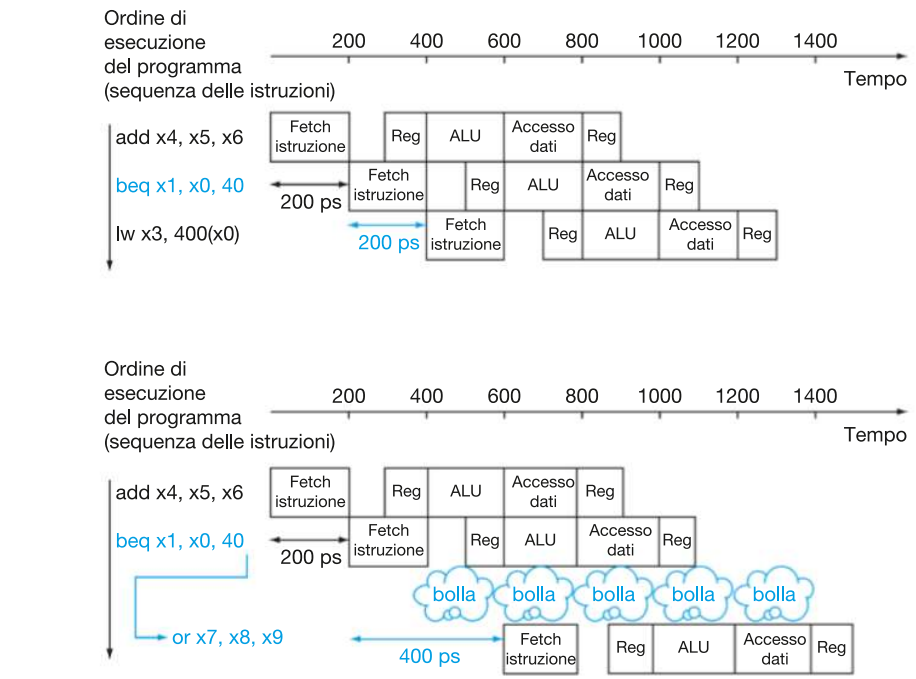


Figura 4.34 Pipeline in cui gli hazard sul controllo vengono risolti predicendo che i salti condizionati non vengano eseguiti. Lo schema mostra la pipeline nel caso in cui il salto non viene eseguito (sopra) e nel caso in cui il salto viene eseguito (sotto). Come sottolineato nella Figura 4.33, l'inserimento di una bolla nel modo mostrato in questo schema costituisce una semplificazione di ciò che accade realmente, almeno durante il primo ciclo di clock successivo alla `beq`; i dettagli verranno mostrati nel Paragrafo 4.9.

Predizione dei salti: un metodo per risolvere gli hazard sul controllo basato sull'ipotesi che il confronto associato a un salto condizionato dia un certo risultato; l'esecuzione procede basandosi su questa ipotesi, invece di aspettare che sia verificato il risultato effettivo del confronto.

 © 978-88-08-19966-9

termine dei cicli e che sono utilizzati per ritornare all'inizio del ciclo: poiché questi salti vanno all'indietro ed è verosimile che siano eseguiti la maggior parte delle volte, il processore potrebbe decidere che i salti verso indirizzi minori vadano sempre eseguiti.

Questi approcci rigidi alla predizione dei salti ipotizzano un comportamento che è sempre lo stesso e non tengono conto delle individualità specifiche di ciascuna istruzione di salto. I predittori hardware *dinamici*, invece, determinano la predizione per ciascun salto in funzione del comportamento precedente di quell'istruzione di salto, e possono modificare la predizione di ogni salto durante l'esecuzione del programma. Tornando all'esempio del bucato, nella predizione dinamica ci sarebbe una persona che esamina quanto erano sporche le divise e predice di conseguenza le impostazioni del lavaggio, modificando la **predizione** per la volta successiva sulla base del successo delle predizioni fatte in precedenza.

Un'implementazione diffusa della predizione dinamica dei salti consiste nel memorizzare la storia di ciascun salto, ricordando quando è stato preso o no, e nell'utilizzare il comportamento recente per predire il futuro. Come vedremo più avanti, la quantità e il tipo di informazione che viene memorizzata a questo scopo sono cresciuti molto negli anni, con il risultato che i circuiti predittori dei salti possono raggiungere attualmente una precisione superiore al 90% (Paragrafo 4.9). Quando la predizione si rivela errata, l'unità di controllo della pipeline deve garantire che le istruzioni che seguivano quella di salto e sono già state caricate nella pipeline non abbiano alcun effetto, e deve far ripartire la pipeline dall'indirizzo corretto. Nell'analogia con la lavanderia, dobbiamo smettere di accettare nuovi bucati in modo da poter far ripartire la pipeline dal bucato lavato in maniera non corretta.

Come avviene in tutte le altre soluzioni degli hazard sul controllo, il problema si aggrava al crescere della lunghezza della pipeline, poiché aumenta il costo delle predizioni errate. Le diverse soluzioni agli hazard sul controllo sono descritte in maggior dettaglio nel Paragrafo 4.9.

Approfondimento. C'è un terzo approccio possibile alla soluzione degli hazard sul controllo, detto *decisione ritardata*. Proseguendo con l'analogia con il bucato, ogni volta che occorre prendere una decisione possiamo mettere nella lavatrice un carico di abiti diverso dalle divise dei calciatori, nell'attesa che le divise del carico precedente si asciughino. A patto di avere a disposizione una sufficiente quantità di abiti sporchi il cui lavaggio non dipende da quello delle divise, questa soluzione funziona correttamente.

Tale soluzione, a cui abbiamo accennato in precedenza, nel mondo dei calcolatori è detta *salto ritardato* ed è la soluzione effettivamente adottata nell'architettura MIPS. In un salto ritardato, l'istruzione successiva all'istruzione di salto viene sempre eseguita e il salto viene eseguito, eventualmente, *dopo* questa istruzione. Questo comportamento è invisibile al programmatore assembler MIPS, poiché l'assemblatore è in grado di riordinare automaticamente la sequenza delle istruzioni in modo da garantire il comportamento dei salti previsto dal programmatore. I compilatori MIPS inseriscono sempre un'istruzione non dipendente dal risultato del salto immediatamente dopo l'istruzione di salto ritardato; quando il salto viene preso, viene modificato l'indirizzo dell'istruzione che *segue* questa istruzione "sicura". Nell'esempio della Figura 4.33, l'istruzione `add` prima della branch non ha influenza sull'istruzione di salto, può quindi essere spostata subito dopo la branch per coprire completamente il ritardo nel salto. Dal momento che i salti ritardati sono utili quando i ritardi sono brevi, nessun processore utilizza questo tipo di salti con un ritardo maggiore di un ciclo di clock. Per ritardi più lunghi solitamente si utilizza la predizione hardware sui salti.

Riepilogo sulla pipeline

La pipeline è una tecnica che sfrutta il **parallelismo** tra le istruzioni di un programma software scritto in maniera sequenziale. Rispetto alla programmazione

dei multiprocessori (Capitolo 6), ha il grosso vantaggio di essere sostanzialmente invisibile al programmatore.

Nei prossimi paragrafi di questo capitolo svilupperemo il concetto di pipeline basandoci sul sottoinsieme di istruzioni RISC-V utilizzato nel Paragrafo 4.4 per l'implementazione a singolo ciclo e mostreremo una versione semplificata della pipeline RISC-V. Esamineremo, quindi, i problemi introdotti dall'uso delle **pipeline** e le prestazioni che possono essere ottenute in condizioni operative tipiche.

I lettori maggiormente interessati al software e all'impatto della pipeline sulle prestazioni hanno già acquisito le conoscenze fondamentali e possono passare al Paragrafo 4.11, che introduce alcuni concetti avanzati su questa tecnica (come la pipeline superscalare e il riordinamento dinamico del codice). Nel Paragrafo 4.12, invece, viene esaminata la pipeline di alcuni dei microprocessori più recenti.

Se invece siete interessati a comprendere come si possa implementare una pipeline e alle sfide poste dalla gestione degli hazard, potete passare a esaminare il progetto di un'unità di elaborazione con pipeline e l'unità di controllo di base associata, trattati nel Paragrafo 4.7. Utilizzando i concetti appresi in questo paragrafo, potete poi passare a esaminare i meccanismi di implementazione della propagazione e dello stallo, trattati nel Paragrafo 4.8. Infine, potete leggere il Paragrafo 4.9 per vedere alcune soluzioni agli hazard sul controllo e il Paragrafo 4.10 per la gestione delle eccezioni.

Autovalutazione

Per ciascuna sequenza di istruzioni riportate nella tabella sottostante, stabilire se occorre mettere in stallo la pipeline, se si possono evitare gli stalli utilizzando la propagazione o se si può eseguire il codice senza stalli né propagazione.

Sequenza 1	Sequenza 2	Sequenza 3
`lw x10, 0(x10)`	`add x11, x10, x10`	`addi x11, x10, 1`
`add x11, x10, x10`	`addi x12, x10, 5`	`addi x12, x10, 2`
	`addi x14, x11, 5`	`addi x13, x10, 3`
		`addi x14, x10, 4`
		`addi x15, x10, 5`

Capire le prestazioni dei programmi

Al di fuori del sottosistema di memoria, la pipeline è di solito l'elemento più importante per determinare il CPI di un processore e quindi le sue prestazioni. Come vedremo nel Paragrafo 4.11, la comprensione delle prestazioni di un moderno processore con pipeline a **esecuzione parallela** è difficile, perché i problemi che si incontrano sono più complessi di quelli di un semplice processore a singola pipeline. In ogni caso, gli hazard strutturali, sui dati e sul controllo rimangono rilevanti sia per le pipeline semplici sia per quelle più sofisticate.

Per le pipeline moderne, gli hazard strutturali spesso riguardano le unità a virgola mobile che possono operare non completamente in pipeline, mentre gli hazard sul controllo sono di solito un problema che riguarda maggiormente i programmi che lavorano su numeri interi (questi tendono ad avere più salti condizionati sui quali è difficile fare previsioni). Gli hazard sui dati possono rappresentare un vero e proprio collo di bottiglia per le prestazioni sia dei programmi che lavorano su numeri interi sia di quelli che lavorano su numeri in virgola mobile.

Spesso è più facile trattare gli hazard sui dati nei programmi in virgola mobile: in questi casi, infatti, per la ridotta frequenza dei salti condizionati e per l'accesso più regolare ai dati in memoria, diventa più facile per il compilatore riordinare la sequenza delle istruzioni in modo da evitare gli hazard. È invece più difficile effettuare questa ottimizzazione nei programmi che lavorano su interi, i quali accedono ai dati in memoria in modo più irregolare, dato che utilizzano maggiormente i puntatori. Come vedremo nel Paragrafo 4.11, esistono tecniche più ambiziose, basate sia sui compilatori sia sull'hardware, per ridurre le dipendenze tra i dati attraverso la riorganizzazione dell'ordine di esecuzione delle istruzioni.

QUADRO D'INSIEME

Latenza (pipeline): numero di stadi di una pipeline o numero di stadi tra due istruzioni successive in esecuzione.

La **pipeline** aumenta il numero di istruzioni che vengono eseguite contemporaneamente e la frequenza con cui viene iniziata e terminata l'esecuzione delle istruzioni. Questa tecnica non riduce il tempo richiesto per completare l'esecuzione della singola istruzione, chiamato anche **latenza**. Per esempio, una pipeline a cinque stadi richiede sempre 5 cicli di clock per completare l'esecuzione di un'istruzione. Utilizzando la terminologia introdotta nel Capitolo 1, la pipeline migliora il *throughput* delle istruzioni ma *non il tempo di esecuzione*, o *latenza*, della singola istruzione. Gli insiemi di istruzioni possono semplificare o rendere particolarmente difficile il compito ai progettisti delle pipeline, i quali devono comunque gestire gli hazard strutturali, sul controllo e sui dati. I meccanismi di **predizione** dei salti condizionati e la propagazione aiutano a rendere veloce l'elaborazione mantenendo un funzionamento corretto del calcolatore. ■

4.7 Unità di elaborazione con pipeline e unità di controllo associata

In questo, c'è meno di quanto vedano gli occhi.
Tallulah Bankhead, riferendosi ad Alexander Woollcott, 1922

La Figura 4.35 mostra l'unità di elaborazione a singolo ciclo progettata nel Paragrafo 4.4 con i cinque stadi della pipeline ben identificati. La suddivisione di un'istruzione in cinque fasi implica una pipeline a cinque stadi, il che significa che in un singolo ciclo di clock ci potranno essere fino a cinque istruzioni in esecuzione contemporaneamente. Occorre quindi separare l'unità di elaborazione in cinque parti, ciascuna delle quali prende il nome dalla fase di esecuzione dell'istruzione corrispondente:

1. IF: fetch dell'istruzione (*instruction fetch*);
2. ID: decodifica dell'istruzione (*instruction decode*) e lettura del register file;
3. EX: esecuzione (*execution*) o calcolo dell'indirizzo;
4. MEM: accesso alla memoria dati (*memory*);
5. WB: scrittura del risultato nel register file (*write-back*).

Queste cinque parti corrispondono approssimativamente al modo in cui è stata disegnata l'unità di elaborazione dei dati della Figura 4.35; durante l'esecuzione, istruzioni e dati si spostano generalmente da sinistra a destra, attraversando le cinque fasi. Tornando all'analogia con il bucato, i panni diventano più puliti, più asciutti e più ordinati mano a mano che si prosegue la lavorazione, e non devono mai ritornare alle fasi precedenti.

Vi sono tuttavia due eccezioni al flusso di esecuzione da sinistra verso destra:

- lo stadio di scrittura del risultato, nel quale viene scritto il risultato dell'esecuzione nel register file al centro dell'unità di elaborazione;

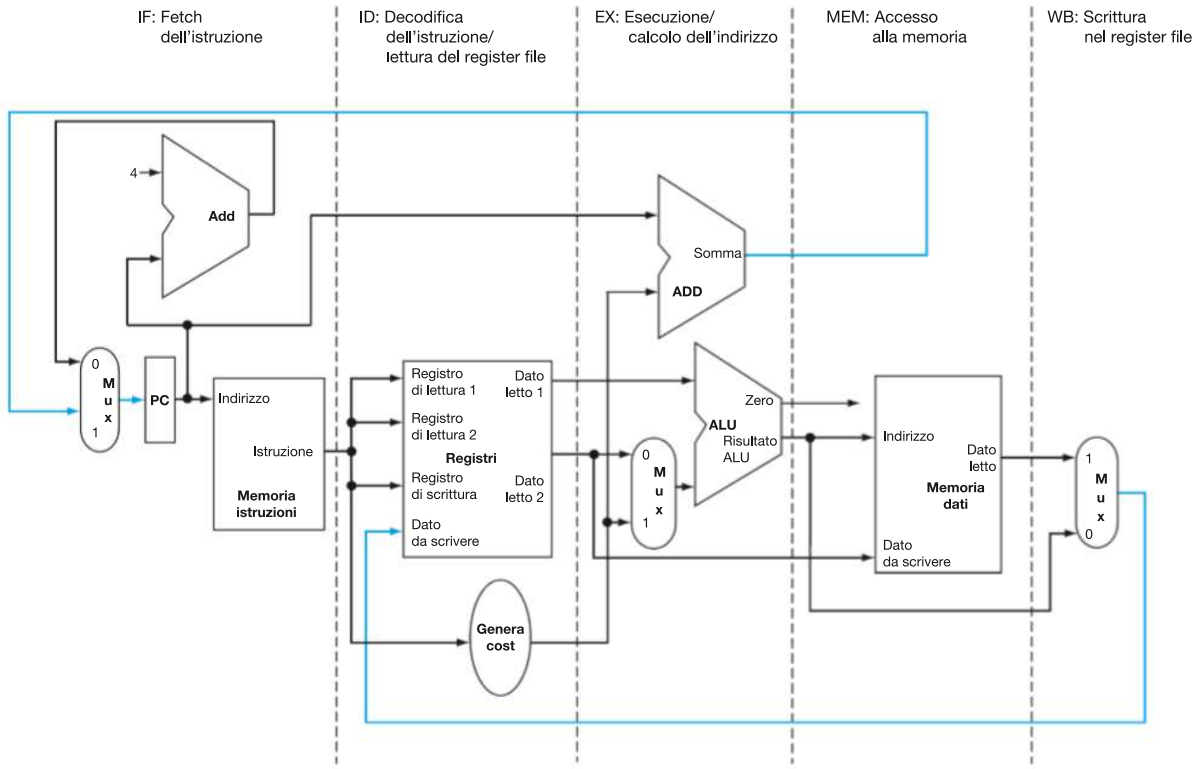


Figura 4.35 L'unità di elaborazione a singolo ciclo del Paragrafo 4.4 (simile a quella della Figura 4.21). Ciascun passaggio dell'esecuzione di un'istruzione può essere messo in corrispondenza con una parte dell'unità di elaborazione, seguendo il flusso di elaborazione da sinistra verso destra. Le sole eccezioni, mostrate in blu, sono costituite dall'aggiornamento del PC e dalla scrittura del risultato: nella fase di WB il risultato calcolato dalla ALU o il dato letto dalla memoria dati viene portato verso sinistra per scriverlo nel register file (di solito utilizziamo il colore blu per evidenziare i segnali di controllo, ma in questa figura vogliamo evidenziare linee dati).

- la selezione del valore successivo del PC, che viene scelto tra il valore del PC incrementato di 4 e l'indirizzo di destinazione del salto proveniente dallo stadio MEM.

Il flusso dei dati da destra a sinistra non ha effetto sull'istruzione corrente: questo spostamento all'indietro dei dati influenzerà solo le istruzioni successive presenti nella pipeline. Si noti che da destra verso sinistra il flusso dei dati nella parte inferiore dello schema può causare hazard sui dati, mentre il flusso nella parte superiore dello schema può provocare hazard sul controllo.

Un modo per descrivere ciò che succede nell'esecuzione con pipeline è quello di ipotizzare che ciascuna istruzione disponga di una propria unità di elaborazione e di inserire tale unità nella corretta posizione lungo l'asse dei tempi, visualizzando le relazioni tra le diverse istruzioni. La **Figura 4.36** mostra l'esecuzione delle stesse istruzioni della Figura 4.29, riportando per ciascuna di esse l'unità di elaborazione posizionata correttamente su un asse unico dei tempi. In questa figura è stata utilizzata la versione schematizzata dell'unità di elaborazione della Figura 4.35.

La Figura 4.36 sembra suggerire che tre istruzioni richiedano tre unità di elaborazione. Invece, possiamo aggiungere dei registri per salvare i dati parziali, in modo tale che le diverse parti di una singola unità di elaborazione possano essere condivise dalle diverse istruzioni durante la loro esecuzione.

Per esempio, come mostrato nella Figura 4.36, la memoria istruzioni viene utilizzata in uno solo dei cinque stadi di un'istruzione, per cui potrà

 © 978-88-08-19966-9

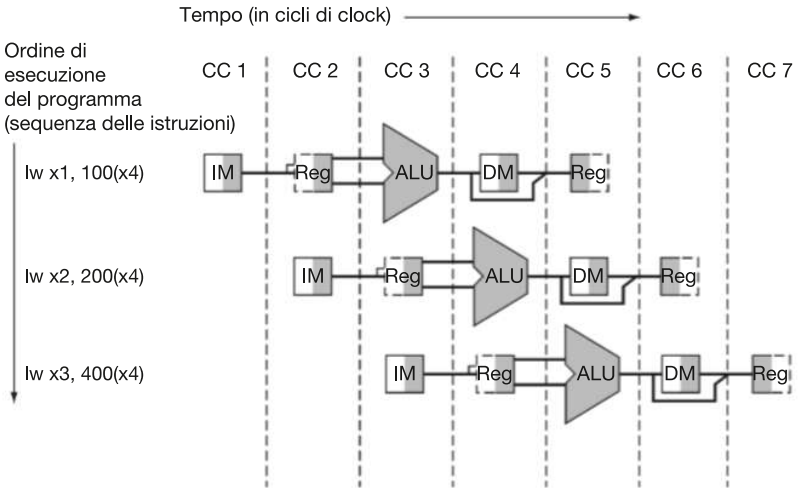


Figura 4.36 Esecuzione in pipeline delle stesse istruzioni eseguite dall'unità di elaborazione a singolo ciclo della Figura 4.35. In modo analogo alle Figure 4.30–4.32, si suppone che ciascuna istruzione disponga della propria unità di elaborazione e si evidenzia la parte utilizzata di ciascuna unità funzionale. A differenza delle figure precedenti, qui i diversi stadi sono etichettati con il nome della risorse fisica utilizzata, che corrisponde a una parte dell'unità di elaborazione della Figura 4.35. *IM* rappresenta la memoria istruzioni, *Reg* sta per register file, che assieme all'unità di estensione del segno viene utilizzato nello stadio di decodifica dell'istruzione e lettura del register file (ID) ecc. Per garantire una corretta sequenza temporale, questa unità di elaborazione schematizzata divide il register file in due parti logiche: i registri letti durante la fase di decodifica (ID) e i registri scritti nella fase finale (WB). Questo duplice impiego del register file viene rappresentato tracciando con una linea tratteggiata, nello stadio ID, il bordo della parte sinistra del register file che non è evidenziato (poiché non avviene alcuna operazione di scrittura in questo stadio); nello stadio WB, invece, è tratteggiato il bordo della parte destra non evidenziata (poiché non avvengono operazioni di lettura). Come già fatto in precedenza, si assume che il register file venga letto nella prima metà del periodo di clock e scritto nella seconda.

essere impiegata dalle istruzioni successive quando l'istruzione passa attraverso gli altri quattro stadi. Per preservare l'istruzione prelevata dalla memoria a beneficio degli altri quattro stadi, essa deve essere salvata in un registro. Considerazioni analoghe valgono per tutti gli altri stadi della pipeline, per cui occorrerà aggiungere dei registri all'interfaccia tra le coppie contigue di stadi della pipeline della Figura 4.35. Ricordando l'analogia con la lavanderia, ci potrebbe essere un cestino tra due stadi di lavorazione dove riporre i panni destinati alla fase successiva.

La Figura 4.37 mostra l'unità di elaborazione con pipeline in cui sono evidenziati i registri della pipeline. In ogni ciclo di clock, tutte le istruzioni procedono da un registro di pipeline a quello successivo. I registri prendono il nome dai due stadi che separano: per esempio, il registro di pipeline inserito tra gli stadi IF e ID è chiamato IF/ID.

Si noti che non c'è alcun registro di pipeline alla fine dello stadio di write-back. Tutte le istruzioni devono modificare qualche elemento dello stato del processore: il register file, la memoria o il PC, per cui un registro di pipeline separato sarebbe ridondante. Per esempio, un'istruzione di load scrive il proprio risultato in uno dei 32 registri del register file, e se l'istruzione successiva richiedesse tale valore potrebbe caricarlo direttamente da lì.

Ovviamente ogni istruzione aggiorna il PC, incrementandolo di 4 o scrivendo l'indirizzo di destinazione del salto. Il PC può essere visto come il registro di pipeline che alimenta lo stadio IF. Tuttavia, a differenza dei registri di pipeline evidenziati nella Figura 4.37, il PC fa parte dello stato dell'architettura, visibile all'esterno: il suo contenuto deve essere salvato quando si verifica un'eccezione, mentre il contenuto dei registri di pipeline veri e propri viene

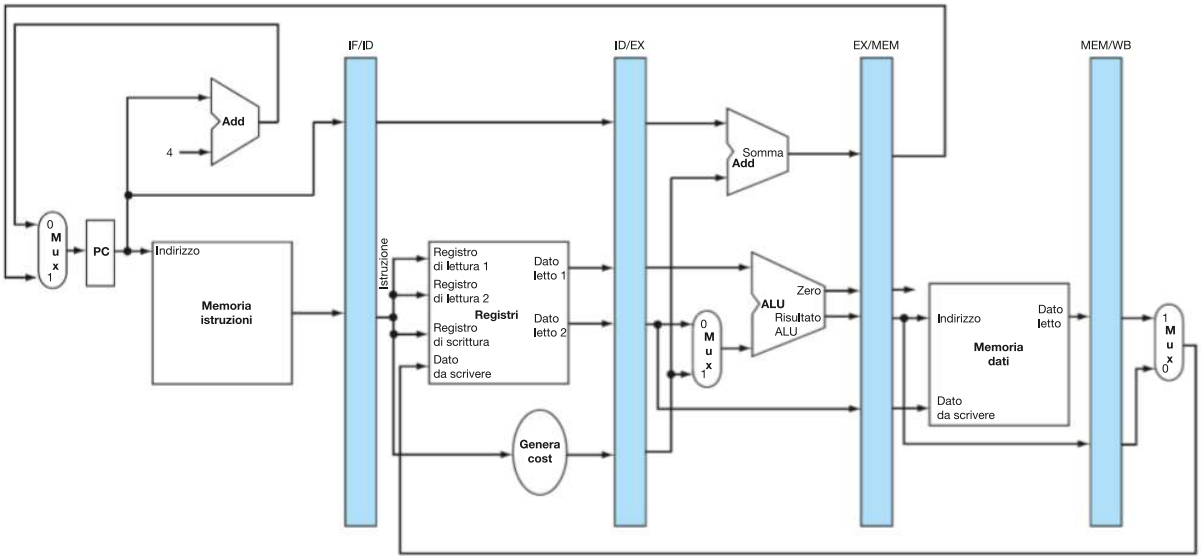


Figura 4.37 Versione con pipeline dell'unità di elaborazione della Figura 4.35. I registri di pipeline, evidenziati in blu, separano i diversi stadi della pipeline e per etichettarli si usa il nome dei relativi stadi che separano; per esempio, il primo registro viene etichettato con *IF/ID*, poiché separa lo stadio IF, di prelevamento dell'istruzione, dallo stadio ID, di decodifica dell'istruzione. I registri devono avere un'ampiezza sufficiente a memorizzare tutti i dati corrispondenti alle linee che li devono attraversare. Il registro IF/ID, per esempio, deve essere ampio 64 bit, poiché deve contenere sia l'istruzione prelevata dalla memoria su 32 bit sia il contenuto del PC incrementato di 4, su 32 bit. Gli altri registri verranno esaminati più in dettaglio nel seguito del capitolo; anticipiamo solamente che gli altri tre registri di pipeline contengono rispettivamente 128, 97 e 64 bit.

perso. Nell'analogia con la lavanderia, il PC rappresenta il cestino che contiene i panni sporchi prima dell'inizio del bucato.

Per mostrare il funzionamento della pipeline, in questo capitolo riporteremo delle sequenze di schemi che rappresentano le operazioni effettuate dalla pipeline in istanti di tempo successivi. Potreste pensare che occorra molto tempo per comprendere il contenuto di tutte queste pagine, ma non è così: le sequenze di schemi si analizzano in maniera assai veloce, poiché gli schemi possono essere semplicemente confrontati tra loro per vedere che cosa cambia in ciascun ciclo di clock. Il Paragrafo 4.8 descrive che cosa succede quando si verificano degli hazard sui dati nelle istruzioni presenti nella pipeline, ma per ora ignoriamo queste complicazioni.

Le Figure 4.38–4.40 costituiscono la prima sequenza di schemi e mostrano (evidenziata in blu) la parte dell'unità di elaborazione attiva nei diversi stadi di esecuzione di un'istruzione di load. Questa istruzione viene esaminata per prima, essendo attiva in tutti e cinque gli stadi. Come nelle Figure 4.32–4.34, verrà evidenziata la *metà destra* dei registri o della memoria quando queste unità vengono *lette*, e la *metà sinistra* quando vengono *scritte*.

In ciascuna figura si riporterà il nome dell'istruzione, `lw`, insieme al nome dello stadio attivo della pipeline. I cinque stadi sono i seguenti.

1. *Fetch dell'istruzione*. La parte superiore della **Figura 4.38** mostra la fase in cui l'istruzione viene prelevata dalla memoria istruzioni (utilizzando l'indirizzo contenuto nel PC) e scritta nel registro di pipeline IF/ID. L'indirizzo contenuto nel PC viene incrementato di 4 e riscritto nel PC stesso, in modo da essere pronto per il ciclo di clock successivo. Il PC viene anche salvato nel registro di pipeline IF/ID, perché potrebbe essere richiesto da un'istruzione successiva, per esempio da una `beq`. Il calcolatore non può prevedere quale istruzione verrà prelevata dalla memoria, per cui deve preparare i dati per l'esecuzione di tutte le possibili istruzioni, trasportando lungo la pipeline tutte le informazioni potenzialmente necessarie.

2. *Decodifica dell'istruzione e lettura del register file.* La parte inferiore della Figura 4.38 mostra che la parte contenente l'istruzione del registro di pipeline IF/ID fornisce il campo immediato su 12 bit, che viene convertito a 32 bit mediante estensione del segno. IF/ID fornisce anche il numero dei due registri da leggere. Il contenuto dei due registri e del campo immediato esteso a 32 bit vengono memorizzati nel registro di pipeline ID/EX insieme

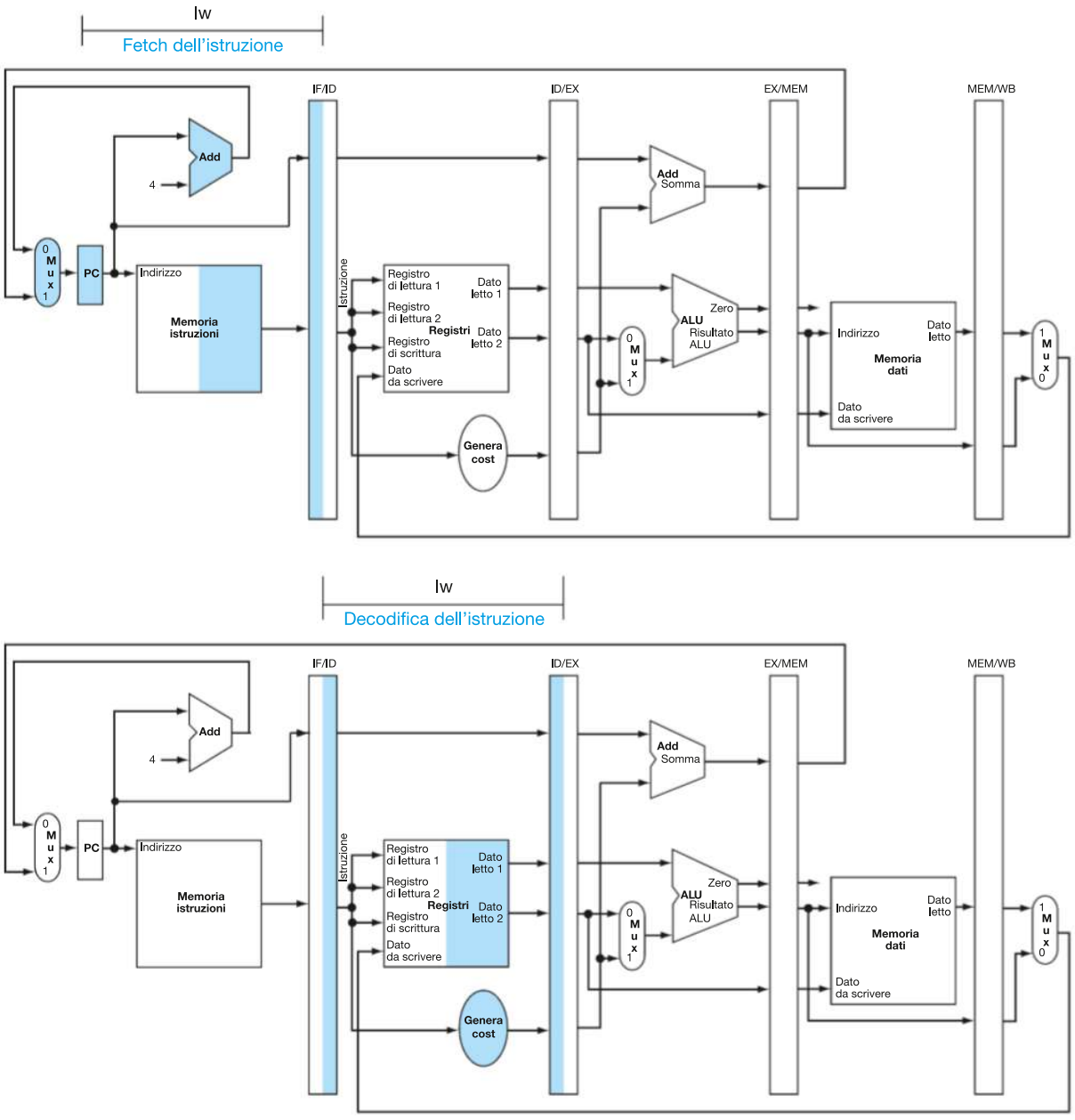


Figura 4.38 IF e ID: il primo e il secondo stadio della pipeline durante l'esecuzione di un'istruzione di load; le parti attive dell'unità di elaborazione mostrata nella Figura 4.37 sono evidenziate in blu. La convenzione per le parti evidenziate è la stessa utilizzata nella Figura 4.30. Come visto nel Paragrafo 4.2, non si può confondere la lettura con la scrittura dei registri, in quanto il loro contenuto varia solamente in corrispondenza del fronte del clock. Sebbene la load nello stadio 2 richieda solo il contenuto del primo registro, il processore non può sapere quale istruzione sta decodificando, per cui estende il segno del contenuto del campo immediato e legge il contenuto di entrambi i registri scrivendolo nel registro di pipeline ID/EX. Tre operandi non servono, ma la realizzazione dell'unità di controllo risulta più semplice se tutti e tre gli operandi vengono comunque scritti nel registro ID/EX.

al valore del PC. Anche in questo stadio, infatti, viene trasferito tutto ciò che potrebbe essere necessario per l'esecuzione delle fasi successive di tutte le possibili istruzioni.

3. *Esecuzione o calcolo dell'indirizzo*. La **Figura 4.39** mostra che l'istruzione di load legge dal registro di pipeline ID/EX il contenuto di un registro e il contenuto del campo immediato, esteso a 32 bit e dotato di segno, li somma utilizzando la ALU e scrive il risultato della somma nel registro di pipeline EX/MEM.
4. *Accesso alla memoria dati*. La parte superiore della **Figura 4.40** mostra l'istruzione di load mentre legge la memoria dati utilizzando come indirizzo il valore letto dal registro di pipeline EX/MEM. Il dato viene quindi scritto nel registro di pipeline MEM/WB.
5. *Scrittura del risultato*. La parte inferiore della Figura 4.40 riporta la fase finale dell'esecuzione: il dato viene letto dal registro di pipeline MEM/WB e scritto nel file posto al centro della figura.

Questa carrellata sull'esecuzione dell'istruzione di load mostra che tutte le informazioni necessarie a uno stadio successivo della pipeline devono essere trasportate attraverso i registri di pipeline. La stessa descrizione dell'esecuzione di un'istruzione di store evidenzia le analogie con la `lw` nell'esecuzione e nel passaggio delle informazioni agli stadi successivi. Ecco i cinque stadi di pipeline per un'istruzione di store.

1. *Fetch dell'istruzione*. L'istruzione viene prelevata dalla memoria utilizzando l'indirizzo contenuto nel PC e viene quindi scritta nel registro di pipeline IF/ID. Questo stadio viene eseguito prima che l'istruzione possa essere identificata, per cui la parte superiore della Figura 4.38 è valida non solo per la load ma anche per la store.
2. *Decodifica dell'istruzione e lettura del register file*. L'istruzione presente nel registro di pipeline IF/ID fornisce il numero dei due registri per

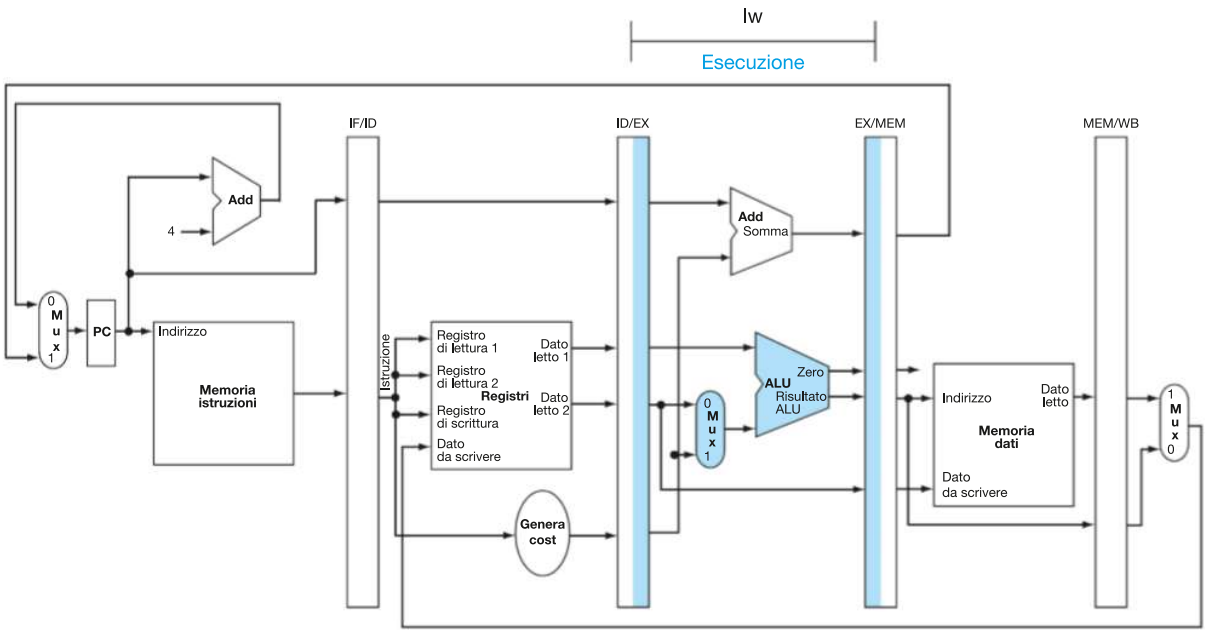


Figura 4.39 EX: il terzo stadio della pipeline durante l'esecuzione di un'istruzione di load; le parti attive dell'unità di elaborazione mostrata nella Figura 4.37 sono evidenziate in blu. Il contenuto del registro viene sommato al contenuto del campo immediato dotato di segno, esteso a 32 bit, e il risultato della somma viene scritto nel registro di pipeline EX/MEM.

leggere il campo immediato dotato di segno ed estenderlo a 32 bit. Il contenuto di questi tre campi a 32 bit, il contenuto dei due registri e il campo immediato esteso, vengono memorizzati nel registro di pipeline ID/EX, insieme al valore del PC. La parte inferiore della Figura 4.38 relativa all'istruzione di load descrive il funzionamento del secondo stadio anche per le istruzioni di store. I primi due stadi funzionano nello stesso modo per tutte le istruzioni, perché è troppo presto per capire quale sia il tipo di istruzione (l'istruzione di store utilizza il campo rs2 per leggere

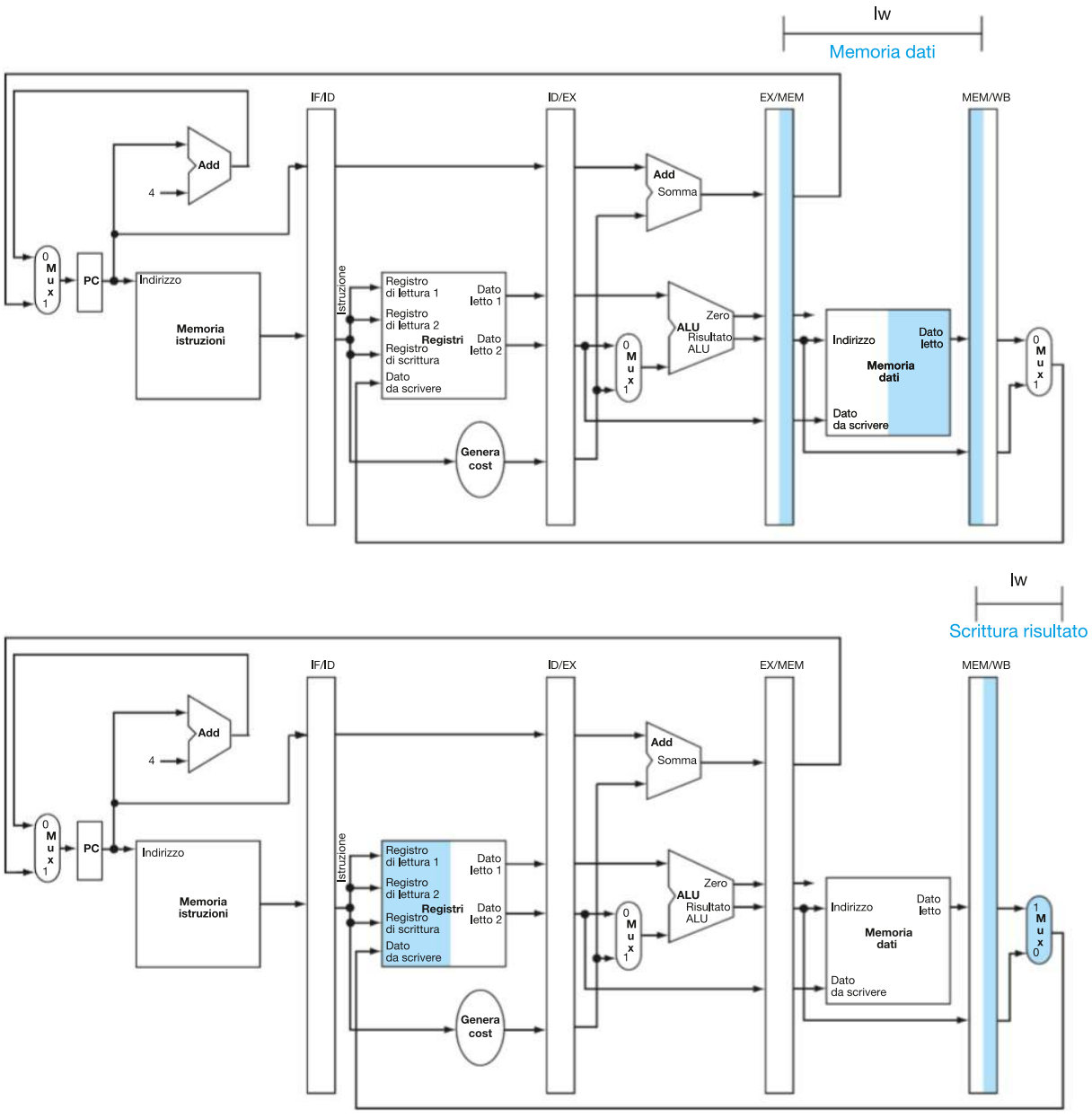


Figura 4.40 MEM e WB: il quarto e il quinto stadio della pipeline durante l'esecuzione di un'istruzione di load; le parti attive dell'unità di elaborazione mostrata nella Figura 4.37 sono evidenziate in blu. La memoria dati viene letta utilizzando l'indirizzo contenuto nel registro di pipeline EX/MEM e il dato letto viene scritto nel registro di pipeline MEM/WB. Poi, nello stadio WB, il dato viene letto dal registro di pipeline MEM/WB e scritto nel register file, al centro della figura. Si noti che c'è un baco in questo schema che verrà corretto nella Figura 4.43.

il secondo registro in questo stadio della pipeline. Questo dettaglio non è riportato in questo diagramma di pipeline, per cui possiamo utilizzare lo stesso schema per entrambi i casi).

3. *Esecuzione e calcolo dell'indirizzo*. La **Figura 4.41** mostra il terzo passo nel quale l'indirizzo della memoria viene scritto nel registro di pipeline EX/MEM.
4. *Accesso alla memoria dati*. La parte superiore della **Figura 4.42** mostra in che modo avviene la scrittura del dato in memoria. Si noti che il registro contenente il dato da scrivere era stato letto nel precedente stadio di decodifica e il suo contenuto era stato memorizzato nel registro ID/EX. L'unico modo per rendere questo dato disponibile anche nello stadio MEM è trasportarlo dal registro ID/EX al registro EX/MEM durante lo stadio EX, scrivendolo in EX/MEM esattamente come viene scritto l'indirizzo della memoria.
5. *Scrittura del risultato*. La parte inferiore della Figura 4.42 riporta la fase finale dell'esecuzione dell'istruzione: si vede che nel caso della store non succede nulla. Poiché l'esecuzione di tutte le istruzioni che seguono la store deve ancora essere completata, non c'è alcun modo per accelerarle; quindi un'istruzione deve sempre passare attraverso tutti gli stadi, anche se a volte non ci sono operazioni da compiere, in quanto le istruzioni successive stanno già procedendo alla massima velocità possibile.

Il funzionamento dell'istruzione di store sottolinea ancora una volta come, per trasportare un'informazione da uno stadio della pipeline a quello successivo, quell'informazione debba essere scritta in un registro di pipeline; se così non fosse, l'informazione verrebbe persa nel momento in cui l'istruzione successiva raggiungesse quella fase. Per l'istruzione di store abbiamo dovuto trasportare il contenuto del secondo dei due registri letti dallo stadio ID fino allo stadio MEM, dove viene scritto in memoria. Il dato proveniente dal register file

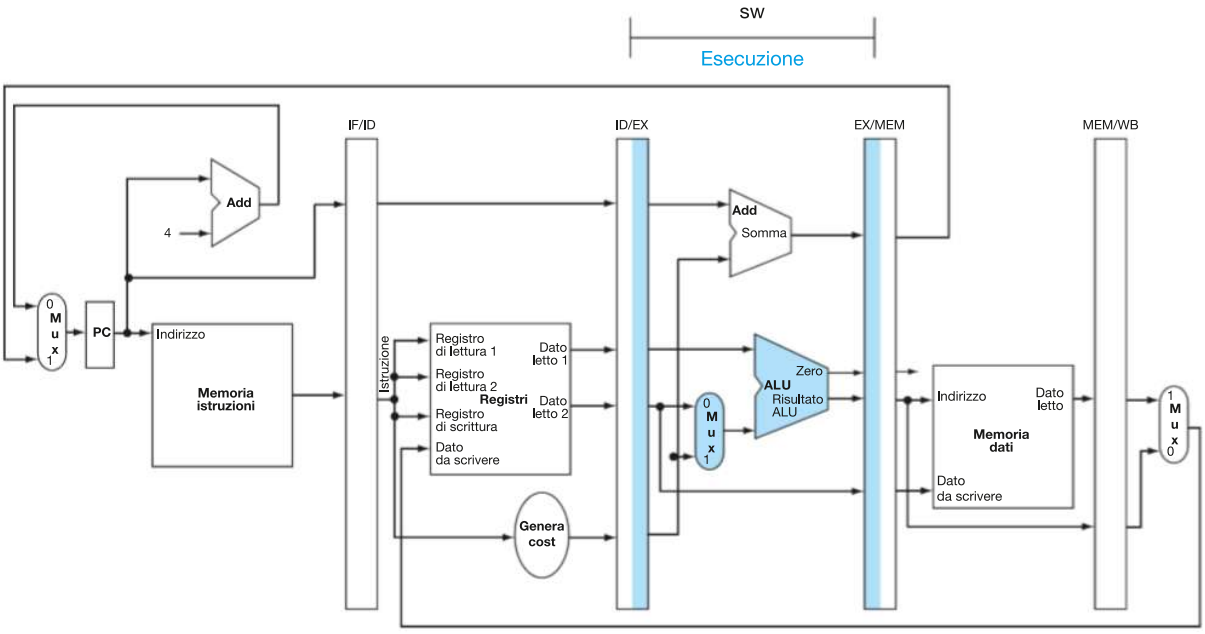


Figura 4.41 EX: il terzo stadio della pipeline durante l'esecuzione di un'istruzione di store. A differenza del terzo stadio di esecuzione dell'istruzione di load mostrato nella Figura 4.39, il contenuto del secondo registro viene scritto nel registro di pipeline EX/MEM per essere utilizzato nello stadio successivo. Sebbene non sia sbagliato scrivere sempre il contenuto del secondo registro nel registro di pipeline EX/MEM, effettueremo questa operazione solo per le istruzioni di store, in modo da rendere più facile la comprensione della pipeline.

viene prima scritto nel registro di pipeline ID/EX e poi copiato nel successivo registro EX/MEM.

Le istruzioni di load e di store permettono di fare una seconda considerazione: ciascun componente funzionale dell'unità di elaborazione, come la memoria istruzioni, le porte di lettura del register file, la ALU, la memoria dati e la porta di scrittura del register file, può essere utilizzato in *un solo* stadio della pipeline; in caso contrario, si incorrerebbe in un *hazard strutturale*

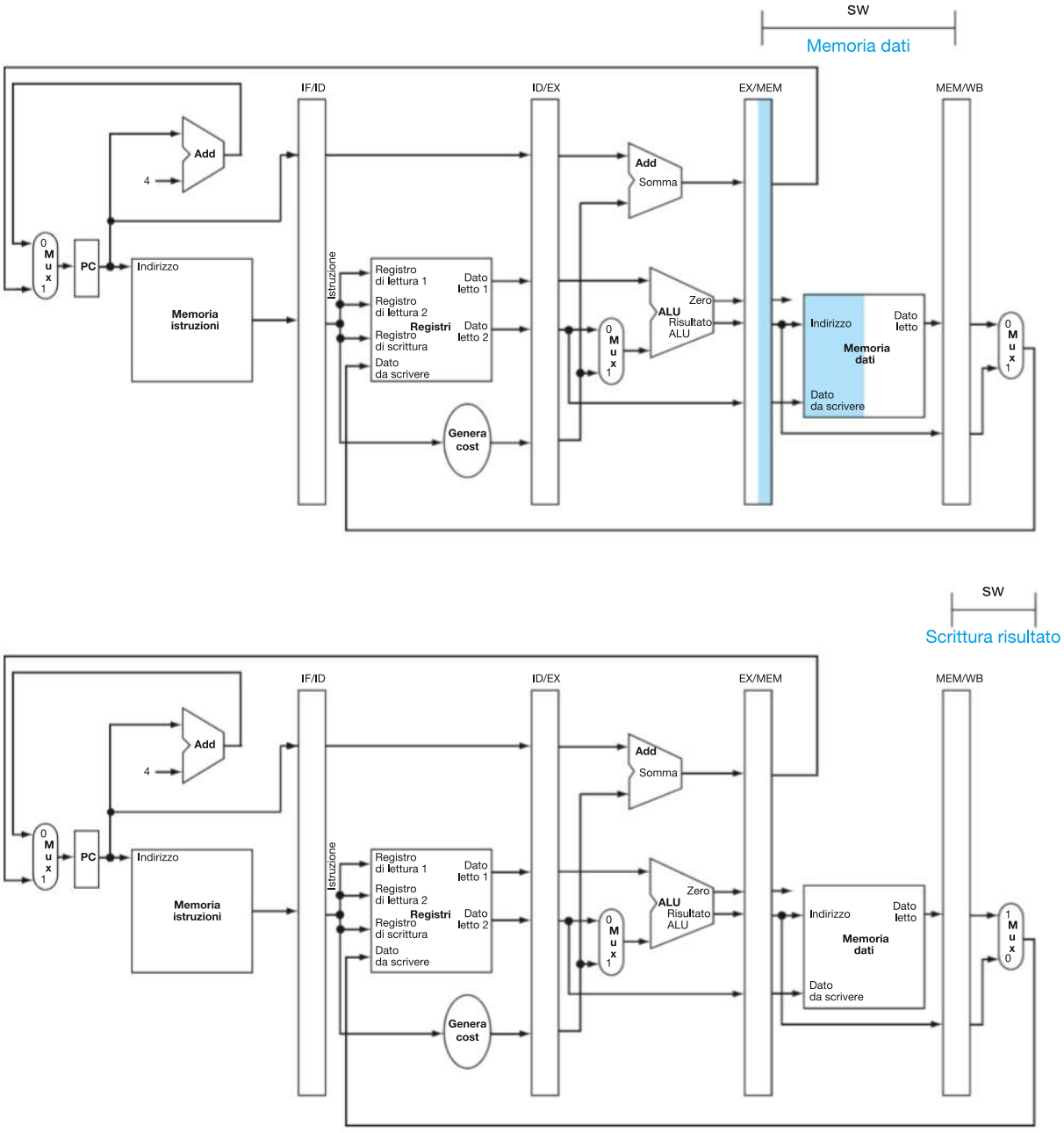


Figura 4.42 MEM e WB: il quarto e il quinto stadio della pipeline durante l'esecuzione di un'istruzione di store. Nel quarto stadio il dato viene scritto nella memoria dati; si noti che esso proviene dal registro di pipeline EX/MEM e che nulla viene modificato nel registro di pipeline MEM/WB. Una volta scritto il dato in memoria, per l'istruzione di store non rimane più nulla da fare, per cui nello stadio 5 non accade niente.

(Paragrafo 4.6). Di conseguenza, questi componenti funzionali e i loro segnali di controllo possono essere associati a un unico stadio della pipeline.

A questo punto, possiamo svelare un baco nell'implementazione hardware dell'istruzione di load che alcuni lettori avranno già notato. Quale registro viene modificato nello stadio finale della load? In particolare, quale istruzione fornisce il numero del registro da scrivere? In questo schema il numero del registro da scrivere è contenuto nel relativo campo dell'istruzione, presente nel registro di pipeline IF/ID. Tuttavia tale numero di registro si riferisce a un'istruzione che è *successiva* alla load!

Quindi, è necessario preservare il numero del registro di destinazione dell'istruzione di load: così come per l'istruzione di store avevamo trasportato il *contenuto* del registro rs2 dal registro di pipeline ID/EX al registro EX/MEM (per poterlo utilizzare nello stadio MEM), così per l'istruzione di load dobbiamo trasportare il *numero* del registro da scrivere da ID/EX, attraverso EX/MEM, fino a MEM/WB per poterlo poi utilizzare nello stadio di WB. Un modo alternativo di vedere questo passaggio del numero del registro da scrivere è pensare che affinché un'unità di elaborazione dotata di pipeline possa essere condivisa da più istruzioni, occorre preservare alcuni campi dell'istruzione prelevata durante lo stadio IF, in modo tale che ciascun registro di pipeline contenga la parte di istruzione che deve essere utilizzata da quello stadio e dagli stadi successivi.

La **Figura 4.43** mostra la versione corretta dell'unità di elaborazione: il numero del registro da scrivere viene dapprima scritto nel registro di pipeline ID/EX, poi nel registro EX/MEM e infine nel registro MEM/WB; viene infine utilizzato nello stadio WB per scrivere il dato caricato dalla memoria.

La **Figura 4.44** mostra in un unico schema l'unità di elaborazione corretta ed evidenzia i componenti funzionali utilizzati nei cinque stadi di esecuzione dell'istruzione load word, riportati nelle Figure 4.38–4.40. Il Paragrafo 4.9 mostra l'implementazione che serve a eseguire correttamente l'istruzione di salto condizionato.

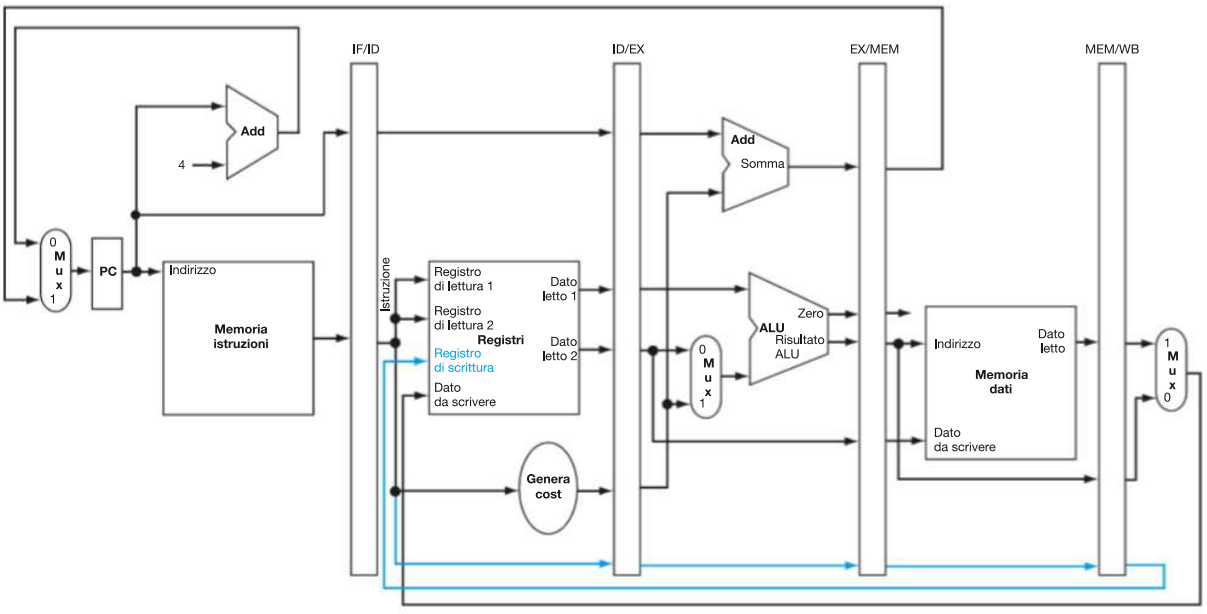


Figura 4.43 L'unità di elaborazione con pipeline modificata per elaborare correttamente l'istruzione load. Il numero del registro da scrivere ora proviene dal registro di pipeline MEM/WB, così come il dato da scrivere. Il numero del registro è stato trasportato dallo stadio ID fino al registro di pipeline MEM/WB; abbiamo quindi aggiunto cinque bit agli ultimi tre registri di pipeline. Il nuovo cammino è mostrato in blu.

 © 978-88-08-19966-9

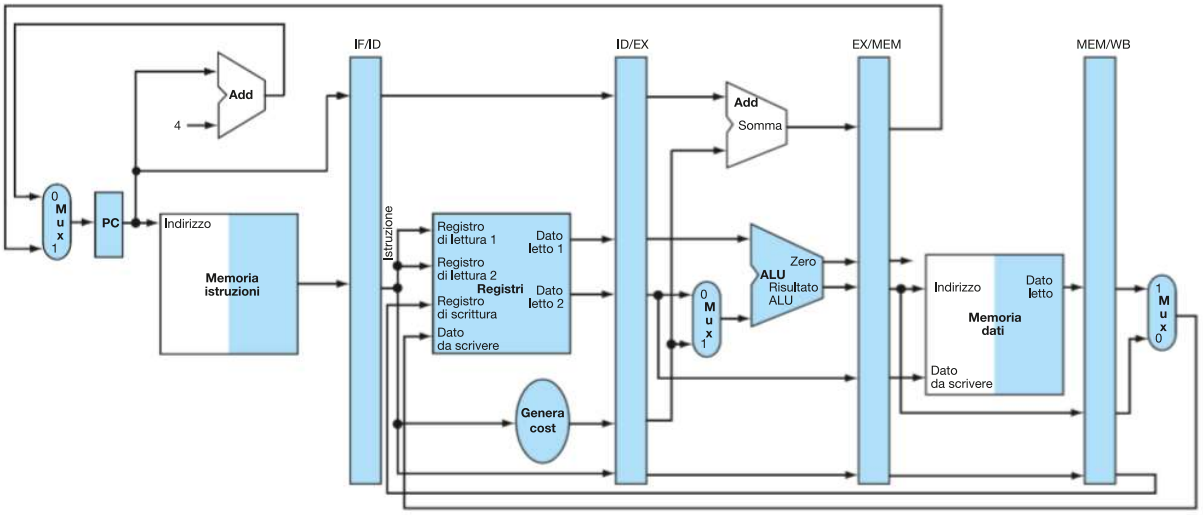


Figura 4.44 La parte di unità di elaborazione della Figura 4.43 utilizzata per l'esecuzione dei cinque stadi di un'istruzione load.

Rappresentazione grafica delle pipeline

Può essere difficile comprendere il funzionamento delle pipeline, perché in ogni ciclo di clock ci sono molte istruzioni che sono eseguite contemporaneamente nella stessa unità di elaborazione. Per facilitare la comprensione, ci sono due modi principali per disegnare i diagrammi che rappresentano le pipeline: i *diagrammi delle pipeline a più cicli di clock*, come quello della Figura 4.36, e i *diagrammi delle pipeline a singolo ciclo di clock*, come quelli mostrati nelle Figure 4.38–4.42. I diagrammi della pipeline a più cicli di clock sono più semplici ma meno dettagliati. Per esempio, consideriamo la seguente sequenza di cinque istruzioni:

```
lw    x10, 40(x1)
sub   x11, x2, x3
add   x12, x3, x4
lw    x13, 48(x1)
add   x14, x5, x6
```

La **Figura 4.45** mostra il diagramma della pipeline a più cicli di clock relativo a tali istruzioni: in questo tipo di diagrammi il tempo scorre da sinistra verso destra, mentre le istruzioni procedono dall'alto verso il basso (come nella pipeline del bucato della Figura 4.27). Per ogni istruzione, lungo l'asse verticale viene rappresentata una pipeline specificando i diversi stadi posizionati nel ciclo di clock corretto: le cinque fasi vengono rappresentate da una versione stilizzata dell'unità funzionale associata a quello stadio, ma un semplice rettangolo con il nome dello stadio andrebbe bene ugualmente. La **Figura 4.46** mostra la versione più tradizionale del diagramma di una pipeline a più cicli di clock; si noti che mentre nella Figura 4.45 si visualizzano le risorse fisiche utilizzate in ciascuno stadio, nella Figura 4.46 viene utilizzato il *nome* di ciascuno stadio.

I diagrammi delle pipeline a singolo ciclo di clock mostrano lo stato dell'intera unità di elaborazione durante un ciclo di clock e, solitamente, le cinque istruzioni caricate nella pipeline vengono identificate da etichette poste sopra lo stadio in cui si trovano. Utilizzeremo questo tipo di diagramma per mostrare i dettagli di ciò che avviene nella pipeline in ciascun periodo di clock, cosicché saranno mostrati gruppi di diagrammi per illustrare il funzionamento della pipeline su più cicli di clock consecutivi. I diagrammi a più cicli di clock

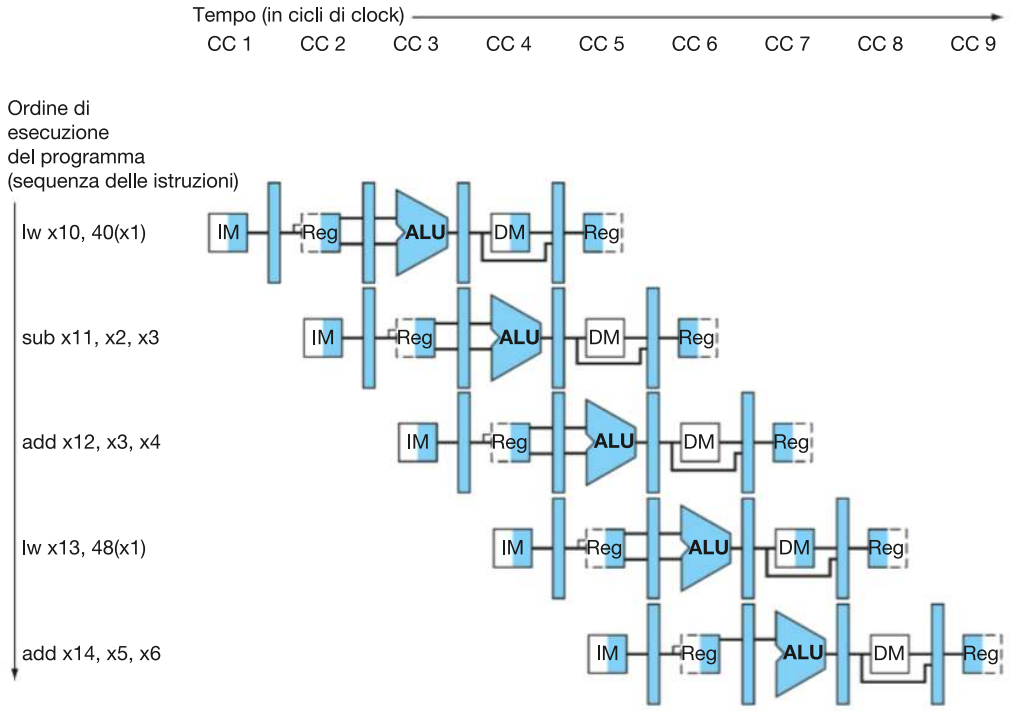


Figura 4.45 Diagramma della pipeline a più cicli di clock durante l'esecuzione di cinque istruzioni. Questo stile di rappresentazione delle pipeline mostra in una sola figura l'esecuzione completa delle istruzioni di un frammento di codice. Le istruzioni sono riportate sull'asse verticale in ordine di esecuzione (dall'alto verso il basso), mentre i cicli di clock procedono da sinistra verso destra. A differenza della Figura 4.30, qui sono indicati i registri di pipeline inseriti tra le coppie contigue di stadi della pipeline. La Figura 4.46 mostra lo schema tradizionale di questo diagramma.

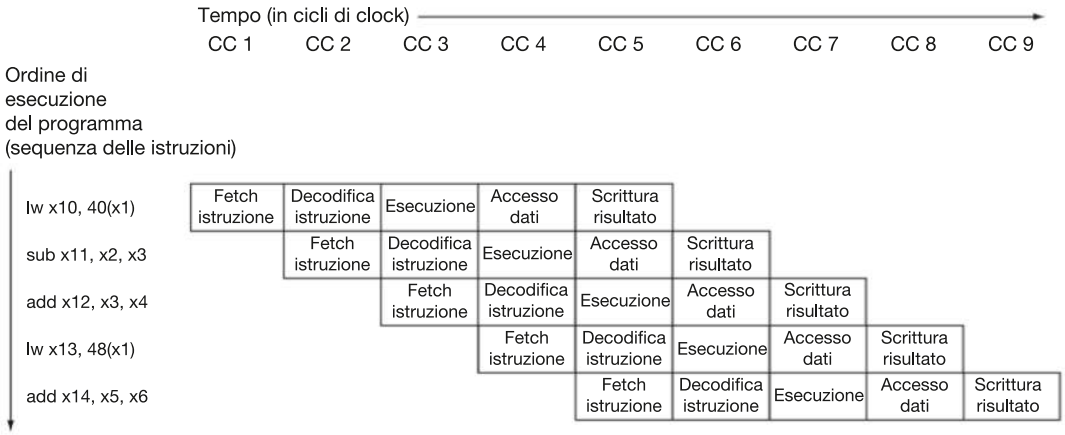


Figura 4.46 Diagramma della pipeline a più cicli di clock in forma tradizionale per le stesse istruzioni della Figura 4.45.

serviranno a fornire un quadro della situazione della pipeline durante l'esecuzione (se siete interessati a vedere ulteriori dettagli della Figura 4.45, nel Paragrafo 4.14 potete trovare altri gruppi di diagrammi a singolo ciclo). Un diagramma a singolo ciclo rappresenta una sezione verticale, in un certo ciclo di clock, di un insieme di diagrammi a più cicli di clock e mostra come viene utilizzata l'unità di elaborazione da ciascuna delle istruzioni caricate nella pipeline in quel determinato ciclo. Per esempio, la Figura 4.47 mostra

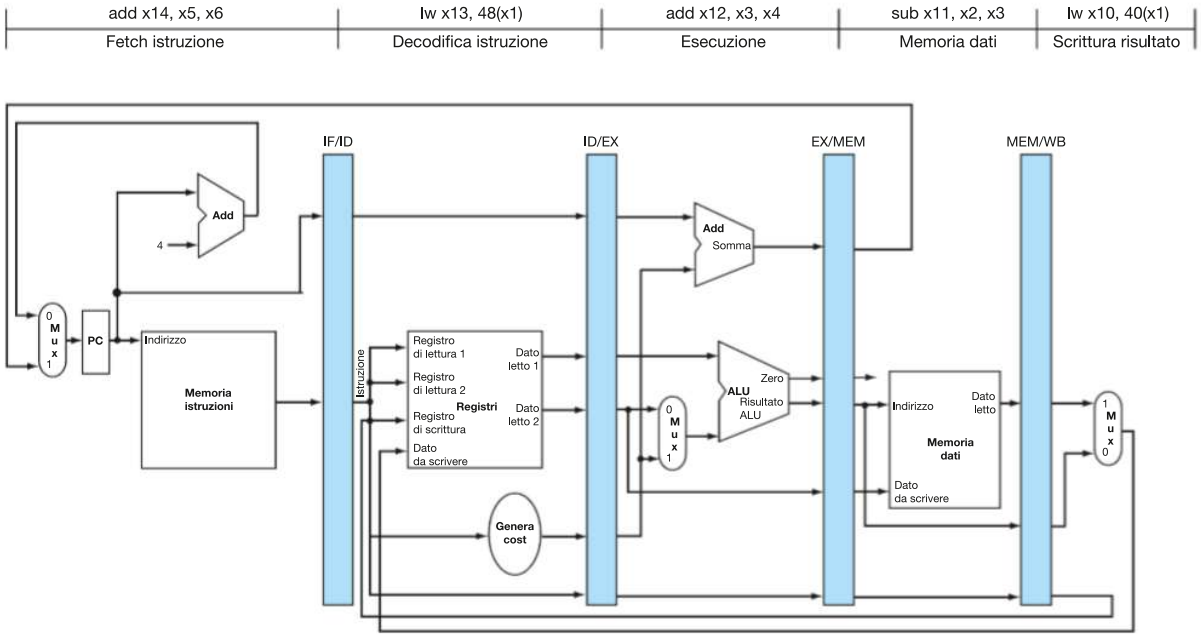


Figura 4.47 Diagramma della pipeline a singolo ciclo di clock corrispondente al quinto ciclo di clock della pipeline delle Figure 4.45 e 4.46. Come si può vedere, un diagramma a singolo ciclo di clock è una sezione verticale di un diagramma a più cicli di clock.

il diagramma a singolo ciclo di clock corrispondente al quinto ciclo di clock delle Figure 4.45 e 4.46. Ovviamente i diagrammi a singolo ciclo contengono più dettagli, ma richiedono anche uno spazio molto maggiore per mostrare lo stesso numero di cicli di clock. Negli esercizi verrà richiesto di disegnare tali diagrammi per altre sequenze di istruzioni.

Autovalutazione

Un gruppo di studenti sta discutendo l'efficienza della pipeline a cinque stadi, quando uno di loro fa notare che non tutte le istruzioni sono attive in ogni stadio della pipeline. Dopo aver deciso di ignorare gli effetti degli hazard, gli studenti fanno le seguenti affermazioni. Quali di queste sono corrette?

1. Permettere ai salti incondizionati e condizionati e alle istruzioni che operano sulla ALU di utilizzare meno stadi dei cinque richiesti dall'istruzione load aumenterà le prestazioni della pipeline in tutte le situazioni.
2. Consentire ad alcune istruzioni di utilizzare meno cicli di clock non aiuta, poiché il throughput è determinato dalla durata del ciclo di clock; il numero degli stadi della pipeline per ciascuna istruzione influenza la latenza, non il throughput.
3. Non si può ridurre il numero di cicli di clock utilizzati dalle istruzioni che operano sulla ALU, per via della scrittura del risultato nella fase di write-back; i salti condizionati e incondizionati possono però utilizzare meno cicli di clock, dunque c'è spazio per un miglioramento.
4. Invece di tentare di fare in modo che le istruzioni durino meno cicli di clock, dovremmo esplorare la possibilità di rendere la pipeline più lunga; le istruzioni, quindi, dovrebbero utilizzare più cicli di clock, ma ciascun ciclo avrebbe una durata inferiore, e ciò produrrebbe un miglioramento delle prestazioni.

Unità di controllo della pipeline

Nel Calcolatore 6600, forse più che in ogni altro calcolatore precedente, era l'unità di controllo a fare la differenza.

James Thornton, *Design of a Computer: The Control Data 6600,* 1970

Aggiungeremo ora l'unità di controllo dell'unità di elaborazione con pipeline analogamente a quanto avevamo fatto nel Paragrafo 4.4 per l'unità di elaborazione a singolo ciclo. Inizieremo con una versione semplificata, in cui non si considerano gli elementi più complessi dell'architettura.

Il primo passo consiste nell'etichettare i segnali di controllo dell'unità di elaborazione esistente, come mostrato nella **Figura 4.48**. Cercheremo di riutilizzare il più possibile l'unità di controllo della CPU a singolo ciclo della Figura 4.21; in particolare, utilizzeremo la stessa unità di controllo della ALU, la stessa logica di controllo dei salti e le stesse linee di controllo. Questi componenti sono descritti nelle Figure 4.12, 4.20 e 4.22 e gli elementi chiave sono riprodotti nelle **Figure 4.49–4.51**.

Come nell'implementazione a singolo ciclo, supporremo che il registro PC venga scritto a ogni ciclo di clock, per cui non ci sarà nessun segnale di controllo esplicito per questa operazione. Per lo stesso motivo non ci saranno segnali di scrittura espliciti per i registri di pipeline (IF/ID, ID/EX, EX/MEM e MEM/WB), dato che anch'essi vengono scritti a ogni ciclo di clock.

Per definire l'unità di controllo della pipeline è sufficiente impostare il valore corretto dei segnali di controllo in ciascuno stadio. Dato che ogni segnale è associato a un componente e che ciascun componente è attivo in uno solo degli stadi della pipeline, si possono suddividere i segnali di controllo in cinque gruppi in base alla fase in cui i segnali sono attivi: vediamoli.

1. *Fetch dell'istruzione*. I segnali di controllo per leggere la memoria istruzioni e per scrivere il PC sono sempre asseriti, quindi non ci sono unità da controllare in questo stadio.
2. *Decodifica dell'istruzione/lettura del register file*. Come nello stadio precedente, a ogni ciclo di clock si verificano sempre le stesse azioni, quindi non ci sono segnali di controllo espliciti associati a questo stadio.

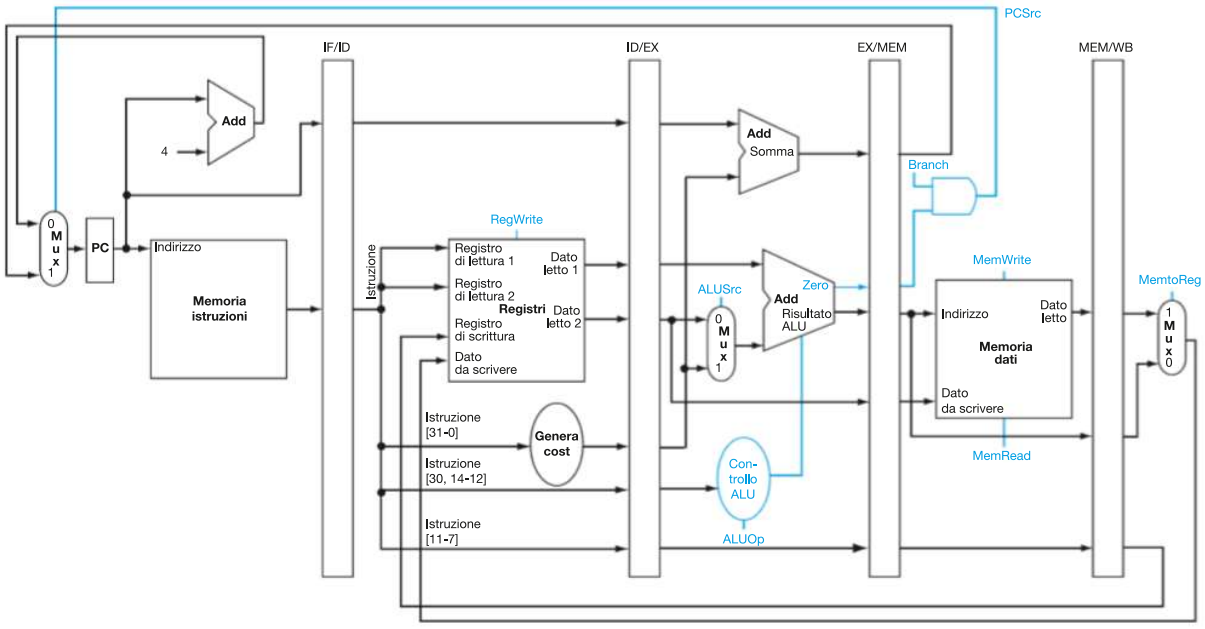


Figura 4.48 L'unità di elaborazione con pipeline della Figura 4.43 completa dei segnali di controllo. L'unità di elaborazione adotta la logica di controllo descritta nel Paragrafo 4.4 per stabilire l'input al PC, il numero del registro destinazione e i segnali di controllo della ALU. Si noti che ora servono i campi funz dell'istruzione nello stadio EX come input del controllore della ALU; occorre quindi salvare anche questi bit nel registro di pipeline ID/EX.

3. *Esecuzione/calcolo dell'indirizzo*. I segnali da impostare sono ALUOp e ALUSrc (Figure 4.49 e 4.50); essi selezionano il registro sul quale scrivere il risultato, l'operazione che deve essere eseguita dalla ALU e scelgono se inviare alla ALU "Dato letto 2" oppure il campo immediato con segno, esteso a 32 bit.
4. *Accesso alla memoria*. Le linee di controllo da impostare per questo stadio sono Branch, MemRead e MemWrite. Questi segnali devono essere asseriti rispettivamente per le istruzioni branch if equal, load e store. Si ricordi che PCSrc (Figura 4.50) seleziona di default l'indirizzo dell'istruzione seguente, a meno che il segnale di Branch non sia asserito e il risultato della ALU valga 0.
5. *Scrittura del risultato*. Le due linee di controllo sono MemtoReg, che sceglie se inviare al register file l'uscita della ALU oppure il valore proveniente dalla memoria, e RegWrite, che comanda la scrittura nel register file del dato selezionato.

Dato che la versione con pipeline dell'unità di elaborazione non modifica il significato dei segnali di controllo, si possono utilizzare gli stessi nomi adottati in precedenza. La Figura 4.51 riporta gli stessi segnali di controllo introdotti

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
lw	00	load di 1 parola	XXXXXXX	XXX	somma	0010
sw	00	store di 1 parola	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001

Figura 4.49 Copia della Figura 4.12. La figura mostra come determinare il valore dei bit di controllo della ALU a partire dai bit di controllo di ALUOp e, per le istruzioni di tipo R, a partire anche dal contenuto del campo funzione.

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nullo	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di PC + 4	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nullo	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nullo	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

Figura 4.50 Copia della Figura 4.20. Viene riportato il funzionamento dei sette segnali di controllo. Quelli della ALU (ALUOp) sono stati definiti nella seconda colonna della Figura 4.49. Quando il segnale di controllo a 1 bit di un multiplexer a due vie viene asserito, il multiplexer seleziona l'ingresso etichettato con 1; in caso contrario, cioè se il segnale di controllo è non asserito, il multiplexer seleziona l'ingresso 0. Si noti che nella Figura 4.48 PCSrc viene controllato dall'uscita di una porta AND. Se il segnale di Branch e il segnale di Zero in uscita dalla ALU assumono entrambi il valore 1, PCSrc viene impostato a 1, altrimenti viene impostato a 0. La logica di controllo imposta il segnale di Branch a 1 solamente in corrispondenza delle istruzioni `beq`; in tutti gli altri casi, quindi, PCSrc viene impostato a 0.

Istruzione	Segnali di controllo dello stadio di esecuzione/calcolo dell'indirizzo		Segnali di controllo dello stadio di accesso alla memoria dati			Segnali di controllo dello stadio di scrittura	
	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
Tipo R	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Figura 4.51 I sette segnali di controllo per gli ultimi tre stadi della pipeline. Si noti che due dei sette segnali di controllo vengono utilizzati nella fase EX, mentre i restanti cinque vengono passati al registro di pipeline EX/MEM, esteso per memorizzare questi segnali. Tre segnali di controllo vengono utilizzati nella fase di MEM, mentre gli ultimi due vengono passati al registro MEM/WB per essere utilizzati nella fase di WB.

nel Paragrafo 4.4, con la differenza che ora le sette linee di controllo sono raggruppate secondo lo stadio della pipeline in cui sono attive.

Implementare l'unità di controllo significa impostare correttamente il valore dei sette segnali di controllo per ciascuno stadio della pipeline e per ciascuna istruzione. Dato che i segnali di controllo espliciti vengono utilizzati a partire dallo stadio EX, essi possono essere determinati durante la decodifica dell'istruzione per poi venire utilizzati negli stadi successivi. Il modo più semplice per propagare questi segnali di controllo è estendere i registri di pipeline per includere l'informazione sul controllo. La **Figura 4.52** mostra che questi segnali di controllo vengono poi utilizzati nello stadio appropriato della pipeline man mano che l'istruzione scorre lungo la pipeline stessa, analogamente a quanto accade per il registro destinazione della load (Figura 4.43). La **Figura 4.53** mostra l'unità di elaborazione completa dei registri di pipeline estesi e dei segnali di controllo associati allo stadio opportuno. Per i lettori interessati ad approfondire l'argomento, nel Paragrafo 4.14 sono riportati altri diagrammi a singolo ciclo che illustrano l'esecuzione di frammenti di programmi RISC-V su hardware dotato di pipeline.

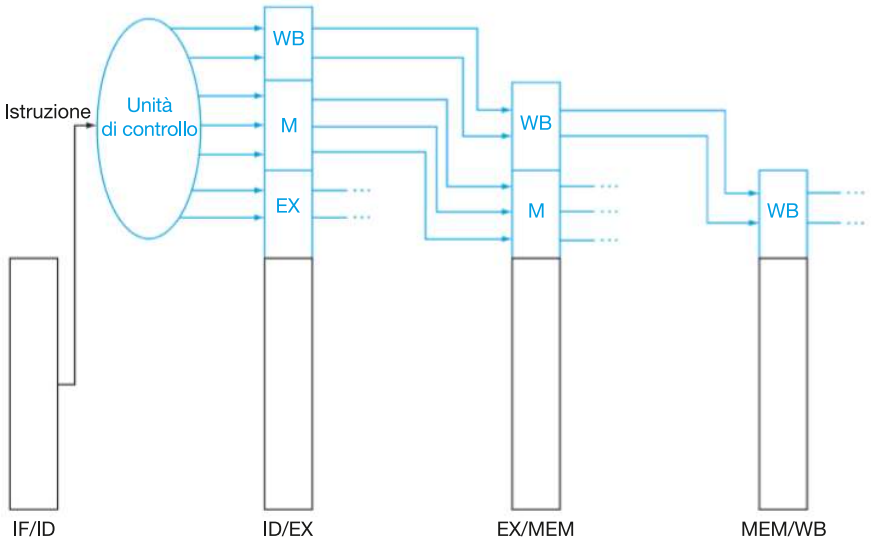


Figura 4.52 I sette segnali di controllo per gli ultimi tre stadi. Si noti che due dei sette segnali di controllo sono utilizzati nello stadio EX, mentre i rimanenti cinque vengono trasportati al registro di pipeline EX/MEM, esteso in modo da poterli contenere. Di questi cinque segnali di controllo, tre vengono impiegati nello stadio MEM, mentre gli altri due vengono trasportati al registro MEM/WB per essere poi utilizzati nello stadio WB.

 © 978-88-08-19966-9

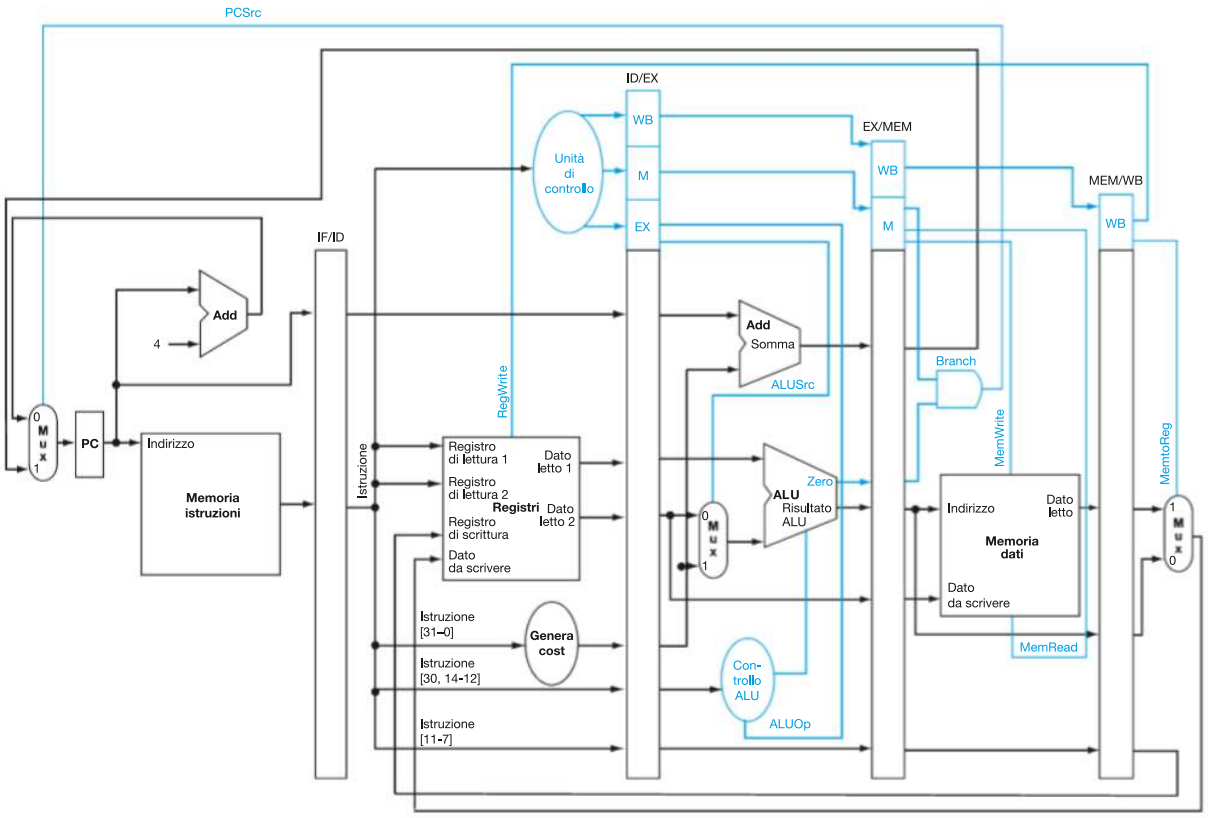


Figura 4.53 L'unità di elaborazione con pipeline della Figura 4.48 completa dei segnali di controllo uscenti dalla porzione dei registri di pipeline dedicata a memorizzare le informazioni sul controllo. I valori dei segnali di controllo degli ultimi tre stadi sono generati durante lo stadio di decodifica dell'istruzione e vengono salvati nel registro di pipeline ID/EX. In ciascuno degli stadi successivi sono impiegati alcuni di questi segnali di controllo, mentre i rimanenti vengono inviati allo stadio successivo della pipeline.

4.8 Hazard sui dati: propagazione o stallo

Cosa vuoi dire, perché dovrebbe essere costruito? È una scorciatoia. Tu devi costruire scorciatoie.

Douglas Adams, *The Hitchhiker's Guide to the Galaxy*, 1979

Gli esempi visti nel paragrafo precedente mostrano le potenzialità dell'esecuzione in pipeline e illustrano come la pipeline possa essere implementata in hardware. A questo punto, però, è necessario abbandonare la situazione ideale e rendersi conto di quello che può accadere durante l'esecuzione di un programma reale. Le istruzioni RISC-V nelle Figure 4.45–4.47 erano indipendenti: nessuna di loro utilizzava i risultati prodotti dalle istruzioni precedenti. Tuttavia abbiamo già visto nel Paragrafo 4.6 che gli hazard sui dati costituiscono un ostacolo all'esecuzione in pipeline.

Esaminiamo una sequenza di istruzioni contenenti molte dipendenze, evidenziate in blu:

```
sub x2,  x1, x3  // sub scrive nel registro x2
and x12, x2, x5  // Il primo operando (x2) dipende da sub
or  x13, x6, x2  // Il secondo operando (x2) dipende da sub
add x14, x2, x2  // Il primo (x2) e il secondo (x2)
                 // operando dipendono da sub
sw  x15, 100(x2) // Il registro base (x2) dipende da sub
```

Le ultime quattro istruzioni dipendono tutte dal risultato della prima istruzione, scritto nel registro x2. Supponiamo che il registro x2 contenga il valore 10 prima dell'istruzione di sottrazione e –20 dopo: il programmatore vorrebbe

che fosse utilizzato il valore –20 in tutte le istruzioni che seguono la sub e che fanno riferimento al registro x2.

Ma come si comporterebbe la sequenza di istruzioni se venisse eseguita sulla pipeline vista in precedenza? La **Figura 4.54** mostra l'esecuzione di queste istruzioni utilizzando un diagramma a più cicli di clock. Per illustrare ciò che accade all'interno della pipeline, la parte in alto della Figura 4.54 riporta il contenuto del registro x2, che cambia solo a metà del quinto ciclo di clock, quando l'istruzione sub scrive il risultato della sottrazione nel register file.

L'hazard potenziale sulla add può essere risolto progettando opportunamente l'hardware che implementa il register file: che cosa succede quando un registro viene letto e scritto nello stesso ciclo di clock? Si può assumere che la scrittura avvenga nella prima metà del ciclo di clock e la lettura nella seconda metà: in questo modo si può leggere nello stesso ciclo di clock il valore che viene scritto. Questa situazione è tipica di molte implementazioni hardware del register file, e in questo caso non si hanno hazard sui dati. La Figura 4.54 mostra che il dato letto dal registro x2 *non* può essere quello prodotto dall'istruzione sub, a meno che l'operazione di lettura non avvenga durante il quinto ciclo di clock o nei cicli successivi, per cui le sole istruzioni che leggerebbero il valore corretto (–20) sarebbero add e sw; and e or leggerebbero invece il valore 10, che è sbagliato! Osservando il diagramma, possiamo affermare che questo tipo di problema si verifica quando c'è una linea di dipendenza che va a ritroso nel tempo.

Come si è detto nel Paragrafo 4.6, il risultato dell'istruzione sub è già disponibile al termine dello stadio EX, cioè nel ciclo di clock 3. Ma quando le istruzioni and e or hanno realmente bisogno di questo dato? All'inizio dello stadio EX, cioè nei cicli di clock 4 e 5, rispettivamente. Di conseguenza, è possibile

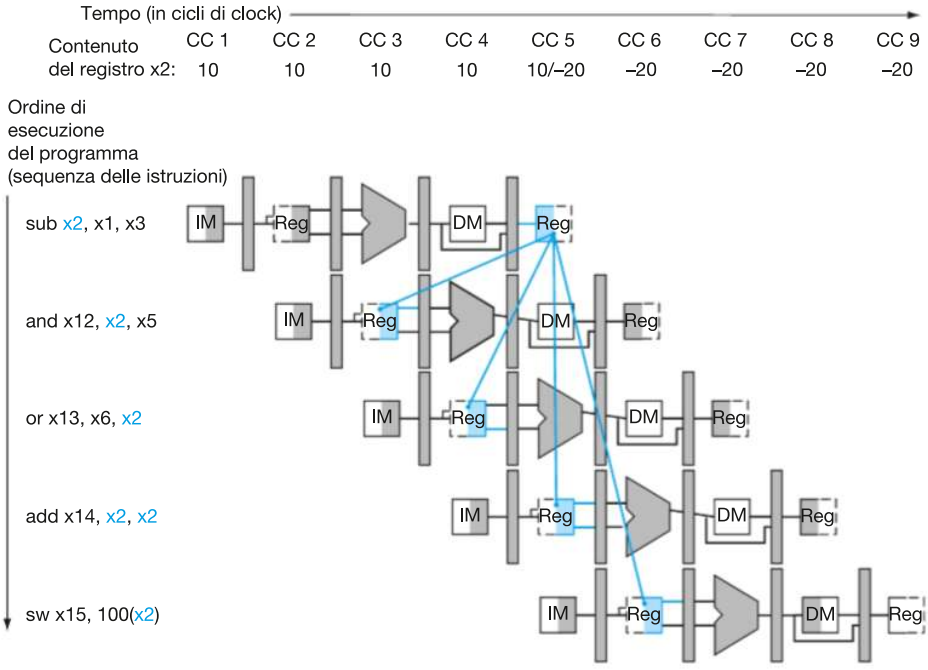


Figura 4.54 Dipendenze in una pipeline che contiene una sequenza di cinque istruzioni: viene utilizzata l'unità di elaborazione semplificata vista finora e le dipendenze sono evidenziate. Tutte le funzioni dipendenti sono disegnate in blu; nella parte alta "CC 1" indica il primo ciclo di clock. La prima istruzione scrive il risultato in x2 e tutte le istruzioni successive devono leggerne il contenuto. L'operazione di scrittura di tale registro avviene solo nel quinto ciclo di clock, quindi il valore corretto non è disponibile prima di questo ciclo. Si ricordi che la lettura di un registro in un certo ciclo di clock restituisce il valore scritto nel registro nella prima metà del ciclo di clock stesso. Le linee blu che partono dalla prima unità di elaborazione e sono dirette verso le unità di elaborazione disegnate sotto mostrano le dipendenze: le linee che vanno a ritroso nel tempo costituiscono degli *hazard sui dati*.

 © 978-88-08-19966-9

eseguire questo frammento di codice senza stalli semplicemente *propagando* il dato a qualsiasi unità che lo richieda, non appena è disponibile, ancor prima che esso sia pronto per essere letto dal register file.

Come funziona la propagazione? Per semplicità, nel seguito di questo paragrafo considereremo solo la propagazione di un dato per un'operazione che deve essere eseguita nello stadio EX, la quale può essere una generica operazione con la ALU oppure il calcolo di un indirizzo. Questo significa che quando un'istruzione nel suo stadio EX deve utilizzare un dato di un registro, che però deve ancora essere scritto da un'istruzione precedente nella sua fase di WB, occorre che il dato venga portato all'ingresso della ALU.

Una notazione che assegna dei nomi ai campi dei registri di pipeline permette di formalizzare le dipendenze con maggior precisione. Per esempio, "ID/EX.RegistroRs1" si riferisce al numero di uno dei registri che si possono trovare nel registro di pipeline ID/EX, cioè il registro associato alla prima porta di lettura del register file. La prima parte del nome, a sinistra del punto, è il nome del registro di pipeline, mentre la seconda parte corrisponde al nome di un campo all'interno del registro. Utilizzando questa notazione, le due coppie di condizioni che generano un hazard sui dati si possono esprimere come segue:

1a. EX/MEM.RegistroRd = ID/EX.RegistroRs1
1b. EX/MEM.RegistroRd = ID/EX.RegistroRs2
2a. MEM/WB.RegistroRd = ID/EX.RegistroRs1
2b. MEM/WB.RegistroRd = ID/EX.RegistroRs2

Il primo hazard nella sequenza di istruzioni dell'esempio che stiamo considerando è sul registro `x2` e si verifica tra la scrittura del risultato dell'istruzione `sub x2, x1, x3` e l'operazione di lettura del primo operando dell'istruzione `and x12, x2, x5`. Tale hazard può essere rilevato quando l'istruzione `and` si trova nello stadio EX e l'istruzione precedente si trova nello stadio MEM; si tratta quindi di un hazard di tipo 1a:

EX/MEM.RegistroRd = ID/EX.RegistroRs1 = x2

Rilevamento delle dipendenze

ESEMPIO

Classificare le dipendenze all'interno della seguente sequenza di istruzioni (identica a quella riportata all'inizio del paragrafo):

```
sub  x2, x1, x3   // sub scrive nel registro x2
and  x12, x2, x5  // Il primo operando (x2)
                  // dipende da sub
or   x13, x6, x2  // Il secondo operando (x2)
                  // dipende da sub
add  x14, x2, x2  // Il primo (x2) e il secondo (x2)
                  // operando dipendono da sub
sd   x15, 100(x2) // Il registro base (x2) dipende da sub
```

SOLUZIONE

Come già detto, l'hazard tra `sub` e `and` è di tipo 1a. Gli altri hazard sono classificabili nel seguente modo:

- tra `sub` e `or` c'è un hazard di tipo 2b:

MEM/WB.RegistroRd = ID/EX.RegistroRs2 = x2

(continua)

- le due dipendenze tra `sub` e `add` non costituiscono hazard, dal momento che il register file fornisce i dati corretti già durante lo stadio ID dell'istruzione `add`;
- non c'è hazard sui dati tra `sub` e `sw`, poiché `sw` legge `x2` nel ciclo di clock *successivo* a quello in cui `sub` scrive in `x2`.

(continua)

Dato che non tutte le istruzioni scrivono il risultato nel register file, questa strategia non è precisa: in alcuni casi si propagherebbero dei dati anche quando non si deve. Una possibile soluzione consiste nel verificare se il segnale RegWrite è attivo; a tal fine, si può controllare se sia asserito il segnale RegWrite contenuto nella porzione dei registri di pipeline EX/MEM e MEM/WB riservata ai segnali di controllo attivi nello stadio di WB. Ricordiamo, inoltre, che il RISC-V richiede che il registro `x0` contenga sempre il valore 0 quando viene utilizzato come operando. Nel caso in cui un'istruzione nella pipeline abbia `x0` come registro destinazione (per es. `addi x0, x1, 2`), si cerca di evitare che il risultato dell'operazione, eventualmente non nullo, sia propagato; in questo modo si evita che il programmatore assembler e il compilatore non possano considerare il registro `x0` come registro destinazione. Le condizioni riportate sopra funzionano quindi correttamente se si aggiunge la condizione EX/MEM.RegistroRd ≠ 0 alla prima condizione di hazard e MEM/WB.RegistroRd ≠ 0 alla seconda.

Ora che siamo in grado di rilevare gli hazard abbiamo risolto metà del problema, ma dobbiamo ancora propagare i dati corretti.

La **Figura 4.55** mostra le dipendenze tra i registri di pipeline e gli ingressi della ALU per la stessa sequenza di istruzioni della Figura 4.54. La differenza

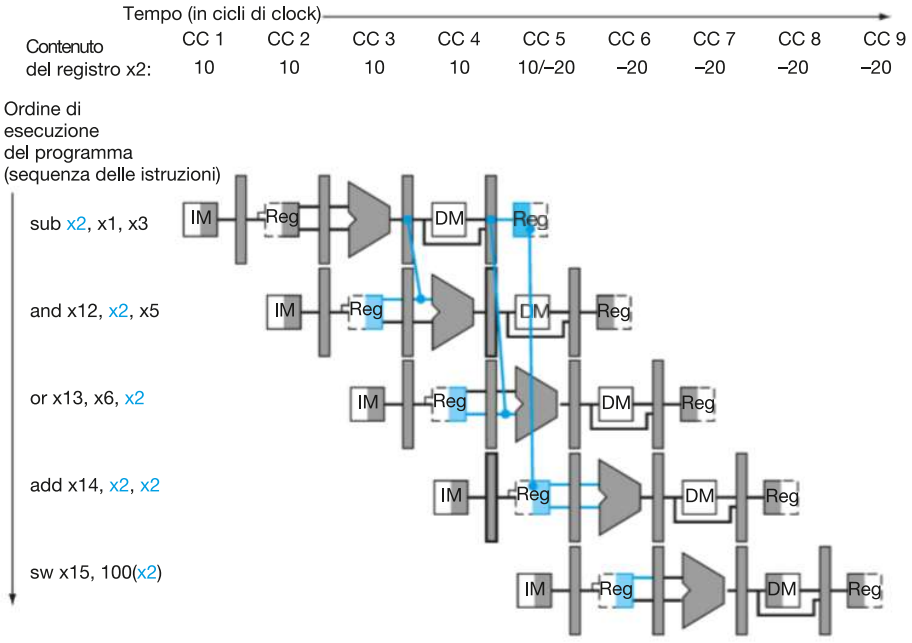
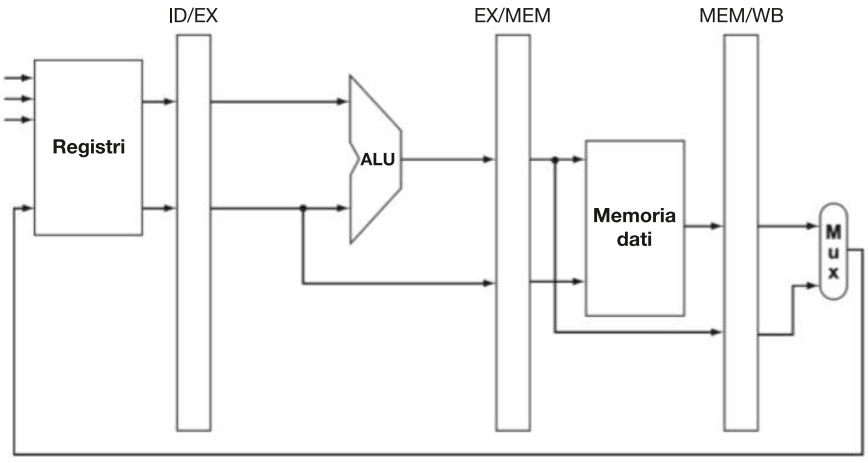


Figura 4.55 Le dipendenze tra i registri di pipeline si muovono in avanti nel tempo; è così possibile fornire alla ALU gli ingressi corretti per l'esecuzione dell'istruzione `and` e dell'istruzione `or`, propagando il risultato della sub presente nei registri di pipeline. Il contenuto dei registri di pipeline mostra che il dato desiderato è disponibile prima che esso venga scritto nel register file. Si suppone che il register file propaghi i valori scritti al suo interno nello stesso ciclo di clock, quindi la `add` non deve essere messa in stallo perché riceve regolarmente i dati dal register file e non serve la propagazione dai registri di pipeline. La propagazione all'interno del register file avviene perché è possibile leggere il dato che viene scritto nello stesso ciclo di clock; questo è il motivo per cui nel quinto ciclo di clock il registro `x2` contiene il valore 10 all'inizio e –20 alla fine.

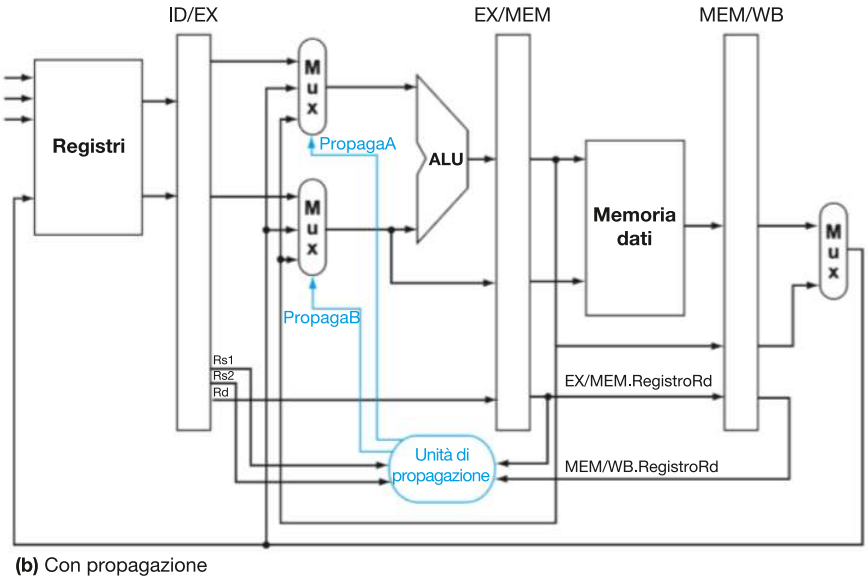
consiste nel fatto che le linee di dipendenza partono dai *registri di pipeline*, anziché dallo stadio WB nel quale viene scritto il register file. Queste linee di dipendenza evidenziano che i dati necessari, che devono essere propagati, sono contenuti nei registri di pipeline e sono quindi disponibili in tempo utile per le istruzioni successive.

Se possiamo prendere gli input della ALU non solo dal registro ID/EX ma anche dagli *altri* registri di pipeline, allora possiamo propagare i dati corretti. Aggiungendo dei multiplexer sugli ingressi della ALU e utilizzando dei segnali di controllo adeguati, è possibile far funzionare la pipeline alla velocità massima anche in presenza delle dipendenze tra i dati che abbiamo evidenziato.

Faremo inizialmente l'assunzione che la propagazione sia necessaria solamente per le quattro istruzioni del formato R: `add`, `sub`, `and` e `or`. La **Figura 4.56**



(a) Nessuna propagazione



(b) Con propagazione

Figura 4.56 Nella parte superiore della figura sono mostrate la ALU e i registri di pipeline prima di aggiungere l'hardware che esegue la propagazione. Nella parte inferiore della figura sono stati inseriti dei multiplexer per la selezione dei cammini di propagazione e viene mostrata l'unità di propagazione. L'hardware aggiunto è evidenziato in blu. La figura è semplificata, in quanto non contiene alcuni dettagli dell'unità di elaborazione completa, come il circuito per l'estensione del segno.

Controllo multiplexer	Sorgente	Spiegazione
PropagaA = 00	ID/EX	Il primo operando della ALU proviene dal register file
PropagaA = 10	EX/MEM	Il primo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente
PropagaA = 01	MEM/WB	Il primo operando della ALU viene propagato dalla memoria dati o da un precedente risultato della ALU
PropagaB = 00	ID/EX	Il secondo operando della ALU proviene dal register file
PropagaB = 10	EX/MEM	Il secondo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente
PropagaB = 01	MEM/WB	Il secondo operando della ALU è propagato dalla memoria dati o da un precedente risultato della ALU

Figura 4.57 Il valore assunto dai segnali di controllo dei multiplexer di propagazione della Figura 4.56. Il campo immediato dotato di segno che costituisce un ulteriore ingresso della ALU viene descritto nella sezione *Approfondimento* al termine di questo paragrafo.

mostra un dettaglio della ALU e dei registri di pipeline prima e dopo l'aggiunta della propagazione, mentre la Figura 4.57 mostra come le linee di controllo dei multiplexer della ALU selezionino il dato sorgente, prendendolo dal register file oppure da uno dei dati propagati.

Dato che i multiplexer di propagazione all'ingresso della ALU si trovano nello stadio EX, il controllo della propagazione avviene in questo stadio. Di conseguenza, per determinare se propagare o meno i dati è necessario trasportare il numero dei due registri utilizzati come operandi dallo stadio ID al registro di pipeline ID/EX. Prima di considerare la propagazione, non c'era alcun motivo di trasportare anche il numero dei registri `rs1` e `rs2` scrivendoli nel registro ID/EX, mentre ora è diventato necessario per poter verificare se c'è un hazard sui dati: occorre quindi aggiungere questi campi al registro ID/EX.

Si possono ora scrivere sia le condizioni per rilevare gli hazard sia i segnali di controllo per risolverle:

1. *Hazard nello stadio EX*:

```
if  (EX/MEM.RegWrite
and (EX/MEM.RegistroRd ≠ 0)
and (EX/MEM.RegistroRd = ID/EX.RegistroRs1)) PropagaA = 10

if  (EX/MEM.RegWrite
and (EX/MEM.RegistroRd ≠ 0)
and (EX/MEM.RegistroRd = ID/EX.RegistroRs2)) PropagaB = 10
```

   In questo schema di risoluzione degli hazard il risultato che arriva dalla precedente istruzione viene propagato a uno dei due ingressi della ALU; se l'istruzione precedente deve scrivere sul register file e il numero del registro di scrittura coincide con quello del registro da cui viene letto il dato A o B in ingresso alla ALU (purché questo non sia il registro 0), allora il multiplexer viene pilotato in maniera da prelevare il dato dal registro di pipeline EX/MEM.

2. *Hazard nello stadio MEM*:

```
if  (MEM/WB.RegWrite
and (MEM/WB.RegistroRd ≠ 0)
and (MEM/WB.RegistroRd = ID/EX.RegistroRs1)) PropagaA = 01

if  (MEM/WB.RegWrite
and (MEM/WB.RegistroRd ≠ 0)
and (MEM/WB.RegistroRd = ID/EX.RegistroRs2)) PropagaB = 01
```

Come già accennato in precedenza, non si può verificare un hazard nello stadio WB, poiché si suppone che il register file fornisca in uscita il dato corretto anche quando l'istruzione nello stadio ID deve leggere il contenuto dello stesso

registro che viene scritto in quel ciclo di clock dall'istruzione che si trova nello stadio WB. Un register file di questo tipo implementa una particolare forma di propagazione che si verifica al suo interno.

Una possibile fonte di problemi è costituita dagli hazard sui dati che si possono verificare tra il risultato di un'istruzione nello stadio WB, il risultato di un'istruzione nello stadio MEM e l'operando sorgente di un'istruzione nello stadio ALU. Per esempio, se si devono sommare tra loro gli elementi di un vettore e scrivere il risultato in un particolare registro, si ottiene una sequenza di istruzioni che leggono e scrivono nello stesso registro:

```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
. . .
```

In questo caso il risultato viene propagato dallo stadio MEM, dal momento che quel risultato è il più recente. Quindi il test da eseguire per rilevare un hazard nello stadio MEM sarà il seguente (le aggiunte sono evidenziate in blu):

```
if   (MEM/WB.RegWrite
and  (MEM/WB.RegistroRd ≠ 0)
and  not(EX/MEM.RegWrite and (EX/MEM.RegistroRd ≠ 0)
        and (EX/MEM.RegistroRd = ID/EX.RegistroRs1))
and  (MEM/WB.RegistroRd = ID/EX.RegistroRs1)) PropagaA = 01

if   (MEM/WB.RegWrite
and  (MEM/WB.RegistroRd ≠ 0)
and  not(EX/MEM.RegWrite and (EX/MEM.RegistroRd ≠ 0)
        and (EX/MEM.RegistroRd = ID/EX.RegistroRs2))
and  (MEM/WB.RegistroRd = ID/EX.RegistroRs2)) PropagaB = 01
```

La **Figura 4.58** mostra l'hardware necessario a implementare la propagazione per le istruzioni che utilizzano nello stadio EX il risultato di istruzioni

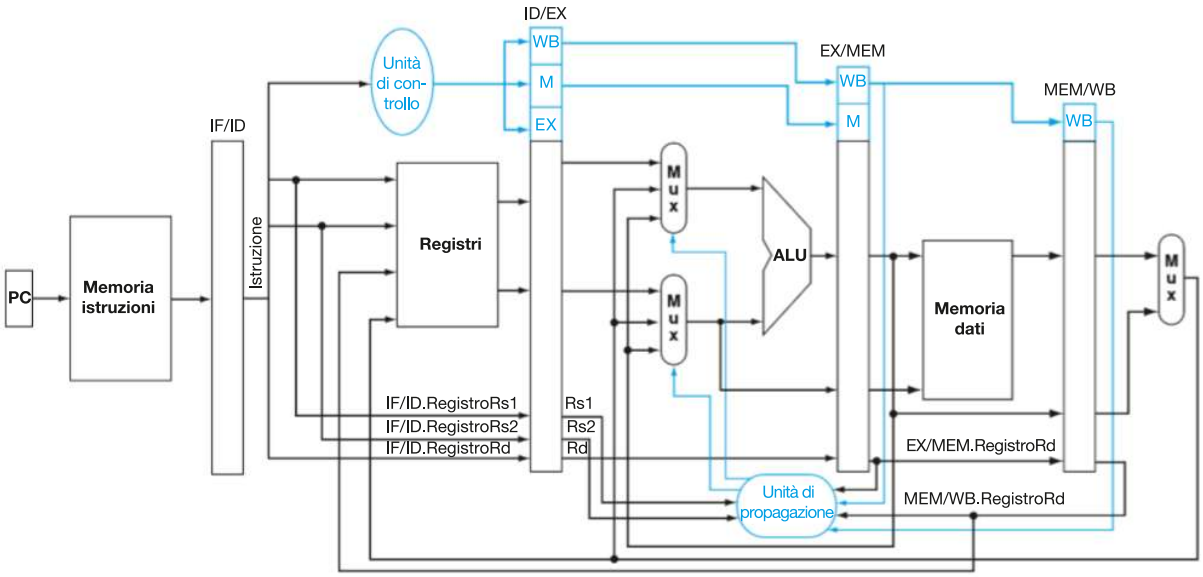


Figura 4.58 L'unità di elaborazione dopo le modifiche apportate per risolvere gli hazard tramite la propagazione. Se si confronta questo schema con l'unità di elaborazione della Figura 4.53, si nota che le aggiunte sono costituite dai multiplexer sugli ingressi della ALU. La figura rappresenta uno schema stilizzato, in cui mancano alcuni dettagli relativi all'implementazione completa che contiene anche la logica dei salti e dell'estensione del segno.

precedenti. Si noti che il campo EX/MEM.RegistroRd contiene il numero del registro di scrittura sia per le istruzioni che operano sulla ALU sia per le istruzioni di load.

Per i lettori interessati ad altri esempi illustrati mediante diagrammi a singolo ciclo, il Paragrafo 4.14 mostra due frammenti di codice RISC-V contenenti hazard che richiedono la propagazione.

Approfondimento. La propagazione può essere di aiuto anche per gli hazard che si verificano quando un'istruzione di store è dipendente da altre istruzioni. Poiché le istruzioni di store utilizzano soltanto un dato durante lo stadio MEM, la propagazione è facile. Si consideri, tuttavia, il caso in cui una load sia immediatamente seguita da alcune istruzioni di store (è il caso in cui si vogliono copiare i dati da memoria a memoria nelle architetture RISC-V). Poiché questo tipo di operazione è frequente, è necessario aggiungere altro hardware per la propagazione per far sì che la copia dei dati da memoria a memoria sia eseguita più velocemente. Se nella Figura 4.55 sostituissimo le istruzioni `sub` e `and`, rispettivamente, con `lw` e `sw`, vedremmo che è possibile evitare lo stallo, poiché il dato che deve essere scritto in memoria dalla `sw` viene scritto nel registro MEM/WB dall'istruzione `lw` precedente, in tempo per essere utilizzato nello stadio MEM dell'istruzione di store. In questo caso, per rendere disponibile il dato alla `sw`, sarebbe necessario aggiungere la propagazione anche all'interno dello stadio di accesso alla memoria. Lasciamo al lettore questa modifica come esercizio.

Inoltre, nell'unità di elaborazione della Figura 4.58 manca il campo immediato dotato di segno che viene richiesto come secondo ingresso della ALU dalle istruzioni load e store. Dal momento che è l'unità di controllo centrale a decidere se selezionare come secondo operando il contenuto di un registro o il campo immediato (mentre l'unità di propagazione sceglie da quale registro di pipeline prendere il secondo input della ALU quando questo deve essere preso da un registro), la soluzione più semplice consiste nell'aggiungere un multiplexer 2:1 che selezioni il secondo ingresso della ALU tra l'uscita PropagaB del multiplexer e il campo immediato dotato di segno. La **Figura 4.59** mostra questa aggiunta.

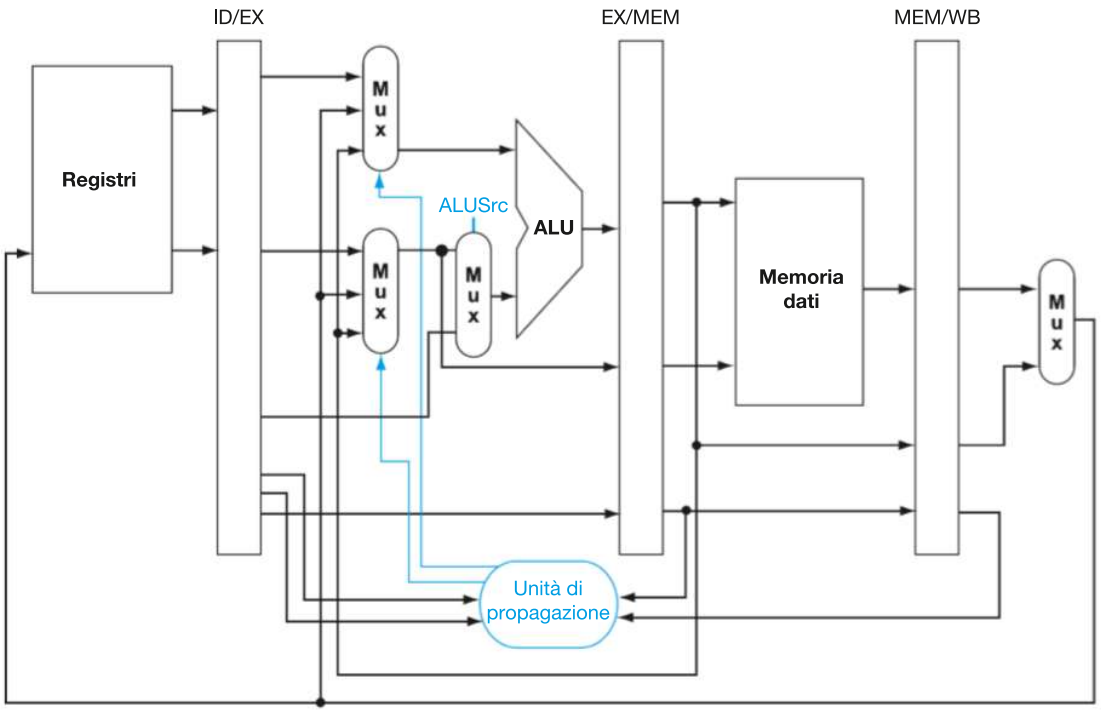


Figura 4.59 Rappresentazione più dettagliata dell'unità di elaborazione della Figura 4.56; essa contiene un multiplexer 2:1, che è stato aggiunto per potere eventualmente selezionare come secondo operando della ALU il campo immediato dotato di segno.

Hazard sui dati e stalli

Se al primo tentativo non hai successo, riformula il concetto di successo.
Anonimo

Come si è detto nel Paragrafo 4.6, un caso nel quale la propagazione non può risolvere il problema è quello in cui un'istruzione tenta di leggere il registro che verrà scritto da un'istruzione di load che la precede; questa situazione è illustrata nella **Figura 4.60**. Durante il quarto ciclo di clock il dato deve ancora essere letto dalla memoria, ma la ALU si appresta già a eseguire l'operazione prevista per l'istruzione successiva. Occorre quindi mettere in stallo la pipeline quando un'istruzione di load è seguita da un'istruzione che utilizza il dato letto.

È quindi necessaria, oltre all'unità di propagazione, anche un'*unità di rilevamento degli hazard* che, durante lo stadio ID di un'istruzione, possa inserire uno stallo tra l'istruzione di load e l'istruzione successiva che dipende dalla load. Per le istruzioni di load, l'unità di rilevamento degli hazard implementa questa semplice condizione logica:

```
if (ID/EX.MemRead and
   ((ID/EX.RegistroRd = IF/ID.RegistroRs1) or
     (ID/EX.RegistroRd = IF/ID.RegistroRs2)))
       metti in stallo la pipeline
```

Ricordate che stiamo utilizzando il RegistroRd per indicare il registro specificato nei bit 11:7 sia per le istruzioni di load che di tipo R. La prima condizione stabilisce se l'istruzione nello stadio EX è una load: questa, infatti, è l'unica istruzione che legge dati dalla memoria. Le successive due condizioni controllano se il numero del registro di scrittura della load coincide con uno dei registri sorgente dell'istruzione nello stadio ID. Se la condizione è verificata,

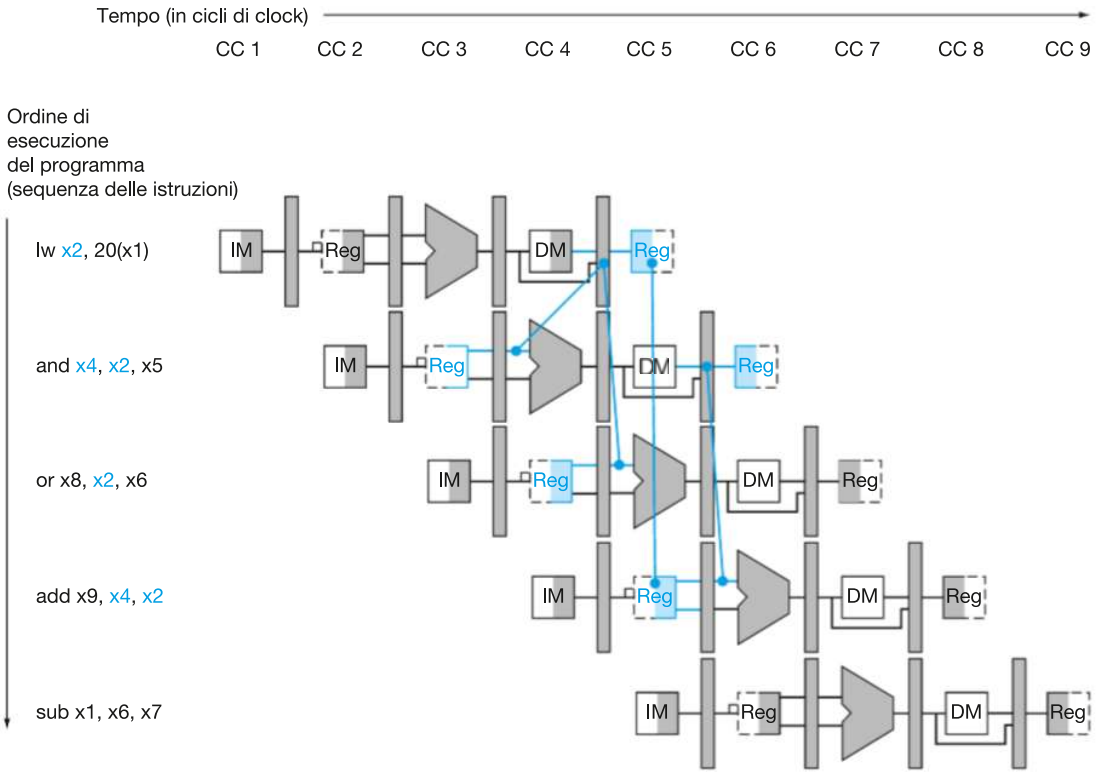


Figura 4.60 Sequenza di istruzioni nella pipeline. Poiché la dipendenza tra l'istruzione di load e l'istruzione successiva (and) è a ritroso nel tempo, questo hazard non può essere risolto con la propagazione, ma occorre che l'unità di rilevamento degli hazard generi uno stallo della pipeline.

l'istruzione nello stadio ID viene messa in stallo per un ciclo di clock. Dopo questo stallo, la dipendenza sui dati può essere gestita dalla logica di propagazione e l'esecuzione può quindi procedere. Se non ci fosse la propagazione le istruzioni della Figura 4.60 richiederebbero lo stallo per un secondo ciclo di clock. Se l'istruzione nello stadio ID viene messa in stallo, anche l'istruzione nello stadio IF deve essere messa in stallo, altrimenti l'istruzione prelevata nella fase di fetch precedente andrebbe persa. Impedire a queste due istruzioni di procedere può essere ottenuto semplicemente impedendo la commutazione del registro PC e del registro di pipeline IF/ID. Se il valore di questi registri viene conservato attraverso il fronte del clock, nel ciclo di clock successivo l'istruzione che verrà letta nello stadio IF verrà prelevata dallo stesso indirizzo contenuto nel PC nel ciclo di clock precedente, e il numero dei registri da leggere nello stadio ID, contenuti nei campi del registro IF/ID, sarà lo stesso del ciclo di clock precedente. Riprendendo l'esempio del bucato, è come se si facesse ripartire la lavatrice con gli stessi vestiti e si facesse lavorare l'asciugatrice a vuoto. Naturalmente gli stadi a valle dello stadio ID, a partire dallo stadio EX, continuano a lavorare, ma come nel caso dell'asciugatrice occorre che eseguano istruzioni che non producono effetti: queste istruzioni sono chiamate **nop** (*not operation*).

Nop: letteralmente "non un'operazione", è un'istruzione che non esegue alcuna operazione che modifichi lo stato della macchina.

Come possiamo inserire una nop, che si comporta come una "bolla d'aria", all'interno della pipeline? Nella Figura 4.51 abbiamo visto che impostando a 0 tutti e sette i segnali di controllo degli stadi EX, MEM e WB è possibile creare un'istruzione che non fa nulla, cioè una nop. Se identifichiamo l'hazard nello stadio ID, possiamo inserire nella pipeline una bolla mettendo a 0 i campi EX, MEM e WB del registro di pipeline ID/EX, i quali contengono i segnali di controllo; questi segnali di controllo per così dire sterilizzati vengono poi trasportati in avanti nella pipeline a ogni colpo di clock producendo l'effetto desiderato: se i segnali di controllo sono tutti a 0 non vengono scritti né i registri né la memoria dati.

La **Figura 4.61** mostra ciò che avviene effettivamente nell'hardware: l'esecuzione dell'istruzione `and` nello stadio EX della pipeline è trasformata nell'esecuzione di una nop e l'esecuzione di tutte le istruzioni successive a quella di load, `and` compresa, viene ritardata di un ciclo di clock. Come una bolla d'aria in un tubo pieno d'acqua, la bolla associata allo stallo ritarda tutto ciò che la segue e procede stadio dopo stadio fino a giungere alla fine della pipeline. Nell'esempio riportato, l'hazard costringe le istruzioni `and` e `or` a ripetere nel quarto ciclo di clock ciò che avevano fatto nel terzo ciclo: l'istruzione `and` legge il contenuto dei registri e viene decodificata, mentre l'istruzione `or` viene prelevata una seconda volta dalla memoria istruzioni. Uno stallo implica quindi la ripetizione di una certa quantità di lavoro, anche se il suo effetto è di allungare il tempo di esecuzione delle istruzioni `and` e `or` e di ritardare il caricamento dell'istruzione `add`.

La **Figura 4.62** mette in evidenza le connessioni dell'unità di rilevamento degli hazard e dell'unità di propagazione all'interno della pipeline. Come in precedenza, l'unità di propagazione controlla i multiplexer all'ingresso della ALU per sostituire eventualmente il valore proveniente da uno dei registri del register file con il contenuto dell'opportuno registro di pipeline. L'unità di rilevamento degli hazard controlla la scrittura del PC e del registro IF/ID oltre al multiplexer che sceglie se scrivere nel registro ID/EX 0 oppure i segnali di controllo dell'istruzione in fase di decodifica. L'unità di rilevamento degli hazard mette in stallo la pipeline e imposta a 0 i segnali di controllo se si verifica la condizione di hazard per l'utilizzo del dato di una load riportata in precedenza. Per i lettori interessati ad approfondire l'argomento, nel Paragrafo 4.14 viene riportato un esempio di codice RISC-V contenente hazard che provocano lo stallo della pipeline.

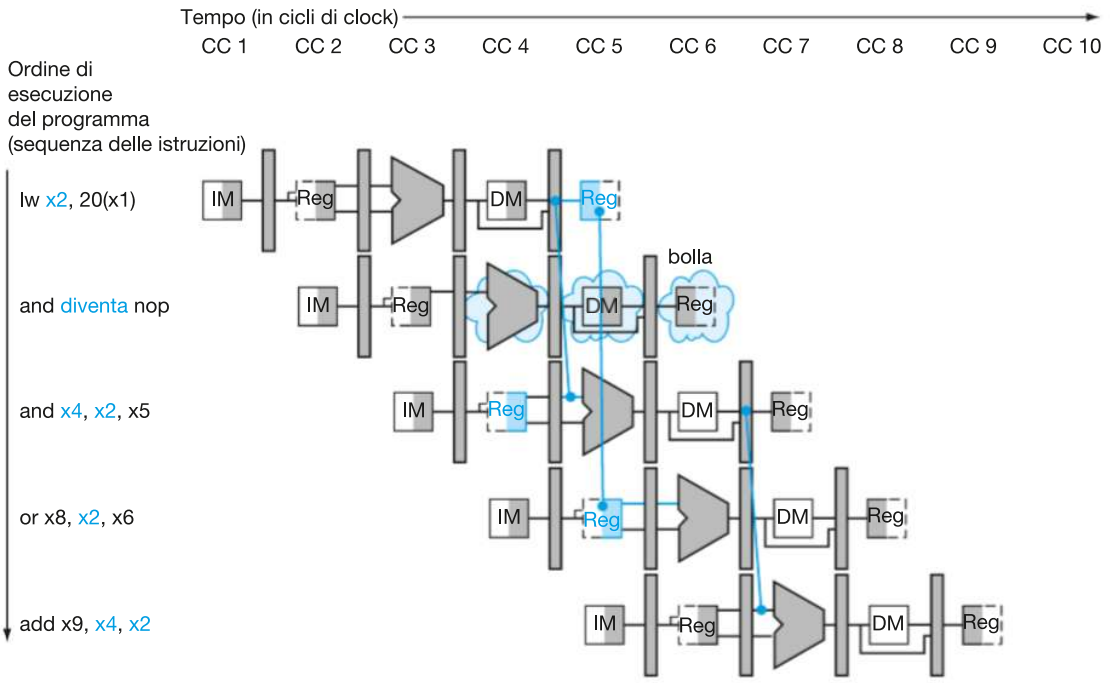


Figura 4.61 Modalità con cui gli stalli vengono realmente inseriti in una pipeline. Una "bolla" viene inserita nel quarto ciclo di clock trasformando l'istruzione `and` in una `nop`. Si noti che l'istruzione `and` viene sì prelevata e poi decodificata rispettivamente nel secondo e terzo ciclo di clock, ma il suo stadio EX viene ritardato fino al quinto ciclo (in assenza di stallo, lo stadio EX verrebbe invece eseguito nel quarto ciclo di clock). Allo stesso modo, l'istruzione `or` viene prelevata nel terzo ciclo di clock, ma il suo stadio ID viene ritardato fino al quinto ciclo di clock (in assenza di stallo lo stadio ID verrebbe eseguito nel quarto ciclo di clock). Dopo l'inserimento della bolla, tutti i dati che generano le dipendenze si spostano in avanti nel tempo e non si verifica più alcun hazard.

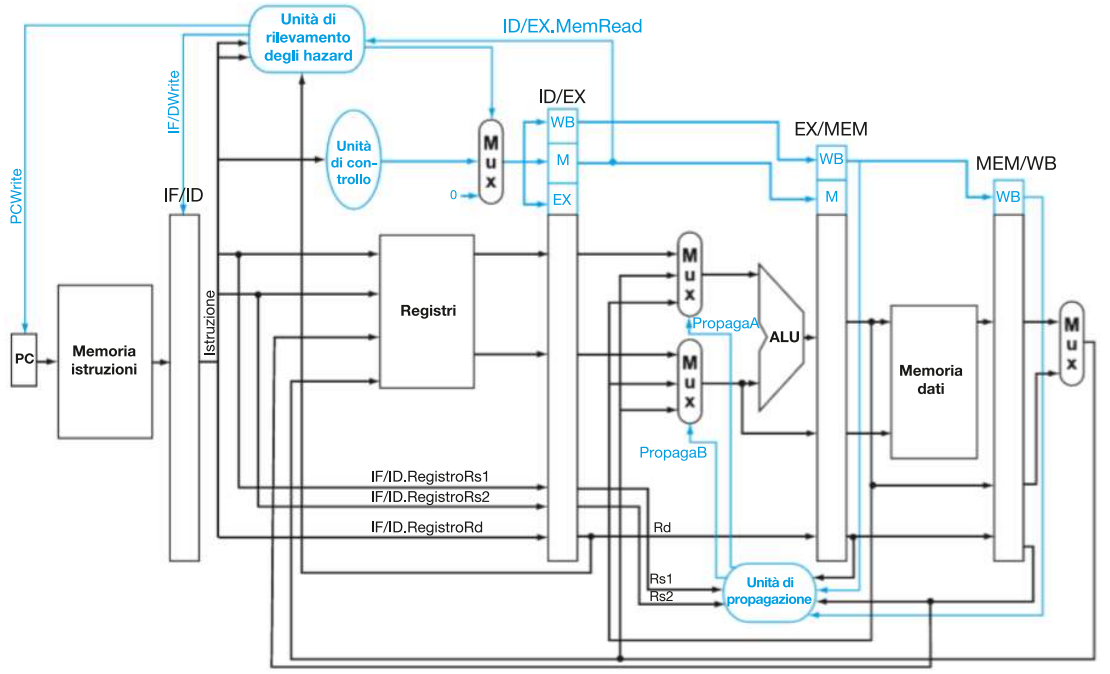


Figura 4.62 Panoramica sul controllo della pipeline: nella figura compaiono i due multiplexer per la propagazione, l'unità di rilevamento degli hazard e l'unità di propagazione. Sebbene gli stadi ID ed EX siano stati semplificati (mancano la logica per l'estensione del segno del campo immediato e la logica dei salti), lo schema mostra gli elementi essenziali dell'hardware per la propagazione.

QUADRO D'INSIEME

Sebbene il compilatore generalmente si basi sull'hardware per risolvere gli hazard e per assicurare l'esecuzione corretta del codice, esso deve comprendere il funzionamento della pipeline per ottenere prestazioni ottimali; altrimenti potrebbero verificarsi stalli inaspettati che riducono le prestazioni del codice compilato. ■

Approfondimento. A proposito della precedente osservazione sull'azzeramento dei segnali di controllo, si noti che per evitare la scrittura nei registri o in memoria i segnali RegWrite e MemWrite devono essere impostati a 0, mentre gli altri segnali di controllo possono assumere un valore indifferente.

4.9 Hazard sul controllo

Sono in migliaia ad accanirsi contro le ramificazioni del male, ma è solo uno a colpirne la radice.
Henry David Thoreau, *Walden*, 1854

Finora ci siamo preoccupati soltanto degli hazard che coinvolgono le operazioni aritmetiche e di trasferimento dati. Tuttavia, come abbiamo visto nel Paragrafo 4.6, ci sono anche hazard della pipeline che riguardano i salti condizionati. La Figura 4.63 mostra una sequenza di istruzioni e indica in quale punto della pipeline verrebbe eseguito il salto. Per alimentare la pipeline è necessario prelevare un'istruzione a ogni ciclo di clock, ma nell'implementazione hardware vista finora le decisioni relative ai salti condizionati non vengono

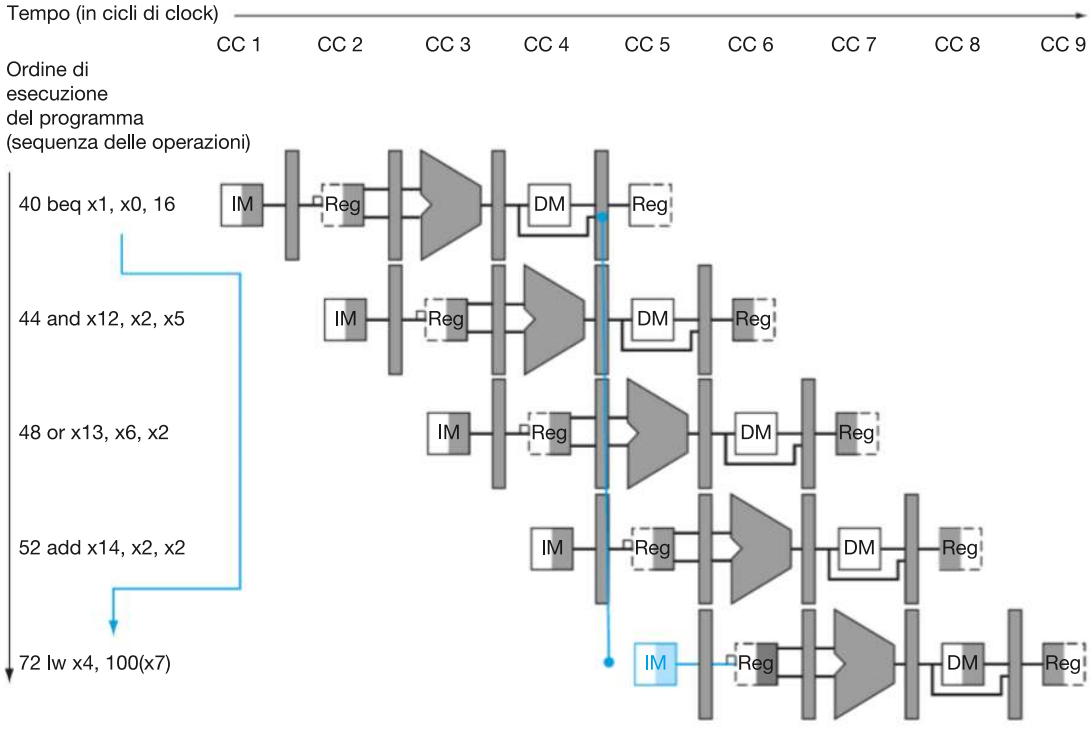


Figura 4.63 Impatto della pipeline sulle istruzioni di salto condizionato. I numeri alla sinistra di ciascuna istruzione (40, 44, ...) rappresentano l'indirizzo delle istruzioni. Dato che l'istruzione di salto condizionato decide se occorre saltare nello stadio MEM, corrispondente al quarto ciclo di clock dell'istruzione `beq`, le tre istruzioni che la seguono vengono comunque prelevate dalla memoria istruzioni e viene avviata la loro esecuzione. Se non si intervenisse, l'esecuzione di queste tre istruzioni inizierebbe prima che `beq` salti eventualmente alla `lw` contenuta all'indirizzo 72. Nella Figura 4.29 si supponeva che esistesse dell'hardware aggiuntivo per ridurre l'hazard sul controllo a un solo ciclo di clock, mentre in questa figura viene rappresentata l'unità di elaborazione non ottimizzata.

prese prima dello stadio MEM della pipeline. Come abbiamo detto nel P aragrafo 4.6, tale ritardo nella determinazione dell'istruzione successiva da prelevare è chiamato *hazard sul controllo* o *hazard sui salti condizionati*, per distinguerlo dagli *hazard sui dati* appena esaminati.

Questo paragrafo è più breve del paragrafo precedente (relativo agli hazard sui dati), poiché gli hazard sul controllo sono relativamente facili da comprendere, si verificano molto meno frequentemente e non c'è nulla di veramente efficace per risolverli. Le modifiche circuitali sono quindi più semplici. In particolare esamineremo due schemi per la risoluzione degli hazard sul controllo e un'ottimizzazione che permette di migliorare tali schemi.

Ipotizzare che il salto condizionato non venga preso

Come abbiamo visto nel Paragrafo 4.6, mettere in stallo la pipeline fino a quando l'esecuzione dell'istruzione di salto condizionato non è stata completata fa perdere troppo tempo. Una tecnica frequentemente utilizzata per risolvere il problema consiste nel **predire** che il salto non debba essere eseguito, continuando quindi a eseguire le istruzioni secondo il loro normale flusso. Se poi risultasse che il salto doveva essere eseguito, le istruzioni che si trovano nella fase di fetch, di decodifica e di calcolo devono essere scartate e l'esecuzione deve riprendere a partire dall'istruzione contenuta all'indirizzo di destinazione del salto. Se i salti condizionati non devono essere eseguiti anche solo nella metà dei casi e se costa poco scartare le istruzioni, questa ottimizzazione permette di dimezzare il costo degli hazard sul controllo.

Per scartare le istruzioni bisogna semplicemente cambiare il valore dei segnali di controllo, forzandoli a 0, analogamente a quanto si è fatto per ottenere lo stallo nel caso degli hazard per l'utilizzo del dato di una load. La differenza consiste nel fatto che, quando l'istruzione di salto condizionato raggiunge lo stadio MEM, è necessario anche eliminare le tre istruzioni presenti negli stadi IF, ID e EX.

Se per le situazioni di stallo legate all'utilizzo del dato di una load era sufficiente mettere a 0 il valore dei segnali di controllo nello stadio ID, lasciando che questi si propagassero lungo la pipeline, ora bisogna eliminare le istruzioni presenti negli stadi IF, ID e EX della pipeline; in termini tecnici occorre effettuare un **flush** (svuotamento) di queste istruzioni.

Flush (di istruzioni): eliminazione di una o più istruzioni da una pipeline, di solito a causa di un evento inaspettato.

Ridurre i ritardi associati ai salti condizionati

Un modo per migliorare le prestazioni sui salti condizionati consiste nel ridurre il costo nel caso in cui un salto condizionato venga erroneamente eseguito. Finora abbiamo supposto (per i salti condizionati) che l'indirizzo successivo da caricare nel PC venga preso dallo stadio MEM; ma se anticipassimo la decisione sul salto all'interno della pipeline, si potrebbe scartare un numero inferiore di istruzioni. Anticipare la decisione su un salto condizionato richiede che due azioni vengano anticipate: calcolare l'indirizzo di destinazione del salto e valutare la condizione logica sulla base della quale il salto condizionato viene effettuato o meno. La parte facilmente realizzabile di questa modifica è l'anticipazione del calcolo dell'indirizzo di destinazione del salto; il contenuto del PC e il campo immediato sono già disponibili nel registro IF/ID della pipeline ed è quindi sufficiente spostare il sommatore che calcola l'indirizzo di salto dallo stadio EX allo stadio ID; naturalmente, il calcolo dell'indirizzo di destinazione del salto sarà effettuato per tutte le istruzioni, ma verrà utilizzato solamente quando richiesto.

La parte più difficile consiste invece nel prendere la decisione sul salto. Se il salto è del tipo branch if equal, si deve confrontare il contenuto dei due registri

letti nello stadio ID e verificare se il loro contenuto è uguale. Il test di uguaglianza può essere eseguito effettuando dapprima uno XOR bit a bit del contenuto dei due registri e inserendo poi in un OR l'uscita degli XOR (un'uscita pari a zero significa che i due registri sono uguali). Spostare il confronto nello stadio ID implica l'aggiunta di altra logica di propagazione e di rilevazione degli hazard; dato che la decisione sul salto dipende dai dati contenuti nella pancia della pipeline, dovremo assicurare che l'istruzione di salto condizionato funzioni correttamente anche con questa ottimizzazione. Per esempio, per implementare l'istruzione di branch if equal (e la sua inversa), abbiamo bisogno di propagare il risultato del test di uguaglianza dello stadio ID. Ci sono due elementi che complicano le cose.

1. Durante lo stadio ID dobbiamo decodificare l'istruzione, decidere se occorra propagare uno dei due operandi all'ingresso del comparatore e valutare l'uguaglianza, in modo che se l'istruzione richiede di saltare si possa scrivere nel PC l'indirizzo di destinazione del salto. Finora la propagazione degli operandi veniva gestita dall'unità di propagazione anche per i salti condizionati, ma l'introduzione del test di uguaglianza nello stadio ID richiede della logica di propagazione aggiuntiva. Si noti che gli operandi di un'istruzione di salto condizionato possono eventualmente essere prelevati dai registri EX/MEM o MEM/WB della pipeline.
2. Poiché i dati su cui fare il confronto sono richiesti già nello stadio ID, ma possono essere prodotti più tardi nella pipeline, è possibile che si verifichi un hazard sui dati e che la pipeline debba essere messa in stallo. Per esempio, se un'istruzione che opera sulla ALU si trova subito prima del salto e produce uno degli operandi per il confronto, ci sarà bisogno di uno stallo, poiché lo stadio EX dell'istruzione che opera sulla ALU è successivo nel tempo allo stadio ID dell'istruzione di salto condizionato. Di conseguenza, se una load è seguita da un'istruzione di salto condizionato e il dato letto dalla memoria è uno dei due operandi, viene richiesto uno stallo di due cicli di clock, dato che il risultato dell'istruzione di load compare nella pipeline solamente alla fine dello stadio MEM, ma viene richiesto all'inizio dello stadio ID dell'istruzione di salto.

Malgrado queste difficoltà, spostare l'esecuzione di un salto condizionato nello stadio ID rappresenta un miglioramento, perché riduce la penalità di un'istruzione di salto condizionato a una sola istruzione da scartare (cioè quella che si trova nella fase di fetch) nel caso in cui il salto debba essere eseguito. L'implementazione dei percorsi di propagazione e di rilevamento degli hazard verrà analizzata dettagliatamente negli esercizi.

Per eliminare l'istruzione in esecuzione dallo stadio IF viene aggiunto un segnale di controllo, denominato IF.Scarta (IF.Flush), che azzera la parte del registro di pipeline IF/ID che contiene l'istruzione. Azzerando questa parte del registro si trasforma l'istruzione appena caricata in una nop, ossia in un'istruzione che non esegue alcuna operazione e non modifica quindi nessun elemento di stato del sistema.

Salti condizionati nella pipeline

ESEMPIO

Mostrare che cosa succede quando all'interno della seguente sequenza di istruzioni il salto condizionato viene eseguito, supponendo che i salti vengano solitamente non eseguiti e che l'esecuzione del salto sia stata spostata nello stadio ID:

(continua)

 © 978-88-08-19966-9

(continua)

```
36 sub x10, x4, x8
40 beq x1,  x3, 16 // Salto relativo al PC: 40 + 16 * 2 = 72
44 and x12, x2, x5
48 or  x13, x2, x6
52 add x14, x4, x2
56 sub x15, x6, x7
. . .
72 lw  x4, 50(x7)
```

SOLUZIONE

La **Figura 4.64** mostra ciò che succede quando viene eseguito il salto. A differenza della Figura 4.63, occorre inserire una sola bolla nella pipeline nel caso in cui il salto venga eseguito.

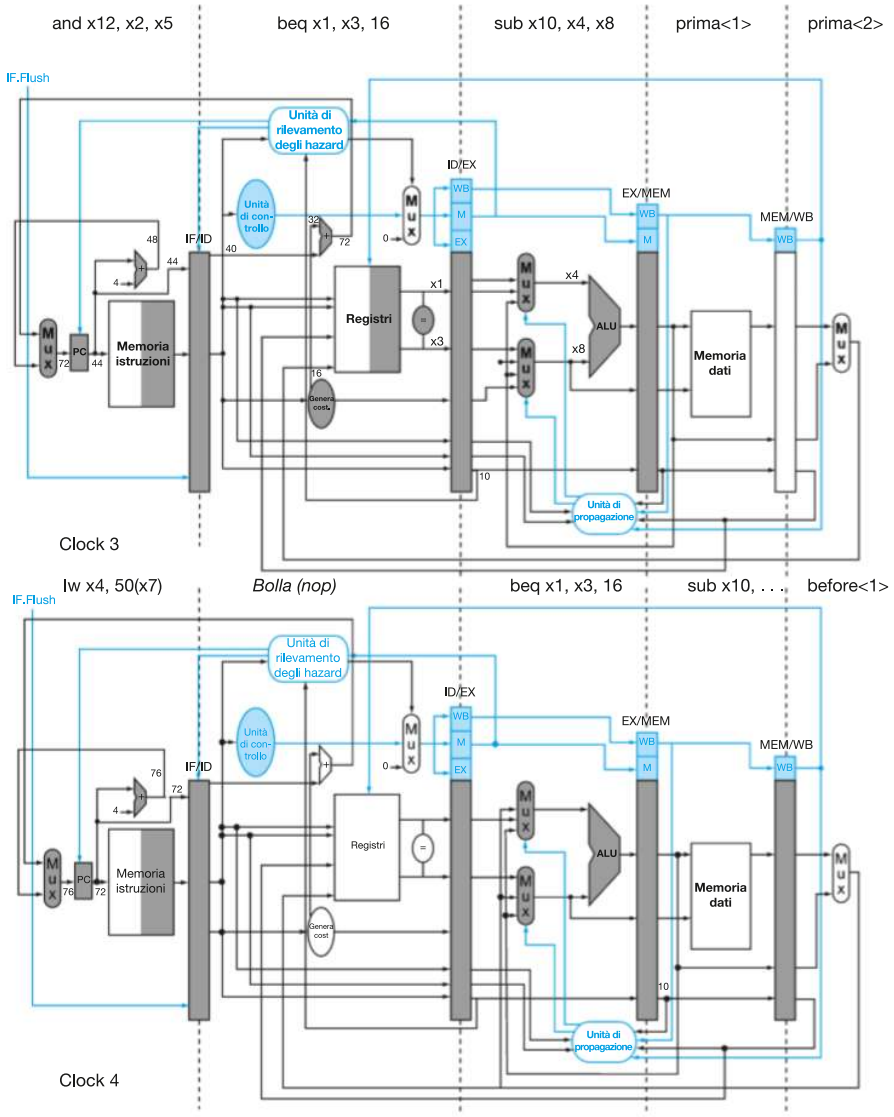


Figura 4.64 Nel terzo ciclo di clock lo stadio ID decide che il salto deve essere eseguito, seleziona quindi 72 come successivo indirizzo da caricare nel PC ed elimina l'istruzione caricata nel registro IF/ID. Nel quarto ciclo di clock viene caricata l'istruzione corrispondente all'indirizzo 72 e si inserisce la bolla corrispondente a un'istruzione `nop` all'interno della pipeline al posto dell'istruzione successiva all'istruzione di salto.

Predizione dinamica dei salti

Quando si suppone che un salto condizionato non debba essere eseguito si realizza una forma rozza di *predizione del salto*; nel caso in cui la supposizione si riveli errata, occorre eliminare le istruzioni contenute nella pipeline. Per una semplice pipeline a cinque stadi questo approccio, possibilmente accoppiato a qualche tecnica di predizione utilizzata dal compilatore, risulta probabilmente adeguato. Con pipeline più profonde il costo dei salti condizionati, misurato in numero di cicli di clock, aumenta. In maniera simile nei processori a esecuzione parallela (Paragrafo 4.11) il costo dei salti condizionati cresce, in termini di numero di istruzioni perse. Ciò significa che in una pipeline "aggressiva" questo semplice schema di predizione statica causerebbe probabilmente una perdita di prestazioni eccessiva. Come accennato nel Paragrafo 4.6, con più hardware a disposizione è possibile cercare di **predire** il comportamento dei salti durante l'esecuzione stessa del programma.

Un possibile approccio consiste nel verificare se l'ultima volta che un'istruzione di salto è stata eseguita il salto sia stato effettivamente eseguito: in caso positivo vengono caricate le istruzioni a partire da quella presente all'indirizzo di destinazione del salto. Questa tecnica viene chiamata **predizione dinamica dei salti**.

Predizione dinamica dei salti: predizione dei salti durante l'esecuzione effettuata utilizzando informazioni raccolte durante l'esecuzione stessa.

Un modo per implementare questa strategia consiste nell'utilizzare un **buffer di predizione dei salti** (*branch prediction buffer*), detto anche **tabella della storia dei salti**. Un buffer di predizione dei salti è una piccola memoria indicizzata attraverso la parte inferiore dell'indirizzo dell'istruzione di branch: questa memoria contiene un bit che indica se il salto era stato eseguito o meno nell'ultima esecuzione.

Buffer di predizione dei salti: detto anche **tabella della storia dei salti**, è una piccola memoria, indicizzata dalla porzione meno significativa dell'indirizzo dell'istruzione di salto, contenente uno o più bit che indicano se in precedenza il salto è stato effettuato.

Questa predizione utilizza il buffer più semplice che si possa immaginare: infatti non possiamo sapere se la predizione sia corretta o meno; per esempio, la predizione potrebbe riguardare un'altra istruzione di salto condizionato la cui parte inferiore dell'indirizzo coincide con quella dell'istruzione di branch che stiamo analizzando. Tuttavia, un errore sulla predizione non influenza la correttezza dell'esecuzione: la predizione è solamente un suggerimento che si spera risulti corretto, su cui è basato il prelevamento dell'istruzione successiva. Se il suggerimento si rivela sbagliato, le istruzioni caricate erroneamente saranno eliminate, verrà invertito il bit di predizione e l'esecuzione ripartirà dal prelevamento dell'istruzione corretta.

Questo semplice schema di predizione a 1 bit ha un inconveniente: anche se un salto venisse quasi sempre effettuato, la predizione risulterebbe sbagliata almeno due volte invece che una volta sola, quando il salto non viene effettuato. L'esempio che segue illustra questo problema.

Cicli e predizione

ESEMPIO

Si consideri un salto condizionato all'interno di un ciclo e si supponga che il salto venga eseguito nove volte di seguito e poi alla decima volta non venga eseguito. Qual è l'accuratezza fornita dalla previsione per questo salto, assumendo che il bit di previsione nel buffer di predizione non venga alterato dalle altre istruzioni?

SOLUZIONE

A regime la previsione non risulterà corretta nella prima e nell'ultima iterazione del ciclo. L'errore sull'ultima iterazione è inevitabile, in quanto la previsione ci dirà che il salto deve essere eseguito, essendo stato eseguito nove volte di seguito fino ad allora. L'errore sulla prima

(continua)

(continua)

iterazione si verifica perché il bit di predizione commuta in occasione della precedente esecuzione dell'ultima iterazione del ciclo, quando il salto non era stato eseguito. Di conseguenza, l'accuratezza della previsione per questo salto (che viene eseguito nel 90% dei casi) è pari all'80% (due previsioni sbagliate e otto corrette).

In linea di principio si vorrebbe che per questi salti altamente regolari l'accuratezza della predizione corrispondesse alla frequenza di esecuzione del salto. Per rimediare a questa inadeguatezza, possiamo utilizzare degli schemi di previsione a più bit. In uno schema a 2 bit la predizione deve risultare sbagliata due volte di seguito prima di essere modificata. La **Figura 4.65** mostra la macchina a stati finiti che implementa questo schema.

Un buffer di predizione dei salti può essere realizzato come un piccolo buffer speciale, accessibile tramite l'indirizzo dell'istruzione durante lo stadio IF della pipeline. Se si prevede che il salto debba essere eseguito, il caricamento delle istruzioni successive sarà effettuato a partire dall'indirizzo di destinazione del salto non appena questo viene scritto nel PC; come detto in precedenza, questo può avvenire già nello stadio ID. In caso contrario, il prelevamento e l'esecuzione delle istruzioni continueranno in modo sequenziale. Se la predizione si rivela sbagliata, i bit di predizione saranno modificati, come mostrato nella Figura 4.61.

Approfondimento. Un predittore dei salti ci dice se un salto debba essere effettuato o meno, ma il calcolo dell'indirizzo di destinazione del salto rimane un problema. In una pipeline a cinque stadi questo calcolo richiede un ciclo di clock. Ciò significa che i salti che vengono effettuati avrebbero il costo di un ciclo. Un approccio consiste nell'utilizzare una memoria cache per salvare l'indirizzo di destinazione del salto o l'istruzione di destinazione, detta anche **buffer degli indirizzi di salto** (*branch target buffer*).

Buffer degli indirizzi di salto: una struttura che memorizza l'indirizzo di destinazione del salto o l'istruzione di destinazione. Di solito viene organizzata come una cache e contiene anche il campo "tag", il che la rende più costosa di un semplice buffer di predizione.

Lo schema di predizione dinamica a 2 bit tiene conto solo delle informazioni sul particolare salto condizionato a essa associato. I ricercatori hanno notato

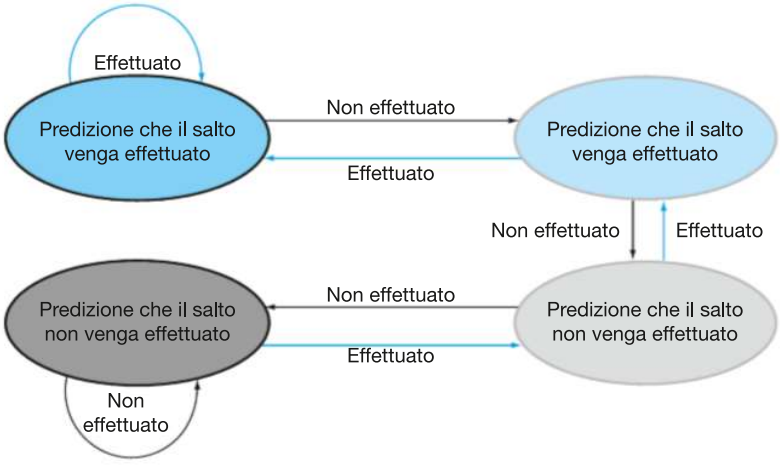


Figura 4.65 Stati possibili in uno schema di predizione su 2 bit. Utilizzando 2 bit anziché 1 bit, una branch che abbia una forte tendenza a eseguire il salto o a non eseguirlo, cosa che accade molto frequentemente, avrà una predizione errata soltanto una volta. I 2 bit sono richiesti per codificare i 4 stati della macchina a stati finiti. Lo schema di predizione su 2 bit è un esempio di predizione basata sui contatori. Questi vengono incrementati quando la predizione si rivela accurata e decrementati in caso contrario; ai contatori vengono associati dei predittori, che utilizzano il valore medio di conteggio per decidere se il salto dovrà essere effettuato o meno.

che utilizzare le informazioni sia sul salto considerato sia sul comportamento globale dei salti eseguiti di recente permette di migliorare notevolmente l'accuratezza della predizione, a parità di bit utilizzati. Questi predittori sono chiamati **predittori correlati** (*correlating predictors*). Un tipico predittore correlato contiene due predittori a 2 bit per ogni salto condizionato e la scelta tra i due predittori viene compiuta in base al risultato (effettuato o meno) dell'ultimo salto. Quindi, la storia globale del salto può essere pensata come un insieme ulteriore di indici per l'accesso alla tabella di predizione.

Un altro approccio alla predizione dei salti è rappresentato dai predittori a torneo. Un **predittore di salto a torneo** (*tournament branch predictor*) utilizza più predittori, tenendo traccia per ciascun salto del predittore migliore. Un tipico predittore a torneo contiene due predizioni per un certo salto: una è basata su informazioni locali, l'altra su informazioni globali. Un circuito di selezione sceglierà quale predittore utilizzare; questo circuito funziona in modo simile a un predittore a 1 o 2 bit e favorisce il predittore che risulta più accurato. Alcuni dei microprocessori più recenti utilizzano questi predittori sofisticati.

Predittore correlato: un predittore di salto che combina il comportamento del salto considerato con informazioni globali associate al comportamento di alcune delle istruzioni di salto condizionato recentemente eseguite.

Predittore di salto a torneo: un predittore di salto con predizioni multiple per ciascun salto, dotato di un meccanismo di selezione che permette di scegliere il predittore più accurato per un certo salto.

Approfondimento. Un modo per ridurre il numero di istruzioni di salto condizionato è quello di aggiungere istruzioni di *spostamento condizionato*. Invece di modificare il contenuto del PC con un'istruzione di salto condizionato, queste istruzioni modificano in modo condizionato il contenuto del registro di destinazione dell'istruzione di spostamento. Per esempio, l'architettura dell'insieme di istruzioni ARMv8 contiene un'istruzione di selezione condizionata chiamata `CSEL`. Questa istruzione definisce un registro destinazione, due registri sorgente e una condizione. Il registro destinazione assume il valore del primo operando se la condizione è vera e del secondo operando se la condizione è falsa. Quindi `CSEL X8, X11, X4, NE` copia il contenuto del registro 11 nel registro 8 se, in accordo con il codice della condizione specificato, il risultato dell'operazione non è zero, e copia il contenuto del registro 11 nel registro 4 se il risultato è zero. Quindi, i programmi che utilizzano l'insieme delle istruzioni ARMv8 possono contenere meno salti condizionati di quello scritti in RISC-V.

Riepilogo sulla pipeline

Siamo partiti dal bucato, mostrando i princìpi della pipeline applicati a un'attività comune della vita quotidiana. Utilizzando questo esempio abbiamo spiegato l'esecuzione in pipeline, passo dopo passo, a partire dall'unità di elaborazione a singolo ciclo a cui abbiamo aggiunto via via i registri di pipeline, i cammini di propagazione, le unità di rilevamento delle criticità, la predizione dei salti condizionati e l'eliminazione delle istruzioni richiesta quando si verificano eccezioni. La **Figura 4.66** mostra l'unità di elaborazione finale con la relativa unità di controllo. Siamo ora pronti ad affrontare il difficile argomento delle eccezioni.

Autovalutazione

Si considerino tre schemi di predizione dei salti: salti non effettuati, salti effettuati e predizione dinamica. Si supponga che tutti e tre non introducano un costo aggiuntivo (in termini di cicli di clock) quando predicono accuratamente il salto e comportino invece due cicli di clock in più quando sbagliano la predizione. Si supponga, inoltre, che l'accuratezza media della predizione sia del 90%. Quale schema di predizione è migliore nelle seguenti situazioni?

1. Un salto condizionato viene eseguito con una frequenza del 5%.
2. Un salto condizionato viene eseguito con una frequenza del 95%.
3. Un salto condizionato viene eseguito con una frequenza del 70%.

 © 978-88-08-19966-9

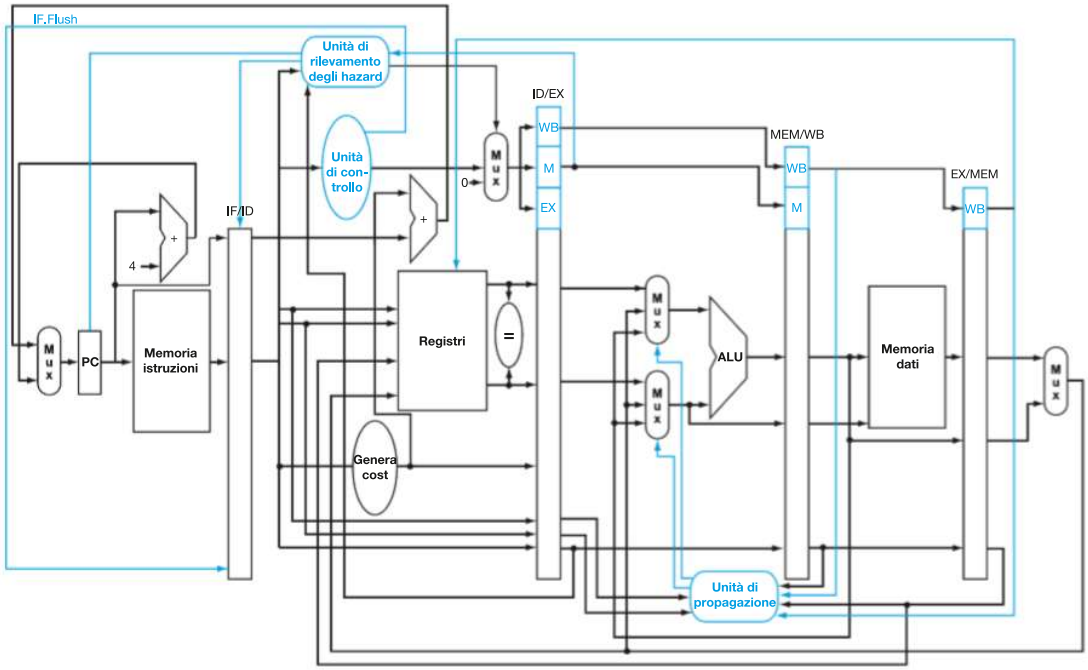


Figura 4.66 L'unità di elaborazione finale e la relativa unità di controllo della pipeline vista in questo capitolo. Si noti che questo è uno schema stilizzato e l'unità di elaborazione non è dettagliata; per esempio, manca il multiplexer pilotato dal segnale di controllo ALUSrc presente nella Figura 4.59 e i segnali di controllo del multiplexer della Figura 4.53.

4.10 Le eccezioni

Far sì che un calcolatore dotato di una gestione automatica degli interrupt si comportasse [in modo sequenziale] non fu facile, dato che, quando si verifica un interrupt, il numero di istruzioni nei diversi stadi di elaborazione può essere molto grande.

Fred Brooks Jr., *Planning a Computer System: Project Stretch*, 1962

Eccezione: detta anche **interrupt** (interruzione), è un evento non previsto che interrompe l'esecuzione di un programma. Può essere utilizzata, per esempio, per segnalare le condizioni di overflow.

Interrupt: eccezione che ha origine all'esterno del processore. Alcune architetture utilizzano il termine *interrupt* per indicare tutti i tipi di eccezioni.

L'unità di controllo di un processore è la parte più difficile da far funzionare correttamente ed è anche la più difficile da rendere veloce. Una delle sfide maggiori è rappresentata dalla gestione delle **eccezioni** e degli **interrupt**, eventi distinti dai salti condizionati che alterano il normale flusso sequenziale di esecuzione delle istruzioni. Inizialmente erano stati concepiti per gestire eventi inaspettati che si verificavano all'interno del processore, come l'overflow aritmetico; lo stesso meccanismo di base fu poi esteso alla comunicazione tra i dispositivi di I/O e il processore come vedremo nel Capitolo 5.

Molte architetture e molti ricercatori non fanno distinzione tra interrupt ed eccezioni, utilizzando spesso entrambi i termini per indicare entrambi i tipi di eventi. Per esempio, l'architettura x86 di Intel utilizza il termine interrupt. In questo testo utilizzeremo il termine *eccezione* per riferirci a un *qualsiasi* cambiamento non previsto del flusso di controllo, senza distinguere se esso abbia cause esterne o interne al processore; utilizzeremo invece il termine *interrupt* per riferirci solo agli eventi che hanno cause esterne. La tabella seguente riporta cinque tipi di eventi, distinguendo se vengono generati all'interno o all'esterno del processore e il nome utilizzato dal RISC-V.

Tipo di evento	Provenienza	Terminologia RISC-V
Reset di sistema	Esterno	Eccezione
Richiesta di un dispositivo di I/O	Esterno	Interrupt
Chiamata al sistema operativo di un programma utente	Interno	Eccezione
Utilizzo di un'istruzione non definita	Interno	Eccezione
Malfunzionamenti hardware	Entrambi	Eccezione o interrupt

Molti dei requisiti per il supporto delle eccezioni provengono dal contesto specifico che le causa. Torneremo perciò su questo argomento nel Capitolo 5, dove comprenderemo meglio la necessità di avere funzionalità aggiuntive per la gestione delle eccezioni. In questo paragrafo verrà descritta l'implementazione di un'unità di controllo in grado di rilevare due tipi di eccezioni che si possono verificare nell'insieme delle istruzioni e nell'architettura che le implementa presentata nei paragrafi precedenti.

Il rilevamento di condizioni eccezionali e la scelta dell'azione opportuna si trovano spesso sul cammino critico di un processore, che determina il periodo del clock e quindi le prestazioni. Occorre porre particolare attenzione alle eccezioni durante la progettazione dell'unità di controllo, perché aggiungere a posteriori la gestione delle eccezioni a un'implementazione hardware complessa può comportare una significativa riduzione delle prestazioni; inoltre, diventerebbe più difficile realizzare un'implementazione hardware che funzioni correttamente.

Gestione delle eccezioni nelle architetture RISC-V

I soli tipi di eccezione che possono essere generati nell'implementazione hardware vista finora sono l'esecuzione di un'istruzione non valida e i malfunzionamenti hardware. In questo paragrafo supporremo che un malfunzionamento hardware si verifichi durante l'esecuzione dell'istruzione `add x11, x12, x11`. L'operazione fondamentale che il processore deve compiere al verificarsi di un'eccezione consiste nel salvare l'indirizzo dell'istruzione che l'ha generata nel *registro supervisione della causa delle eccezioni* (SEPC, *supervision exception cause register*) e trasferire il controllo a un indirizzo specifico del sistema operativo.

Il sistema operativo potrà quindi intraprendere le azioni più opportune, che potrebbero consistere nel fornire alcuni servizi al programma utente, eseguire un'azione predeterminata in risposta al malfunzionamento o terminare l'esecuzione del programma segnalando un errore.

Dopo aver compiuto le azioni necessarie per rispondere all'eccezione, il sistema operativo può terminare il programma o riprenderne l'esecuzione utilizzando il SEPC per determinare il punto da cui ripartire. Nel Capitolo 5 esamineremo in maggior dettaglio come viene fatta ripartire l'esecuzione di un programma.

Per poter gestire correttamente un'eccezione, il sistema operativo deve conoscerne la causa, oltre a sapere quale istruzione l'ha generata. Ci sono due metodi per comunicare la causa di un'eccezione: il metodo utilizzato nell'architettura RISC-V è quello di prevedere un registro dedicato (detto *registro causa di supervisione dell'eccezione* o *SCAUSE, Supervisor Exception Cause Register*) contenente un campo che indica la causa dell'eccezione.

Un metodo alternativo consiste nell'adottare **interrupt vettorizzati**, nei quali l'indirizzo a cui si deve trasferire il controllo viene determinato dalla causa dell'eccezione stessa, che fornisce un numero che viene sommato eventualmente a un registro base che punta a un'area di memoria contenente le istruzioni di risposta agli interrupt vettorizzati. Per esempio, per gestire i due tipi di eccezioni sopra riportati si potrebbero definire i seguenti indirizzi:

Interrupt vettorizzato: un interrupt gestito in modo tale che l'indirizzo del sistema operativo al quale viene trasferito il controllo per la risposta a un'eccezione viene determinato dalla causa stessa dell'eccezione.

Tipo di eccezione	Indirizzo letto dell'eccezione vettorizzata da aggiungere al Registro Base della Tabella dei Vettori
Istruzione non definita	$00\ 0100\ 0000_{due}$
Errore di sistema (malfunzionamento hardware)	$01\ 1000\ 0000_{due}$

Il sistema operativo capisce il motivo dell'eccezione dall'indirizzo da cui inizia la risposta all'eccezione. Se le eccezioni non sono vettorizzate, come nel caso del RISC-V, il sistema operativo inizia a rispondere a tutte le eccezioni dallo stesso indirizzo e deve decodificare il registro causa per capire che cosa abbia generato l'eccezione. Per le architetture con una vettorizzazione delle eccezioni, gli indirizzi potrebbero essere spaziati, per esempio, di 32 byte ovverosia otto istruzioni, e il sistema operativo deve registrare il motivo dell'eccezione e può eseguire un'elaborazione di portata limitata in questo spazio.

Si possono implementare le operazioni richieste per la gestione delle eccezioni aggiungendo all'implementazione base già vista alcuni registri e alcuni segnali di controllo ed estendendo leggermente l'unità di controllo. Supponiamo di implementare il metodo di gestione delle eccezioni con un singolo indirizzo di risposta, l'indirizzo $1C09\ 0000_{esa}$ (l'implementazione delle eccezioni vettorizzate non porrebbe maggiori difficoltà). Occorrerà introdurre due registri aggiuntivi alla nostra unità di elaborazione RISC-V.

- *SEPC*: è un registro a 32 bit utilizzato per memorizzare l'indirizzo dell'istruzione che ha generato l'eccezione; questo registro serve anche nel caso di eccezioni vettorizzate.
- *SCAUSA*: un registro utilizzato per memorizzare la causa dell'eccezione. Nell'architettura RISC-V tale registro è a 32 bit, sebbene alcuni bit siano, a oggi, inutilizzati. Si supponga che all'interno del registro esista un campo di cinque bit che codifica le due possibili cause di eccezione sopra citate: il codice 2 sarà associato all'eccezione di istruzione non definita e il codice 12 al malfunzionamento hardware.

Eccezioni e loro gestione nella pipeline

In un'unità di elaborazione dotata di pipeline le eccezioni vengono trattate come se fossero una forma di hazard sul controllo. Per esempio, supponiamo che si verifichi un overflow aritmetico in un'istruzione di somma; esattamente come facevamo nel paragrafo precedente per i salti condizionati quando venivano presi, dobbiamo eliminare dalla pipeline le istruzioni che seguono la `add` e iniziare a prelevare le istruzioni dal nuovo indirizzo, che è quello di risposta all'eccezione. Utilizzeremo quindi lo stesso meccanismo dei salti condizionati, con la differenza che in questo caso i segnali di controllo devono essere deasseriti.

Quando dovevamo gestire gli errori sulle predizioni dei salti, abbiamo visto come eliminare un'istruzione nello stadio IF convertendola in una `nop`. Per eliminare un'istruzione che si trovava nello stadio ID, invece, utilizziamo il multiplexer già presente che mette a 0 i segnali di controllo, realizzando così di fatto lo stallo della pipeline. Un nuovo segnale di controllo, chiamato ID.Scarta (ID.Flush), viene messo in OR con il segnale di stallo che proviene dall'unità di rilevamento degli hazard per eliminare l'istruzione nello stadio ID. Per eliminare un'istruzione nello stadio EX, viene utilizzato un nuovo segnale di controllo, chiamato EX.Scarta (EX.Flush), che pilota un nuovo multiplexer utilizzato per mettere a 0 i segnali di controllo di questo stadio. Per prelevare l'istruzione successiva dall'indirizzo $1C09\ 0000_{esa}$, che nel RISC-V contiene l'indirizzo della prima istruzione della procedura del sistema operativo di risposta alle eccezioni, occorre semplicemente aggiungere un'altra linea in ingresso, che porta il valore $1C09\ 0000_{esa}$, al multiplexer che seleziona il nuovo valore del PC. Queste modifiche sono mostrate nella **Figura 4.67**.

Questo esempio mostra un problema nella gestione delle eccezioni: se non si interrompe l'istruzione prima della fine della sua esecuzione, il programmatore non sarà più in grado di vedere il valore originale del registro `x1`, dato che tale registro verrà sporcato, poiché è anche il registro destinazione della stessa

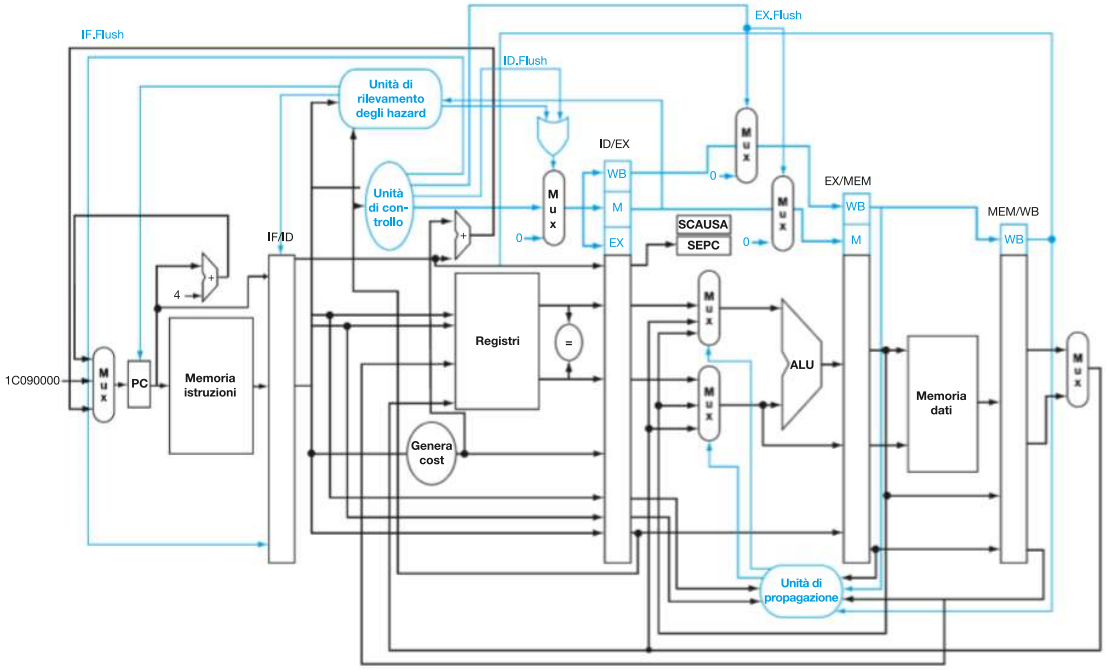


Figura 4.67 L'unità di elaborazione in grado di gestire le eccezioni, con i relativi segnali di controllo. I cambiamenti più importanti sono l'aggiunta di: un nuovo ingresso con valore $1C09\ 0000_{esa}$ al multiplexer che fornisce il nuovo indirizzo al PC, di un registro SCAUSA che memorizza la causa dell'eccezione e di un registro SEPC per salvare l'indirizzo dell'istruzione che ha provocato l'eccezione. L'ingresso $1C09\ 0000_{esa}$ del multiplexer rappresenta l'indirizzo della prima istruzione della procedura del sistema operativo di risposta alle eccezioni. Il segnale di overflow della ALU è uno degli ingressi dell'unità di controllo, anche se in questo schema non viene mostrato.

istruzione `add` che ha generato l'eccezione. Tuttavia, se supponiamo che l'eccezione venga riconosciuta nello stadio EX si può utilizzare il segnale EX.Flush per prevenire che l'istruzione ora nello stadio di EX scriva poi il registro destinazione nello stadio WB. Molte eccezioni richiedono di completare normalmente l'esecuzione dell'istruzione che ha causato l'eccezione come se fossero eseguite normalmente. Il modo più semplice per ottenere ciò è eliminare l'istruzione e farla eventualmente ripartire dall'inizio dopo che l'eccezione è stata gestita.

L'ultimo passo consiste nel salvare l'indirizzo dell'istruzione che ha causato l'eccezione nel *program counter del supervisore delle eccezioni* (SEPC). La Figura 4.67 mostra una versione schematica dell'unità di elaborazione che comprende i circuiti e i componenti necessari per la gestione dei salti condizionati e delle eccezioni.

Eccezioni in un calcolatore dotato di pipeline

ESEMPIO

Supponiamo di avere la seguente sequenza di istruzioni:

```
40esa  sub    x11, x2, x4
44esa  and    x12, x2, x5
48esa  or     x13, x2, x6
4Cesa  add    x1,  x2, x1
50esa  sub    x15, x6, x7
54esa  lw     x16, 100(x7)
. . .
```

(continua)

(continua)

e che la procedura a cui viene trasferito il controllo in caso di eccezione inizi nel modo seguente:

```
1C090000esa  sw    x26, 1000(x10)
1C090004esa  sw    x27, 1004(x10)
. . .
```

Mostrare che cosa avviene all'interno della pipeline se si verifica un'eccezione di malfunzionamento dell'hardware nell'istruzione `add`.

SOLUZIONE

La **Figura 4.68** mostra gli eventi che si susseguono a partire da quando l'istruzione `add` si trova nello stadio EX. Si suppone che il malfunzionamento dell'hardware venga rilevato in questo stadio e che il valore $1C09\,0000_{esa}$ venga forzato nel PC. Il settimo ciclo di clock mostra che la `add` e le istruzioni successive vengono scartate e che viene poi prelevata la prima istruzione della procedura di risposta alle eccezioni. Si noti che si salva l'indirizzo dell'istruzione `add`: $4C_{esa}$.

All'inizio di questo paragrafo abbiamo introdotto alcune possibili eccezioni e vedremo altri esempi nel Capitolo 5. Con cinque istruzioni attive a ogni ciclo di clock, la sfida è associare ciascuna eccezione all'istruzione corretta; inoltre, esiste la possibilità che nello stesso ciclo di clock più eccezioni si verifichino simultaneamente. La soluzione utilizzata consiste nell'associare una priorità alle eccezioni, in modo tale che sia semplice determinare quale eccezione debba essere gestita per prima. Nella maggior parte delle implementazioni RISC-V, l'hardware ordina le eccezioni in modo da interrompere la prima istruzione che ha generato un'eccezione. Le richieste da parte dei dispositivi di I/O e i malfunzionamenti hardware non sono associati a istruzioni specifiche e quindi, in questi casi, è consentita una maggiore flessibilità su quando interrompere la pipeline. Ne consegue che il meccanismo utilizzato per le eccezioni funziona bene anche in questi casi. Il registro SEPC mantiene l'indirizzo delle istruzioni interrotte e il registro SCAUSA, quando si verificano più eccezioni in uno stesso ciclo di clock, memorizza l'eccezione a priorità più elevata.

Interfaccia hardware/software

L'hardware e i sistemi operativi devono lavorare in modo congiunto affinché le eccezioni si comportino nel modo previsto. I compiti dell'hardware sono normalmente i seguenti: bloccare l'istruzione responsabile dell'eccezione prima che venga terminata, far sì che le istruzioni precedenti vengano completate, eliminare tutte le istruzioni successive già caricate nella pipeline, scrivere in un registro la causa dell'eccezione, salvare l'indirizzo dell'istruzione responsabile dell'eccezione e trasferire il controllo a un indirizzo preassegnato. Il compito del sistema operativo consiste invece nell'esaminare la causa dell'eccezione e nell'agire in modo appropriato. Per le eccezioni causate da istruzioni non definite o malfunzionamenti hardware, il sistema operativo normalmente termina (in inglese *kills*) il programma e segnala il motivo dell'interruzione. Nel caso di richieste dei dispositivi di I/O o di chiamate al sistema operativo, quest'ultimo salva lo stato del programma, svolge il compito richiesto e riprende quindi l'esecuzione del programma dal punto in cui era stato lasciato. Nel caso di richieste dei dispositivi di I/O, si sceglie spesso di eseguire un altro programma prima di riprendere l'esecuzione di quello che aveva inviato l'eccezione di I/O, poiché spesso occorre attendere che l'I/O sia stato completato. Questa è la ragione per cui è un passaggio critico salvare e ripristinare lo stato di un qualsiasi programma. Uno degli utilizzi più importanti e frequenti delle eccezioni è la gestione dei page fault e delle eccezioni del TLB che verranno descritte in dettaglio nel Capitolo 5.

Approfondimento. La difficoltà nei calcolatori dotati di pipeline nell'associare sempre correttamente un'eccezione all'istruzione che l'ha provocata ha fatto sì che alcuni progettisti abbiano "ammorbidito" questo vincolo nei casi non critici: si dice che questi processori hanno **interrupt imprecisi** o **eccezioni imprecise**. Nell'esempio precedente, in condizioni normali il PC conterrebbe il valore 58_{esa} all'inizio del ciclo di clock successivo al riconoscimento dell'eccezione, anche se l'istruzione che aveva provocato l'eccezione si trovava all'indirizzo $4C_{esa}$. Un calcolatore con eccezioni imprecise metterebbe 58_{esa} nel registro SEPC, lasciando al sistema operativo il compito di determinare quale istruzione abbia effettivamente provocato l'eccezione. Il RISC-V e la stragrande

Interrupt imprecisi: detti anche **eccezioni imprecise**, sono interrupt oppure eccezioni in calcolatori dotati di pipeline che non sono associati all'istruzione che ha realmente causato l'eccezione o l'interrupt.

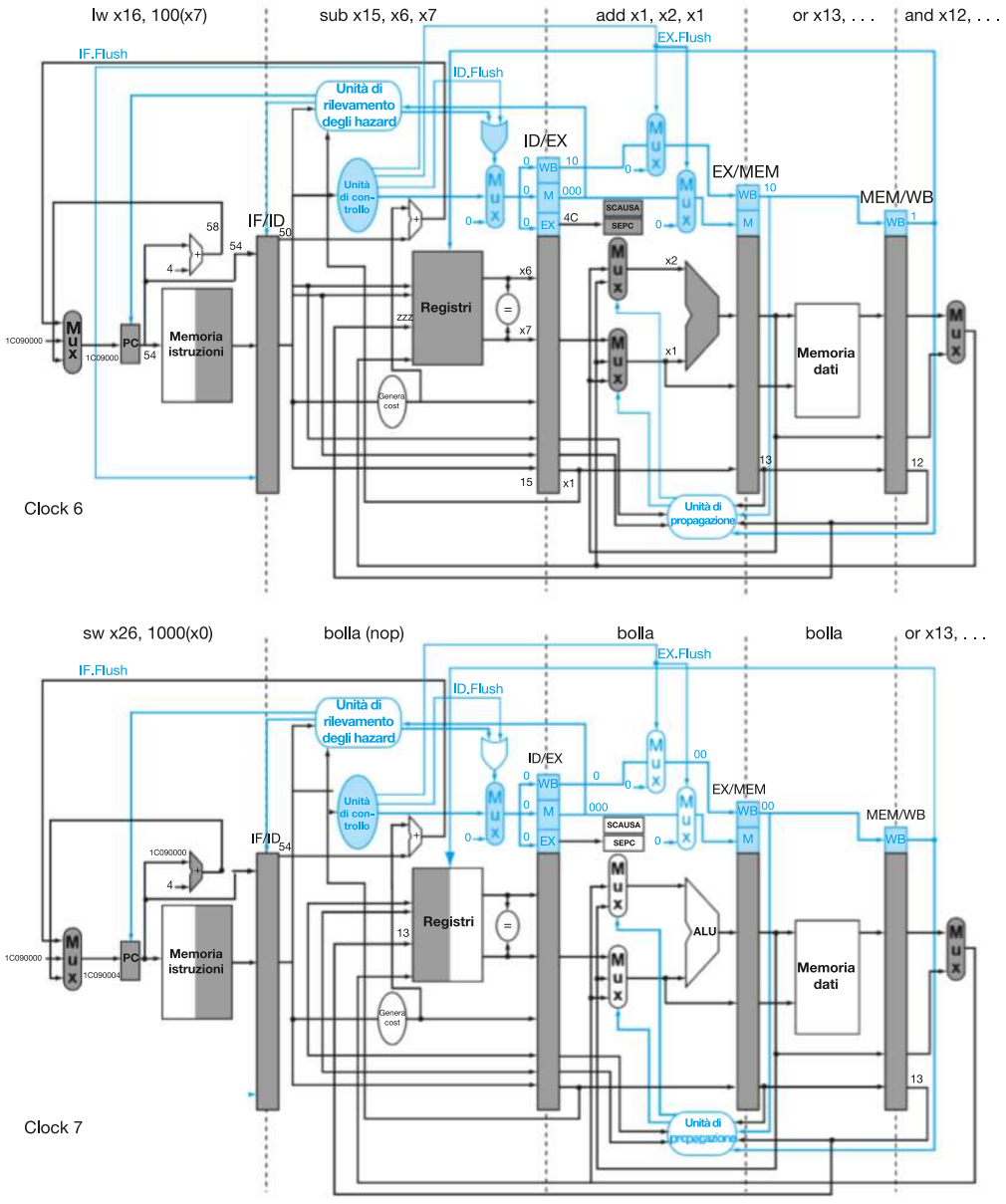


Figura 4.68 Risultato di un'eccezione dovuta a un malfunzionamento dell'hardware in un'istruzione `add`. L'eccezione viene rilevata nello stadio EX del ciclo di clock numero 6 e viene salvato l'indirizzo dell'istruzione `add` nel registro SEPC ($4C_{esa}$). Ciò provoca l'asserimento di tutti i segnali di Flush in prossimità della fine di questo ciclo di clock e il deasserimento (impostazione a 0) dei segnali di controllo associati alla `add`. Nel ciclo di clock numero 7 le istruzioni già presenti nella pipeline risultano convertite in bolle mentre viene caricata in fase di fetch la prima istruzione della funzione di risposta alle eccezioni – `sw x26, 1000(x0)` – dall'indirizzo della memoria istruzioni $1C09\ 0000_{esa}$. Si noti che l'esecuzione delle istruzioni `and` e `or`, caricate prima della `add`, viene comunque completata.

Interrupt precisi: detti anche **eccezioni precise**, sono interrupt oppure eccezioni che vengono sempre associati all'istruzione che ha realmente causato l'eccezione o l'interrupt.

maggioranza dei calcolatori di oggi supportano **interrupt** ed **eccezioni precisi**. Uno dei motivi di questa scelta è che i progettisti di pipeline più profonde potrebbero avere la tentazione di memorizzare in SPEC un valore diverso, cosa che creerebbe un bel mal di testa al sistema operativo. Per evitare ciò, è meglio che le pipeline più profonde memorizzino in SPEC lo stesso valore del PC che verrebbe memorizzato nelle pipeline a cinque stadi che abbiamo esaminato. È più semplice per tutti memorizzare il PC dell'istruzione che ha causato l'eccezione. (Un altro motivo è il supporto alla memoria virtuale, come vedremo nel Capitolo 5.)

Approfondimento. Il RISC-V utilizza come indirizzo della procedura di risposta alle eccezioni l'indirizzo $1C09\,0000_{esa}$, che è stato scelto in modo arbitrario. Molti calcolatori RISC-V memorizzano l'indirizzo di risposta alle eccezioni in un registro speciale chiamato *vettore delle "trappole" del supervisore* (STVEC, *Supervisor Trap Vector*), nel quale il sistema operativo può memorizzare un indirizzo scelto a suo piacimento.

Autovalutazione

Quale eccezione verrà riconosciuta per prima in questa sequenza di istruzioni?

```
1. xxx x11, x12, x11   // istruzione non definita
2. sub x11, x12, x11   // errore hardware
```

4.11 Parallelismo a livello di istruzioni

Parallelismo a livello di istruzioni (ILP): parallelismo tra istruzioni diverse, le quali vengono eseguite in parallelo.

Avvisiamo il lettore che questo paragrafo contiene un'introduzione ad argomenti affascinanti, ma molto avanzati. Per un approfondimento consigliamo di consultare il testo, scritto dagli stessi autori, *Computer Architecture: A Quantitative Approach* (sesta edizione), dove gli argomenti qui appena accennati vengono trattati in oltre 200 pagine (comprese le appendici).

La **pipeline** sfrutta il **parallelismo** potenziale esistente tra le istruzioni. Questa forma di parallelismo viene chiamata, in modo naturale, **parallelismo a livello di istruzioni** (**ILP**, *Instruction-Level Parallelism*). Ci sono due metodi principali per incrementare potenzialmente la quantità di parallelismo a livello di istruzioni. Il primo consiste nell'aumentare la profondità della pipeline per sovrapporre più istruzioni. Utilizzando l'analogia della lavanderia, si può pensare di suddividere il lavaggio tradizionale, supponendo che sia l'operazione più costosa in termini di tempo, in più fasi, utilizzando diverse macchine separate che si occupano di lavare, risciacquare e centrifugare; la pipeline passerebbe in questo caso da quattro a sei stadi e, per ottenere il massimo aumento di velocità, si renderebbe necessario bilanciare nuovamente i restanti stadi in modo tale che abbiano la stessa durata; questo vale sia per il bucato sia per i processori. La quantità di parallelismo è più alta, poiché più istruzioni vengono sovrapposte nel tempo e le prestazioni sono potenzialmente migliori (dato che il ciclo di clock può essere ridotto).

Un altro approccio consiste nel replicare i componenti interni del calcolatore in modo tale che sia possibile lanciare in esecuzione più istruzioni in ogni stadio della pipeline. Il nome generico che si dà a questa tecnica è **esecuzione parallela** (*multiple issue*). In una lavanderia a esecuzione parallela, per esempio, la lavatrice e l'asciugatrice di casa verrebbero sostituite con tre lavatrici e tre asciugatrici; inoltre, sarebbe necessario trovare altra manodopera per stirare e per riporre negli armadi il triplo della biancheria nello stesso lasso di tempo. Lo svantaggio di questo approccio consiste nel lavoro addizionale

Esecuzione parallela (multiple issue): uno schema in base al quale più istruzioni vengono lanciate in esecuzione in uno stesso ciclo di clock.

necessario a mantenere impegnate tutte le macchine e a trasferire i carichi di bucato da uno stadio della pipeline al successivo.

Lanciare in esecuzione più istruzioni nello stesso stadio fa sì che il tasso di esecuzione delle istruzioni sia maggiore della frequenza di clock o, in altri termini, che il CPI sia minore di 1. In alcuni casi è utile invertire l'unità di misura e utilizzare il *numero di istruzioni per ciclo di clock*, detto anche *IPC*. Quindi, un microprocessore a 3 GHz a esecuzione parallela con quattro vie ha una velocità di esecuzione di picco di 12 miliardi di istruzioni al secondo e quindi un CPI massimo di 0,33, cioè un IPC di 3. Supponendo una pipeline a cinque stadi, un processore di questo genere potrebbe eseguire fino a 20 istruzioni contemporaneamente. Gli attuali calcolatori di fascia alta possono arrivare a eseguire da tre a sei istruzioni in ogni stadio della pipeline e i calcolatori di fascia media puntano a ottenere un IPC di picco pari a 2. Tuttavia, ci sono molti vincoli sulle istruzioni che possono essere eseguite in parallelo e su che cosa fare quando si verificano delle dipendenze.

Esistono due modalità principali per realizzare processori a esecuzione parallela, la cui principale differenza consiste nel modo in cui viene suddiviso il lavoro tra il compilatore e l'hardware: le decisioni possono essere prese staticamente (cioè durante la compilazione) o dinamicamente (cioè durante l'esecuzione). Questi due approcci sono a volte chiamati **parallelizzazione statica dell'esecuzione** (*static multiple issue*) e **parallelizzazione dinamica dell'esecuzione** (*dynamic multiple issue*). Come vedremo, entrambi questi approcci hanno altri nomi più comuni che possono risultare, però, meno precisi o più restrittivi.

Ci sono due compiti principali, ben distinti, che una pipeline a esecuzione parallela deve essere in grado di eseguire.

1. Impacchettare le istruzioni in **slot di esecuzione** (*issue slot*). Come fa a sapere il processore quali e quante istruzioni possono essere lanciate in esecuzione in un dato ciclo di clock? Nella maggior parte dei processori dotati di parallelizzazione statica, questo processo viene parzialmente gestito dal compilatore. Nelle architetture dotate di parallelizzazione dinamica, in genere, è il processore a decidere durante l'esecuzione stessa, anche se il compilatore spesso cerca comunque di ottimizzare l'ordine delle istruzioni per aumentare la velocità di esecuzione.
2. Gestire gli hazard sui dati e sul controllo. Nei processori dotati di parallelizzazione statica alcune o tutte le conseguenze degli hazard sui dati e sul controllo vengono gestite staticamente dal compilatore, mentre la maggior parte dei processori dotati di parallelizzazione dinamica tenta di attenuarne gli effetti (almeno di alcuni tipi di hazard) utilizzando tecniche hardware che operano durante l'esecuzione stessa.

Benché questi due approcci vengano qui descritti come distinti, in realtà le tecniche di parallelizzazione sono spesso ibride, perché ciascun approccio prende in prestito alcune delle tecniche tipiche dell'altra modalità; in altri termini, nessun approccio alla parallelizzazione è solamente statico o solamente dinamico.

Parallelizzazione statica dell'esecuzione: un modo di realizzare un processore a esecuzione parallela in cui molte decisioni vengono prese durante la compilazione, quindi prima dell'esecuzione.

Parallelizzazione dinamica dell'esecuzione: un modo di realizzare un processore a esecuzione parallela in cui molte decisioni vengono prese durante l'esecuzione.

Slot di esecuzione: indica la posizione dalla quale un'istruzione può iniziare l'esecuzione in un dato ciclo di clock; è concettualmente analogo al blocco di partenza di una gara di corsa.

Concetto di speculazione

Uno dei metodi più importanti per scoprire e sfruttare al massimo l'ILP è la speculazione. La **speculazione** è basata sulla grande idea della **predizione** ed è un approccio che permette al compilatore o al processore di "indovinare" le caratteristiche di un'istruzione, in modo da permettere l'inizio dell'esecuzione di altre istruzioni che dipendono da quell'istruzione su cui è stata fatta la speculazione. Per esempio, si può speculare sul risultato del confronto associato

Speculazione: un approccio all'esecuzione in cui il compilatore o il processore tentano di indovinare il risultato di un'istruzione per rimuovere la dipendenza di altre istruzioni che dovrebbero essere eseguite successivamente.

 © 978-88-08-19966-9

a un salto condizionato in modo tale che l'esecuzione delle istruzioni successive possa essere anticipata. Oppure, si può speculare se una store, posta subito prima di una load, operi su un indirizzo di memoria differente, il che permetterebbe alla load di essere eseguita prima della store. Il problema è che la speculazione può rivelarsi sbagliata. Perciò, ogni meccanismo di speculazione deve comprendere anche un meccanismo per verificare se la speculazione è corretta e un meccanismo per poter tornare indietro o eliminare gli effetti delle istruzioni eseguite in base alla speculazione. L'eliminazione degli effetti delle istruzioni eseguite in modo speculativo rende la progettazione della pipeline più complicata.

La speculazione può essere fatta dal compilatore oppure dall'hardware. Per esempio, il compilatore può utilizzare la speculazione per riordinare le istruzioni, spostando un'istruzione dopo un salto condizionato oppure invertendo tra loro una load e una store. Il processore può operare le stesse modifiche durante l'esecuzione mediante tecniche che saranno discusse in seguito.

I meccanismi di correzione utilizzati in caso di speculazione errata sono abbastanza differenti tra loro. Nel caso di speculazione software, il compilatore di solito inserisce istruzioni aggiuntive che controllano l'accuratezza dalla speculazione e fornisce una procedura di riparazione da utilizzare quando si verifica un errore. Nel caso di speculazione hardware, il processore di solito memorizza e mantiene in buffer dedicati i risultati dell'esecuzione speculativa, finché non "capisce" se la speculazione è risultata corretta o meno. Se la speculazione risulta corretta, l'esecuzione delle istruzioni viene completata scrivendo il risultato nei registri o nella memoria. Se invece la speculazione si rivela errata, l'hardware elimina il contenuto dei buffer associato all'istruzione e riprende l'esecuzione con la sequenza corretta di istruzioni. Una speculazione errata tipicamente richiede di svuotare la pipeline, o almeno di metterla in stallo, e quindi riduce ulteriormente le prestazioni.

La speculazione introduce un altro genere di problema: essa può introdurre eccezioni che precedentemente non erano presenti. Per esempio, si supponga che sulla base di una speculazione una load venga spostata e che l'indirizzo a cui fa riferimento non sia più valido se la speculazione risultasse sbagliata; in questo caso verrebbe generata un'eccezione di indirizzo non valido che non dovrebbe essere generata. Il problema è reso complicato dal fatto che, quando la load non è oggetto di speculazione, se l'indirizzo risulta non valido, l'eccezione deve essere sicuramente generata! Nella speculazione basata su compilazione, questo problema viene risolto introducendo uno speciale supporto alla speculazione che permette di ignorare queste eccezioni finché non è chiaro se debbano essere sollevate realmente. Nella speculazione hardware, invece, le eccezioni vengono semplicemente memorizzate in un buffer finché la correttezza della speculazione, in base alla quale l'istruzione era stata lanciata in esecuzione, non è stata verificata; solo allora l'esecuzione dell'istruzione può essere terminata. A questo punto l'eccezione viene generata e viene iniziata la procedura di gestione dell'eccezione.

Dato che la speculazione può migliorare le prestazioni, quando viene fatta in maniera appropriata, ma può diminuirle, quando è implementata in modo poco accurato, un grosso sforzo è dedicato a decidere quando sia opportuno speculare. Verso la fine di questo paragrafo esamineremo alcune tecniche, sia hardware sia software, per la speculazione.

Parallelizzazione statica dell'esecuzione

Tutti i processori dotati di parallelizzazione statica dell'esecuzione utilizzano il compilatore per formare i pacchetti di istruzioni e gestire gli hazard. In questo tipo di processore le istruzioni che vengono lanciate in esecuzione in

parallelo nello stesso ciclo di clock costituiscono un **pacchetto di istruzioni** (*issue packet*), che si può intendere come un'istruzione ampia contenente più istruzioni al suo interno. Questa immagine è più di una semplice analogia. Dal momento che, in un processore dotato di parallelizzazione statica, la combinazione di istruzioni che possono essere lanciate in esecuzione nello stesso ciclo di clock è in generale limitata, è utile pensare al pacchetto di istruzioni come a un'istruzione singola nella quale diverse operazioni vengono codificate in campi predefiniti; questa visione ha prodotto il nome originale di questo approccio: **Very Long Instruction Word** (**VLIW**).

Pacchetto di istruzioni: un insieme di istruzioni che vengono lanciate in esecuzione nello stesso ciclo di clock; la composizione del pacchetto può essere determinata sia staticamente dal compilatore sia dinamicamente dal processore.

Very Long Instruction Word (VLIW): un tipo di architettura dell'insieme di istruzioni in grado di lanciare in esecuzione più istruzioni, indipendenti tra loro, attraverso un'istruzione codificata su molti bit, la quale tipicamente contiene diversi codici operativi.

La maggior parte dei processori con parallelizzazione statica si appoggia al compilatore per la gestione degli hazard sui dati e sul controllo. Le responsabilità del compilatore comprendono la predizione statica dei salti e il riordino del codice per ridurre o prevenire gli hazard. Esaminiamo ora una semplice versione del processore RISC-V dotata di parallelizzazione statica prima di descrivere l'utilizzo di queste tecniche in processori più aggressivi.

Un esempio: parallelizzazione statica dell'ISA del RISC-V

Per dare un'idea di come funzioni un processore dotato di parallelizzazione statica dell'esecuzione, consideriamo un semplice processore RISC-V dotato di due cammini di esecuzione, detti anche *vie*: il primo cammino può eseguire istruzioni che operano sulla ALU intera o istruzioni di salto condizionato, il secondo può eseguire istruzioni di load e store. Una pipeline di questo genere viene utilizzata in alcuni processori di tipo embedded. Lanciare in esecuzione due istruzioni per ciclo di clock richiede il fetch e la decodifica di istruzioni su 64 bit. In molti processori a parallelizzazione statica (e praticamente in tutti i processori VLIW) esistono dei vincoli su quali istruzioni si possano inserire all'interno della coppia di istruzioni eseguite contemporaneamente, in modo da semplificare la fase di decodifica e di invio all'esecuzione. Quindi, richiederemo che le istruzioni vengano accoppiate e allineate a parole di 64 bit, con l'istruzione che utilizza la ALU intera o l'istruzione di salto per prima. Inoltre, se una delle due istruzioni non può essere utilizzata, essa deve essere rimpiazzata da una `nop`. Le istruzioni vengono quindi lanciate in esecuzione sempre in coppia, eventualmente con una `nop` al posto di una delle due. La **Figura 4.69** illustra come le istruzioni appaiono quando procedono accoppiate all'interno della pipeline.

I processori dotati di parallelizzazione statica si differenziano in base a come trattano i potenziali hazard sui dati e sul controllo. In alcune architetture

Tipo di istruzione	Stadi della pipeline							
Istruzione ALU o di salto	IF	ID	EX	MEM	WB			
Istruzione di load o di store	IF	ID	EX	MEM	WB			
Istruzione ALU o di salto		IF	ID	EX	MEM	WB		
Istruzione di load o di store		IF	ID	EX	MEM	WB		
Istruzione ALU o di salto			IF	ID	EX	MEM	WB	
Istruzione di load o di store			IF	ID	EX	MEM	WB	
Istruzione ALU o di salto				IF	ID	EX	MEM	WB
Istruzione di load o di store				IF	ID	EX	MEM	WB

Figura 4.69 Unità di elaborazione dotata di parallelizzazione statica a due vie. Le istruzioni che utilizzano la ALU e quelle di trasferimento dati vengono lanciate in esecuzione contemporaneamente. In questa tabella abbiamo supposto una pipeline a cinque stadi simile a quella senza parallelizzazione. Anche se ciò non è strettamente necessario, ha alcuni vantaggi. In particolare, mantenere la scrittura dei registri al termine dell'esecuzione semplifica la gestione delle eccezioni e favorisce l'utilizzo di un preciso modello di gestione delle stesse. Queste funzionalità diventano più complicate nei processori a esecuzione parallela.

il compilatore ha la piena responsabilità di rimuovere *tutti* gli hazard, riordinare il codice e inserire delle istruzioni nop, in modo tale che il codice venga eseguito senza alcun bisogno di rilevare hazard o generare stalli. In altre architetture, è l'hardware a rilevare gli hazard e a generare gli stalli tra due pacchetti di istruzioni consecutivi, mentre viene richiesto al compilatore di eliminare tutte le possibili dipendenze all'interno dello stesso pacchetto di istruzioni. Anche così, un hazard in genere obbliga a mettere in stallo l'intero pacchetto di istruzioni che contiene l'istruzione con la dipendenza. Sia che il software debba gestire tutti gli hazard sia che debba solo cercare di ridurre il numero di hazard tra i diversi pacchetti di istruzioni, l'idea di un'istruzione ampia che codifica più operazioni al suo interno ne esce rafforzata. Faremo riferimento a questo secondo approccio nell'esempio seguente.

Per lanciare in esecuzione una coppia di istruzioni, una che esegue un'operazione aritmetico-logica e una di trasferimento dati, il primo componente hardware aggiuntivo, oltre alla logica di rilevamento degli hazard e di messa in stallo simile a quella già vista, è una coppia addizionale di porte di lettura del register file (**Figura 4.70**). In un singolo ciclo di clock, infatti, occorre leggere due registri per l'istruzione che opera sulla ALU e altri due per l'istruzione di trasferimento dati. Servono, inoltre, due porte di scrittura, una per l'operazione della ALU e l'altra per l'operazione di trasferimento dati dalla memoria. Dato che la ALU è impegnata nell'esecuzione dell'operazione aritmetico-logica, occorre anche un sommatore separato per il calcolo dell'indirizzo della memoria per il trasferimento dei dati. Senza queste risorse aggiuntive, il funzionamento di questa pipeline a due vie sarebbe ostacolato da hazard di tipo strutturale.

Latenza di utilizzo: numero di cicli di clock che devono intercorrere tra un'istruzione di load e un'istruzione a cui serve il dato caricato dalla memoria perché si possa evitare che la pipeline debba essere messa in stallo.

Chiaramente questo semplice processore a due vie può migliorare le prestazioni al massimo di un fattore 2. Tuttavia, con questo semplice parallelismo, si raddoppia il numero di istruzioni in esecuzione che si sovrappongono nel tempo e questo fa crescere le perdite di prestazioni dovute agli hazard sui dati e sul controllo. Per esempio, nella nostra pipeline a cinque stadi le istruzioni di load hanno una **latenza di utilizzo** di un ciclo di clock, per cui un'altra istruzione

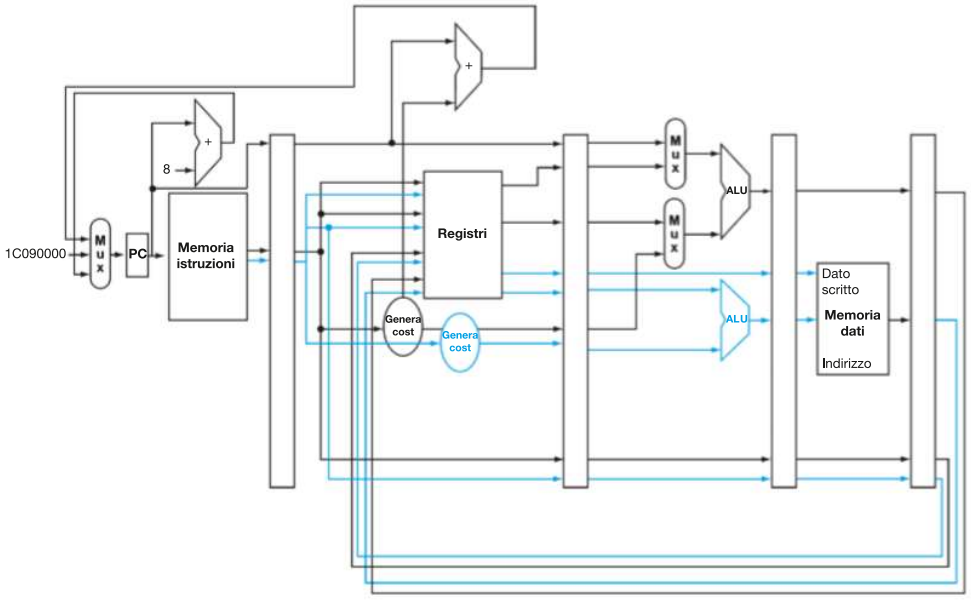


Figura 4.70 Unità di elaborazione dotata di parallelizzazione statica a due vie. Sono evidenziate le aggiunte necessarie per la parallelizzazione dell'esecuzione: 32 bit addizionali dalla memoria istruzioni, due porte di lettura e una di scrittura in più per il register file, e un'altra ALU. Si suppone che la ALU in basso si occupi del calcolo degli indirizzi per il trasferimento dati, mentre quella in alto si occupi di tutto il resto.

non può utilizzare il risultato della load senza uno stallo. Questo significa che in una pipeline a cinque stadi e due vie il risultato di un'istruzione di load non può essere utilizzato nel ciclo di clock *successivo* e le *due* istruzioni seguenti non possono utilizzare il risultato della load senza uno stallo. Inoltre, le istruzioni che utilizzano la ALU e che non hanno latenza di utilizzo nella pipeline semplice ora presentano una latenza di utilizzo di un'istruzione, poiché il loro risultato non può essere utilizzato da una eventuale load o store con cui sono accoppiate. In un processore a esecuzione parallela, per sfruttare il parallelismo in modo efficace sono necessari compilatori più sofisticati o tecniche di riordinamento hardware del codice. I microprocessori a parallelizzazione statica richiedono che il compilatore svolga questi compiti.

Riordinamento di un semplice frammento di codice per la parallelizzazione statica dell'esecuzione

ESEMPIO

Come verrebbe riorganizzato il codice del seguente ciclo in una pipeline RISC-V a due vie a esecuzione parallela?

```
Ciclo: lw   x31, 0(x20)     // x31 = elemento di un vettore
       add  x31, x31, x21   // somma lo scalare contenuto
                            // in x21
       sw   x31, 0(x20)     // memorizza il risultato
       addi x20, x20, -4    // decrementa il puntatore
       blt  x22, x20, Ciclo // confronta con il valore
                            // di fine ciclo, esci dal ciclo
                            // se x20 > x22
```

Riordinare le istruzioni in modo da evitare il numero maggiore di stalli della pipeline. Si supponga che il risultato dei salti condizionati possa essere predetto in modo che gli hazard sul controllo vengano gestiti dall'hardware.

SOLUZIONE

Le prime tre istruzioni, come le due successive, hanno delle dipendenze sui dati. La **Figura 4.71** mostra il migliore riordinamento possibile di queste istruzioni. Si noti che solo una coppia di istruzioni viene eseguita in parallelo. Ogni iterazione del ciclo richiede 5 cicli di clock; con 4 cicli di clock per 5 istruzioni si ottiene un CPI pari a 0,8, contro un CPI massimo di 0,5, ovverosia un IPC di 1,25 contro un IPC massimo di 2,0. Si tratta di valori decisamente deludenti. Si noti che nel calcolare il CPI e l'IPC le nop non vengono contate in quanto non sono istruzioni utili. Se contassimo anche le nop aumenteremmo l'IPC ma non le prestazioni!

	Istruzioni ALU o di salto condizionato	Istruzioni trasferimento dati	Ciclo di clock
Ciclo:		`lw x31, 0(x20)`	1
	`addi x20, x20, -4`		2
	`add x31, x31, x21`		3
	`blt x22, x20, Ciclo`	`sw x31, 4(x20)`	4

Figura 4.71 Codice riordinato come apparirebbe in un processore RISC-V con pipeline a due vie. Si noti che avendo spostato la `addi` prima della `sd`, abbiamo dovuto aggiungere 8 all'offset della `sd`.

Una tecnica importante adottata dai compilatori per ottenere prestazioni migliori nell'esecuzione dei cicli è l'**espansione dei cicli** (*loop unrolling*). Come suggerisce il nome, con questa tecnica vengono generate più copie del corpo del ciclo e istruzioni appartenenti a iterazioni diverse possono essere avviate all'esecuzione contemporaneamente, inserendole nella stessa VLIW.

Espansione dei cicli: una tecnica per ottenere prestazioni migliori sui cicli, specialmente se accedono a vettori, in cui il corpo del ciclo viene replicato più volte e le istruzioni provenienti dalle varie repliche possono essere inserite negli stessi pacchetti di istruzioni.

Espansione dei cicli per pipeline a esecuzione parallela

ESEMPIO

Mostrare l'efficacia della tecnica dell'espansione dei cicli nel caso dell'esempio precedente. Per semplicità, si supponga che l'indice del ciclo sia multiplo di 4.

SOLUZIONE

Per ridurre significativamente i ritardi nel ciclo occorre fare quattro copie del corpo del ciclo. Dopo avere espanso il ciclo e avere eliminato le istruzioni di controllo, che risultano inutili, il ciclo conterrà quattro copie delle istruzioni `lw`, `add` e `sw`, e una copia delle istruzioni `addi` e `blt`. Il codice del ciclo espanso, dopo il riordinamento, è mostrato nella **Figura 4.72**.

Durante il processo di espansione, il compilatore ha introdotto alcuni registri addizionali (`x28`, `x29`, `x30`). L'obiettivo di questa operazione, detta **ridenominazione dei registri** (*register renaming*), è di eliminare le false dipendenze tra i dati che potrebbero generare hazard o limitare il compilatore nel riordino del codice. Si pensi a come apparirebbe il ciclo espanso se si utilizzasse solamente `x31`: si produrrebbero diverse ripetizioni delle istruzioni `lw x31, 0(x20)`, `add x31, x31, x21`, seguite da `sw x31, 8(x20)`. Le diverse ripetizioni di queste tre istruzioni, malgrado utilizzino tutte `x31`, sono in realtà completamente indipendenti: non esiste alcun flusso di dati tra una coppia di istruzioni in un'iterazione e la stessa coppia di istruzioni in un'altra iterazione. Si parla in questo caso di **falsa dipendenza**, o **dipendenza nominale**, che implicherebbe un riordinamento delle istruzioni forzato dall'utilizzo di uno stesso nome e non da una reale dipendenza, chiamata anche *dipendenza effettiva*.

Il processo di ridenominazione dei registri durante l'espansione dei cicli permette al compilatore di inserire una dopo l'altra queste istruzioni tra loro indipendenti, in modo da ottenere una migliore riorganizzazione del codice. La ridenominazione dei registri elimina le false dipendenze ma non quelle vere.

Si noti che 12 delle 14 istruzioni del ciclo vengono eseguite in coppia. Occorrono 8 cicli di clock per eseguire 4 iterazioni del ciclo, cioè 2 cicli di clock per iterazione con un ICP pari a 14/8 = 1,75. L'espansione del ciclo e la riorganizzazione del codice in coppie di istruzioni hanno prodotto un miglioramento delle prestazioni di più di un fattore 2, dovuto in parte alla riduzione del numero di istruzioni di controllo del ciclo e in parte all'esecuzione parallela di coppie di istruzioni. Il costo di questo miglioramento consiste nell'uso di quattro registri temporanei, invece che di uno solo, e in un significativo incremento nella dimensione del codice.

Ridenominazione dei registri: è un processo compiuto dal compilatore o dall'hardware per evitare le false dipendenze.

Falsa dipendenza: detta anche dipendenza nominale, è un riordinamento delle istruzioni causato dall'utilizzo di uno stesso nome, tipicamente di un registro, e non da una vera e propria dipendenza tra i dati delle istruzioni.

Processori dotati di parallelizzazione dinamica dell'esecuzione

I processori dotati di parallelizzazione dinamica dell'esecuzione sono anche conosciuti come processori superscalari o semplicemente **superscalari**. Nei

	Istruzioni ALU o di salto condizionato	Istruzioni trasferimento dati	Ciclo di clock
Ciclo:	addi x20, x20, -32	lw x28, 0(x20)	1
		lw x29, 12(x20)	2
	add x28, x28, x21	lw x30, 8(x20)	3
	add x29, x29, x21	lw x31, 4(x20)	4
	add x30, x30, x21	sw x28, 16(x20)	5
	add x31, x31, x21	sw x29, 12(x20)	6
		sw x30, 8(x20)	7
	blt x22, x20, Ciclo	sw x31, 4(x20)	8

Figura 4.72 Il codice della Figura 4.71 dopo l'espansione del ciclo e il riordinamento del codice, come apparirebbe nell'esecuzione su un RISC-V dotato di parallelizzazione statica a due vie. Gli spazi vuoti sono occupati da istruzioni nop. Dato che x20 viene decrementato di 16 dalla prima istruzione del ciclo, gli indirizzi contenuti nel PC saranno rispettivamente il valore originale di x20, x20 meno 4, meno 8 e meno 12.

processori superscalari più semplici, le istruzioni vengono eseguite in ordine e il processore decide se nessuna, una o più istruzioni possono essere avviate all'esecuzione nello stesso ciclo. Ovviamente, per ottenere buone prestazioni con questo tipo di processore occorre che il compilatore riordini le istruzioni in modo da rimuovere le dipendenze per aumentare il numero di istruzioni che possono essere lanciate in parallelo.

Superscalare: una tecnica di pipeline avanzata che permette al processore di eseguire più istruzioni nello stesso ciclo di clock, selezionando le istruzioni da eseguire in parallelo durante l'esecuzione stessa.

Anche con questo supporto del compilatore c'è una differenza importante tra questo semplice processore superscalare e un processore VLIW: è l'hardware che garantisce che il codice, riordinato o meno, venga eseguito correttamente. Inoltre, il codice in linguaggio macchina verrà eseguito sempre in modo corretto, indipendentemente da quante istruzioni vengono eseguite in parallelo o dalla struttura della pipeline del processore. In alcuni processori VLIW ciò può non essere vero, e diventa quindi necessario ricompilare i programmi quando ci si sposta su modelli diversi dello stesso processore; in altri processori VLIW, il codice può venire eseguito correttamente anche su modelli diversi, ma con prestazioni così basse da rendere di fatto necessaria una ricompilazione del codice.

Molti processori superscalari estendono la versione base dell'unità che gestisce la parallelizzazione dell'esecuzione in modo da includere un **riordinamento dinamico delle istruzioni** (schedulazione dinamica, *dynamic scheduling*). Questo schema permette di decidere quali istruzioni eseguire in un dato ciclo di clock e, allo stesso tempo, di evitare hazard e stalli. Cominciamo con un semplice esempio di hazard sui dati e consideriamo il seguente frammento di codice:

Riordinamento dinamico del codice: supporto hardware al riordino delle istruzioni; viene utilizzato per evitare gli stalli.

```
lw   x31, 0(x21)
add  x1,  x31, x2
sub  x23, x23, x3
andi x5,  x23, 20
```

Anche se l'istruzione sub è pronta per essere eseguita, deve aspettare che l'esecuzione di lw e add sia stata completata, il che può richiedere parecchi cicli di clock se la memoria è lenta (nel Capitolo 5 parleremo delle miss delle cache, che sono il motivo per cui l'accesso alla memoria a volte può essere estremamente lento). La riorganizzazione dinamica del codice nelle **pipeline** permette di evitare questi hazard completamente o parzialmente.

Riorganizzazione dinamica del codice in una pipeline

Unità di consegna: in una pipeline che esegue il codice non nell'ordine scritto, è l'unità che decide quando si può consegnare il risultato di un'istruzione scrivendolo nei registri o nella memoria visibili al programmatore.

Stazione di prenotazione: buffer di un'unità funzionale che conserva gli operandi e l'operazione.

Buffer di riordino: il buffer che conserva i risultati prodotti da un processore con riordinamento dinamico del codice finché non è certo che essi possano essere scritti in memoria o nei registri.

Nella riorganizzazione dinamica del codice si sceglie quali istruzioni eseguire nei cicli di clock successivi, eventualmente riordinandole in modo da evitare stalli. In un processore di questo tipo la pipeline viene suddivisa in tre unità principali: un'unità che preleva l'istruzione dalla memoria e la avvia all'esecuzione, un gruppo di più unità funzionali (una decina o più nei processori di fascia alta nel 2020), e un'**unità di consegna** (*commit unit*). La **Figura 4.73** mostra il modello corrispondente. La prima unità preleva le istruzioni, le decodifica e invia ciascuna di esse alla corrispondente unità funzionale per l'esecuzione. Ciascuna unità funzionale è dotata di buffer, chiamati **stazioni di prenotazione** (*reservation stations*), che conservano gli operandi e l'operazione (nella sezione *Approfondimento* esamineremo una soluzione alternativa alle stazioni di prenotazione che viene impiegata in molti processori recenti). Non appena la stazione di prenotazione contiene tutti gli operandi richiesti e l'unità funzionale è pronta per l'esecuzione, viene eseguita l'operazione e inviato il risultato alle stazioni di prenotazione che lo stanno attendendo e contemporaneamente all'unità di consegna. Questa conserva il risultato dell'operazione fino a quando non avrà il via libera per salvarlo nel register file o, nel caso di un'istruzione di store, nella memoria. Il buffer presente nell'unità di consegna è chiamato **buffer di riordino** (*reorder buffer*) e viene anche utilizzato per fornire gli operandi, un po' come faceva la logica di propagazione nelle pipeline con smistamento statico delle istruzioni. Una volta scritto il risultato nel register file, esso può essere prelevato da lì come in una normale pipeline.

La combinazione del salvataggio degli operandi nei buffer delle stazioni di prenotazione e dei risultati nel buffer di riordino consente una forma di ridenominazione dei registri del tutto analoga a quella utilizzata dal compilatore

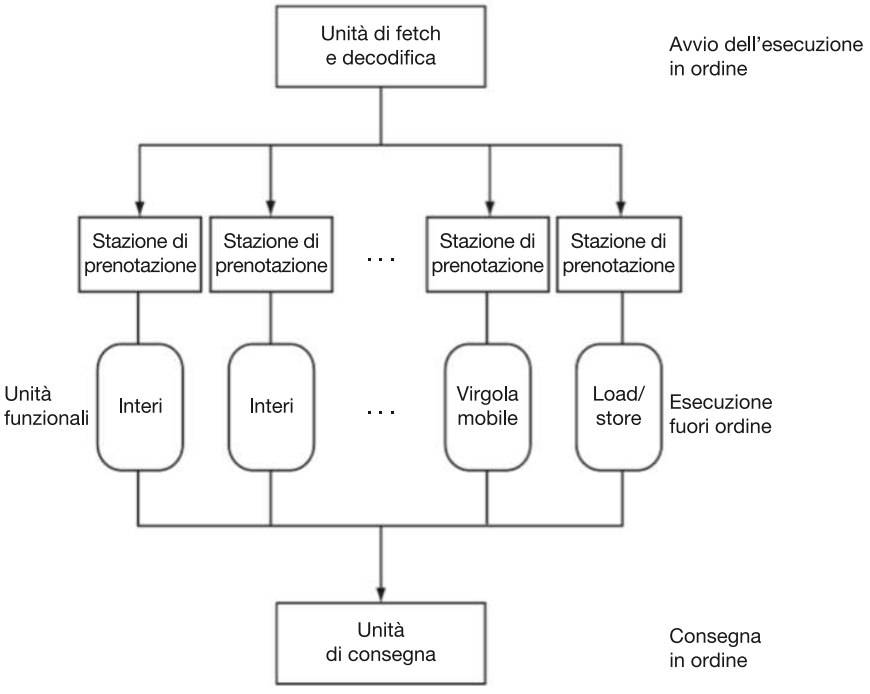


Figura 4.73 Le tre unità fondamentali di una pipeline per il riordinamento dinamico del codice. Il passo finale di aggiornamento dello stato viene chiamato *retirement* (pensionamento) oppure *graduation* (laurea).

nel caso dell'esempio precedente. Per comprenderne meglio il funzionamento da un punto di vista concettuale, consideriamo i seguenti passi.

1. Quando inizia l'esecuzione di un'istruzione, essa viene copiata nella stazione di prenotazione associata all'unità funzionale appropriata e i suoi operandi disponibili copiati dal register file o dal buffer di riordino. L'istruzione rimane all'interno della stazione di prenotazione finché tutti gli operandi non saranno disponibili e l'unità funzionale non sarà pronta. Non è più necessario copiare dai registri gli operandi per l'istruzione che viene avviata all'esecuzione; inoltre, se occorre scrivere i registri che contengono gli operandi, il loro contenuto può essere tranquillamente sovrascritto.
2. Se un operando non si trova nel register file o nel buffer di riordino, l'istruzione deve aspettare che esso sia prodotto da una delle unità funzionali. In questo caso, il lavoro di quell'unità funzionale viene seguito e il risultato prodotto viene copiato direttamente dall'unità funzionale nella stazione di prenotazione, senza aspettare che venga scritto nel registro di destinazione.

In questi due passaggi si utilizzano le stazioni di prenotazione e il buffer di riordino per implementare la ridenominazione dei registri.

Concettualmente, possiamo pensare a una pipeline a riordinamento dinamico del codice come a un'entità che analizza la struttura del flusso dei dati di un programma e poi esegue le istruzioni in un ordine che può anche essere diverso, preservando però il flusso dei dati prodotti. Questo stile di esecuzione è chiamato **esecuzione fuori ordine** (*out-of-order execution*), poiché le istruzioni possono essere eseguite in un ordine diverso da quello con cui erano state prelevate dalla memoria istruzioni.

Esecuzione fuori ordine: la situazione che si verifica in una pipeline quando l'esecuzione di un'istruzione risulta bloccata e le altre istruzioni non dipendenti possono continuare a essere eseguite.

Per fare in modo che i programmi si comportino come se le istruzioni fossero eseguite in ordine, l'unità di fetch e decodifica invia in esecuzione le istruzioni nella sequenza originale, cosa che consente di identificare le dipendenze, e quella di consegna dovrà scrivere i risultati nei registri o in memoria seguendo l'ordine con cui il programma ha prelevato le istruzioni dalla memoria. Questo modello conservativo di esecuzione è detto **completamento in ordine** (*in order commit*). Quindi, nel caso in cui si verifichi un'eccezione, il calcolatore può fare riferimento all'ultima istruzione eseguita, e gli unici registri aggiornati saranno solo quelli scritti dalle istruzioni che precedono l'istruzione che ha generato l'eccezione. Sebbene la parte iniziale della pipeline (fetch e avvio all'esecuzione) e la parte terminale (consegna) eseguano il codice in ordine, le unità funzionali sono invece libere di iniziare e terminare l'esecuzione non appena i dati di cui hanno bisogno si rendono disponibili. Oggi tutte le pipeline dotate di riordinamento dinamico del codice utilizzano il completamento in ordine.

Completamento in ordine: una modalità di consegna dei risultati in cui i risultati dell'esecuzione in una pipeline vengono scritti nei registri visibili al programmatore nello stesso ordine con cui le istruzioni associate erano state prelevate.

Il riordinamento dinamico del codice viene spesso esteso includendo la speculazione hardware, specialmente per i salti condizionati. Prevedendo la direzione di un salto, un processore con riorganizzazione dinamica del codice può prelevare ed eseguire le istruzioni lungo il ramo di esecuzione scelto. Poiché le istruzioni vengono consegnate in ordine, possiamo capire se un salto è stato previsto correttamente prima che venga consegnato il risultato di una qualsiasi istruzione del ramo di esecuzione scelto. Una pipeline speculativa dotata di riordinamento dinamico del codice può anche supportare la speculazione sugli indirizzi delle load, permettendo il riordino delle istruzioni di load-store; l'unità di consegna verrà utilizzata per evitare di scrivere i risultati prodotti dalle istruzioni eseguite quando la speculazione si rivela errata. Nel prossimo paragrafo esamineremo in che modo si utilizza la riorganizzazione dinamica del codice unita alla speculazione nell'architettura Core i7 di Intel.

Interfaccia hardware/software

L'esecuzione fuori ordine crea dei nuovi hazard all'interno della pipeline che non avevamo incontrato nelle pipeline precedenti. Una *dipendenza per nome* (*name dependence)* si verifica quando due istruzioni utilizzano lo stesso registro o la stessa locazione di memoria, chiamata *nome,* ma non esiste uno scambio di dati tra le istruzioni associate a questo *nome.* Ci sono due tipi di dipendenza per nome tra un'istruzione, i, che precede e un'istruzione, j, che segue:

1. Una *anti-dipendenza* tra l'istruzione i e l'istruzione j si verifica quando l'istruzione j scrive un registro o una locazione di memoria che viene letta dall'istruzione i. La sequenza di esecuzione originale deve essere preservata per garantire che i legga il valore corretto;
2. Una *dipendenza di output* si verifica quando le istruzioni i e j scrivono lo stesso registro o locazione di memoria. La sequenza di esecuzione delle due istruzioni deve essere preservata per garantire che il valore contenuto nel registro o nella cella di memoria sia quello scritto dall'istruzione j.

Gli hazard che si potevano verificare nella pipeline originale erano il risultato di quella che viene definita una *dipendenza vera sui dati.*

Ad esempio, nel codice riportato sotto esiste un'anti-dipendenza tra `swc1` e `addiu` sul registro x1 e una dipendenza vera sui dati tra `lwc1` e `add.s` sul registro `f0`. Anche se non ci sono dipendenze di output tra le istruzioni all'interno dello stesso ciclo, si possono verificare sulla stessa istruzione in due cicli di clock diversi; per esempio, tra le istruzioni `addiu` della prima e seconda iterazione.

```
Ciclo:  lwc1, f0, (0x1)     // f0 = elemento del vettore
        add.s f4, f0, f2    // somma lo scalare in $2
        swc1, f4, (0x1)     // salva il risultato
        addiu x1, x1, 4     // decrementa il puntatore di 4 byte
        bne x1, x2, Ciclo   // salta se x1 != x2
```

Un hazard della pipeline si verifica quando esiste una dipendenza per nome o sui dati tra istruzioni, e le istruzioni sono sufficientemente vicine da avere una esecuzione sovrapposta che va a modificare l'ordine di accesso agli operandi coinvolti nella dipendenza. Questo ha portato ad associare dei nomi intuitivi a questi hazard della pipeline:

1. Una *anti-dipendenza* può causare un hazard di Scrittura-Dopo-Lettura (WAR, Write After Read);
2. Una *dipendenza di output* può causare un hazard di Scrittura-Dopo-Scrittura (WAW, Write After Write);
3. Una dipendenza vera sui dati viene chiamata hazard di Lettura-Dopo-Scrittura (RAW, Read After Write).

Non avevamo hazard di tipo WAR o WAW nelle precedenti pipeline perché tutte le istruzioni erano eseguite in ordine e le scritture si verificavano solamente nell'ultimo stadio della pipeline per le istruzioni registro-registro e nello stesso stadio della pipeline per l'accesso ai dati in memoria mediante le istruzioni di load e store.

Capire le prestazioni dei programmi

Dato che i compilatori possono riordinare il codice anche quando ci sono dipendenze tra i dati, potreste chiedervi perché un processore superscalare debba utilizzare il riordinamento dinamico del codice. I motivi sono principalmente tre. In primo luogo, non tutti gli stalli possono essere predetti. In particolare, le miss delle cache (Capitolo 5) nella **gerarchia delle memorie** possono causare degli stalli non prevedibili. La riorganizzazione dinamica del codice consente al processore di nascondere alcuni di questi stalli continuando a eseguire altre istruzioni in attesa che lo stallo dovuto alla lettura abbia termine.

Il secondo motivo è che se il processore speculasse sulla direzione dei salti condizionati utilizzando la **predizione** dinamica dei salti, non potrebbe comunque sapere qual è l'ordine corretto di esecuzione delle istruzioni all'atto della compilazione, poiché questo dipende dal comportamento predetto e da quello effettivo del salto, durante l'esecuzione. Incorporare la speculazione dinamica per sfruttare

un maggiore *parallelismo a livello di istruzioni* (ILP) senza incorporare il riordino dinamico del codice ridurrebbe grandemente i benefici della speculazione.

Infine, poiché la latenza della pipeline e il numero di istruzioni che vengono lanciate in esecuzione in parallelo possono variare molto da un processore all'altro, il modo migliore per compilare una sequenza di istruzioni può cambiare. Per esempio, il modo in cui viene ordinata una sequenza di istruzioni dipendenti è influenzato sia dal numero di istruzioni lanciate in esecuzione in parallelo sia dalla loro latenza. La struttura della pipeline influenza sia il numero di volte che un ciclo deve essere espanso per evitare stalli, sia il processo di ridenominazione dei registri da parte del compilatore. Il riordinamento dinamico del codice consente all'hardware di nascondere la maggior parte di questi dettagli, per cui gli utenti e i distributori di software non devono preoccuparsi di avere versioni diverse dello stesso programma per le differenti implementazioni hardware dello stesso insieme di istruzioni. Analogamente, i vecchi programmi godranno dei miglioramenti delle nuove architetture senza che debba essere ricompilato il codice.

QUADRO D'INSIEME

Sia la **pipeline** sia l'esecuzione parallela incrementano il throughput di picco delle istruzioni e tentano di sfruttare il **parallelismo** a livello di istruzioni (ILP). Le dipendenze tra i dati e sul controllo nei programmi pongono, tuttavia, un limite superiore a prestazioni elevate, poiché il processore a volte deve aspettare che le dipendenze vengano risolte. Gli approcci basati sul software si affidano all'abilità del compilatore nel trovare e ridurre gli effetti di queste dipendenze, mentre gli approcci basati sull'hardware si affidano a estensioni della pipeline e a meccanismi hardware di parallelizzazione. La speculazione, attuata dal compilatore o dall'hardware, può incrementare, attraverso la **predizione**, la parte di ILP che può essere sfruttata, benché si debba fare attenzione alle speculazioni sbagliate che hanno conseguenze assai dannose sulle prestazioni. ■

Interfaccia hardware/software

I microprocessori moderni ad alte prestazioni sono capaci di lanciare in esecuzione molte istruzioni a ogni ciclo di clock; sfortunatamente, mantenere costante questo tasso di riempimento dei pacchetti di istruzioni è molto difficile. Per esempio, malgrado esistano processori con un numero di istruzioni per pacchetto di esecuzione che va da quattro a sei, ben poche applicazioni riescono a sostenere più di due istruzioni per ciclo di clock. I motivi sono principalmente due.

Innanzitutto, nelle pipeline il collo di bottiglia principale è costituito dalle dipendenze che non possono essere eliminate e che riducono il parallelismo tra le istruzioni e quindi il tasso di riempimento dei pacchetti di istruzioni. Anche se si può fare poco contro le dipendenze vere, spesso il compilatore o l'hardware non sono in grado di stabilire con precisione se una dipendenza sia reale o meno, e quindi devono ipotizzare, in modo conservativo, che la dipendenza ci sia. Per esempio, il codice basato sui puntatori, soprattutto quando puntano ad aree di memoria parzialmente sovrapposte, porta a dipendenze più complesse da analizzare. Al contrario, la maggiore regolarità dell'accesso agli elementi dei vettori consente spesso al compilatore di capire quando non esistono dipendenze reali. Analogamente, i salti condizionati che non possono essere predetti accuratamente durante l'esecuzione, o durante la compilazione, limitano fortemente la possibilità di sfruttare l'ILP. Spesso sarebbe possibile utilizzare un ILP maggiore, ma l'abilità del compilatore o dell'hardware nel trovare istruzioni che possono essere molto distanti, a volte anche migliaia di istruzioni, è limitata.

In secondo luogo, le perdite che scaturiscono dalla **gerarchia delle memorie** (Capitolo 5) limitano anch'esse la capacità di mantenere la pipeline sempre piena. In alcuni sistemi di memoria gli stalli possono essere nascosti, ma un ILP minore limita anche la capacità di nascondere gli stalli.

Efficienza energetica e pipeline avanzate

L'incremento del parallelismo a livello di istruzioni mediante parallelizzazione dinamica del codice ha come rovescio della medaglia il potenziale abbassamento dell'efficienza energetica. Ciascuna innovazione discussa in questo paragrafo ha sfruttato il crescente numero di transistor per incrementare le prestazioni, ma spesso ciò è avvenuto in modo poco efficace. Ora che la barriera dell'energia è stata raggiunta, si cominciano a produrre architetture con più processori sullo stesso chip, nelle quali ciascun processore ha una pipeline meno profonda ed è dotato di una speculazione meno aggressiva rispetto ai precedenti.

Nonostante i processori più semplici non siano tanto veloci quanto i loro sofisticati fratelli, infatti, essi hanno prestazioni migliori per joule e possono quindi garantire prestazioni più elevate per chip quando i vincoli sul progetto sono dettati più dall'energia assorbita che dal numero di transistor.

La **Figura 4.74** mostra il numero di stadi di pipeline, l'ampiezza del pacchetto di istruzioni, il livello di speculazione, la frequenza di clock, il numero di core per singolo chip e la potenza assorbita di alcuni processori vecchi e recenti. Si noti la diminuzione del numero di stadi di pipeline e della potenza dissipata con l'avvento dei microprocessori multicore.

Approfondimento. Un'unità di consegna controlla l'aggiornamento del register file e della memoria. Alcuni processori dotati di riordinamento dinamico del codice aggiornano il register file immediatamente durante l'esecuzione, utilizzando dei registri aggiuntivi per implementare la ridenominazione e preservare il vecchio contenuto di questi registri fintanto che l'istruzione che ha scritto il register file non viene considerata corretta. Altri processori, invece, memorizzano il risultato in buffer dedicati, che, come abbiamo visto, hanno tipicamente la struttura di un buffer di riordino, e l'aggiornamento effettivo del register file avviene solamente più tardi, durante la fase di consegna del risultato. In questo caso i dati scritti in memoria dalle store devono essere contenuti nel buffer fino al momento della loro consegna; questo buffer può essere un *buffer di scrittura* (Capitolo 5) oppure il buffer di riordino. L'unità di consegna consente all'istruzione di store di scrivere in memoria dal buffer non appena al buffer viene associato un indirizzo valido e contiene dati validi e le dipendenze dai salti condizionati sono state risolte.

Microprocessore	Anno	Frequenza del clock	Stadi di pipeline	Ampiezza del pacchetto	Esecuzione fuori ordine/Speculazione	Core per chip	Potenza assorbita
Intel 486	1989	25 MHz	5	1	No	1	5 W
Intel Pentium	1993	66 MHz	5	2	No	1	10 W
Intel Pentium Pro	1997	200 MHz	10	3	Sì	1	29 W
Intel Pentium 4 Willamette	2001	2000 MHz	22	3	Sì	1	75 W
Intel Pentium 4 Prescott	2004	3600 MHz	31	3	Sì	1	103 W
Intel Core	2006	2930 MHz	14	4	Sì	2	75 W
Intel Core i7 Nehalem	2008	3600 MHz	14	4	Sì	2–4	87 W
Intel Core Westmere	2010	3730 MHz	14	4	Sì	6	130 W
Intel Core i7 Ivy Bridge	2012	3400 MHz	14	4	Sì	6	130 W
Intel Core i7 Broadwell	2014	3700 MHz	14	4	Sì	10	140 W
Intel Core i9 Skylake	2016	3100 MHz	14	4	Sì	14	165 W
Intel Ice Lake	2018	4200 MHz	14	4	Sì	16	185 W

Figura 4.74 Complessità della pipeline, numero di core e potenza dissipata da alcuni microprocessori Intel. Gli stadi della pipeline del Pentium 4 non comprendono gli stadi dell'unità di consegna. Se includessimo anche questi, la profondità della pipeline del Pentium 4 sarebbe ancora maggiore.

Approfondimento. Gli accessi a memoria sono avvantaggiati dall'utilizzo di *cache non bloccanti*, che continuano a gestire gli accessi alla cache anche durante la gestione di una miss della cache (Capitolo 5). I processori con esecuzione fuori ordine hanno bisogno di circuiti aggiuntivi della cache per consentire di eseguire altre istruzioni durante una miss.

Autovalutazione

Stabilire se le seguenti tecniche e i seguenti componenti sono associati principalmente all'approccio software o hardware di sfruttamento dell'ILP. In alcuni casi la risposta corretta può essere entrambi gli approcci.

1. Predizione dei salti
2. Parallelizzazione dell'esecuzione
3. VLIW
4. Superscalare
5. Parallelizzazione dinamica
6. Esecuzione fuori ordine
7. Speculazione
8. Buffer di riordino
9. Ridenominazione dei registri

4.12 La pipeline del Core i7 Intel e del Cortex-A53 ARM

In questo paragrafo, esploriamo l'architettura di due processori multiple-issue: il Cortex A-53 ARM, che viene utilizzato come motore di molti tablet e telefoni cellulari, e il Core i7 6700 Intel, un processore di fascia alta, a schedulazione dinamica e dotato di speculazione, pensato per i desktop di fascia alta e le applicazioni su server. Iniziamo con la descrizione del processore più semplice. Questo paragrafo è basato sul Paragrafo 3.12 della sesta edizione del libro *Computer Architecture: A Quantitative Approach* degli stessi autori.

Cortex-A53 ARM

Il Cortex A-53 è un processore superscalare con due cammini di esecuzione, a schedulazione statica, con identificazione dinamica dei pacchetti di istruzioni che gli consente di avviare a esecuzione due istruzioni per ciclo di clock. La **Figura 4.75** mostra l'ossatura della sua pipeline. Per le istruzioni intere diverse dai salti condizionati sono presenti otto stadi: F1, F2, D1, D2, D3 / ISS, EX1, EX2 e WB, come descritto nella legenda della figura. La pipeline esegue le istruzioni in ordine, per cui un'istruzione può iniziare l'esecuzione solamente quando i suoi dati sono disponibili e quando le istruzioni precedenti sono già state avviate a esecuzione. Quindi, se due istruzioni consecutive sono dipendenti, possono procedere ciascuna verso il proprio cammino di esecuzione della pipeline, ma verranno serializzate quando arrivano all'inizio del cammino: solo quando la logica di controllo della pipeline indica che il risultato della prima istruzione è disponibile, la seconda istruzione può iniziare l'esecuzione.

La fase di fetch richiede quattro cicli di clock e contiene un'unità che genera l'indirizzo dell'istruzione successiva incrementando il contenuto del PC oppure scrivendo nel PC l'indirizzo prodotto da uno dei quattro predittori:

1. Una cache della destinazione dei salti con un singolo elemento, che contiene le due istruzioni che verranno eseguite successivamente (le due istruzioni

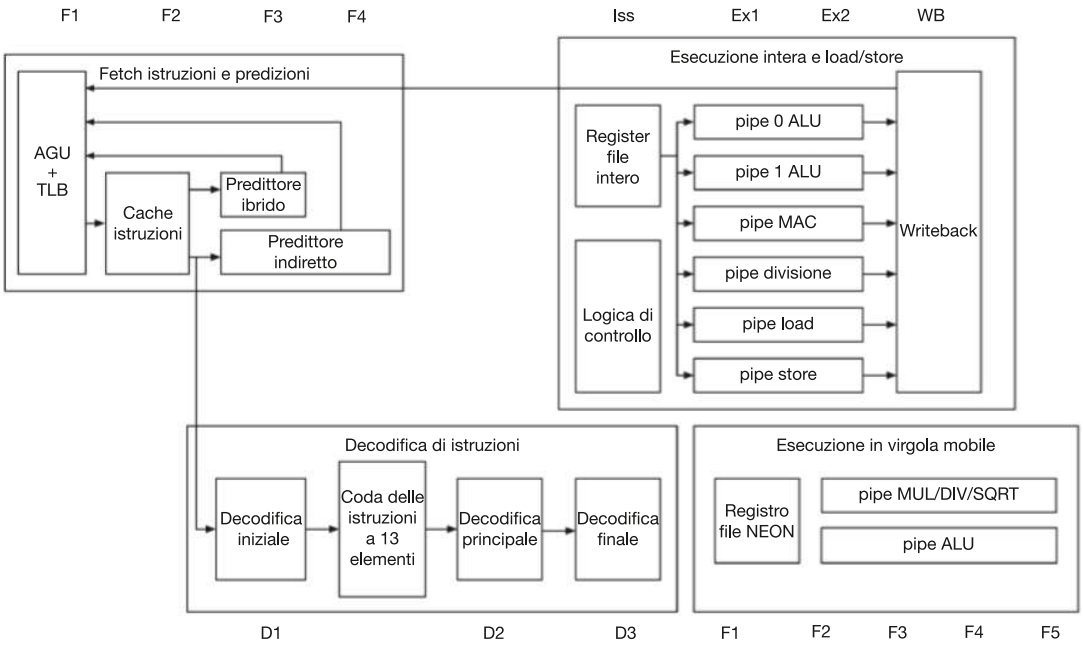


Figura 4.75 L'ossatura della pipeline intera dell'A-53 è costituita da otto stadi: F1 e F2 eseguono il fetch delle istruzioni, D1 e D2 la decodifica di base, mentre D3 esegue la decodifica delle istruzioni più complesse e si sovrappone nel tempo al primo stadio del cammino di esecuzione della pipeline (ISS). Dopo ISS, gli stadi EX1, EX2 e WB completano la pipeline per la parte di elaborazione di numeri interi. Le istruzioni di salto condizionato utilizzano quattro diversi predittori. La pipeline per l'elaborazione di numeri in virgola mobile è profonda cinque cicli di clock oltre ai cinque cicli di clock necessari per la fase di fetch e decodifica, per un totale di dieci cicli di clock. AGU sta per *Address Generation Unit* (Unità di Generazione degli Indirizzi) e TLB per Translation Lookaside Buffer (Capitolo 5). L'unità funzionale NEON esegue le istruzioni SIMD ARM che iniziano con lo stesso nome. (Da Hennessy JL, Patterson: *Computer Architecture: A Quantitative Approach*, sesta edizione, Cambridge MA, 2018, Morgan Kaufmann).

successive all'istruzione di salto condizionato, supponendo che la predizione sia corretta). Questa cache viene controllata durante il primo ciclo di clock della fase di fetch; se si verifica una hit, le due istruzioni successive vengono fornite da questa cache di destinazione dei salti. Nel caso si verifichi una hit e la predizione risulti corretta, il salto condizionato viene eseguito senza cicli di ritardo.

2. Un predittore ibrido a 3072 elementi, utilizzato per tutte le istruzioni che non producono una hit nella cache di destinazione dei salti. Questo predittore lavora nello stadio F3 e i salti condizionati gestiti da questo predittore introducono un ritardo di due cicli di clock.
3. Un predittore indiretto di salto condizionato a 256 elementi, che lavora nello stadio F4. I salti condizionati gestiti da questo predittore introducono un ritardo di tre cicli di clock quando vengono predetti correttamente.
4. Uno stack di ritorno profondo 8 elementi, che lavora nello stadio F4 e produce un ritardo di tre cicli di clock.

Le decisioni sui salti condizionati sono prese nel cammino di esecuzione 0, contenente una ALU intera; se la predizione risulta errata, la penalità associata sarà di otto cicli di clock. La **Figura 4.76** mostra la frequenza con cui avvengono le predizioni errate nei programmi del benchmark SPECint2006. La quantità di lavoro che viene persa dipende sia dalla frequenza delle predizioni errate sia dalla frequenza di invio a esecuzione dei pacchetti di istruzioni durante il ciclo di clock successivo a un salto condizionato la cui predizione è risultata errata. Come mostrato nella **Figura 4.77**, in generale, la quantità di lavoro perso segue la frequenza delle predizioni errate, anche se in alcuni casi può rivelarsi minore o maggiore.

Trabocchetto: *non tenere in giusta considerazione la struttura dell'insieme delle istruzioni può influire in modo negativo sulle pipeline.*

Molte difficoltà nel progettare le pipeline nascono dalle complicazioni causate dagli insiemi di istruzioni. Ecco alcuni esempi.

- Lunghezza e tempo di esecuzione fortemente variabili da istruzione a istruzione possono portare a sbilanciamenti tra i diversi stadi della pipeline; inoltre, complicano di molto il riconoscimento degli hazard quando la progettazione della pipeline viene effettuata sull'insieme delle istruzioni. Questo problema è stato affrontato e superato per la prima volta nei calcolatori DEC VAX 8500 degli anni '80, utilizzando uno schema basato su micro-operazioni e micropipeline simile a quello impiegato oggi dal Core i7 Intel. Ovviamente, rimane il costo aggiuntivo richiesto per tradurre e mantenere la corrispondenza tra le micro-operazioni e le istruzioni.
- Modalità di indirizzamento sofisticate possono condurre a diversi tipi di problemi. Per esempio, modalità di indirizzamento che aggiornano i registri complicano il riconoscimento degli hazard. Altre modalità di indirizzamento, che richiedono accessi multipli alla memoria, rendono significativamente più complessa l'unità di controllo della pipeline e rendono difficile ottenere un flusso costante delle istruzioni nella pipeline.
- L'esempio migliore è forse quello dei DEC Alpha e dei DEC NVAX. Nonostante i due processori fossero basati su tecnologie confrontabili, il nuovo insieme di istruzioni Alpha consentì un'implementazione hardware le cui prestazioni erano doppie in velocità rispetto a quelle del NVAX. Un altro esempio è fornito da Bhandarkar e Clark (1991) che confrontarono le prestazioni del MIPS M/2000 con quelle del VAX 8700 contando i cicli di clock relativi ai benchmark SPEC; essi trovarono che, benché il MIPS M/2000 eseguisse un maggior numero di istruzioni, il VAX utilizzava in media un numero di cicli di clock pari a 2,7 volte quello del MIPS; il MIPS risultò quindi più veloce.

4.16 Note conclusive

La saggezza consiste per nove decimi nell'essere saggi al momento giusto.
Proverbio popolare americano

Come abbiamo visto in questo capitolo, sia l'unità di elaborazione sia l'unità di controllo di un processore possono essere progettate a partire dall'architettura dell'insieme delle istruzioni e da una conoscenza delle caratteristiche di base della tecnologia. In particolare, nel Paragrafo 4.3 abbiamo visto come l'unità di elaborazione dei processori RISC-V possa essere costruita a partire dall'architettura delle istruzioni e dalla decisione di costruire un processore che esegua un'istruzione per ogni ciclo di clock. Naturalmente la tecnologia utilizzata influenza molte delle scelte di progetto, suggerendo quali componenti possano essere impiegati nell'unità di elaborazione e se l'implementazione a singolo ciclo abbia senso o meno.

L'utilizzo delle **pipeline** aumenta il throughput, ma non il tempo di esecuzione intrinseco delle singole istruzioni o **latenza dell'istruzione**; per alcune istruzioni, tale latenza ha un valore simile a quello dell'approccio a singolo ciclo di clock. L'esecuzione di più istruzioni in parallelo richiede l'aggiunta di altro hardware in modo tale da poter avviare a esecuzione più istruzioni per ogni ciclo di clock al prezzo di un aumento della latenza effettiva della singola istruzione. La pipeline è stata introdotta per ridurre il tempo di esecuzione rispetto all'unità di elaborazione a singolo ciclo di clock, mentre il parallelismo dell'esecuzione punta chiaramente a ridurre il *numero di cicli di clock per istruzione* (CPI).

PIPELINE

Latenza dell'istruzione: il tempo di esecuzione intrinseco di un'istruzione.

La pipeline e il parallelismo dell'esecuzione tentano entrambi di sfruttare il parallelismo a livello di istruzioni. Le dipendenze sui dati e sul controllo (che possono diventare hazard) rappresentano la limitazione principale allo sfruttamento del parallelismo. Il riordinamento del codice e la speculazione, mediante **predizione** sia hardware sia software, sono le tecniche principali utilizzate per ridurre l'impatto delle dipendenze sulle prestazioni.

Abbiamo mostrato che l'espansione del ciclo della funzione DGEMM di 4 volte consente di esporre più istruzioni che possono essere sfruttate dal motore di esecuzione fuori ordine del Core i7 per ottenere più del raddoppio delle prestazioni.

Il passaggio a pipeline più profonde, alla parallelizzazione dinamica dell'esecuzione e al riordinamento dinamico del codice, avvenuto a metà degli anni '90, ha contribuito a mantenere il miglioramento annuale delle prestazioni del 60%, iniziato a partire dagli anni '80. Come si è detto nel Capitolo 1, i microprocessori hanno mantenuto il modello sequenziale di scrittura del codice, ma si sono scontrati con la barriera dell'energia. Perciò, l'industria è stata costretta a sperimentare i multiprocessori, che sfruttano il parallelismo a un livello più elevato (Capitolo 6). Questa tendenza ha anche richiesto che i progettisti rivalutassero le implicazioni sul rapporto potenza-prestazioni di alcune delle invenzioni introdotte a partire dalla metà degli anni '90; questo ha portato a una semplificazione delle pipeline nelle versioni più recenti delle microarchitetture.

Considerato il tasso di miglioramento delle prestazioni nei processori paralleli, la legge di Amdahl suggerisce che un'altra parte del sistema diventerà il collo di bottiglia. Il collo di bottiglia in questione è l'argomento del prossimo capitolo: la **gerarchia delle memorie**.

4.17 Inquadramento storico e approfondimenti

Questo paragrafo, disponibile online, tratta la storia dei primi processori dotati di pipeline e dei primi processori superscalari. Inoltre, illustra lo sviluppo delle tecniche di esecuzione fuori ordine e di speculazione e gli importanti sviluppi nella tecnologia dei compilatori.

4.18 Aiuto allo studio

Mentre le pipeline dei processori di fascia alta sono molto più lunghe di cinque stadi, alcuni processori a bassissimo costo o a basso consumo energetico hanno pipeline più corte. Si supponga la stessa temporizzazione dei componenti funzionali posti sul cammino di elaborazione delle Figure 4.43 e 4.44.

Pipeline a tre stadi. In quali stadi si potrebbe suddividere il cammino di esecuzione di una pipeline che abbia tre stadi invece di cinque?

Frequenza del clock. Ignorando l'impatto sulla durata del periodo del clock dei registri di pipeline e della logica di propagazione, quale sarebbe la frequenza di clock di una pipeline a cinque stadi e di una a tre stadi? Si supponga la stessa temporizzazione dei componenti funzionali posti sul cammino di elaborazione delle Figure 4.43 e 4.44.

Hazard sulla lettura/scrittura dei dati nei registri? Esistono ancora questi hazard nella pipeline a tre stadi? In caso affermativo, la propagazione può risolvere il problema?

Hazard sui dati nelle load? Esistono ancora questi hazard nella pipeline a tre stadi? Occorre mettere in stallo la pipeline oppure si può utilizzare la propagazione per risolvere il problema?

Hazard sul controllo? Esistono ancora questi hazard nella pipeline a tre stadi? In caso affermativo, come si può ridurre il loro impatto?

CPI. Il clock della pipeline a tre stadi avrà un periodo maggiore o minore di quello della pipeline a cinque stadi?

Risposte ai quesiti

Pipeline a tre stadi. Anche se ci sono molte soluzioni possibili, una suddivisione ragionevole è la seguente:

1. Fetch delle istruzioni, lettura dei registri (300 ps);
2. ALU (200 ps);
3. Accesso ai dati, scrittura nei registri (300 ps).

Frequenza del clock. La Figura 4.44 mostra che il periodo del clock della pipeline a cinque stadi è di 200 ps, per cui la frequenza di clock è di 1/(200 ps) cioè 5 GHz. Lo stadio peggiore per questa pipeline a tre stadi dura 300 ps per cui la frequenza di clock sarà di 1/(300 ps) cioè 3,33 GHz.

Hazard sulla lettura/scrittura dei dati nei registri. Come si può vedere dallo schema della pipeline mostrato in precedenza, esistono ancora gli hazard di lettura/scrittura. La prima istruzione scrive il dato nel registro nel terzo stadio ma l'istruzione successiva ha bisogno di quel dato nel suo secondo stadio. La soluzione basata sulla propagazione descritta nel Paragrafo 4.8 funziona bene per la pipeline a tre stadi, poiché il risultato della ALU dell'istruzione precedente è pronto prima dell'inizio del secondo stadio dell'istruzione che ha bisogno di quel dato.

Hazard sui dati nelle load. Anche nel caso di pipeline a tre stadi occorre mettere in stallo la pipeline per un ciclo di clock quando si verifica un hazard su una load come mostrato nel Paragrafo 4.8. Infatti il dato non è disponibile prima del terzo stadio dell'istruzione di load, ma l'istruzione successiva ha bisogno di quel dato all'inizio del secondo stadio.

Hazard sul controllo. Questo hazard è dove la pipeline a tre stadi brilla. Si può utilizzare la stessa ottimizzazione descritta nel Paragrafo 4.9 per calcolare l'indirizzo di salto e verificare se il contenuto dei due registri è uguale, prima dello stadio di esecuzione della ALU, come mostrato nella Figura 4.62. Questo calcolo viene eseguito prima del fetch dell'istruzione successiva per cui la logica dei salti posizionata all'inizio risolve gli hazard sul controllo senza penalità.

CPI. Il numero medio di cicli di clock per istruzione diminuirà nella pipeline a tre stadi per le seguenti ragioni.

- Dato che il periodo di clock è maggiore, occorreranno meno cicli di clock per accedere alla memoria DRAM, e questo influenzerà il CPI quando si verificherà una miss nella cache (Capitolo 5).

- I salti condizionati verranno sempre eseguiti in un ciclo di clock, mentre qualsiasi schema software o hardware il cui obiettivo sia accelerare, su una pipeline a cinque stadi, l'esecuzione dei salti condizionati, fallirà in alcuni casi, aumentando così il CPI effettivo.
- Il periodo del clock è più lungo per lo stadio della ALU; questo può consentire di eseguire nella pipeline a cinque stadi alcune operazioni complesse che potrebbero richiedere più di un ciclo di clock. Per esempio la moltiplicazione o divisione intera potrebbero richiedere un numero minore di questi cicli di clock più lunghi rispetto alla pipeline a cinque cicli di clock.

4.19 Esercizi

4.1 Si consideri la seguente istruzione:

Istruzione: `and rd, rs1, rs2`
Interpretazione: `Reg[rd] = Reg[rs1] AND reg[rs2]`

4.1.1 [5] <4.3> Determinare il valore dei segnali di controllo generati per questa istruzione dall'unità di controllo della Figura 4.10.

4.1.2 [5] <4.3> Quali risorse (blocchi funzionali) eseguono il lavoro utile per l'esecuzione di questa istruzione?

4.1.3 [10] <4.3> Quali risorse (blocchi funzionali) producono comunque un output, anche se il loro output non viene poi utilizzato dall'istruzione? Quali risorse non producono alcun output per questa istruzione?

4.2 [10] <4.4> Spiegare il significato di tutti i termini "indifferente" riportati nella Figura 4.22.

4.3 Si consideri la seguente combinazione di istruzioni:

Tipo R	Tipo I (non load)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

4.3.1 [5] <4.4> Quale percentuale di tutte le istruzioni fa uso della memoria dati?

4.3.2 [5] <4.4> Quale percentuale di tutte le istruzioni fa uso della memoria istruzioni?

4.3.3 [5] <4.4> Quale percentuale di tutte le istruzioni fa uso dell'estensione del segno?

4.3.4 [5] <4.4> Cosa fa l'estensione del segno nei cicli nei quali il suo output non serve?

4.4 Quando i chip di silicio vengono fabbricati, difetti nei materiali (per es. nel silicio) ed errori di fabbricazione possono fare sì che i circuiti risultino difettati. Un difetto molto comune è la rottura di una pista: il segnale da essa trasportato rimane sempre bloccato a 0, indipendentemente da quale sia il valore logico che dovrebbe assumere. Questo viene spesso chiamato collegamento forzato a 0 ("fault stuck-at-0")

4.4.1 [5] <4.4> Quali istruzioni non funzionerebbero più correttamente se il bus del segnale `MemtoReg` fosse bloccato a zero?

4.4.2 [5] <4.4 Quali istruzioni non funzionerebbero più correttamente se il bus del segnale `ALUSrc` fosse bloccato a zero?

4.5 In questo esercizio verrà esaminato in dettaglio come viene eseguita un'istruzione da un'unità di elaborazione a singolo ciclo. I problemi di questo esercizio si riferiscono al ciclo di clock nel quale il processore carica dalla memoria istruzioni la seguente parola: `0x00c6ba23`.

4.5.1 [10] <4.4> Qual è il valore degli ingressi dell'unità di controllo della ALU per questa istruzione?

4.5.2 [5] <4.4> Qual è il nuovo indirizzo del PC dopo che l'istruzione è stata eseguita? Evidenziare il cammino che calcola il nuovo indirizzo.

4.5.3 [10] <4.4> Mostrare il valore fornito in ingresso e in uscita da ciascun multiplexer durante l'esecuzione di questa istruzione. Riportare il valore in uscita a `Reg[xn]`.

4.5.4 [10] <4.4> Determinare quali siano i dati in ingresso alla ALU e ai due addizionatori.

4.5.5 [10] <4.4> Determinare tutti i valori in ingresso al register file.

4.6 Il Paragrafo 4.4 non tratta le istruzioni di tipo I, quali `addi` e `andi`.

Mem-I/ Mem-D	Register File	Mux	ALU	Addiz	Porta logica singola	Lettura Registro	Impostazione Registro	Estensione Segno	Controllo
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

4.6.1 [5] <4.4> Quali blocchi logici aggiuntivi sono eventualmente necessari per aggiungere le istruzioni di tipo I alla CPU mostrata nella Figura 4.21? Aggiungere i blocchi eventualmente necessari alla Figura 4.21 e spiegare la loro funzione.

4.6.2 [10] <4.4> Riportare i valori dei segnali generati dall'unità di controllo per le istruzioni di `addi`. Spiegare il ragionamento che sta dietro alla specifica di "indifferente" per alcuni segnali di controllo.

4.7 Nei problemi di questo esercizio si suppone che i blocchi logici richiesti per implementare l'unità di elaborazione di un processore abbiano la latenza riportata nella tabella riportata in testa alla pagina. "Lettura Registro" si riferisce al tempo che intercorre tra il fronte di salita del clock e l'istante in cui il valore contenuto nel registro compare all'uscita del registro stesso. Questo tempo si applica solo al program counter. "Impostazione Registro" si riferisce al tempo per cui il dato in input a un registro deve rimanere stabile prima del fronte di salita del clock. Questo tempo si applica sia al PC sia al Register File.

4.7.1 [5] <4.4> Qual è la latenza di un'istruzione di tipo R (cioè quanto deve durare il periodo del clock per essere sicuri che l'istruzione venga eseguita correttamente)?

4.7.2 [10] <4.4> Qual è la latenza di un'istruzione di `lw`? (Fate attenzione: molti studenti inseriscono dei multiplexer in più sul cammino critico).

4.7.3 [10] <4.4> Qual è la latenza di un'istruzione di `sw`? (Fate attenzione: molti studenti inseriscono dei multiplexer in più sul cammino critico).

4.7.4 [5] <4.4> Qual è la latenza di un'istruzione di `beq`?

4.7.5 [5] <4.4> Qual è la latenza di un'istruzione aritmetica, logica o di shift (non di load) di tipo I?

4.7.6 [5] <4.4> Qual è il periodo di clock minore possibile per questo PC?

4.8 [10] Si supponga che si possa costruire una CPU in cui il periodo di clock sia diverso per le diverse istruzioni. Quale sarà lo l'incremento di velocità (speed-up) di questa CPU rispetto alla CPU presentata nella Figura 4.21 per la combinazione di istruzioni qui riportata?

Tipo R/Tipo I (non lw)	lw	sw	beq
52%	25%	11%	12%

4.9 Si supponga di aggiungere un moltiplicatore alla CPU mostrata nella Figura 4.25. Questo moltiplicatore aggiungerebbe 300 ps alla latenza della ALU, ma ridurrebbe il numero di istruzioni del 5% (perché non ci sarebbe più la necessità di emulare le istruzioni di moltiplicazione).

4.9.1 [5] <4.4> Quale sarebbe il periodo di clock con e senza questo miglioramento?

4.9.2 [10] <4.4> Quale sarebbe l'incremento di velocità ottenuto con questo miglioramento?

4.9.3 [10] <4.4> Di quanto potrebbe essere più lenta al massimo la nuova ALU, pur mantenendo un miglioramento delle prestazioni?

4.10 Quando i progettisti dei processori valutano un possibile miglioramento dell'unità di elaborazione, la decisione finale dipende di solito dal rapporto costo/prestazioni. Nei prossimi tre problemi si supponga di partire dall'unità di elaborazione della Figura 4.25, dalle latenze dell'Esercizio 4.7 e dai costi presenti nella tabella riportata in fondo alla pagina. Si consideri che raddoppiare il numero di registri di uso generico da 32 a 64 possa ridurre il numero di istruzioni di `lw` e `sw` del 12% ma aumenti la latenza del register file da 150 ps a 160 ps e raddoppi il costo da 200 a 400. (Si utilizzi la combinazione di istruzioni dell'Esercizio 4.8).

4.10.1 [5] <4.4> Qual è l'incremento di velocità ottenuto con questo miglioramento?

4.10.2 [10] <4.4> Confrontare le variazioni in termini di velocità con l'incremento in termini di costo.

Mem-I	Register File	Mux	ALU	Addiz	Mem-D	Registro Singolo	Estensione Segno	Porta logica singola	Controllo
1000	200	10	100	30	2000	5	100	1	500

4.10.3 [10] <4.4> Dati i rapporti costo/velocità appena calcolati, descrivere una situazione in cui abbia senso aggiungere altri registri e descriverne un'altra in cui non abbia senso aggiungere altri registri.

4.11 Esaminare la difficoltà ad aggiungere un'istruzione di `lwi.d rd, rs1, rs2` ("Load con incremento") al RISC-V.

Interpretazione:

```
Reg[rd] = Mem[Reg[rs1]+Reg[rs2]]
```

4.11.1 [5] <4.4> Quali nuovi blocchi funzionali servirebbero eventualmente per questa nuova istruzione?

4.11.2 [5] <4.4> Quali blocchi funzionali esistenti dovrebbero eventualmente essere modificati per questa nuova istruzione?

4.11.3 [5] <4.4> Quali nuovi cammini interni all'unità di elaborazione dovrebbero eventualmente essere introdotti per l'esecuzione di questa nuova istruzione?

4.11.4 [5] <4.4> Quali nuovi segnali di controllo dovrebbero eventualmente essere introdotti per supportare l'esecuzione di questa nuova istruzione?

4.12 Esaminare la difficoltà ad aggiungere un'istruzione di `swap rs1, rs2` al RISC-V.

Interpretazione:

```
Reg[rs2] = Reg[rs1]; Reg[rs1] = Reg[rs2]
```

4.12.1 [5] <4.4> Quali nuovi blocchi funzionali servirebbero eventualmente per questa nuova istruzione?

4.12.2 [10] <4.4> Quali blocchi funzionali esistenti dovrebbero eventualmente essere modificati per questa nuova istruzione?

4.12.3 [5] <4.4> Quali nuovi cammini interni all'unità di elaborazione dovrebbero eventualmente essere introdotti per l'esecuzione di questa nuova istruzione?

4.12.4 [5] <4.4> Quali nuovi segnali di controllo dovrebbero eventualmente essere introdotti per supportare l'esecuzione di questa nuova istruzione?

4.12.5 [5] <4.4> Modificare la Figura 4.25 per dimostrare una possibile implementazione di questa nuova istruzione.

4.13 Esaminare la difficoltà ad aggiungere un'istruzione di `ss rs1, rs2` ("Store della Somma") al RISC-V.

Interpretazione:

```
Mem[Reg[rs1]]=Reg[rs2]+immediato
```

4.13.1 [10] <4.4> Quali nuovi blocchi funzionali servirebbero eventualmente per questa nuova istruzione?

4.13.2 [10] <4.4> Quali blocchi funzionali esistenti dovrebbero eventualmente essere modificati per questa nuova istruzione?

4.13.3 [5] <4.4> Quali nuovi cammini interni all'unità di elaborazione dovrebbero eventualmente essere introdotti per l'esecuzione di questa nuova istruzione?

4.13.4 [5] <4.4> Quali nuovi segnali di controllo dovrebbero eventualmente essere introdotti per supportare l'esecuzione di questa nuova istruzione?

4.13.5 [5] <4.4> Modificare la Figura 4.25 per dimostrare una possibile implementazione di questa nuova istruzione.

4.14 [5] <4.4> Per quali istruzioni (se ce ne sono) il blocco funzionale Genera cost si trova sul cammino critico?

4.15 L'istruzione di `lw` ha la latenza maggiore nella CPU del Paragrafo 4.4. Se modificassimo le istruzioni di `lw` e `sw` in modo che la CPU non debba calcolare l'offset (cioè che l'indirizzo da/a cui debba essere trasferito il dato sia calcolato e scritto nel registro `rs1` prima di chiamare l'istruzione di `lw/sw`), non ci sarebbero istruzioni che debbano utilizzare sia la ALU sia la memoria dati. Questo ci consentirebbe di ridurre il periodo del clock, ma aumenterebbe il numero di istruzioni dato che molte `lw` e `sd` dovrebbero essere sostituite dalla coppia di istruzioni `lw/add` o `sw/add`.

4.15.1 [5] <4.4> Quale sarebbe il nuovo periodo del clock?

4.15.2 [10] <4.4> Il programma costituito dalla combinazione di istruzioni presentata nell'Esercizio 4.7 verrebbe eseguito più velocemente o più lentamente da questa nuova CPU? Di quanto? (Per semplicità, si supponga di sostituire ciascuna istruzione di `lw` e `sw` con una coppia di istruzioni `lw/add` e `sw/add`.

4.15.3 [5] <4.4> Qual è il fattore principale che determina se un programma viene eseguito più velocemente o più lentamente sulla nuova CPU?

4.15.4 [5] <4.4> Pensate che la CPU originaria (quella mostrata nella Figura 4.25) sia un progetto complessivamente migliore oppure pensate che sia migliore la nuova CPU? Perché?

 © 978-88-08-19966-9

4.16 In questo esercizio verrà esaminato come la pipeline influenzi il periodo del clock di processore. I problemi di questo esercizio suppongono che i singoli stadi dell'unità di elaborazione abbiano la seguente latenza:

IF	ID	EX	MEM	WB
250 ps	350 ps	150 ps	300 ps	200 ps

Si assuma inoltre che le istruzioni eseguite dal processore siano suddivise come segue:

ALU/Logiche	Salti incondiz/condiz	Load	Store
45%	20%	20%	15%

4.16.1 [5] <4.6> Qual è il periodo del clock di un processore dotato di pipeline e di un processore senza pipeline?

4.16.2 [10] <4.6> Qual è la latenza totale di un'istruzione `lw` in un processore dotato di pipeline e in uno senza pipeline?

4.16.3 [10] <4.6> Se si potesse suddividere uno stadio dell'unità di elaborazione dotata di pipeline in due nuovi stadi, ciascuno avente metà latenza dello stadio originale, quale stadio sarebbe meglio suddividere e quale sarebbe la nuova durata del ciclo di clock del processore?

4.16.4 [10] <4.6> Supponendo che non ci siano stalli o hazard, qual è la percentuale di utilizzo della memoria dati?

4.16.5 [10] <4.6> Supponendo che non ci siano stalli o hazard, qual è la percentuale di utilizzo della porta di scrittura del register file?

4.17 [10] <4.6> Qual è il minimo numero di cicli di clock richiesti per portare a termine l'esecuzione di `n` istruzioni su una CPU con una pipeline a `k` stadi? Giustificare la formula ricavata.

4.18 [5] <4.6> Si supponga che `x11` venga inizializzato a 11 e `x12` venga inizializzato a 22. Si supponga di eseguire il codice riportato sotto sulla realizzazione della pipeline mostrata nel Paragrafo 4.6 che non gestisce gli hazard sui dati (cioè il programmatore è responsabile di gestire gli hazard sui dati inserendo istruzioni `NOP` quando serve). Quale sarà il contenuto finale dei registri `x13` e `x14`?

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
```

4.19 [10] <4.6> Si supponga che `x11` venga inizializzato a 11 e `x12` venga inizializzato a 22. Si supponga di eseguire il codice seguente sulla realizzazione della pipeline mostrata nel Paragrafo 4.6 che non gestisce gli hazard sui dati (cioè il programmatore è responsabile di gestire gli hazard sui dati inserendo istruzioni `NOP` quando serve). Quale sarà il contenuto finale del registro `x15`? Si supponga che il register file venga scritto all'inizio del ciclo di clock e letto alla fine di una ciclo. Quindi, lo stadio ID fornisce in uscita il contenuto dello stadio WB durante lo stesso ciclo di clock. Si esaminino il Paragrafo 4.8 e la Figura 4.53 per ulteriori dettagli.

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
add   x15, x11, x11
```

4.20 [5] <4.6> Aggiungere un'istruzione `NOP` al codice riportato sotto in modo tale che funzioni correttamente su una pipeline che non gestisce gli hazard.

```
addi  x11, x12, 5
add   x13, x11, x12
addi  x14, x11, 15
add   x15, x13, x12
```

4.21 Si consideri una realizzazione della pipeline del Paragrafo 4.6 che non gestisce gli hazard sui dati (cioè il programmatore è responsabile di gestire gli hazard sui dati inserendo istruzioni `NOP` quando serve). Si supponga che (dopo l'ottimizzazione) un programma tipico contenente `n` istruzioni richieda altre `4*n` istruzioni `NOP` per gestire correttamente gli hazard sui dati.

4.21.1 [5] <4.6> Si supponga che il periodo del ciclo di clock di questa pipeline senza la propagazione sia di 250 ps. Si supponga anche che l'aggiunta di hardware per la propagazione riduca il numero di `NOP` da `0,4*n` a `0,05*n`, ma aumenti il periodo del clock a 300 ps. Qual è l'aumento di velocità di questa nuova pipeline rispetto a quella senza la propagazione?

4.21.2 [10] <4.6> Programmi diversi richiedono un numero diverso di `NOP`. Quante `NOP` (in percentuale delle istruzioni del codice) possono rimanere prima che il tipico programma riportato sopra venga eseguito più lentamente sulla pipeline dotata di propagazione?

4.21.3 [10] <4.6> Ripetere l'Esercizio 4.21.2; tuttavia, questa volta considerare `x` il numero di `NOP` rispetto a `n` (nell'Esercizio 4.21.2, `x` era uguale a `0,4`). La risposta sarà funzione di `x`.

4.21.4 [10] <4.6> Può un programma contenente solo `0,075*n` `NOP` essere eseguito più velocemente da

una pipeline dotata di propagazione? Sia in caso affermativo che negativo, spiegare il motivo.

4.21.5 [10] <4.6> Qual è la percentuale minima, rispetto al numero di istruzioni del codice, di NOP che un programma deve avere prima che possa venire eseguito più velocemente sulla pipeline dotata di propagazione?

4.22 [10] <4.6> Si consideri il seguente frammento di codice assembler RISC-V:

```
sw      x29,12(x16)
lw      x29,8(x16)
sub     x17,x15,x14
beqz    x17,Etichetta
add     x15,x11,x14
sub     x15,x30,x14
```

Si supponga di modificare la pipeline in modo che abbia solamente una memoria (in grado di gestire sia i dati sia le istruzioni). In questo caso, si verificherà un hazard strutturale ogni volta che un programma ha bisogno di caricare un'istruzione durante lo stesso ciclo di clock in cui un'altra istruzione ha bisogno di accedere ai dati in memoria.

4.22.1 [5] <4.6> Disegnare lo schema di una pipeline e indicare dove viene messo in stallo il frammento di codice riportato sopra.

4.22.2 [5] <4.6> In generale, è possibile ridurre il numero di stalli/NOP che risultano da questo hazard strutturale riordinando il codice?

4.22.3 [5] <4.6> Questo hazard strutturale deve essere gestito dall'hardware? Abbiamo visto che gli hazard sui dati possono venire eliminati aggiungendo un'istruzione NOP al codice. Si può fare la stessa cosa con questo hazard strutturale? Se sì, spiegare come. Se no, spiegare perché.

4.22.4 [5] <4.6> Approssimativamente, quanti stalli ci si può aspettare che vengano generati da un programma tipico? (Si utilizzi la combinazione di istruzioni dell'Esercizio 4.8.)

4.23 Se modifichiamo le istruzioni di load/store per utilizzare un registro (senza l'offset) per costruire l'indirizzo, queste istruzioni non hanno più bisogno di utilizzare la ALU (Esercizio 4.15). Di conseguenza, gli stadi MEM ed EX si possono sovrapporre e la pipeline sarebbe costituita da solo quattro stadi.

4.23.1 [10] <4.6> Come viene influenzato il periodo di clock dalla riduzione della profondità della pipeline?

4.23.2 [5] <4.6> Come potrebbe questo cambiamento migliorare le prestazioni della pipeline?

4.23.3 [5] <4.6> Come potrebbe questo cambiamento degradare le prestazioni della pipeline?

4.24 [10] <4.8> Quale dei due diagrammi di esecuzione descrive meglio il funzionamento dell'unità di rilevamento degli hazard?

Scelta 1:

```
lw  x11,  0(x12):     IF ID EX ME WB
add x13,  x11, x14    IF ID EX..ME WB
or  x15,  x16, x17:   IF ID..EX ME WB
```

Scelta 2:

```
lw  x11,  0(x12):     IF ID EX ME WB
add x13,  x11, x14    IF ID..EX ME WB
or  x15,  x16, x17:   IF..ID EX ME WB
```

4.25 Si consideri il ciclo seguente:

```
CICLO:  lw    x10, 0(x13)
        lw    x11, 8(x13)
        add   x12, x10, x11
        addi  x13, x13, 16
        bnez  x12, CICLO
```

Si supponga una predizione esatta delle branch e quindi che non si verifichino stalli dovuti a hazard sul controllo. Si supponga anche che non ci siano delay slot e che la pipeline abbia un supporto completo alla propagazione. Infine, si supponga che i salti condizionati vengano risolti nello stadio EX invece che nello stadio ID.

4.25.1 [10] <4.8> Mostrare lo schema di esecuzione della pipeline per le prime due iterazioni di questo ciclo.

4.25.2 [10] <4.8> Identificare gli stadi della pipeline che non fanno lavoro utile. Con quale frequenza, espressa come percentuale del numero totale di cicli di clock, si verificano cicli di clock in cui tutti e cinque gli stadi della pipeline compiono lavoro utile? (Iniziare dal ciclo di clock in cui la addi si trova nello stadio IF e terminare con il ciclo nel quale la bnez si trova nello stadio IF.)

4.26 Questo esercizio è stato pensato per aiutarvi a capire il compromesso tra costo, complessità e prestazioni della propagazione in un processore dotato di pipeline. I problemi si riferiscono all'unità di elaborazione dotata di pipeline descritta nella Figura 4.47 supponendo che, tra tutte le istruzioni eseguite dal processore, una parte esibisca una particolare dipendenza tra i dati RAW (dati grezzi).

EX a 1° Solo	MEM a 1° Solo	EX a 2° Solo	MEM a 2° Solo	EX a 1° e EX a 2°
5%	20%	5%	10%	10%

Questa dipendenza viene identificata dallo stadio che produce il risultato richiesto (EX o MEM) e dall'istruzione che deve utilizzare questo risultato: la prima istruzione immediatamente successiva all'istruzione che produce il risultato, la seconda, o entrambe. Si suppone che la scrittura dei registri avvenga nella prima metà del periodo di clock e che la lettura avvenga nella seconda metà, per cui la dipendenza tra l'uscita dallo stadio EX e gli operandi della terza istruzione e tra l'uscita dallo stadio MEM e gli operandi della seconda istruzione non viene considerata un hazard sui dati. Inoltre, si supponga che i salti condizionati siano risolti nello stadio EX e che il CPI del processore sia uguale a 1 se non si verificano hazard sui dati (tabella in alto alla pagina). Si supponga no le latenze riportate nella tabella in fondo alla pagina per i singoli stadi della pipeline. Per lo stadio EX, la latenza viene fornita separatamente per un processore senza propagazione e per un processore dotato di diversi tipi di propagazione.

4.26.1 [5] <4.8> Per ciascuna delle dipendenze GREZZE riportate nella tabella in testa alla pagina, fornire una sequenza di almeno tre istruzioni assembler che esibiscono quella dipendenza.

4.26.2 [5] <4.8> Per ciascuna delle dipendenze GREZZE riportate nella tabella in testa alla pagina, quante `NOP` sarebbe necessario inserire per consentire che il codice dell'Esercizio 4.26.1 venga eseguito correttamente su una pipeline che non sia dotata né di propagazione né di rilevamento degli hazard? Mostrare dove si possano inserire le `NOP`.

4.26.3 [10] <4.8> Analizzando ciascuna istruzione indipendentemente, verrebbe sovrastimato il numero di `NOP` necessarie per il corretto funzionamento del programma su una pipeline non dotata né di unità di propagazione né di rilevamento degli hazard. Scrivere una sequenza di tre istruzioni assembler tale per cui la somma del numero di stalli quando si analizza ciascuna istruzione separatamente è maggiore del numero di stalli richiesti effettivamente dalla sequenza per evitare hazard sui dati.

4.26.4 [5] <4.8> Se si supponesse che non ci siano altri hazard, quale sarebbe il CPI del programma descritto nella tabella riportata sopra quando verrebbe eseguito su una pipeline senza propagazione? Quale sarebbe la percentuale di cicli in cui la pipeline è in stallo? (Per semplicità, assumere che tutti i casi possibili siano riportati nella tabella sopra e che possano essere trattati indipendentemente.)

4.26.5 [5] <4.8> Quale sarebbe il CPI se utilizzassimo la propagazione completa (propagazione di tutti i risultati che possono essere propagati)? Qual è la percentuale dei cicli di clock in cui la pipeline è in stallo?

4.26.6 [10] <4.8> Si supponga di non potersi permettere un multiplexer a tre input come quelli richiesti dalla propagazione completa. In questo caso occorre decidere se sia meglio propagare il dato dal registro di pipeline EX/MEM (propagazione dal ciclo di clock precedente), oppure solamente dal registro di pipeline MEM/WB (propagazione da due cicli di clock precedenti). Quale è il CPI per ciascuna delle due soluzioni?

4.26.7 [5] <4.8> Per la probabilità degli hazard e la latenza dei diversi stadi di pipeline riportati, quale incremento di velocità si otterrebbe con i diversi tipi di propagazione (EX/MEM, MEM/WB, completo) rispetto a una pipeline che sia sprovvista di propagazione?

4.26.8 [5] <4.8> Quale sarebbe l'incremento di velocità ulteriore (relativo al processore più veloce dell'Esercizio 4.26.7) se aggiungessimo una propagazione "attraverso il tempo" che eliminasse tutti gli hazard sui dati? Si supponga che il circuito di "propagazione attraverso il tempo", ancora da inventare, aggiunga 100 ps alla latenza dello stadio EX della pipeline dotato di propagazione completa.

4.26.9 [5] <4.8> La tabella dei tipi di hazard riporta separatamente i tempi per le condizioni da EX a 1° e da EX a 1° e 2°. Perché non ci sono valori separati per la condizione da MEM a 1° e da MEM a 2°?

IF	ID	EX (no Propag)	EX (Propag completa)	EX (Propag solo da EX/MEM)	EX (Propag solo da MEM/WB)	MEM	WB
120 ps	100 ps	110 ps	130 ps	120 ps	120 ps	120 ps	100 ps

4.27 I problemi di questo esercizio si riferiscono alla seguente sequenza di istruzioni e suppongono che il codice venga eseguito su un'unità di elaborazione dotata di pipeline a cinque stadi:

```
add  x15, x12, x11
lw   x13, 4(x15)
lw   x12, 0(x2)
or   x13, x15, x13
sd   x13, 0(x15)
```

4.27.1 [5] <4.8> Inserire opportunamente delle `NOP` per assicurare la corretta esecuzione del codice, supponendo che non ci siano né propagazione né rilevamento degli hazard.

4.27.2 [10] <4.8> Ora, modificare e/o riordinare il codice per minimizzare il numero di `NOP` richieste. Si utilizzi il registro `x17` per memorizzare il contenuto di variabili temporanee aggiunte nel codice modificato.

4.27.3 [10] <4.8> Supponendo che il processore sia dotato di un'unità di propagazione ma non di un'unità di rilevamento degli hazard, che cosa succederebbe durante l'esecuzione della sequenza di istruzioni originaria sopra riportata?

4.27.4 [20] <4.8> Supponendo che il processore sia dotato di propagazione, specificare quali segnali vengano asseriti dall'unità di rilevamento delle criticità e dall'unità di propagazione disegnate nella Figura 4.62 in ciascuno dei primi sette cicli di clock dell'esecuzione della sequenza di istruzioni riportata sopra.

4.27.5 [10] <4.8> Se non ci fosse la propagazione, quali segnali di ingresso e di uscita aggiuntivi sarebbero richiesti per l'unità di riconoscimento degli hazard della Figura 4.62? Utilizzando la sequenza di istruzioni riportata come esempio, spiegare per ciascun segnale il motivo per cui occorre aggiungerlo.

4.27.6 [20] <4.8> Per la nuova unità di rilevamento degli hazard dell'Esercizio 4.27.5, specificare quali segnali di uscita vengono asseriti in ciascuno dei primi cinque cicli di clock dell'esecuzione di questa sequenza di istruzioni.

4.28 L'importanza di avere un buon predittore dei salti condizionati dipende da quanto spesso i salti condizionati vengono eseguiti. Questo esercizio consentirà di valutare sia l'accuratezza dei predittori dei salti sia il tempo che viene perso negli stalli dovuti agli errori nella predizione dei salti condizionati. Si assuma che la suddivisione delle istruzioni nelle diverse categorie sia la seguente:

Tipo R	beqz/ bnez	jal	lw	sw
40%	25%	5%	25%	5%

Inoltre, si supponga che la predizione dei salti condizionati abbia la seguente accuratezza:

Sempre effettuato	Sempre non effettuato	Su 2 bit
45%	55%	85%

4.28.1 [10] <4.8> I cicli di clock associati agli stalli dovuti a errori nella predizione delle branch aumentano il CPI. Qual è l'aumento del CPI per errori nella predizione delle branch quando viene utilizzato un predittore del tipo "salto sempre effettuato"? Si supponga che il risultato del confronto delle branch venga determinato nello stadio di ID e applicato nello stadio EX, che non ci siano hazard sui dati e che non vengano utilizzati delay slot.

4.28.2 [10] <4.8> Risolvere l'Esercizio 4.28.1 considerando un predittore che suppone che i salti siano sempre non effettuati.

4.28.3 [10] <4.8> Risolvere l'Esercizio 4.28.1 considerando un predittore su 2 bit.

4.28.4 [10] <4.8> Quando si utilizza un predittore su 2 bit, quale incremento di velocità si otterrebbe se si convertissero metà delle istruzioni di branch in 2 istruzioni che utilizzano la ALU? Si supponga che le istruzioni di salto condizionato che vengono predette correttamente o non correttamente abbiano la stessa probabilità di essere sostituite.

4.28.5 [10] <4.8> Quando si utilizza un predittore su 2 bit, quale incremento di velocità si otterrebbe se si convertissero metà delle istruzioni di branch in una coppia di istruzioni che utilizzano la ALU? Si supponga che le istruzioni di salto condizionato che vengono predette correttamente o non correttamente abbiano la stessa probabilità di essere sostituite.

4.28.6 [10] <4.8> Alcune istruzioni di salto condizionato sono più predicibili di altre. Sapendo che l'80% dei salti condizionati che vengono effettuati sono facili da predire perché saltano indietro all'inizio di un ciclo e vengono predetti quasi sempre correttamente, quale sarebbe l'accuratezza del predittore a 2 bit sul rimanente 20% delle istruzioni di salto condizionato?

4.29 Questo esercizio esamina l'accuratezza di vari predittori di salti condizionati per la seguente sequenza di risultati ripetuti (per esempio all'interno di cicli cicli) della predizione: `P, NP, P, P, NP`.

4.29.1 [5] <4.9> Qual è l'accuratezza della predizione "sempre preso" e "sempre non preso" per la sequenza di risultati riportata?

4.29.2 [5] <4.9> Qual è l'accuratezza del predittore a 2 bit quando il risultato delle prime quattro branch è quello riportato e il predittore, all'inizio, si trova nello stato riportato in basso a sinistra della Figura 4.63 ("predetto non preso")?

4.29.3 [10] <4.9> Qual è l'accuratezza del predittore a 2 bit se la sequenza di risultati riportata si ripetesse all'infinito?

4.29.4 [30] <4.9> Progettare un predittore che raggiunga un'accuratezza del 100% se la sequenza dei risultati del confronto è ripetuta sempre allo stesso modo. Il predittore dovrà essere costituito da un circuito sequenziale che fornisce in uscita la predizione (1 se il salto deve essere effettuato, 0 altrimenti) e riceve in ingresso solamente il clock e un segnale di controllo che indica quando l'istruzione in esecuzione è un salto condizionato.

4.29.5 [10] <4.9> Qual è l'accuratezza del predittore costruito per risolvere l'Esercizio 4.29.4 quando la sequenza dei risultati del confronto, associata alle istruzioni di salto condizionato, è esattamente l'opposto di quella riportata?

4.29.6 [20] <4.9> Risolvere l'Esercizio 4.29.4 realizzando un predittore che sia capace, dopo un periodo di prova durante il quale alcune predizioni possono rivelarsi sbagliate, di predire perfettamente il risultato dei confronti sia per la sequenza riportata sia per la sua opposta. Il predittore dovrà avere un input che gli dice quale sia la predizione corretta. (Suggerimento: questo segnale deve permettere al predittore di capire quale delle due sequenze sia quella in ingresso.)

4.30 Questo esercizio esplora come la gestione delle eccezioni influenzi il progetto di una pipeline. I primi tre problemi si riferiscono alla seguente coppia di istruzioni:

Istruzione 1	Istruzione 2
`beqz x11, ETICHETTA`	`lw x11, 0(x12)`

4.30.1 [5] <4.10> Quali eccezioni possono essere generate da queste istruzioni? Specificare in quale stadio della pipeline può essere identificata ciascuna di queste eccezioni.

4.30.2 [10] <4.10> Se la procedura di risposta alle eccezioni avesse un indirizzo diverso per ogni eccezione, mostrare come si dovrebbe modificare la pipeline per poter gestire ciascuna eccezione. Si può supporre che l'indirizzo di ciascuna procedura sia stato specificato quando il processore è stato progettato.

4.30.3 [10] <4.10> Descrivere che cosa succederebbe nella pipeline se la seconda istruzione della coppia venisse prelevata dalla memoria subito dopo la prima istruzione e la prima istruzione generasse la prima eccezione identificata nell'Esercizio 4.30.1. Mostrare il diagramma di esecuzione della pipeline dal momento in cui la prima istruzione viene prelevata dalla memoria istruzioni fino a quando la prima istruzione della procedura di risposta alle eccezioni viene completata.

4.30.4 [20] <4.10> Nella risposta alle eccezioni vettorizzata, la tabella che contiene gli indirizzi delle procedure di risposta alle diverse eccezioni si trova in una locazione nota (prefissata) della memoria. Modificare la pipeline per implementare questo meccanismo di risposta alle eccezioni. Risolvere l'Esercizio 4.30.3 utilizzando questa pipeline modificata e il meccanismo vettorizzato di risposta alle eccezioni.

4.30.5 [15] <4.10> Si vuole emulare la risposta vettorizzata alle eccezioni (descritta nell'Esercizio 4.30.4) su una macchina che prevede un unico indirizzo, prefissato, per la procedura di risposta alle eccezioni. Scrivere la sequenza di istruzioni che deve essere poi caricata in memoria a partire da questo indirizzo.) Suggerimento: il codice dovrà identificare l'eccezione, leggere l'indirizzo corretto nella tabella del vettore delle eccezioni e trasferire l'esecuzione alla procedura corretta per la gestione di quell'eccezione.)

4.31 Questo esercizio confronterà le prestazioni dei processori senza parallelizzazione dell'esecuzione a una via con quelli dotati di parallelizzazione a due vie tenendo conto delle modifiche richieste ai programmi per ottimizzare l'esecuzione su un processore che esegua in parallelo due istruzioni. I problemi si riferiscono a questo ciclo scritto in C:

```
for(i=0;i!=j;i+=2)
    b[i]=a[i]-a[i+1];
```

Un compilatore facendo poco o nulla per ottimizzare il codice potrebbe produrre la sequenza di istruzioni RISC-V seguente:

```
        addi    x12, x0, 0
        jal     FINE
INIZIO: slli    x5, x12, 3
        add     x6, x10, x5
        lw      x7, 0(x6)
        lw      x29, 4(x6)
        sub     x30, x7, x29
        add     x31, x11, x5
        sw      x30, 0(x31)
        addi    x12, x12, 2
FINE:   bne     x12, x13, INIZIO
```

Questo codice utilizza i registri seguenti:

i	j	a	b	Valori temporanei
x12	x13	x10	x11	x5–x7, x29–x31

Si supponga che il processore a 2 vie, con smistamento statico delle istruzioni, abbia per questo esercizio le seguenti proprietà:

1. Un'istruzione deve essere un'operazione sulla memoria; l'altra può essere un'istruzione aritmetico-logica o un salto condizionato.
2. Il processore possiede tutti i possibili cammini di propagazione tra i diversi stadi (compresi i cammini allo stadio ID per la risoluzione degli hazard associati ai salti condizionati).
3. Il processore possiede una predizione perfetta dei salti condizionati.
4. Due istruzioni non possono risiedere nello stesso pacchetto di esecuzione se una dipende dall'altra (Paragrafo 4.11.)
5. Se uno stallo diventa necessario, entrambe le istruzioni del pacchetto di esecuzione incriminato devono andare in stallo (Paragrafo 4.11.)

Svolgendo questi esercizi, si osservi quale sforzo venga richiesto per generare del codice che produca un incremento delle prestazioni vicino a quello ottimale.

4.31.1 [30] <4.11> Disegnare un diagramma di pipeline che mostri come il codice RISC-V riportato sopra venga eseguito sul processore a due vie. Si supponga che il programma esca dal ciclo dopo due iterazioni.

4.31.2 [10] <4.11> Qual è il guadagno di prestazioni passando da un processore a una via a un processore a due vie? (Si supponga che il ciclo venga eseguito migliaia di volte.)

4.31.3 [10] <4.11> Riordinare/riscrivere il codice RISC-V riportato sopra per ottenere prestazioni migliori su un processore a una via. (Suggerimento: utilizzare l'istruzione `beqz x13, TERMINATO` per saltare completamente il ciclo quando `j = 0`.)

4.31.4 [20] <4.11> Riordinare/riscrivere il codice RISC-V riportato sopra per ottenere prestazioni migliori su un processore a due vie (senza srotolare i cicli.)

4.31.5 [30] <4.11> Ripetere l'Esercizio 4.31.1, ma questa volta utilizzare il codice ottimizzato dell'Esercizio 4.31.4.

4.31.6 [10] <4.101> Qual è l'incremento di velocità nel passaggio dal processore a una via al processore a due vie quando si utilizza il codice prodotto per gli Esercizi 4.31.3 e 4.31.4?

4.31.7 [10] <4.11> Srotolare il codice RISC-V dei cicli dell'Esercizio 4.31.3 in modo tale che ciascuna iterazione del ciclo srotolato contenga due iterazioni del ciclo originario. Poi riordinare/riscrivere il codice srotolato per ottenere prestazioni migliori sul processore a una via. Si può supporre che `j` sia un multiplo di 4.

4.31.8 [20] <4.11> Srotolare il codice RISC-V dei cicli dell'Esercizio 4.31.4 in modo tale che ciascuna iterazione del ciclo srotolato contenga due iterazioni del ciclo originario. Poi riordinare/riscrivere il codice srotolato per ottenere prestazioni migliori sul processore a due vie. Si può supporre che `j` sia un multiplo di 4. (Suggerimento: riorganizzare il ciclo in modo tale che i calcoli compaiano al di fuori del ciclo e alla fine del ciclo. Si può supporre che i valori contenuti nei registri temporanei non siano richiesti dopo il ciclo.)

4.31.9 [10] <4.11> Qual è l'incremento di velocità quando si passa da un processore a una via a un processore a due vie quando viene eseguito il codice non srotolato e non ottimizzato dell'Esercizio 4.31.7 e 4.31.8?

4.31.10 [30] <4.11> Ripetere gli Esercizi 4.31.8 e 4.31.9, ma questa volta supponendo che il processore a due vie possa eseguire contemporaneamente due istruzioni aritmetico-logiche. (In altre parole, la prima istruzione di un pacchetto può essere di tipo qualsiasi, ma la seconda deve essere un'istruzione aritmetica o un'istruzione logica. Due operazioni sulla memoria non possono essere avviate a esecuzione nello stesso istante.)

4.32 Questo esercizio riguarda l'efficienza energetica e la sua relazione con le prestazioni. I problemi di questo esercizio ipotizzano il seguente consumo di energia da parte di queste unità funzionali: memoria istruzioni, registri e memoria dati. Si può supporre che gli altri componenti funzionali dell'unità di elaborazione consumino una quantità trascurabile di energia ("Lettura registro" e "Scrittura registro" si riferiscono solo al register file).

Mem-I	Lettura 1 registro	Scrittura registro	Lettura Mem-D	Scrittura Mem-D
140 pJ	70 pJ	60 pJ	140 pJ	120 pJ

Si supponga che i componenti dell'unità di elaborazione abbiano le seguenti latenze (si può supporre che gli altri componenti dell'unità di elaborazione abbiano latenze trascurabili):

Mem-J	Unità controllo	Lettura/Scrittura registri	ALU	Mem-D L/S
200 ps	150 ps	90 ps	90 ps	250 ps

4.32.1 [5] <4.3, 4.7, 4.15> Quanta energia viene richiesta per eseguire un'istruzione `add` in una CPU a singolo ciclo e in una pipeline a cinque stadi?

4.32.2 [10] <4.7, 4.15> Qual è l'istruzione RISC-V peggiore in termini di energia consumata, e a quanto ammonta l'energia richiesta per la sua esecuzione?

4.32.3 [10] <4.7, 4.15> Se la riduzione di energia fosse l'elemento più importante, come dovrebbe essere modificata la pipeline? Quale sarebbe la quantità di energia consumata da un'istruzione `ld` dopo la modifica?

4.32.4 [10] <4.7, 4.15> Quali altre istruzioni potrebbero potenzialmente beneficiare dei cambiamenti discussi nell'Esercizio 4.32.3?

4.32.5 [10] <4.7, 4.15> Qual è l'impatto sulle prestazioni di una CPU dotata di pipeline delle modifiche introdotte nell'Esercizio 4.32.3?

4.32.6 [10] <4.7, 4.15> Si può eliminare il segnale di controllo di lettura della memoria dati (`MemRead`) e quindi leggere la memoria dati a ogni ciclo di clock; avremo cioè `MemRead = 1` permanentemente. Spiegare perché il processore funzionerebbe ancora correttamente dopo questa modifica. Qual è l'impatto di questa modifica sul periodo di clock e sull'energia consumata?

4.33 Quando i chip di silicio vengono fabbricati, difetti nei materiali (per es. nel silicio) ed errori di fabbricazione possono fare sì che i circuiti risultino difettati. Un difetto molto comune è chiamato "cross-talk" (interferenza) e si verifica quando un segnale che transita su una pista influenza un altro segnale che transita su un'altra pista. Un tipo particolare di malfunzionamento dovuto al cross-talk si manifesta quando uno dei due segnali è trasportato da una pista erroneamente collegata a un valore logico costante, per esempio all'alimentazione elettrica. In questo caso si ottiene un segnale che rimane sempre bloccato a 0 o a 1, indipendentemente da quale sia il valore logico che dovrebbe assumere. I problemi successivi si riferiscono al bit 0 del Registro di Scrittura del Register File della Figura 4.25.

4.33.1 [10] <4.3, 4.4> Si supponga che la verifica del funzionamento del processore venga effettuata: (1) scrivendo dei dati, a piacimento, nel PC, nei registri, nella memoria dati e nella memoria istruzioni; (2) eseguendo una sola istruzione; (3) leggendo al termine dell'esecuzione il contenuto del PC, delle memorie e dei registri, e questi valori verranno esaminati per determinare se si sia verificato un qualche malfunzionamento. Progettare un test, cioè determinare un valore opportuno da scrivere nel PC, nelle memorie e nei registri, che consenta di determinare se il bit analizzato è bloccato a 0.

4.33.2 [10] <4.3, 4.4> Risolvere l'Esercizio 4.33.1 nel caso in cui si voglia determinare se il bit analizzato è bloccato a 1. È possibile utilizzare un unico test per verificare se il bit è bloccato a 0 o a 1? Se sì, spiegare come; se non è possibile, spiegare perché.

4.33.3 [10] <4.3, 4.4> Se sapessimo che in un processore il bit analizzato è bloccato a 1, potremmo comunque utilizzare il processore? Per potere utilizzare comunque questo processore occorrerebbe convertire i programmi che vengono eseguiti su un RISC-V normale in programmi che possano essere eseguiti su questo processore, tenendo conto del malfunzionamento. Si supponga che ci sia abbastanza memoria istruzioni per aggiungere delle istruzioni e abbastanza memoria dati libera per memorizzare altri dati.

4.33.4 [10] <4.3, 4.4> Risolvere l'Esercizio 4.33.1 supponendo che il malfunzionamento da identificare sia sul segnale di controllo `MemRead`, il quale diventa 0 se il segnale `branch` è anch'esso 0; non ci sono malfunzionamenti in tutti gli altri casi.

4.33.5 [10] <4.3, 4.4> Risolvere l'Esercizio 4.33.1 supponendo che il malfunzionamento da identificare sia sul segnale di controllo `MemRead`, il quale diventa 1 se il segnale di controllo `RegRd` diventa 1; non ci sono malfunzionamenti in tutti gli altri casi. (Suggerimento: questo problema richiede delle conoscenze sui sistemi operativi. Identificate cosa possa causare un errore di segmentazione della memoria o *segmentation fault*.)

Risposte alle domande di autovalutazione

Paragrafo 4.1, pagina 223 – 3 o 5: unità di controllo, unità di elaborazione, memoria. Mancano input e output.

Paragrafo 4.2, pagina 227 – Falso. Gli elementi di stato sincronizzati sui fronti rendono possibile e non ambigua la lettura e scrittura simultanea.

Paragrafo 4.3, pagina 233 – I. a., II. c.

Paragrafo 4.4, pagina 246 – Sì, i segnali di Branch e ALUOP0 sono identici. Inoltre si può utilizzare la flessibilità offerta dai bit il cui valore è indifferente

per raggruppare assieme altri segnali. ALUSrc e MemReg possono essere raggruppati in un unico segnale se si impostano a 1 e 0 rispettivamente i due bit associati al segnale MemtoReg. ALUOp1 e MemtoReg possono essere raggruppati dopo avere negato uno dei due, se si impostano i bit indifferenti di MemtoReg a 1. Non è necessario un inverter: è sufficiente cambiare l'ordine degli input nel multiplexer associato a MemtoReg!

Paragrafo 4.5, online – Forse; se il Segnale "PCSource" viene sempre impostato a zero quando è specificato il valore indifferente (cosa che succede nella maggior parte delle situaizoni); in questo caso "PCSource" è identico al segnale "PCWriteCond".

Paragrafo 4.6, pagina 259 – 1. Stallo sul risultato della `lw`. 2. Evitare lo stallo nella terza istruzione per l'hazard sul dato per la lettura dopo la scrittura su `x11` propagando il risultato della `add`. 3. Non occorrono stalli, anche senza propagazione.

Paragrafo 4.7, pagina 272 – Le affermazioni 2 e 4 sono corrette; le altre due sono errate.

Paragrafo 4.9, pagina 293 – 1. Predetto non preso. 2. Predetto preso. 3. Predizione dinamica.

Paragrafo 4.10, pagina 300 – La prima istruzione, dato che viene eseguita prima delle altre.

Paragrafo 4.11, pagina 313 – 1. Entrambi. 2. Entrambi. 3. Software. 4. Hardware. 5. Hardware. 6. Hardware. 7. Entrambi. 8. Hardware. 9. Entrambi.

Paragrafo 4.13, pagina 322 – Le prime due affermazioni sono false, mentre le ultime due affermazioni sono vere.

5 Grande e veloce: la gerarchia delle memorie

Idealmente si desidererebbe possedere una memoria indefinitamente grande tale che ogni particolare … parola risulti immediatamente disponibile … Siamo … costretti a riconoscere la possibilità di costruire una gerarchia delle memorie, ognuna delle quali abbia una capacità maggiore rispetto alla precedente, ma sia accessibile meno velocemente.

A.W. Burks, H.H. Goldstine e J. von Neumann,
Discussione preliminare sul progetto logico di uno strumento elettronico di computazione, 1946

5.1 Introduzione

Fin dal principio dell'era dell'informatica i programmatori hanno sperato di avere a disposizione una quantità illimitata di memoria che fosse al contempo veloce. Tutti gli argomenti che saranno illustrati in questo capitolo sono incentrati sulle tecniche che permettono di dare ai programmatori quest'illusione, ossia di poter usufruire di una memoria che sia contemporaneamente veloce e di dimensioni illimitate. Prima di esaminare come venga effettivamente realizzata questa illusione, consideriamo una semplice analogia che illustra i princìpi chiave e i meccanismi che vengono utilizzati.

Immaginiamo uno studente che debba scrivere una relazione sulle tappe principali della storia dell'architettura dei calcolatori. Lo studente è seduto a una scrivania della biblioteca di ingegneria o di informatica e sta esaminando una serie di testi che ha prelevato dagli scaffali. A un certo punto si accorge che molti dei calcolatori che deve descrivere sono illustrati nei libri che ha davanti, ma non trova alcuna informazione sull'EDSAC; così, si alza e torna a guardare sugli scaffali, terminando la ricerca solo dopo aver trovato un testo dedicato ai primi calcolatori inglesi che descrive l'EDSAC. Dopo che lo studente ha selezionato un certo numero di libri e li ha riposti sulla sua scrivania, ci sarà un'alta probabilità che la maggior parte degli argomenti a cui è interessato sia trattata in questi libri; egli passerà perciò la maggior parte del tempo a consultare i libri raccolti, senza dover tornare ogni volta agli scaffali. Avere a disposizione molti libri sulla scrivania consente allo studente di risparmiare

molto tempo rispetto ad avere un solo libro e doversi continuamente alzare per riporlo sullo scaffale e prenderne un altro.

Lo stesso principio ci permette di creare l'illusione di avere a disposizione in un calcolatore una memoria di grandi dimensioni a cui poter accedere con la stessa velocità con cui si accede a una memoria di piccole dimensioni. Proprio come lo studente non ha bisogno di dover consultare, con la stessa probabilità, tutti i libri della biblioteca, un programma non deve accedere con la stessa probabilità a tutte le sue istruzioni e a tutti i suoi dati contemporaneamente. Se così non fosse, sarebbe impossibile pensare di rendere veloce l'accesso alla memoria e, allo stesso tempo, avere una memoria di grandi dimensioni, così come è impossibile per lo studente mettere tutti i libri della biblioteca sulla sua scrivania e pensare ancora di riuscire a trovare velocemente le informazioni che gli interessano.

Il *principio di località* sta alla base del comportamento dei programmi in un calcolatore ed è del tutto simile al modo di cercare informazioni in una biblioteca. Questo principio afferma che un programma, in un certo istante di tempo, accede soltanto a una porzione relativamente piccola del suo spazio di indirizzamento, proprio come lo studente accede solo a una piccola porzione dei libri nella biblioteca. Esistono due diversi tipi di località:

- **località temporale** (località nel tempo): quando si fa riferimento a un elemento, c'è la tendenza a fare riferimento allo stesso elemento dopo poco tempo; se lo studente ha appena portato un libro sulla scrivania per consultarlo, è probabile che abbia bisogno di consultarlo ancora poco dopo;
- **località spaziale** (località nello spazio): quando si fa riferimento a un elemento, c'è la tendenza a fare riferimento poco dopo ad altri elementi che hanno l'indirizzo vicino a esso. Per esempio, quando lo studente ha prelevato il libro sui primi calcolatori costruiti in Inghilterra per trovare informazioni sull'EDSAC, ha anche notato che il testo a fianco riguardava le prime calcolatrici meccaniche; così, ha deciso di portare sulla sua scrivania anche questo libro, in cui più tardi potrà trovare delle informazioni utili. I bibliotecari dispongono i libri su uno stesso argomento sullo stesso scaffale per aumentare la località spaziale. Più avanti in questo capitolo vedremo come la località spaziale venga sfruttata dalle gerarchie delle memorie.

Località temporale: principio secondo cui se si accede a una determinata locazione di memoria, è molto probabile che vi si acceda di nuovo dopo poco tempo.

Località spaziale: principio secondo cui se si accede a una determinata locazione di memoria, è molto probabile che si acceda alle locazioni vicine a essa dopo poco tempo.

La località emerge in modo naturale dalle strutture di controllo semplici e tipiche dei programmi, proprio come l'accesso ai libri della biblioteca mostra un comportamento naturalmente locale. Per esempio, la maggior parte dei programmi contiene dei cicli; le istruzioni e i dati che si trovano all'interno di queste strutture di controllo vengono letti ripetutamente dalla memoria, dimostrando così un elevato livello di località temporale. Dato che le istruzioni di un programma normalmente vengono caricate in sequenza dalla memoria, i programmi presentano anche un'elevata località spaziale. Anche l'accesso ai dati è intrinsecamente caratterizzato da una località spaziale; per esempio, è naturale che l'accesso agli elementi consecutivi di un vettore o di una struttura dati abbia un elevato grado di località spaziale.

Il principio di località viene sfruttato strutturando la memoria di un calcolatore in forma gerarchica. La **gerarchia delle memorie** consiste in un insieme di livelli di memoria, ciascuno caratterizzato da una diversa velocità e dimensione: a parità di capacità, le memorie più veloci hanno un costo più elevato per singolo bit di quelle più lente, perciò, di solito, sono più piccole.

La **Figura 5.1** mostra che la memoria più veloce è posta vicino al processore e quella più lenta, meno costosa, è posizionata più lontano. L'obiettivo è di fornire all'utente una quantità di memoria pari a quella disponibile nella tecnologia più economica, consentendo allo stesso tempo una velocità di accesso pari a quella garantita dalla memoria più veloce.

Gerarchia delle memorie: una struttura che utilizza più livelli di memoria; all'aumentare della distanza dal processore, crescono sia la dimensione sia il tempo di accesso alla memoria mentre diminuisce il costo per bit.

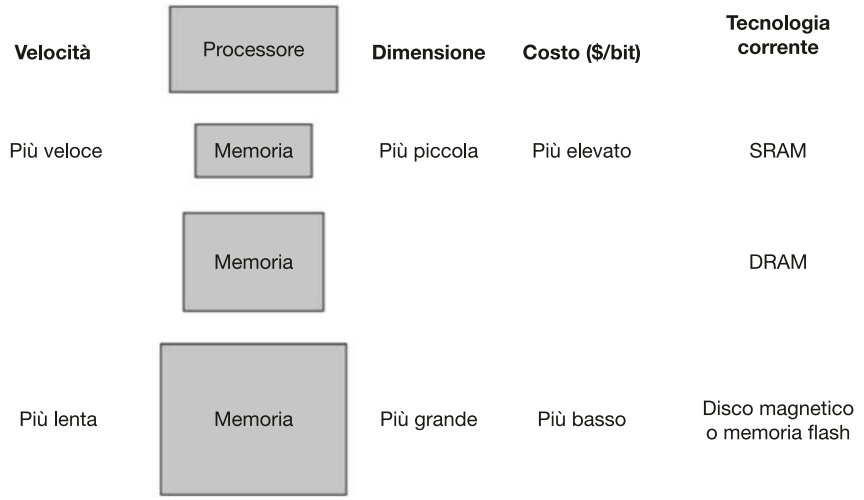


Figura 5.1 Struttura base di una gerarchia delle memorie. Realizzando il sistema di memoria in maniera gerarchica, l'utente ha l'impressione di utilizzare una memoria grande come quella del livello contenente la memoria di dimensioni maggiori, ma con una velocità di accesso pari a quella della memoria realizzata con la tecnologia più veloce. Le memorie flash hanno sostituito i dischi in molti dispositivi embedded e potrebbero portare in breve tempo all'introduzione di un nuovo livello di memoria nei calcolatori desktop e nei server (Paragrafo 5.2).

Anche i dati sono organizzati in modo gerarchico: un livello più vicino al processore contiene in generale un sottoinsieme dei dati memorizzati in ognuno dei livelli sottostanti, e tutti i dati si trovano memorizzati nel livello più basso della memoria. Si consideri ancora l'analogia con la biblioteca: i libri sulla scrivania rappresentano il sottoinsieme dei libri della biblioteca su cui lo studente sta lavorando; a sua volta la biblioteca è un sottoinsieme di tutte le biblioteche dell'università. Quanto più ci si allontana dal processore, tanto più aumenta il tempo necessario per accedere ai dati, proprio come accade per la consultazione dei libri in una gerarchia di biblioteche universitarie.

Blocco: detto anche linea, è l'unità minima di informazione che può essere presente o assente in una cache.

Una gerarchia delle memorie può essere composta da più livelli, ma i dati vengono di volta in volta trasferiti solo tra due livelli vicini; possiamo perciò concentrare la nostra attenzione su due soli livelli di memoria. Quello superiore, più vicino al processore, è più piccolo e veloce del livello inferiore, poiché utilizza una tecnologia più costosa. La più piccola quantità di informazione che può essere presente o assente in questa gerarchia su due livelli è denominata **blocco**, o **linea**, come illustrato nella **Figura 5.2**; nell'analogia con la biblioteca, il blocco di informazione corrisponde a un libro.

Se il dato richiesto dal processore è contenuto in uno dei blocchi presenti nel livello superiore, si dice che la richiesta ha avuto successo e si denota questo evento con il termine inglese *hit* (letteralmente, "colpito"); una hit corrisponde allo studente che trova le informazioni che cerca in uno dei libri che ha a disposizione sulla scrivania. Se il dato non viene trovato nel livello superiore della gerarchia, si dice che la richiesta fallisce e si indica questo evento con il termine *miss* (letteralmente, "mancato"). In questo secondo caso, per trovare il blocco che contiene il dato richiesto occorre accedere al livello inferiore della gerarchia. Proseguendo con l'analogia della biblioteca, lo studente si alza dalla scrivania e va a cercare il libro che gli serve sugli scaffali. La frequenza di hit, chiamata **hit rate** (letteralmente, "frequenza delle hit"), è la frazione degli accessi alla memoria nei quali il dato desiderato è stato trovato nel livello superiore; spesso l'hit rate viene utilizzato come indice delle prestazioni della gerarchia delle memorie. La frequenza delle miss, indicata con il termine

Hit rate: detto anche frequenza delle hit, è la frazione degli accessi alla memoria nei quali la parola cercata viene trovata in un certo livello della gerarchia.